

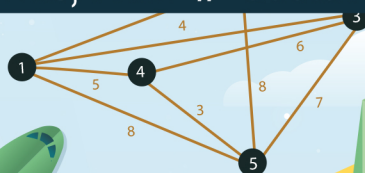
APRENDE MODELANDO:

Programación Lineal Entera-Mixta

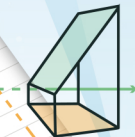
Guía de aprendizaje desde el problema a la solución con AMPL

Daniel Morillo, Ph.D. // Gustavo Gatica, Ph.D.

AMPL: A Mathematical Programming Language



(a)



x

Minimizar $z = \sum_{i \in P} \sum_{j \in T} x_{ij} \cdot c_{ij}$

Sujeto a:

$$\sum_{j \in T} x_{ij} = 1, \quad i \in P$$
$$\sum_{i \in P} t_{ij} \cdot x_{ij} = \sum_{i \in T} t_{ij} \cdot x_{ij}, \quad j \in T$$

(4.52)



Prólogo de Paolo Toth

Aprende Modelando: Programación Lineal Entera-Mixta

Guía de aprendizaje, desde el problema a la
solución con AMPL



Pontificia Universidad
JAVERIANA
Cali



**Universidad
Andrés Bello®**

Santiago de Cali, 2022

Aprende Modelando: Programación Lineal Entera-Mixta

Guía de aprendizaje, desde el problema a la
solución con AMPL

Daniel Morillo-Torres, Ph.D.

Pontificia Universidad Javeriana Cali, Colombia

Gustavo Gatica, Ph.D.

Universidad Andrés Bello, Chile



Pontificia Universidad
JAVERIANA
Cali



**Universidad
Andrés Bello®**

Santiago de Cali, 2022

Morillo-Torres, Daniel

Aprende modelando : programación lineal entera-mixta : guía de aprendizaje, desde el problema a la solución con AMPL / Daniel Morillo Torres, Gustavo Gatica. -- Santiago de Cali : Pontificia Universidad Javeriana, Sello Editorial Javeriano, 2022.

271 páginas: ilustraciones, tablas ; 24 cm

Incluye referencias bibliográficas.

ISBN: 978-628-7536-55-5

ISBN(e): 978-628-7536-57-9

1. Optimización matemática 2. Programación lineal 3. Optimización combinatoria 4. Algoritmos 5. Análisis numérico 6. Investigación operacional 7. Programación (matemáticas) 8. Modelos matemáticos I. Morillo-Torres, Daniel II. Gatica, Gustavo III. Facultad de Ingeniería y Ciencias. Departamento de Ingeniería Civil e Industrial. SCDD 519.6 ed. 23

CO-CaPUJ
lmc/2022



Pontificia Universidad Javeriana Cali - Colombia
Universidad Andrés Bello Santiago de Chile - Chile

Aprende Modelando: Programación Lineal Entera-Mixta
Guía de aprendizaje, desde el problema a la solución con AMPL

Editores

© Daniel Morillo-Torres, Ph.D.

© Gustavo Gatica, Ph.D.

ISBN: 978-628-7536-55-5

ISBN: (e): 978-628-7536-57-9

Formato: 17 x 24 cms

Coordinación editorial: Claudia Lorena González González

Asistente editorial: Jennifer Ramírez Martínez

Diagramación: Luisa Fernanda Panteves Ospina y Jennifer Ramírez Martínez

Diseño de portada: Karol Andrea Rojas Solarte

Corrección de estilo: Luisa Fernanda Panteves Ospina y María Victoria Gutiérrez Padilla

Pontificia Universidad Javeriana Cali

Calle 18 N°118-250

Teléfonos (57-2) 3218200

Santiago de Cali, Colombia, 2022

Universidad Andrés Bello

República 239

Teléfonos (56) 600 228 6262

Santiago de Chile, Chile, 2022

El contenido de esta publicación es responsabilidad absoluta de su autor y no compromete el pensamiento de la Institución. Este libro no podrá ser reproducido por ningún medio impreso o de reproducción sin permiso escrito de los titulares del *copyright*.

*Dedicado a
mi amada compañera.*
Daniel Morillo-Torres

*Dedicado a
esposa, hijas y familia.*
Gustavo Gatica

Agradecimientos

Los autores agradecen a cada una de las unidades académicas por apoyar la labor docente de un investigador. En particular, agradecemos a la Editorial de la Pontificia Universidad Javeriana Cali (PUJ Cali), por la logística, edición y distribución de nuestro libro.

Un factor relevante y no menos importante es el equipo de colegas y estudiantes que contribuyeron en la selección, programación y comentarios. En particular, agradecemos a Paolo Toth, Ph.D. por sus consejos, a la ingeniera María Victoria Gutiérrez Padilla por su constante apoyo e invaluable dedicación, a Consuelo Castro, M.Sc. por sus reflexiones, y a las ingenieras: Aura María Vargas Quintero, Sara Lucía Londoño Barreto e Ivana Valentina Brand Mutis por su apoyo.

Presentación

Daniel Morillo-Torres recibió el grado de Ingeniería Industrial en el 2010, el grado de Maestría en Ingeniería de Sistemas en 2012, ambos en la Universidad Nacional de Colombia (Medellín, Colombia), y el grado de Doctor en Informática, con reconocimiento *cum laude*, de la Universidad Politécnica de Valencia, en España. Ganador en el año 2013 de la iniciativa Enlaza Mundos de la Agencia de Educación Superior de Medellín (Sapiencia), y ganador de la convocatoria Doctorado en el Exterior 2015 del Departamento Administrativo de Ciencia, Tecnología e Innovación (Colciencias). El profesor Morillo ha trabajado los últimos años en la optimización de problemas de programación de tareas y ruteo de vehículos. Sus intereses de investigación incluyen: investigación de operaciones, programación de tareas, algoritmos metaheurísticos, inteligencia artificial, optimización multi-objetivo y programación lineal entera-mixta. Actualmente el Doctor Morillo es profesor tiempo completo e investigador en el Departamento de Ingeniería Civil e Industrial de la Facultad de Ingeniería en la Pontificia Universidad Javeriana Cali, en Colombia.

Gustavo Gatica es Ingeniero en Computación de la Universidad de La Serena y Doctor en Ciencias de la Ingeniería de la Universidad de Santiago de Chile. El profesor Gatica inició su carrera académica en el año 2003 y actualmente es investigador de la Universidad Andrés Bello de Chile. Su línea de investigación se complementa con el desarrollo de aplicaciones informáticas que integran clásicos problemas de ruteo de vehículos, programación de tareas y logística, con algoritmos heurísticos y metaheurísticos para resolverlos. El Doctor Gatica tiene formación en análisis de datos, *Big Data* e inteligencia artificial, en donde desarrolla modelos de aprendizaje para problemas altamente combinatorios, ya sea a nivel de optimización multi-objetivo o programación lineal entera-mixta mono-objetivo.

Prologue

This book illustrates several optimization problems dealing with a large range of different real-world applications, and gives particular emphasis to the development of the mathematical models able to determine the optimal solutions of the considered problems. In particular, optimization problems that can be formulated through Linear Programming (LP) or Mixed- Integer Linear Programming (MILP) models are presented. The problems are introduced according to an increasing degree of difficulty, so as to smoothly guide the reader, step by step, on how to construct complex mathematical models.

While the majority of the books concerning optimization problems are mainly devoted to the design of heuristic and exact algorithms for the effective solution of the considered problems, the goal of this book is to facilitate the difficult task of the proper modelling of the problems to be tackled. To

this end, for each problem, the following phases are performed:

1. Correct and complete description of the problem.
2. Definition of the values of the necessary parameters (input data).
3. Choice of the decisional variables used in the mathematical model, and clear explanation of this choice.
4. Proper definition of the objective function to be minimized (or maximized) by the optimal solution.
5. Description of the constraints that must be satisfied to determine a feasible solution.

In addition, for each model the corresponding code (written in the language AMPL) is presented, and the values of the decisional variables corresponding to the optimal solution (found through the execution of the MILP CPLEX or Gurobi Solver) are reported.

The task of teaching how to construct a correct mathematical model is very difficult, much more difficult than that of teaching theorems and how to prove them. The best didactic technique is to gradually guide the students along a way which allows them to understand the similarities and the differences among the components of the different models which have been presented. And this is indeed the technique followed in this excellent book.



Paolo Toth, Ph.D.

Professor emeritus

Department of Computer Science and Engineering

Alma Mater Studiorum - Università di Bologna

Prólogo

Este libro ilustra varios problemas de optimización, que abordan una gran variedad de aplicaciones del mundo real, y hace especial hincapié en el desarrollo de los modelos matemáticos capaces de determinar las soluciones óptimas de los problemas considerados. En particular, se presentan problemas de optimización que pueden formularse mediante modelos de Programación Lineal (LP, por su sigla en inglés) o de Programación Lineal entera-mixta (MILP, por su sigla en inglés). Los problemas se introducen según un grado de dificultad creciente, para guiar al lector, paso a paso, en la construcción de modelos matemáticos complejos.

Mientras que la mayoría de los libros relativos a los problemas de optimización se dedican principalmente al diseño de algoritmos heurísticos y exactos para la solución efectiva de los problemas considerados, el objetivo de este libro es facilitar la difícil tarea de la adecuada modelización de los problemas a abordar. Para ello, para cada problema, se realizan las siguientes fases:

1. Descripción correcta y completa del problema.
2. Definición de los valores de los parámetros necesarios (datos de entrada).
3. Elección de las variables de decisión utilizadas en el modelo matemático, y explicación clara de esta elección.
4. Definición adecuada de la función objetivo que debe ser minimizada (o maximizada) por la solución óptima.
5. Descripción detallada de las restricciones que deben satisfacerse para determinar una solución factible.

Además, para cada modelo se presenta el código correspondiente (escrito en el lenguaje AMPL) y se reportan los valores de las variables de decisión correspondientes a la solución óptima (encontrada mediante la ejecución del motor de solución CPLEX o Gurobi).

La tarea de enseñar a construir un modelo matemático correcto es muy difícil, mucho más que la de enseñar teoremas y cómo demostrarlos. La mejor técnica didáctica es guiar gradualmente a los alumnos por un camino que les permita comprender las similitudes y diferencias entre los componentes de los distintos modelos que se han presentado. Y esta es, efectivamente, la técnica que se sigue en este excelente libro.



Paolo Toth, Ph.D.

Professor emeritus

Department of Computer Science and Engineering

Alma Mater Studiorum - Università di Bologna

Índice general

Capítulo 1	Introducción	1
1.1	Estructura del Libro	3
Capítulo 2	Modelos de baja complejidad	5
2.1	Problema del carpintero	5
2.2	Problema de producción tecnológica	10
2.3	Problema pesquero	13
2.4	Problema de los vehículos refrigerados (adaptado de [1])	15
2.5	Problema de asignación de buses (adaptado de [2])	17
2.6	Problema de asignación de pilotos	20
2.7	Problema de transporte	28
2.8	Problema de la inversión (adaptado de [3])	34
2.9	Problema del restaurante italiano	37
2.10	Problema de la dieta	42
2.11	Problema de compra de zapatos	48
2.12	Problema de la mochila (knapsack problem)	52
2.13	Problema de jugadores de fútbol	54
2.14	Problema de un baby shower	59

2.15 Problema de cobertura (<i>set covering problem</i>)	62
Capítulo 3 Modelos de complejidad media	69
3.1 Problema de suministro para la guerra (adaptado de [4])	69
3.2 Problema de corte de madera (adaptado de [5])	73
3.3 Problema del inventario de tapabocas	76
3.4 Problema de distribución en un avión (adaptado de [3])	79
3.5 Problema de mezcla de combustible (adaptado de [5])	83
3.6 Problema de localización de planta textil (adaptado de [5]) ...	88
3.7 Problema de producción minera	93
3.8 Problema del escape de prisión (<i>shortest path</i>)	97
3.9 Problema de planeación de producción de puertas (adaptado de [3])	101
3.10 Problema de mezcla de aleaciones	107
3.11 Problema de la repostería	112
3.12 Problema de asignación de enfermeros 1	116
3.13 Problema de los libros	120
3.14 Problema de espadas y hachas	123
3.15 Problema de la construcción del barco	127
Capítulo 4 Modelos de complejidad alta	133
4.1 Problema de los bonos	133
4.2 Problema de las metas (adaptado de [3])	138
4.3 Problema del centro de cómputo	143
4.4 Problema de los bombarderos (adaptado de [3])	146
4.5 Problema del plan agregado	153
4.6 Problema del arancel (adaptado de [3])	158
4.7 Problema de programación de actividades (<i>job shop problem</i>) .	164
4.8 Problema de las divisas	169
4.9 Problema de los muelles	172

4.10 Problema del plan de extracción de una mina	178
4.11 Problema de las rutas de pizza	183
4.12 Problema de asignación de enfermeros 2	186
4.13 Problema del aeropuerto	191
4.14 Problema de las maderas	198
4.15 Problema del túnel y la mafia	202
Capítulo 5 Guía Rápida - Uso de AMPL	211
5.1 Preguntas frecuentes	211
5.2 Instalación y ejecución	213
5.3 Interfaz de AMPL IDE	214
5.4 Archivos necesarios	215
5.5 Código para el archivo <.run>	217
5.6 Código para el archivo <.mod>	223
5.7 Código para el archivo <.dat>	229
5.8 ¿Cómo ejecutar el modelo en AMPL?	233
5.9 Un ejemplo completo	235
Bibliografía	237

Índice de figuras

2.1: Una alineación de jugadores óptima.	58
2.2: Mapa de la ciudad de Cali y las posibles localizaciones de las estaciones.	63
2.3: Solución a. plan óptimo de construcción de estaciones de bomberos.	66
3.1: Patrones de corte eficientes.	74
3.2: Distribución de peso en el avión.	83
3.3: Mezcla de crudos para el componente 1.	86
3.4: Esquema de patio de Alcatraz.	98
3.5: Balance de operarios en una semana t	106
3.6: Diagrama de <i>Gantt</i> de la solución encontrada.	131
4.1: Probabilidades de éxito y fracaso en el ataque con 3 bombarderos.	149
4.2: Efecto del logaritmo natural en una función polinómica.	151
4.3: Diagrama de <i>Gantt</i> de la solución encontrada.	169
4.4: Distribución óptima de productos en los muelles.	178
4.5: Esquema de la mina <i>Mano de Midas</i> y sus vetas de oro.	180
4.6: Diagrama de los clientes para el envío.	183
4.7: Mapa de las ciudades visitadas por la aerolínea.	192

4.8: Esquema del terreno donde se planea hacer el túnel.	204
4.9: Esquema de la ruta de escape óptima.	209
5.1: Dinámica de AMPL al resolver un problema.	212
5.2: Partes de AMPL IDE.	214
5.3: Formas de crear archivos en AMPL IDE.	215
5.4: Asistente para la creación de archivos en AMPL IDE.	216
5.5: Ejecutar un modelo en AMPL usando el directorio activo. ...	234

Índice de tablas

2.1: Tiempos de procesamiento, disponibilidad y utilidades.....	11
2.2: Demanda de autobuses según intervalo de tiempo.	17
2.3: Calificación obtenida por los pilotos para cada nave.....	21
2.4: Costos de transporte [<i>COP/und</i>] y demanda de mercados [<i>und</i>]. 28	
2.5: Solución al problema de transporte.	33
2.6: Utilidad y demanda de las especialidades.....	37
2.7: Cantidades de ingredientes requeridos para elaborar una especiali- dad.....	38
2.8: Solución al problema del restaurante italiano.....	41
2.9: Requerimiento mínimo nutricional diario.	42
2.10: Contenido nutricional y costo de los alimentos.....	43
2.11: Cantidad de alimentos mínimos para sobrevivir.....	47
2.12: Consumo vs. requerimiento en la solución óptima.....	48
2.13: Especificación de zapatos por caja.	48
2.14: Volumen y utilidad de los artículos.	52
2.15: Evaluación de cada jugador.	55
2.16: Lista de regalos y su asignación a los invitados.....	59
2.17: Costos fijos por localización de las estaciones de bomberos.	64
2.18: Población de barrios.	64

2.19: Esquema de A_{ij} que representa la cobertura de los barrios por las estaciones.....	65
3.1: Plan de producción de tapabocas.....	79
3.2: Capacidad de compartimentos.....	80
3.4: Composición de cada crudo.....	83
3.5: Especificaciones de componentes para cada tipo de gasolina.....	84
3.6: Costos (USD) de transporte unitarios entre origen y su destino.....	89
3.7: Solución problema de localización.....	92
3.8: Capacidad de explotación en toneladas.....	93
3.9: Costos en dólares por tonelada extraída.....	93
3.10: Costos fijos de explotación en dólares.....	94
3.11: Pronóstico de demanda en toneladas por cuatrimestre.....	94
3.12: Plan de extracción al mínimo costo.....	97
3.13: Tiempos en minutos para viajar de una puerta a otra.....	98
3.14: Planeación óptima de producción de puertas.....	107
3.15: Características de las minas, metales y costos.....	108
3.16: Costos de los elementos manufacturados o comprados.....	113
3.17: Tiempo de manufactura de los elementos.....	113
3.18: Plan de producción y compras para los componentes de los pasteles.....	116
3.19: Definición de turnos y demanda de enfermeros por turno....	117
3.20: Solución del problema de asignación de enfermeros.....	119
3.21: Costos de transporte de enviar una caja desde cada planta a cada cliente.....	120
3.22: Tiempos, costos y precedencias de las actividades.....	128
4.1: Datos de los bonos disponibles.....	134
4.2: Plan de inversión óptimo.....	138

4.3: Contribución de los productos al cumplimiento de los objetivos.....	139
4.4: Contribución de los productos al cumplimiento de los objetivos.	142
4.5: Costos de operación (USD) de los trabajos en la CPU 1 según la posición.....	143
4.6: Costos de operación (USD) de los trabajos en la CPU 2 según la posición.....	144
4.7: Distancia desde la base central a las plantas enemigas.	147
4.8: Probabilidad de éxito en el ataque.....	147
4.9: Productos disponibles para producir y sus utilidades.....	153
4.10: Horas requeridas por estación de trabajo para la fabricación de los productos.....	153
4.11: Días laborables y costo de inventario por mes.....	154
4.12: Demanda máxima de cada producto dependiente del mes. ..	154
4.13: Plan de producción óptimo para los siguientes 6 meses.	158
4.14: Secuencias para el desarrollo de los proyectos.	164
4.15: Tiempos de procesamiento de los proyectos dependientes del programador.....	165
4.16: Tasas de cambio entre divisas.....	170
4.17: Costos en USD de manejo e inventario por cada 100 unidades de cada producto dependiendo del muelle.	173
4.18: Capacidad y demanda en cientos estimada de cada producto.	174
4.19: Precio de venta en dólares de las vetas de oro en cada semana.	179
4.20: Costo en dólares por pizza transportada entre nodos.....	184
4.21: Turnos disponibles y demandas de enfermeros.	187
4.22: Enfermeros disponibles en la base de datos.....	187
4.23: Asignación óptima de enfermeros a los turnos y sus salarios reales.....	191
4.24: Capacidad máxima en las aduanas de cada ciudad de paso.	193

4.25: Distancia en kilómetros entre parejas de ciudades.	193
4.26: Plan de vuelo de aviones para dos días.	197
4.27: Precio de los tablones de madera en función a su cantidad..	199
4.28: Tiempo de extracción de las piedras (horas).	203

Lista de píldoras informativas

2.1: Elementos básicos de un modelo de programación lineal.....	6
2.2: Diferentes tipos de modelos lineales.....	7
2.3: Símbolos de unión ($\cup$) y de pertenece ($\in$)	8
2.4: Variables enteras ($\mathbb{Z}^+$) y continuas ($\mathbb{R}^+$).....	9
2.5: Coherencia de unidades.....	11
2.6: Dinámica entre la función objetivo y las restricciones	14
2.7: Definición de nuevas variables.....	18
2.8: Soluciones múltiples	20
2.9: Notación extendida y algebraica	21
2.10: Variables de decisión binarias o booleanas	21
2.11: Sumatorias (Σ) y para todo ($\forall$)	25
2.12: Balance en el problema de transporte.....	29
2.13: Igualdad en problemas de transporte.....	33
2.14: Componente temporal.....	39
2.15: Modelo simple vs. complejo, ¿un subíndice o dos?.....	49
2.16: Conjuntos complementarios.....	56
2.17: Restricciones conjuntas	61
2.18: Nuevos parámetros.....	65
3.1: Capacidad porcentual	71

3.2: Definición de nuevas variables 2.....	74
3.3: Balance de inventarios.....	78
3.4: Variables de decisión auxiliares.....	81
3.5: Nombre de la función objetivo.....	81
3.6: Restricciones de mezcla.....	85
3.7: Supuestos en las restricciones.....	89
3.8: Relación entre variables binarias y continuas.....	90
3.9: La gran <i>M</i> (<i>the big M</i>).....	91
3.10: Variables con diferentes subíndices.....	95
3.11: Símbolo de diferencia de conjuntos ($\setminus$).....	99
3.12: Balance en grafos (o redes).....	99
3.13: Balance de inventario 2.....	105
3.14: Alcance de una variable de decisión.....	109
3.15: Variables binarias con más información.....	110
3.16: Evitar conjuntos innecesarios.....	124
3.17: Conjuntos auxiliares.....	128
3.18: Subíndices nuevos.....	130
4.1: Variables con consecuencias en el tiempo.....	135
4.2: Variables de desviación.....	140
4.3: Conjuntos reutilizables.....	144
4.4: Probabilidad en modelos deterministas lineales.....	148
4.5: Linealizar funciones mediante el logaritmo natural.....	150
4.6: Linealizar restricciones.....	160
4.7: Linealizar la multiplicación de una variable continua y una variable binaria.....	161
4.8: Posición de los subíndices como notación.....	165
4.9: Restricciones disyuntivas.....	167
4.10: Completar parámetros.....	171
4.11: Varios objetivos.....	175

4.12: Multiplicar por subíndices.....	181
4.13: Restricciones redundantes	185
4.14: Variable no restringida en signo	188
4.15: Complementar la relación entre variables continuas y binarias	195
4.16: Modos de ejecución	200
4.17: Cotas no explícitas	204
4.18: Variables de decisión como contenedores.....	206
4.19: Funciones objetivo que no representan exactamente el objetivo	206
5.1: Nombres de los archivos.....	216
5.2: El <i>gap</i> de un modelo entero.....	221
5.3: Relajación de un modelo de programación lineal.....	222

Capítulo 1

Introducción

“Si tuviese solo una hora para salvar al mundo, dedicaría 55 minutos a definir el problema”.

Frase atribuida a Albert Einstein (1879-1955), físico teórico.

Desde sus inicios la investigación de operaciones está en constante evolución. Tomó protagonismo en un contexto doloroso, como es la Segunda Guerra Mundial, dado que con estas técnicas se pudieron planificar, secuenciar, localizar y optimizar las primeras redes de suministro. En la actualidad sigue siendo una poderosa herramienta que permite optimizar sistemas.

De forma general, la investigación de operaciones se puede definir como una rama de las matemáticas, que usa en conjunto la estadística y la ciencia de la computación para aprovechar al máximo los sistemas que son influenciados por decisiones humanas, usualmente sometidos a un uso limitado de recursos.

Dentro de la investigación de operaciones, se encuentra el campo de la optimización. Sus métodos responden a herramientas cuantitativas para enfrentar, modelar y resolver problemas reales, presentes en los más diversos ámbitos, como biología, medicina, matemáticas, química, ingeniería, e incluso sistemas sociales. Actualmente, la optimización de los procesos productivos es fundamental para el éxito de las organizaciones, ya sean públicas o privadas, más aun teniendo en cuenta la constante

exigencia del mercado para implementar los principios de la Industria 4.0: transformación digital, sistemas productivos inteligentes, uso de la inteligencia artificial para la toma de decisiones y generación de valor a través de la información.

Por otra parte, la docencia tradicional de la investigación de operaciones, en particular en pregrado, considera el modelar y resolver diversos tipos de problemas, donde destacan casos de producción, cobertura, mezclas, inversiones y transporte. Para luego, en una segunda unidad, estudiar los clásicos algoritmos de resolución de problemas. Sin embargo, una vez el profesional se encuentra bajo un problema presente en el ejercicio de su labor, o bien formándose a nivel de posgrado, se enfrenta a problemas altamente complejos, con notación matemática de alto nivel, que exigen grandes esfuerzos de cómputo en herramientas científicas de vanguardia. Problemas que el profesional, generalmente, no sabe cómo resolver.

Esta brecha surge, principalmente, porque el modelar matemáticamente implica caracterizar un sistema, para formalizarlo en lenguaje matemático, labor en la que es indispensable desarrollar habilidades lógico-matemáticas, que solo se pueden obtener mediante la práctica y el estudio de diversos ejemplos. Además, cada problema puede requerir artilugios matemáticos poco explicados, que la literatura en investigación de operaciones no cubre en un único texto. La curva de aprendizaje en optimización es compleja y la docencia tradicional explora problemas con pocas variables y complejidad moderada. Por lo anterior, y dado que constantemente surgen nuevos problemas de optimización, es necesario proveer suficientes herramientas para modelar estos desafíos.

Por otra parte, resolver problemas manualmente, en el área de optimización, puede ser una actividad altamente repetitiva y extenuante. Por esta razón, la resolución se debe ejecutar con herramientas computacionales presentes en el mercado. Destacan los motores ©CPLEX y ©Gurobi. Cada herramienta tiene una vasta documentación y cualidades que lo caracterizan. Pero, a pesar de sus bondades, estas herramientas no suelen hacer parte del plan de estudios de los clásicos cursos de Investigación de Operaciones.

Convencidos que el aprendizaje debe ser dinámico, soportado en herramientas computacionales, y con el propósito de desarrollar habilidades en optimización, los autores proponen esta guía de aprendizaje, donde se modelan y resuelven 45 problemas. Estos han sido seleccionados y clasificados en casos de baja, mediana y alta complejidad. Así, el lector podrá comprender el procedimiento de modelado, aplicar la lógica y modelar nuevos problemas con escenarios más cercanos a la vida real. La explicación de cada modelo

se facilita con la incorporación de *píldoras informativas*, que describen y justifican el porqué de las decisiones de modelado. Como herramienta de programación matemática, y dada la usabilidad y disponibilidad de licencias educacionales, se ha optado por ©AMPL, que permite utilizar ©CPLEX o ©Gurobi como motor de resolución.

Este libro, como dijo el maestro del profesor Gatica, el sacerdote Barnabita Juan Battista Picetti Serughetti, (1927-2021) “está dedicado a los niñ@s, ... que aprenden jugando”. En este caso queremos que nuestros estudiantes aprendan programando en AMPL.

Esperamos que este libro sea de vuestro agrado, y que nuestro esfuerzo en formalizar lo realizado en optimización a lo largo de los años permita desarrollar modelos cada vez más cercanos a la realidad, en donde ustedes, los futuros profesionales y maestros, puedan contribuir en sus empresas y países.

Daniel Morillo-Torres y Gustavo Gatica.

1.1 Estructura del Libro

Esta guía de estudio está compuesta por cinco capítulos. El primer capítulo es la introducción, los capítulos 2, 3 y 4 componen el cuerpo del documento. Cada uno de ellos propone 15 problemas cuidadosamente seleccionados de la bibliografía del área, adaptados y complementados con los realizados en el aula, para proporcionar una curva de aprendizaje gradual. Cada problema se resuelve detallando la definición de las variables y describiendo el funcionamiento de las restricciones. Finalmente, se presenta el modelo matemático en AMPL, con su respectiva solución. Además, en algunos problemas se podrán encontrar variantes propuestas para poner en práctica las habilidades de los estudiantes. En el Capítulo 5 se incorpora una guía rápida para el manejo de AMPL, se explica cómo ejecutar y escribir los modelos.

El acceso a todos los archivos de la solución de los problemas en AMPL (modelos, *scripts* y datos) se encuentra en “Aprende Modelando - GitHub”. [Online]. Disponible en: <https://github.com/dmorill/AprendeModelandoMILP>.

Capítulo 2

Modelos de baja complejidad

En este capítulo se proponen un conjunto de 15 problemas elementales, con una dificultad incremental en cuanto a su formulación matemática. Se considerarán diversas temáticas y contextos en cada uno de los problemas. Además, en los primeros casos se usará la notación matemática extendida y, paulatinamente, se mostrará una transición hacia la formulación algebraica. En cada problema se encontrará el código del modelo implementado en AMPL y la solución correspondiente. Este capítulo se recomienda para estudiantes que deben formular sus primeros modelos de programación lineal.

Los archivos correspondientes a la solución en AMPL de los problemas planteados en este capítulo se pueden encontrar en “Aprende Modelando - GitHub”. [Online]. Disponible en: <https://github.com/dmorill/AprendeModelandoMILP>.

2.1 Problema del carpintero

Geppeto, el carpintero de un lejano pueblo, se ha dado cuenta de que la demanda de mecedoras y de taburetes este verano será muy alta. Por esta razón desea diseñar un plan de producción de estos artículos en su taller. La elaboración de cada producto requiere de piezas de madera en diferentes tamaños: pequeñas, medianas y grandes. Una mecedora requiere 18 piezas pequeñas, 9 medianas y 2 grandes; por otra parte, un taburete requiere 2 piezas pequeñas, 2 medianas y 1 grande.

Revisando su bodega, Geppeto tiene 54, 30 y 30 piezas pequeñas, medianas y grandes, respectivamente. Finalmente, el precio de venta de cada producto es de 70 USD para mecedoras y 15 USD para taburetes.

¿Cuántas mecedoras y taburetes debe fabricar Geppeto para maximizar sus utilidades con la materia prima disponible? (asuma que se venderá todo lo que se produzca).

2.1.1 Solución

Píldora informativa 2.1: Elementos básicos de un modelo de programación lineal

- **Variables de decisión:** son letras o símbolos que representan aquello controlable en un problema de programación lineal, es decir, **son las elecciones a tomar** (los productos a fabricar, la instalación a construir, el inventario a almacenar, el camino a tomar, empleados a asignar, etc.). Sus valores son desconocidos y el propósito en un modelo matemático es encontrar los valores para alcanzar un objetivo.
- **Subíndices:** son números o letras que acompañan a una variable, generalmente de un tamaño menor, se usan para **agrupar múltiples variables** con características similares. Por ejemplo, se podrían tener dos variables X y Y , que representen la fabricación de un producto 1 y un producto 2, respectivamente. Aunque son variables diferentes, las dos representan la fabricación de productos, por tanto se pueden agrupar en una única variable con un subíndice: X_1 y X_2 . Esto se puede extrapolar a variables con múltiples subíndices que agrupan información relacionada. Por ejemplo, la variable X_{ijkt} podría representar la cantidad de producto tipo i a fabricar en la máquina j en el turno k el día t .
- **Parámetros:** son los elementos **constantes y conocidos** (por lo tanto incontrolables) del problema, como por ejemplo, un precio de venta establecido, una cantidad máxima de horas disponibles, un número de empleados en una empresa, la cantidad de material para realizar una aleación, la imposibilidad de ir de una ciudad a otra, la incompatibilidad entre cultivos, etc.

- **Función objetivo:** es una expresión matemática **lineal**, compuesta por variables y parámetros, que representa **el propósito del problema**, lo buscado con la toma de decisiones. Generalmente se busca maximizar o minimizar la expresión. Se suele representar con la letra zeta minúscula (z).
- **Restricciones:** ecuaciones ($=$) o desigualdades ($\leq$, $\geq$) **lineales** que expresan **las limitantes de un problema**. Son las expresiones que impiden que la función objetivo tome un valor infinito cuando se maximiza, o un valor menos infinito (o cero) al minimizar.

Definición de las variables de decisión

- x_1 : cantidad de mecedoras a fabricar [und].
- x_2 : cantidad de taburetes a fabricar [und].

Píldora informativa 2.2: Diferentes tipos de modelos lineales

En la investigación de operaciones existen diferentes términos que hacen referencia a modelos con características particulares, pero todos ellos son lineales.

- **Modelo de programación lineal** (*linear programming, LP*). Aunque es el nombre general, normalmente se usa para describir modelos lineales solo con variables continuas.
- **Modelo de programación entera** (*integer programming, IP*). Modelo lineal exclusivamente con variables enteras.
- **Programación binaria o programación lineal cero-uno** (*zero-one linear programming, ZOLP o binary integer programming, BIP*). Modelo lineal solo con variables binarias o booleanas $\{0, 1\}$.
- **Modelo de programación lineal entera-mixta** (*linear mixed-integer programming, MILP*). Modelo lineal con variables continuas y enteras.



En este libro se usará el término **modelo de programación lineal** desde una perspectiva más general y puede referirse cualquiera de estas variantes.

Modelo de programación lineal

Píldora informativa 2.3: Símbolos de unión ($\cup$) y de pertenece ($\in$)

- $\cup$: la unión de conjuntos permite combinar el contenido de dos conjuntos en solo uno:

$$\{1, 2, 3\} \cup \{4, 5\} = \{1, 2, 3, 4, 5\}$$

- $\in$: cuando un subíndice pertenece a un conjunto, indica que sus posibles valores solo pueden ser aquellos elementos de dicho conjunto:

$$A = \{1, 2, 3\}, \quad \text{si } i \in A \text{ entonces } i = 1 \text{ o } i = 2 \text{ o } i = 3$$

$$\text{Maximizar } z = 70 \cdot x_1 + 15 \cdot x_2 \tag{2.1}$$

Sujeto a:

$$18 \cdot x_1 + 2 \cdot x_2 \leq 54 \tag{2.2}$$

$$9 \cdot x_1 + 2 \cdot x_2 \leq 30 \tag{2.3}$$

$$2 \cdot x_1 + 1 \cdot x_2 \leq 30 \tag{2.4}$$

$$x_1, x_2 \in \mathbb{Z}^+ \cup \{0\} \tag{2.5}$$

La expresión (2.1) representa la función objetivo, que indica la utilidad al vender cada mecedora y taburete fabricado. El análisis dimensional corresponde a:

$$[USD] = \left[\frac{USD}{und} \right] \cdot [und]$$

La expresión (2.2) representa la restricción asociada a las piezas pequeñas, donde se considera el número de piezas (pequeñas) necesarias para fabricar cada producto. Estas no deben exceder el *stock* de materia prima de piezas pequeñas, que alcanza 54 unidades. De manera análoga, las expresiones (2.3) y (2.4) representan las restricciones asociadas a las

piezas medianas y grandes, respectivamente. Finalmente, la expresión (2.5) representa las restricciones de no negatividad de las variables (no tendría sentido fabricar un número negativo de productos) y su naturaleza entera (su dominio está definido en el conjunto de números enteros incluyendo el cero). Esta última restricción también se suele escribir de la forma: $x_1, x_2 \geq 0$, entero.

El Algoritmo 1 contiene el código del modelo en AMPL. La solución obtenida con Gurobi en AMPL es fabricar $x_1 = 2$ mecedoras y $x_2 = 6$ taburetes, con una utilidad neta de $z = 230$ USD. Es importante notar que, a pesar de que el precio de venta de las mecedoras es significativamente mayor al de los taburetes, el plan óptimo de producción opta por fabricar más taburetes que mecedoras posibles. Esto se debe a que los taburetes hacen mejor uso de los recursos disponibles.

Algoritmo 1: archivo Carpintero.mod

```
1: var x1 >= 0 integer;
2: var x2 >= 0 integer;
3: maximize z : 70*x1 + 15*x2;
4: s.t.
5: r1: 18*x1 + 2*x2 <= 54;
6: r2: 9*x1 + 2*x2 <= 30;
7: r3: 2*x1 + 1*x2 <= 30;
```

Píldora informativa 2.4: Variables enteras ($\mathbb{Z}^+$) y continuas ($\mathbb{R}^+$)

El valor óptimo de un problema puede cambiar significativamente si se consideran las variables enteras ($\mathbb{Z}^+$) o continuas ($\mathbb{R}^+$). Por ejemplo, el Problema 2.1 tiene un valor óptimo de 230 USD al considerar que el número de mecedoras y taburetes son enteros. Sin embargo, si se supone que las variables son continuas (es decir que pueden tomar valores decimales), la solución óptima sería de 2,667 mecedoras y 3 taburetes con una utilidad de 231,667 USD.

Esto se debe a que la condición de valores enteros limita aún más los posibles valores de cada variable. Por tal motivo, siempre **un modelo que considere variables continuas podrá encontrar un mejor valor** (o igual en el peor de los casos) que el **modelo entero**.

La decisión de usar o no variables enteras depende completamente del contexto del problema. Por ejemplo, en este caso no tendría sentido fabricar 2,667 mecedoras o 0,5 taburetes, puesto que se tendrían productos incompletos.

2.1.2 Variación propuesta para el estudiante

Introduzca un nuevo artículo construido solo por 4 piezas pequeñas y 4 piezas medianas a un precio de venta de 45 USD. Reformular el problema, ¿la solución óptima cambia?, ¿el valor de la función objetivo podría ser de menor valor que la encontrada en el problema original?

2.2 Problema de producción tecnológica

Una compañía dedicada al mercado tecnológico debe programar su producción para maximizar el beneficio semanal¹. Actualmente, la compañía produce y ensambla dos productos diferentes: televisores y equipos de sonido. Ambos tipos de artículos se deben producir, armar y, posteriormente, ser probados por personal especializado.

El distribuidor que adquiere estos artículos necesita, entre ambos productos, no menos de 100 unidades semanales. Además, la cantidad de equipos de sonido a entregar no puede ser menor a la cuarta parte de televisores a entregar, pero en ningún caso puede superar en más de 150 artículos al número de televisores a entregar.

En la Tabla 2.1 se presentan los tiempos (en minutos) que requieren los especialistas para producción, ensamble y prueba de los productos. Esta también contiene la disponibilidad de horas por semana para cada uno de los procesos y la utilidad al vender los productos.

Se busca desarrollar un modelo de programación lineal que permita maximizar la utilidad semanal de la compañía, pero teniendo en cuenta sus limitaciones.

¹Se usará el símbolo *sem* para representar las semanas.

	Televisores	Equipos de sonido	Disponibilidad
Producción y armado	10 min	12 min	55 h/sem
Prueba	30 min	6 min	100 h/sem
Utilidad	200 USD/und	120 USD/und	

Tabla 2.1: Tiempos de procesamiento, disponibilidad y utilidades.

2.2.1 Solución

Definición de las variables de decisión

- x_1 : unidades de televisores a producir semanalmente [und/sem].
- x_2 : unidades de equipos de sonido a producir semanalmente [und/sem].

Modelo de programación lineal

Píldora informativa 2.5: Coherencia de unidades

En todo modelo matemático, las restricciones se deben expresar **en unidades iguales para ambos lados de la desigualdad**. En este problema los tiempos de procesamiento están en minutos, mientras que la capacidad se encuentra en horas. Por tanto, los tiempos se deben transformar en una misma unidad.

$$\text{Maximizar } z = 200 \cdot x_1 + 120 \cdot x_2 \quad (2.6)$$

Sujeto a:

$$10 \cdot x_1 + 12 \cdot x_2 \leq 3300 \quad (2.7)$$

$$30 \cdot x_1 + 6 \cdot x_2 \leq 6000 \quad (2.8)$$

$$x_1 + x_2 \geq 100 \quad (2.9)$$

$$x_2 \geq \frac{1}{4} \cdot x_1 \quad (2.10)$$

$$x_2 \leq x_1 + 150 \quad (2.11)$$

$$x_1, x_2 \in \mathbb{Z}^+ \cup \{0\} \quad (2.12)$$

La ecuación (2.6) es la función objetivo del problema y expresa la utilidad semanal al vender televisores (x_1) y equipos de sonido (x_2). A continuación, se detalla el análisis dimensional:

$$\left[\frac{USD}{sem} \right] = \left[\frac{USD}{und} \right] \cdot \left[\frac{und}{sem} \right]$$

Las restricciones (2.7) y (2.8) indican, al lado izquierdo, los minutos utilizados para el proceso de producción y armado, respectivamente. Mientras que en el lado derecho se expresa la disponibilidad de tiempo de cada proceso. Se ha decidido representar las restricciones en minutos. Sin embargo, nótese que el tiempo disponible de los procesos (Tabla 2.1) se encuentra en horas, por tanto se debe realizar la conversión a minutos, multiplicando cada disponibilidad por 60 (minutos por hora). El análisis dimensional sería el siguiente:

$$\left[\frac{min}{und} \right] \cdot \left[\frac{und}{sem} \right] \leq \left[\frac{min}{sem} \right]$$

La restricción (2.9) hace alusión a la cantidad total de aparatos a entregar, que debe ser al menos 100 unidades. La restricción (2.10) se refiere al total de equipos de sonido para la venta, que debe ser al menos la cuarta parte de los televisores. Así mismo, la restricción (2.11) limita el número de equipos de sonido para no exceder el número de televisores en más de 150 unidades. Finalmente, la restricción (2.12) representa la no negatividad y la limitación a valores enteros de las variables.

El Algoritmo 2 contiene el código del modelo en AMPL. La solución obtenida con CPLEX en AMPL expresa que, al vender $x_1 = 174$ televisores, y $x_2 = 130$ equipos de sonido se genera un beneficio semanal máximo de $z = 50400$ USD.

Algoritmo 2: archivo AparatosTecnologicos.mod

```

1: var x1 >= 0 integer;
2: var x2 >= 0 integer;
3: maximize z: 200*x1 + 120*x2;
4: s.t.
5: r1: 10*x1 + 12*x2 <= 3300;
6: r2: 30*x1 + 6*x2 <= 6000;
7: r3: 1*x1 + 1*x2 >= 100;
8: r4: x2 >= 0.25*x1;
9: r5: x2 <= x1 + 150;

```

2.2.2 Variación propuesta para el estudiante

Asuma que es posible separar el proceso de producción y armado, dividiendo a la mitad los tiempos requeridos por los productos, si la disponibilidad también se divide a la mitad, ¿se puede encontrar una mejor función objetivo con estas nuevas condiciones?

2.3 Problema pesquero

La empresa *Caballito de Mar* tiene 2 sectores de pesca: el océano Pacífico y el océano Atlántico. En cada viaje hacia el océano Pacífico se pescan 300 toneladas de atún: 150 de atún *Big Eye*, 100 toneladas de atún *Yellowfin* y 50 toneladas de atún *Skipjack*. Por otro lado, en cada viaje hacia el océano Atlántico se pescan 400 toneladas de atún: 100 toneladas de *Big Eye*, 200 de *Yellowfin* y 100 de *Skipjack*. El costo de operación por viaje hacia el océano Pacífico es de 13 000 USD y de 10 000 USD hacia el océano Atlántico.

¿Cuántos viajes se deben hacer en cada sector de pesca para satisfacer la demanda, garantizando el menor costo, si se requieren 1 500 toneladas de atún *Big Eye*, 600 de atún *Yellowfin* y 700 de atún *Skipjack*?

2.3.1 Solución

Definición de las variables de decisión

- x_1 : número de viajes hacia el océano Pacífico [v]².
- x_2 : número de viajes hacia el océano Atlántico [v].

² v es el símbolo de viajes.

Modelo de programación lineal

Píldora informativa 2.6: Dinámica entre la función objetivo y las restricciones

Todo problema de programación lineal debe tener **una dinámica coherente entre la función objetivo (FO) y las restricciones**. Asumiendo que todas las variables son positivas, si la FO busca minimizar y las restricciones son todas de tipo menor o igual ($\leq$), la solución óptima sería que las variables de decisión tomen el valor de cero (solución trivial). De forma similar, si la FO busca maximizar y las restricciones son todas de tipo mayor o igual ($\geq$), la solución óptima sería que las variables tomen un valor infinito (que realmente no es posible). Este último caso se conoce como **problema no acotado**.

Por esto se pueden presentar dos casos para garantizar factibilidad del problema:

- Si la FO busca maximizar $\rightarrow$ debe haber al menos una restricción del tipo menor o igual ($\leq$) o del tipo igual ($=$).
- Si la FO busca minimizar $\rightarrow$ debe haber al menos una restricción del tipo mayor o igual ($\geq$) o del tipo igual ($=$).

$$\text{Minimizar } z = 13000 \cdot x_1 + 10000 \cdot x_2 \quad (2.13)$$

Sujeto a:

$$150 \cdot x_1 + 100 \cdot x_2 \geq 1500 \quad (2.14)$$

$$100 \cdot x_1 + 200 \cdot x_2 \geq 600 \quad (2.15)$$

$$50 \cdot x_1 + 100 \cdot x_2 \geq 700 \quad (2.16)$$

$$x_1, x_2 \in \mathbb{Z}^+ \cup \{0\} \quad (2.17)$$

La expresión (2.13) corresponde a la función objetivo del problema, que minimiza los costos de operación por viaje pesquero. El análisis dimensional corresponde a:

$$[USD] = \left\lceil \frac{USD}{v} \right\rceil \cdot [v]$$

Las restricciones (2.14), (2.15) y (2.16) aseguran el cumplimiento del requerimiento (en toneladas) de atún de cada tipo. Las dimensiones de las restricciones son:

$$[v] \cdot \left\lceil \frac{ton}{v} \right\rceil \geq [ton]$$

Finalmente, la expresión (2.17) indica la no negatividad y la limitación a valores enteros de las variables de decisión.

El Algoritmo 3 contiene el código del modelo en AMPL. La solución obtenida con CPLEX en AMPL expresa que el número de viajes al océano Pacífico deben ser 8 y el número de viajes al océano Atlántico debe ser de 3 para un costo mínimo de 134000 USD.

Algoritmo 3: archivo ProblemaPesquero.mod

```

1: var x1 >= 0 integer;
2: var x2 >= 0 integer;
3: minimize z: 13000*x1 + 10000*x2;
4: s.t.
5: r1: 150*x1 + 100*x2 >= 1500;
6: r2: 100*x1 + 200*x2 >= 600;
7: r3: 50*x1 + 100*x2 >= 700;

```

2.3.2 Variación propuesta para el estudiante

¿Cuál sería la variable de decisión adecuada si se considera que existe una tecnología para pescar el tipo de pescado deseado?

2.4 Problema de los vehículos refrigerados (adaptado de [1])

Una compañía dedicada al alquiler de camiones dispone de dos tipos de vehículos: el tipo A, que tiene 20 pies cúbicos de espacio para producto refrigerado y 40 pies cúbicos de espacio para producto no refrigerado; el tipo

B, que tiene 30 pies cúbicos de espacio para producto refrigerado y la misma cantidad de espacio para producto no refrigerado.

Una fábrica de alimentos debe transportar 900 pies cúbicos de producto refrigerado y 1200 pies cúbicos de producto no refrigerado. ¿Cuántos camiones de cada tipo se deben alquilar si el camión A se alquila a 30 USD y el camión B a 40 USD, y se desea alcanzar el mínimo costo? (asumir que todo camión alquilado utiliza el 100 % de su capacidad).

2.4.1 Solución

Definición variables de decisión

- x : número de camiones tipo A necesarios $[cam]^3$.
- y : número de camiones tipo B necesarios $[cam]$.

Modelo de programación lineal

$$\text{Minimizar } z = 30 \cdot x + 40 \cdot y \quad (2.18)$$

Sujeto a:

$$20 \cdot x + 30 \cdot y \geq 900 \quad (2.19)$$

$$40 \cdot x + 30 \cdot y \geq 1200 \quad (2.20)$$

$$x, y \in \mathbb{Z}^+ \cup \{0\} \quad (2.21)$$

La expresión (2.18) representa la minimización del costo de alquiler para los camiones de tipo A y B. La dimensión de esta expresión es:

$$[USD] = \left[\frac{USD}{cam} \right] \cdot [cam]$$

La restricción (2.19) indica que el espacio para producto refrigerado, de la totalidad de los camiones alquilados, debe ser suficiente para cubrir el volumen de producto refrigerado requerido por la empresa. La restricción (2.20) satisface la misma condición para la cantidad de producto no refrigerado. Finalmente, la expresión (2.21) representa la restricción de no

³ cam es el símbolo de camiones.

negatividad y la limitación a valores enteros de las variables.

El Algoritmo 4 contiene el código del modelo matemático en AMPL. Según la solución obtenida con CPLEX en AMPL, la empresa debe alquilar 15 vehículos tipo A (x) y 20 tipo B (y), con un costo de 1250 USD.

Algoritmo 4: archivo TiposVehiculos.mod

```

1: var x >= 0 integer;
2: var y >= 0 integer;
3: minimize z: 30*x + 40*y;
4: s.t.
5: r1: 20*x + 30*y >= 900;
6: r2: 40*x + 30*y >= 1200;

```

2.5 Problema de asignación de buses (adaptado de [2])

La empresa Transporte y Tránsito Global (TTG) estudia la factibilidad de introducir un sistema de autobuses de transporte masivo, que aliviará el tránsito de la ciudad. El estudio inicial busca determinar el mínimo número de autobuses que pueden suplir las necesidades de transporte en la ciudad.

Luego de un análisis de datos, se identificó que el número de autobuses requerido se puede suponer constante para intervalos de cuatro horas, como se presenta en la Tabla 2.2. La empresa TTG decidió que, para hacer el mantenimiento diario requerido, cada autobús debe operar solo ocho horas consecutivas al día. ¿Cuántos buses se deben asignar y en qué turnos deben trabajar para cumplir con la demanda al mínimo costo?

Intervalo de tiempo	Autobuses requeridos
12 a.m. - 4 a.m.	14
4 a.m. - 8 a.m.	6
8 a.m. - 12 p.m.	12
12 p.m. - 4 p.m.	5
4 p.m. - 8 p.m.	11
8 p.m. - 12 a.m.	8

Tabla 2.2: Demanda de autobuses según intervalo de tiempo.

2.5.1 Solución

Píldora informativa 2.7: Definición de nuevas variables

Es común que los problemas requieran la **definición de variables que no están de forma explícita** en el enunciado. En este caso, se deben definir los **turnos factibles** considerando que cada uno de ellos debe ser de ocho horas consecutivas.

Es importante comentar que no es apropiado definir los turnos cada cuatro horas (como en la Tabla 2.2), pues de hacerlo no se garantizaría que el turno sea de ocho horas consecutivas. Por ejemplo, se podría tener el turno inválido de 8 a.m. a 12 p.m. y 4 p.m. a 8 p.m.

Definición de las variables de decisión

- x_1 : número de autobuses asignados al turno 1, es decir, de 12 a.m. a 8 a.m. [*Autobuses*].
- x_2 : número de autobuses asignados al turno 2, es decir, de 4 a.m. a 12 p.m. [*Autobuses*].
- x_3 : número de autobuses asignados al turno 3, es decir, de 8 a.m. a 4 p.m. [*Autobuses*].
- x_4 : número de autobuses asignados al turno 4, es decir, de 12 p.m. a 8 p.m. [*Autobuses*].
- x_5 : número de autobuses asignados al turno 5, es decir, de 4 p.m. a 12 a.m. [*Autobuses*].
- x_6 : número de autobuses asignados al turno 6, es decir, de 8 p.m. a 4 a.m. [*Autobuses*].

Modelo de programación lineal

$$\text{Minimizar } z = x_1 + x_2 + x_3 + x_4 + x_5 + x_6 \quad (2.22)$$

Sujeto a:

$$x_1 + x_6 \geq 14 \quad (2.23)$$

$$x_1 + x_2 \geq 6 \quad (2.24)$$

$$x_2 + x_3 \geq 12 \quad (2.25)$$

$$x_3 + x_4 \geq 5 \quad (2.26)$$

$$x_4 + x_5 \geq 11 \quad (2.27)$$

$$x_5 + x_6 \geq 8 \quad (2.28)$$

$$x_1, \dots, x_6 \in \mathbb{Z}^+ \cup \{0\} \quad (2.29)$$

La expresión (2.22) representa la función objetivo e indica el número total de autobuses asignados en el día. Las restricciones (2.23) a (2.28) indican el cumplimiento de la demanda de autobuses en cada franja horaria (una restricción por cada franja). Finalmente, la restricción (2.29) expresa la no negatividad y la limitación a valores enteros de las variables.

El Algoritmo 5 contiene el código del modelo en AMPL. Según la solución obtenida con CPLEX en AMPL, se deben asignar 14 autobuses en el turno 1, 9 autobuses en el turno 2, 3 autobuses en los turnos 3 y 4, y 8 autobuses en el turno 5. En el turno 6 no se deben asignar autobuses. Dicha asignación daría un total de 37 autobuses por día.

Algoritmo 5: archivo AsignacionBuses.mod

```

1: var x1 >= 0 integer;
2: var x2 >= 0 integer;
3: var x3 >= 0 integer;
4: var x4 >= 0 integer;
5: var x5 >= 0 integer;
6: var x6 >= 0 integer;
7: minimize z: x1 + x2 + x3 + x4 + x5 + x6;
8: s.t.
9: r1: x1 + x6 >= 14;
10: r2: x1 + x2 >= 6;
11: r3: x2 + x3 >= 12;
12: r4: x3 + x4 >= 5;
13: r5: x4 + x5 >= 11;
14: r6: x5 + x6 >= 8;
```

Píldora informativa 2.8: Soluciones múltiples

Es importante tener en cuenta que un problema de programación lineal **puede tener soluciones múltiples**, es decir, soluciones con **diferentes valores en las variables, pero con el mismo valor en la función objetivo**.

Por ejemplo, para el Problema 2.5 también se pueden asignar 6 autobuses en los turnos 2 y 3, 11 autobuses en el turno 5 y 14 autobuses en el turno 6, y obtener un número total de autobuses igual al reportado antes (37).

2.5.2 Variación propuesta para el estudiante

Reformule este problema considerando que las horas de trabajo para los autobuses pueden ser separables, es decir, en lugar de ser consecutivas, pueden estar repartidas en diferentes horas, pero no sobrepasar 8 horas en total.

2.6 Problema de asignación de pilotos

Una empresa prestadora de servicios aeronáuticos cuenta con 4 pilotos expertos en su equipo de trabajo. Cada piloto debe ser asignado a la nave aérea que más se ajuste a sus habilidades de pilotaje, según la disponibilidad de naves. Para ello, cada piloto ha realizado un examen en el que se valoró la habilidad de conducción para cada una de las naves, en una escala de 1 a 10. Los resultados se muestran en la Tabla 2.3.

Usted, como ingeniero, ha sido contratado para determinar la asignación óptima de pilotos a cada nave, para maximizar la habilidad de conducción según el examen.

Piloto	Dirigible	Avión	Jet	Helicóptero
Piloto 1	6	2	8	5
Piloto 2	9	3	5	8

Piloto	Dirigible	Avión	Jet	Helicóptero
Piloto 3	4	8	3	4
Piloto 4	6	7	6	4

Tabla 2.3: Calificación obtenida por los pilotos para cada nave.

Píldora informativa 2.9: Notación extendida y algebraica

La **notación extendida**, utilizada hasta ahora en los modelos de programación lineal, no hace uso de conjuntos o símbolos matemáticos complejos. Su **limitante es que no es práctica para representar modelos de gran tamaño**.

La **notación algebraica** permite expresar modelos con un gran número de variables y restricciones, **con un número pequeño de expresiones matemáticas**. Esta notación se basa en el uso de conjuntos en las variables y los parámetros, junto con símbolos matemáticos como *sumatorias* y *para todo*.



En este problema, y en los cinco siguientes, se presenta el modelo matemático en notación extendida y algebraica. **La intención es hacer una transición paulatina hacia el uso de modelos algebraicos.**

Píldora informativa 2.10: Variables de decisión binarias o booleanas

En muchos problemas **existen decisiones que tienen solo dos posibilidades**: ejecutar dicha alternativa (*true*) o no hacerlo (*false*). Un ejemplo práctico son los problemas de asignación en general, pues presentan como únicas alternativas asignar el recurso o no hacerlo. En estos casos se usa un tipo de variable de decisión especial, llamada **variable binaria**, cuyos únicos posibles valores son 0 y 1. Además son adimensionales. Cuando se usan en un modelo matemático, debe garantizar que al integrar estas variables no se afecte la coherencia de las unidades.

2.6.1 Modelo extendido

Definición de las variables de decisión

$$x_{11} : \begin{cases} 1, \text{ asignar el piloto 1 al dirigible} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{12} : \begin{cases} 1, \text{ asignar el piloto 1 al avión} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{13} : \begin{cases} 1, \text{ asignar el piloto 1 al jet} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{14} : \begin{cases} 1, \text{ asignar el piloto 1 al helicóptero} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{21} : \begin{cases} 1, \text{ asignar el piloto 2 al dirigible} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{22} : \begin{cases} 1, \text{ asignar el piloto 2 al avión} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{23} : \begin{cases} 1, \text{ asignar el piloto 2 al jet} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{24} : \begin{cases} 1, \text{ asignar el piloto 2 al helicóptero} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{31} : \begin{cases} 1, \text{ asignar el piloto 3 al dirigible} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{32} : \begin{cases} 1, \text{ asignar el piloto 3 al avión} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{33} : \begin{cases} 1, \text{ asignar el piloto 3 al jet} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{34} : \begin{cases} 1, \text{ asignar el piloto 3 al helicóptero} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{41} : \begin{cases} 1, \text{ asignar el piloto 4 al dirigible} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{42} : \begin{cases} 1, \text{ asignar el piloto 4 al avión} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{43} : \begin{cases} 1, \text{ asignar el piloto 4 al jet} \\ 0, \text{ en otro caso} \end{cases}$$

$$x_{44} : \begin{cases} 1, \text{ asignar el piloto 4 al helicóptero} \\ 0, \text{ en otro caso} \end{cases}$$

Modelo de programación lineal

$$\begin{aligned} \text{Maximizar } z = & 6 \cdot x_{11} + 2 \cdot x_{12} + 8 \cdot x_{13} + 5 \cdot x_{14} + 9 \cdot x_{21} & (2.30) \\ & + 3 \cdot x_{22} + 5 \cdot x_{23} + 8 \cdot x_{24} + 4 \cdot x_{31} + 8 \cdot x_{32} \\ & + 3 \cdot x_{33} + 4 \cdot x_{34} + 6 \cdot x_{41} + 7 \cdot x_{42} + 6 \cdot x_{43} \\ & + 4 \cdot x_{44} \end{aligned}$$

Sujeto a:

$$x_{11} + x_{12} + x_{13} + x_{14} = 1 \quad (2.31)$$

$$x_{21} + x_{22} + x_{23} + x_{24} = 1 \quad (2.32)$$

$$x_{31} + x_{32} + x_{33} + x_{34} = 1 \quad (2.33)$$

$$x_{41} + x_{42} + x_{43} + x_{44} = 1 \quad (2.34)$$

$$x_{11} + x_{21} + x_{31} + x_{41} = 1 \quad (2.35)$$

$$x_{12} + x_{22} + x_{32} + x_{42} = 1 \quad (2.36)$$

$$x_{13} + x_{23} + x_{33} + x_{43} = 1 \quad (2.37)$$

$$x_{14} + x_{24} + x_{34} + x_{44} = 1 \quad (2.38)$$

$$x_{11}, x_{12}, \dots, x_{43}, x_{44} \in \{0, 1\} \quad (2.39)$$

A continuación, se presenta el modelo algebraico.

2.6.2 Modelo algebraico

Definición de conjuntos

- A : conjunto de pilotos $\{1, 2, 3, 4\}$.
- C : tipos de naves $\{1, 2, 3, 4\}$ para dirigible, avión, jet y helicóptero, respectivamente.

Definición de parámetros

- e_{ij} : calificación del piloto $i \in A$ obtenida en el examen de habilidad de conducción en la nave $j \in C$.

Definición variables de decisión

- x_{ij} : $\begin{cases} 1, & \text{asignar al piloto } i \in A \text{ a la nave } j \in C \\ 0, & \text{en otro caso} \end{cases}$

Modelo algebraico de programación lineal

Píldora informativa 2.11: Sumatorias (Σ) y para todo ($\forall$)

Las sumatorias (Σ) permiten representar **la suma de varios elementos de una restricción, o de la función objetivo, en una sola expresión matemática**. Es indispensable indicar en cada sumatoria el inicio y fin de la misma. A continuación, se muestra un ejemplo del uso de la sumatoria:

$$\sum_{i=1}^6 x_i \cdot c_i = (x_1 \cdot c_1) + (x_2 \cdot c_2) + \cdots + (x_5 \cdot c_5) + (x_6 \cdot c_6)$$

Cuando hay más de una sumatoria, estas se deben recorrer fijando los valores de las sumatorias más externas y variando los valores de la más interna; luego se fija el siguiente valor de las sumatorias externas, y así se repite el proceso:

$$\sum_{i=1}^2 \sum_{j=1}^2 x_{ij} = \underbrace{x_{11} + x_{12}}_{i=1} + \underbrace{x_{21} + x_{22}}_{i=2},$$

Un ejemplo con tres sumatorias:

$$\sum_{i=1}^2 \sum_{j=1}^2 \sum_{k=1}^2 x_{ijk} = \underbrace{x_{111} + x_{112}}_{j=1} + \underbrace{x_{121} + x_{122}}_{j=2} + \underbrace{x_{211} + x_{212}}_{j=1} + \underbrace{x_{221} + x_{222}}_{j=2}$$

Mientras que los para todo ($\forall$) permiten representar **varias restricciones en una sola expresión matemática**, por ejemplo:

$$r : x_i \leq c_i, \forall i \in \{1, \dots, 4\}$$

Representa:

Restricción # 1 (cuando $i = 1$) $\Rightarrow r : x_1 \leq c_1$,

⋮

Restricción # 4 (cuando $i = 4$) $\Rightarrow r : x_4 \leq c_4$

Cuando hay más de un $\forall$ (para todo), se deben expresar las restricciones con todas las posibles combinaciones de los subíndices:

$$r : x_{ij} \leq c_{ij}, \forall i \in \{1, 2\}, \forall j \in \{1, 2\}$$

Representa:

Restricción # 1 (cuando $i = 1, j = 1$) $\Rightarrow r : x_{11} \leq c_{11}$,

Restricción # 2 (cuando $i = 1, j = 2$) $\Rightarrow r : x_{12} \leq c_{12}$,

Restricción # 3 (cuando $i = 2, j = 1$) $\Rightarrow r : x_{21} \leq c_{21}$,

Restricción # 4 (cuando $i = 2, j = 2$) $\Rightarrow r : x_{22} \leq c_{22}$.

$$\text{Maximizar } z = \sum_{i \in A} \sum_{j \in C} e_{ij} \cdot x_{ij} \quad (2.40)$$

Sujeto a:

$$\sum_{j \in C} x_{ij} = 1, \quad \forall i \in A \quad (2.41)$$

$$\sum_{i \in A} x_{ij} = 1, \quad \forall j \in C \quad (2.42)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in A, \forall j \in C \quad (2.43)$$

La expresión (2.40) corresponde a la función objetivo del problema, que maximiza la calificación obtenida por los pilotos. La restricción (2.41) indica que cada piloto debe ser asignado a una sola nave, mientras que la restricción (2.42) indica que una nave solo puede ser asignada a un piloto. Finalmente, la restricción (2.43) indica la naturaleza binaria de las variables.

El Algoritmo 6 contiene el código del modelo algebraico en AMPL. La solución obtenida con Gurobi en AMPL, con una calificación óptima de 30 unidades, indica que:

- Al piloto 1 se le asigna el jet.
- Al piloto 2 se le asigna el helicóptero.
- Al piloto 3 se le asigna el avión.
- Al piloto 4 se le asigna el dirigible.

Algoritmo 6: archivo AsignacionPilotos.mod

```
1: set A:={1..4};
2: set C:={1..4};
3: param e{A, C};
4: var x{A, C} binary;
5: maximize z: sum{i in A, j in C} e[i,j]*x[i,j];
6: s.t.
7: r1{i in A}: sum{j in C} x[i,j] = 1;
8: r2{j in C}: sum{i in A} x[i,j] = 1;
```



Para la resolución de los modelos en AMPL se debe tener en cuenta el tipo de notación matemática:

- Notación extendida: esta requiere de un archivo `<.mod>`, que contiene el modelo matemático, y un archivo `<.run>`, que contiene los comandos de ejecución (la forma de comunicarse con AMPL).
- Notación algebraica: esta requiere de un archivo `<.mod>` que contiene el modelo matemático, un archivo `<.dat>`, que contiene los datos usados por el modelo, y un archivo `<.run>`, que contiene los comandos de ejecución.
- Notación mixta (extendida y algebraica): en este caso se requieren los mismos archivos que con la notación algebraica.

Para mayor detalle se remite al lector Capítulo 5, Sección 5.4.

2.7 Problema de transporte

Daar S. A. es un fabricante de teléfonos móviles de MioXi de última tecnología. La empresa tiene dos plantas de producción, una en Cali y otra en Medellín, además, tiene un almacén ubicado en Bogotá. Daar vende sus teléfonos en seis mercados: Pasto (PSO), Medellín (MDE), Cartagena (CTG), Cali (CLO), Manizales (MZL) y Bogotá (Btá).

Para satisfacer la demanda semanal del mercado, la gerente de distribución debe determinar un plan para el envío desde las plantas y su almacén hacia los distintos mercados, al mínimo costo. Ella tiene conocimiento de que existen 600 unidades de MioXi disponibles para ser enviados desde la planta de Cali, 600 desde la planta de Medellín y 250 desde el almacén en Bogotá. El plan de envío no puede sobrepasar estos valores desde las plantas y el almacén.

Las demandas del mercado se indican en la Tabla 2.4, junto con el costo de transporte asociado al envío de una unidad de teléfono MioXi.

Origen	Destinos					
	PSO	MDE	CTG	CLO	MZL	Btá
Planta 1 Cali	13 150	9 650	19 950	5 000	9 650	15 950
Planta 2 Medellín	19 950	6 500	19 950	9 650	11 150	15 950
Almacén Bogotá	17 150	15 950	19 950	15 950	15 950	7 000
Demanda	280	250	240	170	210	300

Tabla 2.4: Costos de transporte [COP/und] y demanda de mercados [und].

- Verificar la viabilidad del problema de transporte de Daar, es decir, ¿la oferta será suficiente para satisfacer la demanda total?
- Formular un modelo matemático que asegure un costo de transporte total mínimo al realizar los envíos de teléfonos MioXi desde las plantas y el almacén, hacia los mercados.

Píldora informativa 2.12: Balance en el problema de transporte

En un problema de transporte, **si la demanda es igual a la oferta, la solución lineal óptima siempre es entera**, aunque en el modelo matemático no se especifique. Esto se denomina **balance** en un problema de transporte.

Un problema desbalanceado (mayor oferta que demanda o viceversa) se puede balancear adicionando nuevos nodos de demanda u oferta, según sea necesario. Sin embargo, cabe resaltar que en el caso donde la demanda supera la oferta, el problema es directamente infactible.

- a. Para verificar la viabilidad del problema, es necesario comprobar que la oferta es suficiente para satisfacer la demanda de los mercados. En este caso, las unidades totales de teléfonos MioXi disponibles son 1450 *versus* las necesidades del mercado totales que son 1450. Por lo tanto, la oferta disponible de Daar es suficiente para satisfacer la demanda actual.
- b. A continuación, se presenta el modelo matemático del problema tanto en notación extendida como algebraica.

2.7.1 Modelo extendido*Definición de variables de decisión*

- x_{11} : cantidad de teléfonos a enviar de planta 1 a mercado 1 (PSO).
- x_{12} : cantidad de teléfonos a enviar de planta 1 a mercado 2 (MDE).
- x_{13} : cantidad de teléfonos a enviar de planta 1 a mercado 3 (CTG).
- x_{14} : cantidad de teléfonos a enviar de planta 1 a mercado 4 (CLO).
- x_{15} : cantidad de teléfonos a enviar de planta 1 a mercado 5 (MZL).
- x_{16} : cantidad de teléfonos a enviar de planta 1 a mercado 6 (Btá).

- x_{21} : cantidad de teléfonos a enviar de planta 2 a mercado 1 (PSO).
- x_{22} : cantidad de teléfonos a enviar de planta 2 a mercado 2 (MDE).
- x_{23} : cantidad de teléfonos a enviar de planta 2 a mercado 3 (CTG).
- x_{24} : cantidad de teléfonos a enviar de planta 2 a mercado 4 (CLO).
- x_{25} : cantidad de teléfonos a enviar de planta 2 a mercado 5 (MZL).
- x_{26} : cantidad de teléfonos a enviar de planta 2 a mercado 6 (Btá).
- x_{31} : cantidad de teléfonos a enviar de almacén a mercado 1 (PSO).
- x_{32} : cantidad de teléfonos a enviar de almacén a mercado 2 (MDE).
- x_{33} : cantidad de teléfonos a enviar de almacén a mercado 3 (CTG).
- x_{34} : cantidad de teléfonos a enviar de almacén a mercado 4 (CLO).
- x_{35} : cantidad de teléfonos a enviar de almacén a mercado 5 (MZL).
- x_{36} : cantidad de teléfonos a enviar de almacén a mercado 6 (Btá).

Modelo de programación lineal

$$\begin{aligned}
 \text{Minimizar } z = & 13\,150 \cdot x_{11} + 9\,650 \cdot x_{12} + 19\,950 \cdot x_{13} & (2.44) \\
 & + 5\,000 \cdot x_{14} + 9\,650 \cdot x_{15} + 15\,950 \cdot x_{16} \\
 & + 19\,950 \cdot x_{21} + 6\,500 \cdot x_{22} + 19\,950 \cdot x_{23} \\
 & + 9\,650 \cdot x_{24} + 11\,150 \cdot x_{25} + 15\,950 \cdot x_{26} \\
 & + 17\,150 \cdot x_{31} + 15\,950 \cdot x_{32} + 19\,950 \cdot x_{33} \\
 & + 15\,950 \cdot x_{34} + 15\,950 \cdot x_{35} + 7\,000 \cdot x_{36}
 \end{aligned}$$

Sujeto a:

$$x_{11} + x_{12} + x_{13} + x_{14} + x_{15} + x_{16} \leq 600 \quad (2.45)$$

$$x_{21} + x_{22} + x_{23} + x_{24} + x_{25} + x_{26} \leq 600 \quad (2.46)$$

$$x_{31} + x_{32} + x_{33} + x_{34} + x_{35} + x_{36} \leq 250 \quad (2.47)$$

$$x_{11} + x_{21} + x_{31} + x_{41} + x_{51} + x_{61} \geq 280 \quad (2.48)$$

$$x_{12} + x_{22} + x_{32} + x_{42} + x_{52} + x_{62} \geq 250 \quad (2.49)$$

$$x_{13} + x_{23} + x_{33} + x_{43} + x_{53} + x_{63} \geq 240 \quad (2.50)$$

$$x_{14} + x_{24} + x_{34} + x_{44} + x_{54} + x_{64} \geq 170 \quad (2.51)$$

$$x_{15} + x_{25} + x_{35} + x_{45} + x_{55} + x_{65} \geq 210 \quad (2.52)$$

$$x_{16} + x_{26} + x_{36} + x_{46} + x_{56} + x_{66} \geq 300 \quad (2.53)$$

$$x_{11}, x_{12}, \dots, x_{56}, x_{66} \in \mathbb{Z}^+ \cup \{0\} \quad (2.54)$$

A continuación, se presenta el modelo algebraico.

2.7.2 Modelo algebraico

Definición de conjuntos

- I : nodos de origen $\{1, 2, 3\}$.



En un conjunto, es común usar los tres puntos (...) para indicar una sucesión de números entre uno inicial y uno final. Por ejemplo, $\{1, \dots, 4\}$ es igual a $\{1, 2, 3, 4\}$.

- J : mercados $\{1, \dots, 6\}$.

Definición de parámetros

- c_{ij} : costo de transportar un teléfono MioXi desde el origen $i \in I$ al mercado $j \in J$.
- o_i : oferta disponible del origen $i \in I$.
- d_j : demanda del mercado $j \in J$.

Definición variables de decisión

- x_{ij} : unidades de teléfonos MioXi a distribuir, desde el origen $i \in I$ al mercado $j \in J$.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} c_{ij} \cdot x_{ij} \quad (2.55)$$

Sujeto a:

$$\sum_{j \in J} x_{ij} \leq o_i, \quad \forall i \in I \quad (2.56)$$

$$\sum_{i \in I} x_{ij} \geq d_j, \quad \forall j \in J \quad (2.57)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (2.58)$$

La expresión (2.55) es la función objetivo del modelo, que busca minimizar los costos de transportar los teléfonos desde las fuentes de origen hasta los distintos mercados. La dimensión de esta expresión corresponde a:

$$[COP] = \left[\frac{COP}{und} \right] \cdot [und]$$

La restricción (2.56) garantiza que los teléfonos enviados a los mercados no superen la cantidad disponible en las plantas o el almacén. La expresión (2.57) expresa la demanda de los mercados que debe ser cumplida desde las plantas y la bodega. Por último, la expresión (2.58) indica la no negatividad y la limitación a valores enteros de las variables de decisión.

Píldora informativa 2.13: Igualdad en problemas de transporte

Como el problema **está balanceado**, las desigualdades (2.56) y (2.57) se podrían **reemplazar por ecuaciones (igualdades)**, y el resultado sería el mismo.

El Algoritmo 7 contiene el código del modelo algebraico en AMPL. En la Tabla 2.5 se presenta la solución obtenida por CPLEX en AMPL, logrando satisfacer la demanda de teléfonos MioXi en los mercados, a un costo mínimo de 15609000 COP.

Algoritmo 7: archivo ProblemaTransporte.mod

```

1: set I:={1..3};
2: set J:={1..6};
3: param c{I,J};
4: param o{I};
5: param d{J};
6: var x{I,J} >= 0 integer;
7: minimize z: sum{i in I, j in J} c[i,j]*x[i,j];
8: s.t.
9: Oferta{i in I}: sum{j in J} x[i,j] <= o[i];
10: Demanda{j in J}: sum{i in I} x[i,j] >= d[j];

```

Variable	Unidades a enviar $[und]$	Variable	Unidades a enviar $[und]$	Variable	Unidades a enviar $[und]$
x_{11}	280	x_{21}	0	x_{31}	0
x_{12}	0	x_{22}	250	x_{32}	0
x_{13}	0	x_{23}	240	x_{33}	0
x_{14}	170	x_{24}	0	x_{34}	0
x_{15}	150	x_{25}	60	x_{35}	0
x_{16}	0	x_{26}	50	x_{36}	250

Tabla 2.5: Solución al problema de transporte.



Para interpretar la solución mostrada en la Tabla 2.5 se debe usar la propia definición de las variables. Así, por ejemplo, $x_{14} = 170$ nos indica que se deben enviar 170 teléfonos desde la planta 1 al mercado 4 (CLO).

2.8 Problema de la inversión (adaptado de [3])

Cuatro personas viajan a Panamá y disponen de un presupuesto global de 3000000 COP para inversiones. Hay tres alternativas de inversión en artefactos eléctricos: comprar televisores, equipos de sonido y/o aspiradoras, cuyos precios unitarios son 250000 COP, 180000 COP y 70000 COP, respectivamente. Estos equipos se venden en el mercado fácilmente a 400000 COP, 270000 COP y 130000 COP, respectivamente. La aduana no permite que una persona transporte más de 12 artefactos eléctricos.

Determine cómo se debe invertir el dinero para obtener la mayor utilidad posible.

2.8.1 Modelo extendido

Definición de las variables de decisión

- x_{11} : unidades de televisores a comprar por persona uno.
- x_{12} : unidades de televisores a comprar por persona dos.
- x_{13} : unidades de televisores a comprar por persona tres.
- x_{14} : unidades de televisores a comprar por persona cuatro.
- x_{21} : unidades de equipos de sonido a comprar por persona uno.
- x_{22} : unidades de equipos de sonido a comprar por persona dos.
- x_{23} : unidades de equipos de sonido a comprar por persona tres.
- x_{24} : unidades de equipos de sonido a comprar por persona cuatro.
- x_{31} : unidades de aspiradoras a comprar por persona uno.
- x_{32} : unidades de aspiradoras a comprar por persona dos.
- x_{33} : unidades de aspiradoras a comprar por persona tres.

- x_{34} : unidades de aspiradoras a comprar por persona cuatro.

Modelo de programación lineal

$$\begin{aligned} \text{Maximizar } z = & (400\,000 - 250\,000) \cdot (x_{11} + x_{12} + x_{13} + x_{14}) & (2.59) \\ & + (270\,000 - 180\,000) \cdot (x_{21} + x_{22} + x_{23} + x_{24}) \\ & + (130\,000 - 70\,000) \cdot (x_{31} + x_{32} + x_{33} + x_{34}) \end{aligned}$$

Sujeto a:

$$250\,000 \cdot (x_{11} + x_{12} + x_{13} + x_{14}) + \quad (2.60)$$

$$180\,000 \cdot (x_{21} + x_{22} + x_{23} + x_{24}) +$$

$$70\,000 \cdot (x_{31} + x_{32} + x_{33} + x_{34}) \leq 3\,000\,000$$

$$x_{11} + x_{21} + x_{31} \leq 12 \quad (2.61)$$

$$x_{12} + x_{22} + x_{32} \leq 12 \quad (2.62)$$

$$x_{13} + x_{23} + x_{33} \leq 12 \quad (2.63)$$

$$x_{14} + x_{24} + x_{34} \leq 12 \quad (2.64)$$

$$x_{11}, x_{12}, \dots, x_{33}, x_{34} \in \mathbb{Z}^+ \cup \{0\} \quad (2.65)$$

2.8.2 Modelo algebraico

Definición de conjuntos

- I : alternativas de inversión $\{1, 2, 3\}$.
- J : personas $\{1, 2, 3, 4\}$.

Definición de parámetros

- C_i : valor inversión en artefacto $i \in I$.
- PV_i : precio de venta de alternativa de inversión $i \in I$.

Definición variables de decisión

- x_{ij} : unidades de artefactos eléctricos a invertir en alternativa $i \in I$ por persona $j \in J$.

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i=1}^3 \sum_{j=1}^4 PV_i \cdot x_{ij} - \sum_{i=1}^3 \sum_{j=1}^4 C_i \cdot x_{ij} \quad (2.67)$$

Sujeto a:

$$\sum_{i=1}^3 \sum_{j=1}^4 C_i \cdot x_{ij} \leq 3\,000\,000 \quad (2.68)$$

$$\sum_{i=1}^3 x_{ij} \leq 12, \quad \forall j \in J \quad (2.69)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (2.70)$$

La expresión (2.66) corresponde a la función objetivo del problema, que maximiza la utilidad obtenida. La dimensión de esta expresión es:

$$[COP] = \left[\frac{COP}{und} \right] \cdot [und]$$

La restricción (2.67) indica que el valor total de la inversión realizada por cada persona y en todas las alternativas de inversión no debe superar el presupuesto global disponible. La restricción (2.68) garantiza que cada persona solo puede transportar doce o menos artefactos. Finalmente, la expresión (2.69) del modelo algebraico indica la limitación a valores enteros y no negatividad de las variables.

El Algoritmo 8 contiene el código del modelo en AMPL. La solución encontrada usando CPLEX en AMPL, es que los inversionistas 1, 2 y 4 deben comprar 12 aspiradoras y el inversionista 3 solo 6 aspiradoras. Con esta estrategia se obtiene una utilidad máxima de 2520 000 COP.

Algoritmo 8: archivo Inversion.mod

```

1: set I:={1..3};
2: set J:={1..4};
3: param C{I};
4: param PV{I};
5: var x{I,J} >=0 integer;
6: maximize z: sum{i in I,j in J} PV[i]*x[i,j] - sum{i in I,j in J}
  C[i]*x[i,j];
7: s.t.
8: r1: sum{i in I,j in J} C[i]*x[i,j] <= 3000000;
9: r2{j in J}: sum{i in I} x[i,j] <= 12;

```

2.8.3 Variación propuesta para el estudiante

1. Formular este problema usando la siguiente variable de decisión:
 x_i cantidad total de artefactos tipo i a comprar (sin distinguir qué inversionista hace la compra).
2. ¿Cómo cambia el modelo matemático si los inversionistas 1, 2, 3 y 4 tuvieran disponibles 3000000, 4000000, 2500000 y 1900000 pesos, respectivamente?

2.9 Problema del restaurante italiano

Un restaurante italiano cuenta con cinco especialidades i en su menú: tallarines, raviolos, lasaña, corbatitas y fideos. Cada una genera distintas utilidades u_i en dólares y tiene una demanda máxima diaria de especialidades d_i . Ambos son datos estimados mediante un estudio de mercado, que se resume en la Tabla 2.6.

Especialidades	Tallarines	Raviolos	Lasaña	Corbatitas	Fideos
Utilidad [USD]	50	65	40	70	55
Demanda [und]	15	12	32	18	10

Tabla 2.6: Utilidad y demanda de las especialidades.

Cada especialidad i puede estar compuesta por los siguientes j ingredientes: tomate, cebolla, huevo y pimienta. Así, cada especialidad i requiere cantidades diferentes de ingredientes j para su elaboración (a_{ij}), como muestra la Tabla 2.7.

	Tomate	Cebolla	Huevo	Pimienta
Tallarines	1	1	0	1
Ravioles	4	2	0	1
Lasaña	2	1	1	2
Corbatitas	0	0	2	0
Fideos	1	1	0	1

Tabla 2.7: Cantidades de ingredientes requeridos para elaborar una especialidad.

Adicionalmente, el restaurante tiene una política donde solo puede cocinar con ingredientes frescos (m_j), por lo tanto, compra diariamente a un campesino 80, 75, 10 y 30 unidades de tomate, cebolla, huevo, y pimienta, respectivamente. Al finalizar el día se desechan los ingredientes sobrantes.

El objetivo del problema es calcular la cantidad x_i a producir de cada especialidad para maximizar la utilidad diaria del restaurante. Un supuesto a considerar es que la mano de obra y maquinaria se pueden adaptar sin inconvenientes a los valores obtenidos.

2.9.1 Modelo extendido

Definición de variables de decisión

- x_1 : número de platos de tallarines diarios⁴ $\left\lceil \frac{und}{d} \right\rceil$.
- x_2 : número de platos de ravioles diarios $\left\lceil \frac{und}{d} \right\rceil$.
- x_3 : número de lasañas diarias $\left\lceil \frac{und}{d} \right\rceil$.
- x_4 : número de platos de corbatitas diarios $\left\lceil \frac{und}{d} \right\rceil$.

⁴Se usará la letra d como símbolo de día. No confundir con d_i en este problema, este último representa la demanda.

- x_5 : número de platos de fideos diarios $\left[\frac{und}{d}\right]$.

Píldora informativa 2.14: Componente temporal

En general los problemas suelen tener un **componente temporal**, **que las variables de decisión lo heredan**. Por ejemplo, en este caso el número de platos (independiente del platillo) es diario. A veces, el componente temporal puede ser un ciclo, una ruta, un viaje, una ejecución, entre otros.

La existencia de este componente siempre está presente aunque en un problema no se establezca de forma explícita. Pero **siempre se debe tener en cuenta para que las variables y parámetros guarden coherencia**.

Modelo de programación lineal

$$\text{Maximizar } z = 50 \cdot x_1 + 65 \cdot x_2 + 40 \cdot x_3 + 70 \cdot x_4 + 55 \cdot x_5 \quad (2.70)$$

Sujeto a:

$$1 \cdot x_1 + 4 \cdot x_2 + 2 \cdot x_3 + 0 \cdot x_4 + 1 \cdot x_5 \leq 80 \quad (2.71)$$

$$1 \cdot x_1 + 2 \cdot x_2 + 1 \cdot x_3 + 0 \cdot x_4 + 1 \cdot x_5 \leq 75 \quad (2.72)$$

$$0 \cdot x_1 + 0 \cdot x_2 + 1 \cdot x_3 + 2 \cdot x_4 + 0 \cdot x_5 \leq 10 \quad (2.73)$$

$$1 \cdot x_1 + 1 \cdot x_2 + 2 \cdot x_3 + 0 \cdot x_4 + 1 \cdot x_5 \leq 30 \quad (2.74)$$

$$x_1 \leq 15 \quad (2.75)$$

$$x_2 \leq 12 \quad (2.76)$$

$$x_3 \leq 32 \quad (2.77)$$

$$x_4 \leq 18 \quad (2.78)$$

$$x_5 \leq 10 \quad (2.79)$$

$$x_1, x_2, x_3, x_4, x_5 \in \mathbb{Z}^+ \cup \{0\} \quad (2.80)$$

2.9.2 Modelo algebraico

Definición de conjuntos

- I : tipos de especialidades $\{1, \dots, 5\}$, para tallarines, raviolos, lasañas, corbatitas y fideos, respectivamente.
- J : ingredientes $\{1, \dots, 4\}$, para tomate, cebolla, huevo y pimienta, respectivamente.

Definición de parámetros

- a_{ij} : cantidad de unidades del ingrediente $j \in J$ para elaborar especialidad $i \in I$.
- m_j : cantidad, en unidades, disponible del ingrediente $j \in J$.
- d_i : demanda máxima del especialidad $i \in I$.

Definición de las variables de decisión

- x_i : número de especialidad i diaria a producir $\left\lceil \frac{und}{d} \right\rceil$.

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in I} u_i \cdot x_i \quad (2.81)$$

Sujeto a:

$$\sum_{i \in I} a_{ij} \cdot x_i \leq m_j, \quad \forall j \in J \quad (2.82)$$

$$x_i \leq d_i, \quad \forall i \in I \quad (2.83)$$

$$x_i \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I \quad (2.84)$$

La expresión (2.81) es la función objetivo del modelo, que busca maximizar las utilidades del restaurante. Las dimensiones de esta expresión están representadas por:

$$\left[\frac{USD}{d} \right] = \left[\frac{USD}{und} \right] \cdot \left[\frac{und}{d} \right]$$

La expresión (2.82) garantiza que los ingredientes a utilizar en la especialidad no excedan la cantidad disponible comprada diariamente. La restricción (2.83) garantiza no exceder la demanda máxima diaria de las especialidades. Por último, la expresión (2.84) indica la no negatividad y la limitación a valores enteros de las variables de decisión.

El Algoritmo 9 contiene el código del modelo en AMPL. En la Tabla 2.8 se presenta la solución obtenida por CPLEX en AMPL, con una utilidad máxima de 2080 USD.

Algoritmo 9: archivo RestauranteItaliano.mod

```

1: set I:={1..5};
2: set J:={1..4};
3: param u{I};
4: param d{I};
5: param a{I,J};
6: param m{J};
7: var x{I} >=0 integer;
8: maximize z:sum{i in I} x[i]*u[i];
9: s.t.
10: r1{j in J}: sum{i in I} a[i,j]*x[i] <= m[j];
11: r2{i in I}: x[i] <= d[i];

```

Variables	Valores $[und]$
Tallarines (x_1)	8
Ravioles (x_2)	12
Lasañas (x_3)	0
Corbatitas (x_4)	5
Fideos (x_5)	10
Valor z	2080 USD

Tabla 2.8: Solución al problema del restaurante italiano.

2.9.3 Variación propuesta para el estudiante

Debido a la reputación del restaurante como representante de la gastronomía italiana, el gerente desea que mínimo se elaboren 3 platos de cada receta. Reformular el problema teniendo en cuenta esta solicitud. ¿La utilidad cambia respecto a la solución anterior?

2.10 Problema de la dieta

Debido a la recesión económica pos Covid-19, usted ha decidido seguir una dieta estricta y económica que logre aportar el requerimiento mínimo diario nutricional para sobrevivir, y así ahorrar para empezar un emprendimiento. Asumiendo que usted es una mujer con actividad física moderada, mayor de 18 años y con un peso de 60 kg, su necesidad nutricional promedio diaria⁵ sería como se ilustra en la Tabla 2.9.

Macronutriente o micronutriente	Requerimiento mínimo
Proteína (g/día)	65
Lípidos o grasas (g/día)	66
Hidratos de carbono (g/día)	362
Vitamina A (μ g/día)	700
Vitamina C (mg/día)	75
Folato (mg/día)	400
Calcio (mg/día)	1 000
Hierro (mg/día)	18

Tabla 2.9: Requerimiento mínimo nutricional diario.

Teniendo en cuenta el costo, usted desea elaborar una dieta con los alimentos más económicos. Estos y sus unidades de medida pueden resumirse en: leche entera [1/2 taza] (LE), huevo [2 unidades] (H), porotos [2 tazas] (P), arroz [2 tazas] (A), fideos [2 tazas] (F), zanahoria [1 unidad] (Z), manzana [1 unidad] (M), naranja [1 unidad] (N) y mantequilla [4 cucharadas] (Mant). El aporte nutricional de cada uno de estos alimentos con respecto a sus

⁵Datos tomados de *Food and Agriculture Organization of the United Nations* y la Fundación Española de la Nutrición.

unidades de medida se muestran en la Tabla 2.10.

Alimento	LE	H	P	A	F	Z	M	N	Mant
Proteína (g/und)	3,3	13,5	20,6	6,4	12,2	0,9	0,3	0,7	0
Lípidos o grasas (g/und)	3,2	10	1,6	0,8	0,3	0,5	0,3	0,3	82,9
Hidratos de carbono (g/und)	4,8	4	57,3	79,7	74,6	8,1	14,5	8,7	0
Vitamina A (μg /und)	91	226	85	0	0	1346	4	40	5,6
Vitamina C (mg/ und)	4,5	0	26	0	0	6	10	50	0
Folato (mg/und)	14	51,2	88	12	4	10	5	37	3
Calcio (mg/und)	123	100	260	14	24	41	6	36	0
Hierro (mg/und)	0,1	2,4	2	0,8	3,6	0,7	0,4	0,3	0
Costo (COP por porción)	660	1180	2100	950	560	440	1440	400	800

Tabla 2.10: Contenido nutricional y costo de los alimentos.

A partir de la información anterior, determine la cantidad de alimentos a comprar diariamente para cumplir con sus necesidades alimenticias al mínimo costo.

2.10.1 Modelo extendido

Definición de variables de decisión

- x_1 : cantidad de leche a comprar por día $\left[\frac{\text{tazas}}{d}\right]$.
- x_2 : cantidad de huevos a comprar por día $\left[\frac{\text{und}}{d}\right]$.

- x_3 : cantidad de porotos a comprar por día $\left[\frac{tazas}{d}\right]$.
- x_4 : cantidad de arroz a comprar por día $\left[\frac{tazas}{d}\right]$.
- x_5 : cantidad de fideos a comprar por día $\left[\frac{tazas}{d}\right]$.
- x_6 : cantidad de zanahorias a comprar por día $\left[\frac{und}{d}\right]$.
- x_7 : cantidad de manzanas a comprar por día $\left[\frac{und}{d}\right]$.
- x_8 : cantidad de naranjas a comprar por día $\left[\frac{und}{d}\right]$.
- x_9 : cantidad de mantequilla a comprar por día $\left[\frac{cucharadas}{d}\right]$.

Modelo de programación lineal

$$\begin{aligned} \text{Minimizar } z = & 660 \cdot x_1 + 1180 \cdot x_2 + 2100 \cdot x_3 + 950 \cdot x_4 + 560 \cdot x_5 \\ & + 440 \cdot x_6 + 1440 \cdot x_7 + 400 \cdot x_8 + 800 \cdot x_9 \end{aligned} \quad (2.85)$$

Sujeto a:

$$3,3 \cdot x_1 + 13,5 \cdot x_2 + 20,6 \cdot x_3 + 6,4 \cdot x_4 + 12,2 \cdot x_5 \quad (2.86)$$

$$+ 0,9 \cdot x_6 + 0,3 \cdot x_7 + 0,7 \cdot x_8 + 0 \cdot x_9 \geq 65$$

$$3,2 \cdot x_1 + 10 \cdot x_2 + 1,6 \cdot x_3 + 0,8 \cdot x_4 + 0,3 \cdot x_5 \quad (2.87)$$

$$+ 0,5 \cdot x_6 + 0,3 \cdot x_7 + 0,3 \cdot x_8 + 82,9 \cdot x_9 \geq 66$$

$$4,8 \cdot x_1 + 4 \cdot x_2 + 57,3 \cdot x_3 + 79,7 \cdot x_4 + 74,6 \cdot x_5 \quad (2.88)$$

$$+ 8,1 \cdot x_6 + 14,5 \cdot x_7 + 8,7 \cdot x_8 + 0 \cdot x_9 \geq 362$$

$$91 \cdot x_1 + 226 \cdot x_2 + 85 \cdot x_3 + 0 \cdot x_4 + 0 \cdot x_5 \quad (2.89)$$

$$+ 1346 \cdot x_6 + 4 \cdot x_7 + 40 \cdot x_8 + 5,6 \cdot x_9 \geq 700$$

$$4,5 \cdot x_1 + 0 \cdot x_2 + 26 \cdot x_3 + 0 \cdot x_4 + 0 \cdot x_5 + 6 \cdot x_6 \quad (2.90)$$

$$+ 10 \cdot x_7 + 50 \cdot x_8 + 0 \cdot x_9 \geq 75$$

$$14 \cdot x_1 + 51,2 \cdot x_2 + 88 \cdot x_3 + 12 \cdot x_4 + 4 \cdot x_5 \quad (2.91)$$

$$+ 10 \cdot x_6 + 5 \cdot x_7 + 37 \cdot x_8 + 3 \cdot x_9 \geq 400$$

$$123 \cdot x_1 + 100 \cdot x_2 + 260 \cdot x_3 + 14 \cdot x_4 + 24 \cdot x_5 \quad (2.92)$$

$$+ 41 \cdot x_6 + 6 \cdot x_7 + 36 \cdot x_8 + 0 \cdot x_9 \geq 1000$$

$$0, 1 \cdot x_1 + 2,4 \cdot x_2 + 2 \cdot x_3 + 0,8 \cdot x_4 + 3,6 \cdot x_5 \quad (2.93)$$

$$+ 0, 7 \cdot x_6 + 0,4 \cdot x_7 + 0,3 \cdot x_8 + 0 \cdot x_9 \geq 18$$

$$x_1, x_3, x_4, x_5, x_9 \in \mathbb{R}^+ \cup \{0\} \quad (2.94)$$

$$x_2, x_6, x_7, x_8 \in \mathbb{Z}^+ \cup \{0\} \quad (2.95)$$

2.10.2 Modelo algebraico

Definición de conjuntos

- I : tipo de alimentos $\{1, 3, 4, 5, 9\}$, para leche, porotos, arroz, fideos, mantequilla, respectivamente. Nótese que los elementos del conjunto excluyen a algunos números.
- A : tipo de alimentos $\{2, 6, 7, 8\}$, para huevo, zanahoria, manzana, naranja, respectivamente. Nótese que los elementos del conjunto excluyen a algunos números.



El motivo para separar los alimentos es debido a que unos son discretos (cantidades con valore enteros) y otros continuos (cantidades con valores reales o decimales).

- J : $\{1, ..., 8\}$ para proteína, lípidos o grasas, hidratos de carbono, vitamina A, vitamina C, folato, calcio y hierro, respectivamente.

Definición de parámetros

- c_i : costo de alimento $i \in I \cup A$.
- a_{ij} : alimento $i \in I \cup A$ con aporte nutricional $j \in J$.
- d_j : requerimiento mínimo de nutrientes o vitaminas $j \in J$ en el cuerpo humano.

Definición variables de decisión

- x_i : cantidad de alimento $i \in I$ a comprar diariamente.
- y_i : cantidad de alimento $i \in A$ a comprar diariamente.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} c_i \cdot x_i + \sum_{i \in A} c_i \cdot y_i \quad (2.96)$$

Sujeto a:

$$\sum_{i \in I} a_{ij} \cdot x_i + \sum_{i \in A} a_{ij} \cdot y_i \geq d_j, \quad \forall j \in J \quad (2.97)$$

$$x_i \in \mathbb{R}^+ \cup \{0\}, \quad \forall i \in I \quad (2.98)$$

$$y_i \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in A \quad (2.99)$$

La expresión (2.96) es la función objetivo del modelo, que permite minimizar los costos asociados a la compra de alimentos. Las variables x y y surgen debido a las unidades de medida de los alimentos, siendo algunos continuos (que pueden tomar valores decimales, por ejemplo, las tazas de arroz) y otros discretos (que solo pueden tomar valores enteros, por ejemplo, la naranja). La dimensión de la expresión está dada por:

$$\left[\frac{COP}{d} \right] = \left[\frac{COP}{continuo} \right] \cdot \left[\frac{continuo}{d} \right] + \left[\frac{COP}{discreto} \right] \cdot \left[\frac{discreto}{d} \right]$$

La expresión (2.97) permite que usted se alimente diariamente con el aporte nutricional necesario de proteína, lípidos o grasas, hidratos de carbono, vitamina A, vitamina C, folato, calcio y hierro. Las expresiones (2.98) y (2.99) representan la no negatividad de las variables de decisión para el caso continuo y discreto, respectivamente.

El Algoritmo 10 contiene el código del modelo en AMPL. En la Tabla 2.11 se presenta la solución obtenida por CPLEX en AMPL, que minimiza los costos a 9794,85 COP/d.

Algoritmo 10: archivo ProblemaDeLaDieta.mod

```

1: set I:={1, 3, 4, 5, 9};
2: set A:={2, 6, 7, 8};
3: set J:={1..8};
4: param c{I union A};
5: param d{J};
6: param a{I union A,J};
7: var x{I} >= 0;
8: var y{A} >=0 integer;
9: minimize z:sum{i in I} c[i]*x[i] + sum{i in A} c[i]*y[i];
10: s.t.
11: r1{j in J}: sum{i in I} a[i,j]*x[i] + sum{i in A} a[i,j]*y[i]>= d[j];

```

Variables	Valores
Leche entera (1/2 taza)	4,071
Huevo (2 unidades)	0
Porotos (2 tazas)	0,022
Arroz (2 tazas)	0
Fideos (2 tazas)	3,846
Zanahoria (1 unidad)	1
Manzana (1 unidad)	0
Naranja (1 unidad)	10
Mantequilla (4 cucharadas)	0,582

Tabla 2.11: Cantidad de alimentos mínimos para sobrevivir.

De acuerdo al resultado obtenido, su consumo nutricional diario *versus* el requerimiento mínimo nutricional diario se muestra en la Tabla 2.12.

2.10.3 Variación propuesta para el estudiante

Este problema solo tuvo en cuenta los mínimos límites nutricionales, pero no consideró los máximos permitidos. Complemente la formulación incluyendo restricciones de valores máximos para cada macronutriente/micronutriente (generar los parámetros necesarios).

Macronutriente/ micronutriente	Consumo	Requerimiento mínimo
Proteína (g/día)	68,732	65
Lípidos o grasas (g/día)	66	66
Hidratos de carbono (g/día)	402,895	362
Vitamina A (μ g/día)	1 753,32	700
Vitamina C (mg/día)	507,816	75
Folato (mg/día)	400	400
Calcio (mg/día)	1 000	1 000
Hierro (mg/día)	18	18

Tabla 2.12: Consumo vs. requerimiento en la solución óptima.

2.11 Problema de compra de zapatos

Zapato a tu medida es una empresa de zapatos que cuenta con un presupuesto de 50 000 USD para un proyecto de innovación. Para ello, acude a 2 proveedores: Nikis y Pinki. Debido a que es una compra al por mayor, ambos proveedores venden los zapatos en paquetes (cajas) de varias unidades, que pueden variar dependiendo del tipo de zapato. La empresa desea comprar la misma cantidad de cajas a ambos proveedores, pero la compra debe ser de mínimo 30 cajas en total. Adicionalmente, la empresa busca que el total de pares de zapatos generados por la compra de las cajas sea al menos de 250 pares.

En la Tabla 2.13 se observan los datos de costos, número de pares de zapatos por caja y la utilidad que generaría para cada paquete y fabricante, respectivamente.

Caja	Fabricante	Costo [USD]	Número de pares de zapatos	Utilidad por caja [USD/caja]
Tenis	Nikis	2 700	45	4 500
Botín	Nikis	2 400	30	2 550
Tacones	Pinki	640	16	960
Sandalias	Pinki	625	25	1 250

Tabla 2.13: Especificación de zapatos por caja.

¿Cuántas cajas de zapatos debe comprar para obtener la máxima utilidad?

2.11.1 Modelo extendido

Píldora informativa 2.15: Modelo simple vs. complejo, ¿un subíndice o dos?

Uno de los principios de la programación lineal es **preferir un modelo simple a uno complejo**. Por simple se entiende un modelo con menos variables y/o restricciones (también se conoce como modelo esbelto).

En este problema se podría definir una variable con dos subíndices: x_{ij} como la cantidad de cajas i compradas al proveedor j . Sin embargo, esto no es necesario, porque **cada fabricante produce una referencia de zapatos distinta**, por tanto, solo con saber qué tipo de zapato se compra se puede deducir de qué proveedor viene. Si ambos fabricantes tuvieran referencias iguales, sí se debería usar dos subíndices.

Definición de variables de decisión

- x_1 : número de cajas de tenis a comprar [*cajas*].
- x_2 : número de cajas de botines a comprar [*cajas*].
- x_3 : número de cajas de tacones a comprar [*cajas*].
- x_4 : número de cajas de sandalias a comprar [*cajas*].

Modelo de programación lineal

$$\text{Maximizar } z = 4500 \cdot x_1 + 2550 \cdot x_2 + 960 \cdot x_3 + 1250 \cdot x_4 \quad (2.100)$$

Sujeto a:

$$2700 \cdot x_1 + 2400 \cdot x_2 + 640 \cdot x_3 \quad (2.101)$$

$$+ 625 \cdot x_4 \leq 50000$$

$$x_1 + x_2 + x_3 + x_4 \geq 30 \quad (2.102)$$

$$x_1 + x_2 = x_3 + x_4 \quad (2.103)$$

$$45 \cdot x_1 + 30 \cdot x_2 + 16 \cdot x_3 + 25 \cdot x_4 \geq 250 \quad (2.104)$$

$$x_1, x_2, x_3, x_4 \in \mathbb{Z}^+ \cup \{0\} \quad (2.105)$$

2.11.2 Modelo algebraico

Definición de conjuntos

- I : tipo de caja $\{1, 2, 3, 4\}$ para tenis, botines, tacones y sandalias, respectivamente.

Definición de parámetros

- c_i : costo de comprar la caja de tipo $i \in I$.
- s_i : pares de zapatos máximos en la caja de tipo $i \in I$.
- u_i : utilidad de la caja $i \in I$.

Definición variables de decisión

- x_i : cantidad de cajas de tipo $i \in I$ a comprar.

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in I} u_i \cdot x_i \quad (2.106)$$

Sujeto a:

$$\sum_{i \in I} c_i \cdot x_i \leq 50000 \quad (2.107)$$

$$\sum_{i \in I} x_i \geq 30 \quad (2.108)$$

$$\sum_{i=1}^2 x_i = \sum_{i=3}^4 x_i \quad (2.109)$$

$$\sum_{i \in I} s_i \cdot x_i \geq 250 \quad (2.110)$$

$$x_i \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I \quad (2.111)$$

La expresión (2.106) es la función objetivo del modelo, que busca maximizar la utilidad de las cajas compradas. La dimensión de la expresión está dada por:

$$[USD] = \left[\frac{USD}{caja} \right] \cdot [caja]$$

La expresión (2.107) restringe el monto de la inversión con el presupuesto disponible. La expresión (2.108) indica comprar como mínimo 50 cajas. La expresión (2.109) permite que la cantidad de cajas compradas al proveedor Nikis sea igual a las compradas al proveedor Pinki. La expresión (2.110) indica que la cantidad de pares de zapatos comprados sea de al menos 250 pares. Finalmente, la expresión (2.111) indica la no negatividad y la limitación a valores enteros de las variables de decisión.

El Algoritmo 11 contiene el código del modelo en AMPL. La solución obtenida por CPLEX en AMPL alcanza un costo total de inversión de 49875 USD y una utilidad de 86250 USD, al comprar 15 cajas de tenis al fabricante Nikis y 15 sandalias al fabricante Pinki.

Algoritmo 11: archivo Mezcla.mod

```
1: set I := {1..4};
2: param c{I};
3: param s{I};
4: param u{I};
5: var x{I} >= 0 integer;
6: maximize z: sum{i in I} u[i]*x[i];
7: s.t.
8: r1: sum{i in I} c[i]*x[i] <= 50000;
9: r2: sum{i in I} x[i] >= 30;
10: r3: sum{i in {1,2}} x[i] = sum{i in {3,4}} x[i];
11: r4: sum{i in I} s[i]*x[i] >= 250;
```

2.12 Problema de la mochila (*knapsack problem*)

Un ladrón decide entrar sigilosamente a la tienda *El Faraón del Loro* cuando está seguro de que no hay personal en ella. Cuenta con un vehículo para su huida, cuyo baúl tiene un volumen de $1,1 \text{ m}^3$, por ello debe decidir cuidadosamente qué artículos sustraer para maximizar su utilidad al revenderlos. En la Tabla 2.14 se muestran los artículos que puede robarse, el volumen y el beneficio neto de cada uno.

Producto	Volumen $\left[\frac{\text{m}^3}{\text{und}}\right]$	Utilidad $\left[\frac{\text{COP}}{\text{und}}\right]$
Caja de teléfonos	0,17	2000 000
Docena de <i>Laptops</i>	0,5	6000 000
Caja de libros	0,32	900 000
Caja de cremas	0,09	1 100 000
Equipo de sonido	0,2	900 000
Batería para carro	0,25	1 600 000
Microondas	0,01	300 000
Televisor	0,4	2 500 000
Parlantes	0,28	1 300 000

Tabla 2.14: Volumen y utilidad de los artículos.

2.12.1 Solución

Definición de conjuntos

- P : productos que puede sustraer $\{1, \dots, 9\}$, según el orden de la Tabla 2.14.

Definición de parámetros

- V_i : volumen del producto $i \in P$ $\left[\frac{\text{m}^3}{\text{und}}\right]$.
- U_i : utilidad del producto $i \in P$ $\left[\frac{\text{COP}}{\text{und}}\right]$.

Definición variables de decisión

$$\blacksquare \quad x_i : \begin{cases} 1, & \text{si decide sustraer el producto } i \in P \\ 0, & \text{en otro caso} \end{cases}$$

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in P} U_i \cdot x_i \quad (2.112)$$

Sujeto a:

$$\sum_{i \in P} V_i \cdot x_i \leq 1,1 \quad (2.113)$$

$$x_i \in \{0, 1\} \quad i \in P \quad (2.114)$$

La expresión (2.112) es la función objetivo del problema, que representa la maximización de la utilidad obtenida al revender los artículos robados.



Como los análisis dimensionales de los problemas son similares a los presentados hasta ahora a partir de este problema solo se realizarán cuando tengan elementos de mayor complejidad.

La restricción (2.113) indica que el volumen total de los productos robados no puede exceder el volumen del baúl, en este caso 1,1 m^3 . Finalmente, la expresión (2.114) indica que las variables son del tipo binarias.

El Algoritmo 12 contiene el código del modelo en AMPL. Según la solución obtenida por CPLEX en AMPL, el ladrón debe sustraer la caja de celulares ($x_1 = 1$), la docena de *Laptops* ($x_2 = 1$), la caja de cremas ($x_4 = 1$), la batería para carro ($x_6 = 1$) y el microondas ($x_7 = 1$). Al revender estos artículos, él podrá obtener una ganancia total de 11 millones COP.

Algoritmo 12: archivo ProblemaMochila.mod

```

1: set P := {1..9};
2: param V{P};
3: param U{P};
4: var x{P} binary;
5: maximize z: sum{i in P} U[i]*x[i];
6: s.t.
7: Volumen: sum{i in P} V[i]*x[i] <= 1.1;
```


2.12.2 Variación propuesta para el estudiante

Asuma que los siguientes artículos solo se pueden vender si se ofrecen en parejas: la Caja de teléfonos junto con los Parlantes, las *Laptops* junto con el equipo de sonido, y Microondas junto con el Televisor. Reformular el problema ajustándose a esta nueva información.

2.13 Problema de jugadores de fútbol

El entrenador del equipo de fútbol *Atlético Andes* desea escoger la alineación inicial para su nuevo equipo de fútbol. El equipo cuenta con 14 jugadores, de los cuales dos son arqueros y juegan un tiempo cada uno⁶.

Para esto, el entrenador realizó una evaluación de los jugadores con notas de 1 a 4, donde 1 es deficiente y 4 excelente. Los puntos evaluados fueron: habilidades con el balón, disparos al arco, velocidad y habilidades defensivas. Por otra parte, las posiciones que un jugador puede ocupar en la cancha son tres: defensa, medio-campo y delantero (D1, M y D2, respectivamente). En la Tabla 2.15 se presentan los resultados de las evaluaciones de cada jugador en las distintas categorías y posiciones que pueden ocupar.

La alineación inicial de los 10 jugadores de cancha (sin incluir al arquero) tiene que satisfacer las siguientes restricciones:

1. Por lo menos 4 de los jugadores deben ser capaces de jugar en la defensa, 3 en el medio-campo y 3 en la delantera.
2. El nivel promedio en la habilidad con el balón y la velocidad debe ser mayor o igual a 2 unidades para cada uno.
3. Si el jugador 5 está en la alineación, entonces el jugador 11 no puede estar, y viceversa.
4. Los jugadores 6 y 9 comienzan en el juego, solo si el jugador 1 lo hace también (si el jugador 1 no comienza el juego, ni el jugador 6 ni el 9 lo harán).

⁶Un partido usual de fútbol consiste en dos juegos de 90 minutos de duración. A cada uno de estos se le llama un tiempo: primer y segundo.

5. Puede jugar el jugador 2 o el jugador 4, pero no ambos al mismo tiempo.
6. El equipo debe estar compuesto por 10 jugadores y 2 que son asignados al banquillo.

Jugador	Posición	Habilidad con el balón	Disparo al arco	Velocidad	Habilidades defensivas
1	D1-M	1	2	4	2
2	M-D2	3	3	2	2
3	D1-M-D2	3	3	4	3
4	M-D2	4	4	3	1
5	D1-M	2	1	2	4
6	D1-M	4	2	2	2
7	D2	3	4	2	1
8	D1-M-D2	4	2	3	2
9	M-D2	2	3	4	1
10	D1-M-D2	2	3	4	1
11	M	3	4	3	4
12	M-D2	2	1	2	4

Tabla 2.15: Evaluación de cada jugador.

Dadas estas restricciones, el entrenador desea maximizar la capacidad total de disparo al arco de la formación inicial. Formular un modelo de programación lineal que ayude al entrenador a escoger su equipo.

2.13.1 Solución

Definición de conjuntos

- I : conjunto de Jugadores $\{1, \dots, 12\}$.
- J : posiciones a jugar $\{1, 2, 3\}$ para D1, M y D2, respectivamente.

Píldora informativa 2.16: Conjuntos complementarios

En los modelos algebraicos es común definir conjuntos complementarios, que no están de forma explícita en el enunciado, pero **ayudan a la formulación. Generalmente, agrupan información relevante y están compuestos por más de un único elemento.**

En este problema, por ejemplo, es de gran utilidad definir un nuevo conjunto cuyos elementos sean una dupla o pareja (i, j) , que contengan las posiciones prohibidas de cada jugador.

- A : parejas (i, j) de jugadores i que no pueden desempeñarse en la posición j : $\{(1, 3), (2, 1), (4, 1), (5, 3), (6, 3), (7, 1), (7, 2), (9, 1), (11, 1), (11, 3), (12, 1)\}$.

Definición de parámetros

- HB_i : habilidad con el balón del jugador $i \in I$.
- D_i : disparo al arco del jugador $i \in I$.
- V_i : velocidad del jugador $i \in I$.
- HD_i : habilidades defensivas del jugador $i \in I$ (se define el parámetro de forma ilustrativa, el problema no establece alguna condicional respecto a esta habilidad).
- $MinPlay_j$: mínimo de jugadores necesarios en posición $j \in J$.

Definición de las variables de decisión

- $x_{ij} : \begin{cases} 1, & \text{si el Jugador } i \in I \text{ es asignado a jugar en la posición } j \in J \\ 0, & \text{de lo contrario} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in I} \sum_{j \in J} D_i \cdot x_{ij} \quad (2.115)$$

Sujeto a:

$$\sum_{i \in I} x_{ij} \geq \text{MinPlay}_j, \quad \forall j \in J \quad (2.116)$$

$$\frac{\sum_{i \in I} \sum_{j \in J} HB_i \cdot x_{ij}}{10} \geq 2 \quad (2.117)$$

$$\frac{\sum_{i \in I} \sum_{j \in J} V_i \cdot x_{ij}}{10} \geq 2 \quad (2.118)$$

$$\sum_{j \in J} x_{5j} = 1 - \sum_{j \in J} x_{11,j} \quad (2.119)$$

$$\sum_{j \in J} x_{1j} = \sum_{j \in J} x_{6j} \quad (2.120)$$

$$\sum_{j \in J} x_{1j} = \sum_{j \in J} x_{9j} \quad (2.121)$$

$$\sum_{j \in J} x_{2j} = 1 - \sum_{j \in J} x_{4j} \quad (2.122)$$

$$\sum_{i \in I} \sum_{j \in J} x_{ij} = 10 \quad (2.123)$$

$$\sum_{j \in J} x_{ij} \leq 1, \quad \forall i \in I \quad (2.124)$$

$$x_{ij} = 0, \quad \forall (i, j) \in A \quad (2.125)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J \quad (2.126)$$

La expresión (2.115) corresponde a la función objetivo del problema, que maximiza la capacidad de disparo al arco, obtenida al realizar la asignación de los jugadores en las posiciones correspondientes.

La restricción (2.116) indica el número mínimo de jugadores que deben ser asignados en cada posición del campo. La restricciones (2.117) y (2.118) garantizan la limitación con respecto a la capacidad del balón y la velocidad promedio en el campo de juego.

Las expresiones (2.119) a (2.123) representan las restricciones para la selección de los jugadores (basado en las condiciones del problema). La expresión (2.124) obliga a que cada jugador no pueda desempeñar más una posición. La expresión (2.125) limita las posiciones que no pueden ser desempeñadas por cada jugador. Finalmente, la restricción (2.126) indica la naturaleza binaria las variables de decisión.

El Algoritmo (13) contiene el código del modelo en AMPL. La solución obtenida por CPLEX en AMPL, logrando una capacidad de disparo de 28 unidades, se muestra en la Figura 2.1.



Figura 2.1: Una alineación de jugadores óptima.

Algoritmo 13: archivo Jugadores.mod

```

1: set I := {1..12};
2: set J := {1..3};
3: set A := {(1,3), (2,1), (4,1), (5,3), (6,3), (7,1), (7,2), (9,1),
  (11,1), (11,3), (12,1)};
4: param HB{I};
5: param D{I};
6: param V{I};
7: param HD{I};
8: param MinPlay{J};
9: var x{I, J} binary;
10: maximize z: sum{i in I, j in J} D[i]*x[i,j];
11: s.t.
12: r1{j in J}: sum{i in I} x[i,j] >= MinPlay[j];
13: r2: (sum{i in I, j in J} HB[i]*x[i,j])/10 >= 2;
14: r3: (sum{i in I, j in J} V[i]*x[i,j])/10 >= 2;
15: r4: sum{j in J} x[5,j] = 1 - sum{j in J} x[11,j];
16: r5: sum{j in J} x[1,j] = sum{j in J} x[6,j];
17: r6: sum{j in J} x[1,j] = sum{j in J} x[9,j];
18: r7: sum{j in J} x[2,j] = 1 - sum{j in J} x[4,j];
19: r8: sum{i in I, j in J} x[i,j] = 10;
20: r9{i in I}: sum{j in J} x[i,j] <= 1;
21: r10{(i,j) in A}: x[i,j] = 0;

```

2.14 Problema de un *baby shower*

Usted desea realizar un *Baby Shower* para el recibimiento de su bebé y para ello, en consenso con sus posibles invitados, ha decidido hacer una rifa para asignar el regalo que cada invitado deberá llevar. En la Tabla 2.16 se muestra una lista resumen con el resultado de la asignación de regalos que usted clasificó como los más importantes.

Regalo	Costo [USD]	Invitado
Juego de teteros	62	José Alvarado
Paca de pañales	30	Ana Castro
Coche de bebé	85	Daniel Reyes
Extractor de leche	82	Valentina Acosta
Silla comedor	77	Camila Torres

Tabla 2.16: Lista de regalos y su asignación a los invitados.

Para hacer las invitaciones usted debe tener en cuenta lo siguiente: José Alvarado solo deseará asistir si asiste Valentina Acosta, pero no asistirá si van Ana Castro y Daniel Reyes (ambos). Además, Ana Castro no asistirá si Camila Torres está invitada, y viceversa, pero usted desea que una de las dos asista. Finalmente, Ana Castro no asistirá a menos que Daniel Reyes y Valentina Acosta asistan juntos.

Formular un modelo con variables binarias que permita seleccionar la lista de invitados para maximizar la contribución total obtenida en el *baby shower*.

2.14.1 Solución

Definición de conjuntos

- I : invitados $\{1, 2, 3, 4, 5\}$ para José, Ana, Daniel, Valentina y Camila, respectivamente.

Definición de parámetros

- U_i : costo de regalo $i \in I$. Definición de las variables de decisión
- x_i : $\begin{cases} 1, & \text{si la persona } i \in I \text{ es invitada al } baby \text{ shower} \\ 0, & \text{de lo contrario} \end{cases}$

Modelo de programación lineal

$$\text{Maximizar } z = \sum_{i \in I} U_i \cdot x_i \quad (2.127)$$

Sujeto a:

$$x_1 \leq x_4 \quad (2.128)$$

$$x_1 \leq 2 - x_2 - x_3 \quad (2.129)$$

$$x_2 = 1 - x_5 \quad (2.130)$$

$$x_4 + x_3 \leq 1 + x_2 \quad (2.131)$$

$$x_2 \leq x_4 \quad (2.132)$$

$$x_2 \leq x_3 \quad (2.133)$$

$$x_i \in \{0, 1\}, \quad \forall i \in I \quad (2.134)$$

La expresión (2.127) corresponde a la función objetivo del problema, que maximiza el beneficio obtenido por la asistencia de los invitados. La restricción (2.128) indica que José Alvarado puede asistir, si Valentina Acosta es invitada. La restricción (2.129) expresa que José Alvarado no asistirá, si van tanto Ana Castro como Daniel Reyes. La expresión (2.130) representa que Ana Castro no asistirá al *baby shower*, si Camila Torres lo hace, y viceversa.

De la restricción (2.131) a la (2.133) se expresa que Ana Castro no asistirá a menos que Daniel Reyes y Valentina Acosta asistan juntos. Finalmente, la restricción (2.134) del modelo indica la naturaleza binaria de las variables.

Píldora informativa 2.17: Restricciones conjuntas

Es usual que cada restricción sea independiente de las demás y represente una condición particular del problema. Sin embargo, algunas restricciones son **conjuntas**, es decir, **se complementan entre sí**. En el ejemplo, se encuentran dos casos:

- **Caso 1:** Valentina y Daniel asisten ($x_4 = 1$ y $x_3 = 1$), por tanto, Ana debería asistir. En este caso, la restricción (2.131) se encarga por sí sola de obligar a la variable de Ana (x_2) a tomar únicamente el valor de 1, para cumplir con la ecuación. Pero esta restricción no funciona en todos los casos. Por su parte, las restricciones (2.132) y (2.133) no restringen el valor de x_2 .

Este caso se muestra en la siguiente inecuación. En la parte inferior de cada variable se revela un número que representa el valor fijo o un símbolo de incógnita (?), que representa la variable a analizar y los posibles valores que pueda tomar dependiendo de la restricción.

$$\underset{1}{x_4} + \underset{1}{x_3} \leq 1 + \underset{? \rightarrow 1}{x_2}$$

- **Caso 2:** solo Valentina asiste ($x_4 = 1$ y $x_3 = 0$), o solo Daniel asiste ($x_4 = 0$ y $x_3 = 1$), por tanto, Ana **no** debería asistir. En este caso la restricción (2.131) no obliga a la variable de Ana (x_2), a tomar el valor de 0, porque la restricción admite tanto 0 como 1. Sin pérdida de generalidad, tomaremos el caso en el que solo Valentina asiste:

$$\underset{1}{x_4} + \underset{0}{x_3} \leq 1 + \underset{? \rightarrow 0/1}{x_2}$$

Por tanto, se necesita de las restricciones (2.132) y (2.133) para obligar a x_2 a tomar el valor de 0.

$$\underset{? \rightarrow 0/1}{x_2} \leq \underset{1}{x_4}, \text{ } x_2 \text{ puede tomar cualquier valor}$$

$$\underset{? \rightarrow 0}{x_2} \leq \underset{0}{x_3}, \text{ } x_2 \text{ solo puede ser 0}$$

El caso donde solo Daniel asiste es similar, pero la interpretación de las restricciones se intercambian. La primera restringiría el valor de x_2 mientras que la segunda no lo haría. Por esto son necesarias las dos restricciones, para que funcione en ambos casos.

El Algoritmo 14 contiene el código del modelo en AMPL. Según la solución obtenida por CPLEX, la asignación de los invitados al *baby shower* son: José Alvarado, Valentina Acosta y Camila Torres. Obteniendo así un beneficio de 221 USD.

Algoritmo 14: archivo baby.mod

```

1: set I:= {1..5};
2: param U{I};
3: var x{I} binary;
4: maximize z: sum{i in I} U[i]*x[i];
5: s.t.
6: r1: x[1] <= x[4];
7: r2: x[1] <= 2-x[2]-x[3];
8: r3: x[2] = 1 - x[5];
9: r4: x[4] + x[3] <= 1 + x[2];
10: r5: x[2] <= x[4];
11: r6: x[2] <= x[3];

```

2.15 Problema de cobertura (*set covering problem*)

Usted, como gerente de los bomberos *La Llama Eterna*, se encuentra frente a una gran encrucijada: le acaban de encomendar la construcción de las estaciones de bomberos en la ciudad de Cali. Para este fin, le han entregado el siguiente mapa (ver Figura 2.2), donde puede ver los 22 barrios (comunas) y las 24 posibles localizaciones para ubicar las estaciones de bomberos (nodos circulares). Cada localización (estación de bomberos) puede atender a los barrios adyacentes (que tocan). Por ejemplo, la localización 5 puede prestar sus servicios de bomberos a los barrios 10, 17 y 19. La construcción en cada localización tiene un costo fijo que se detalla en la Tabla 2.17.

- a. Formular este problema mediante programación lineal, para determinar en cuáles localizaciones se debe asignar una estación de bomberos, de modo que se minimicen los costos fijos. Considerar que todos los barrios deben ser atendidos.

- b. Formular este problema mediante programación lineal, para determinar en cuáles localizaciones se debe asignar una estación de bomberos, de modo que se maximice la población atendida. La información complementaria se presenta en la Tabla 2.18. En este caso, no necesariamente todos los barrios deben ser cubiertos y solo se pueden abrir máximo 4 estaciones de bomberos.

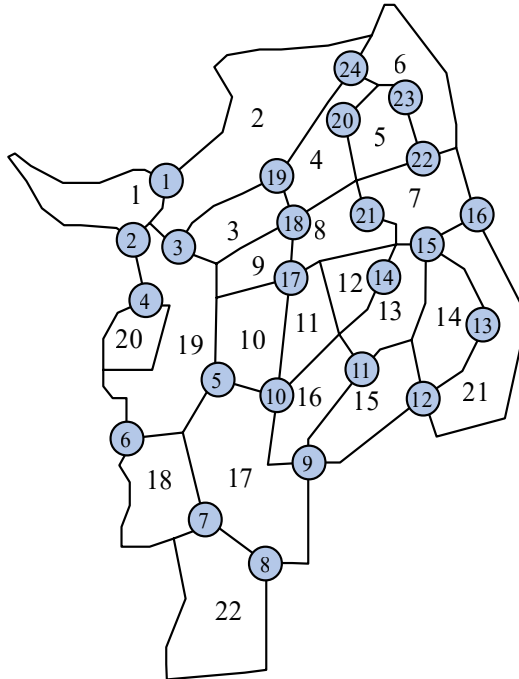


Figura 2.2: Mapa de la ciudad de Cali y las posibles localizaciones de las estaciones.

Estación de bomberos	Costo fijo [miles <i>USD</i>]	Estación de bomberos	Costo fijo [miles <i>USD</i>]
1	529	13	565
2	678	14	503
3	816	15	731
4	866	16	759
5	370	17	827
6	357	18	555

Estación de bomberos	Costo fijo [miles <i>USD</i>]	Estación de bomberos	Costo fijo [miles <i>USD</i>]
7	788	19	412
8	767	20	600
9	640	21	810
10	340	22	568
11	575	23	776
12	387	24	798

Tabla 2.17: Costos fijos por localización de las estaciones de bomberos.

Barrio	Habitantes	Barrio	Habitantes
1	65 333	12	67 638
2	103 022	13	169 659
3	44 088	14	151 544
4	56 933	15	126 496
5	100 358	16	94 383
6	166 906	17	103 975
7	78 097	18	100 276
8	96 991	19	98 257
9	47 830	20	65 440
10	103 087	21	92 170
11	98 172	22	8 971

Tabla 2.18: Población de barrios.

2.15.1 Solución a. Modelo para minimizar costos

Definición de conjuntos

- L : localización de estaciones $\{1, \dots, 24\}$.
- B : barrios (comunas) $\{1, \dots, 22\}$.

Definición de parámetros

- C_i : costo fijo por ubicar estación de bomberos en localización $i \in L$ [\$].
- $A_{ij} : \begin{cases} 1, \text{ si la estación de bomberos ubicada en } i \in L \text{ puede atender} \\ \text{al barrio } j \in B \\ 0, \text{ en otro caso} \end{cases}$

Píldora informativa 2.18: Nuevos parámetros

Es común que para la formulación se deben definir **nuevos parámetros**, es decir, **información que se encuentra en el enunciado pero no de forma explícita**.

Por ejemplo, en este problema el parámetro A_{ij} corresponde a una matriz de 24×22 (número de localizaciones o estaciones por número de barrios), donde se registra 1 si la localización $i \in L$ puede atender a $j \in B$, 0 en otro caso. En la Tabla 2.19 se muestra un esquema de este parámetro.

		Barrios										
Estaciones		1	2	3	4	5	6	...	19	20	21	22
	1	1	1	0	0	0	0	...	0	0	0	0
	2	1	0	0	0	0	0	...	1	0	0	0
	3	0	1	1	0	0	0	...	1	0	0	0
	4	0	0	0	0	0	0	...	1	1	0	0
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	...	⋮	⋮	⋮	⋮
	23	0	0	0	0	1	1	...	0	0	0	0
	24	0	1	0	1	0	1	...	0	0	0	0

Tabla 2.19: Esquema de A_{ij} que representa la cobertura de los barrios por las estaciones.

Definición de variables de decisión

- $x_i : \begin{cases} 1, \text{ si se decide asignar una estación de bomberos en la} \\ \text{localización } i \in L \\ 0, \text{ en otro caso} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in L} C_i \cdot x_i \quad (2.135)$$

Sujeto a:

$$\sum_{i \in L} A_{ij} \cdot x_i \geq 1, \quad \forall j \in B \quad (2.136)$$

$$x_i \in \{0, 1\}, \quad \forall i \in L \quad (2.137)$$

La expresión (2.135) es la función objetivo, que representa la minimización de los costos fijos por estación de bomberos asignada a una determinada localización. La restricción (2.136) garantiza que todos los barrios deben ser atendidos por al menos una estación de bomberos. Finalmente, la expresión (2.137) indica la naturaleza binaria de las variables.

El Algoritmo 15 contiene el código del modelo algebraico en AMPL. Según la solución obtenida con CPLEX, se logra un costo mínimo de 4536 miles de USD con la construcción de las estaciones como se muestra en la Figura 2.3.

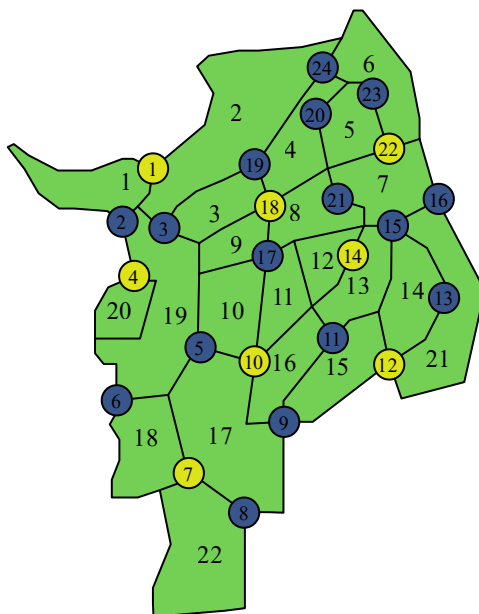


Figura 2.3: Solución a. plan óptimo de construcción de estaciones de bomberos.

Algoritmo 15: archivo Cobertura1

```

1: set L := {1..24};
2: set B := {1..22};
3: param C{L};
4: param A{L,B};
5: var x{L} binary;
6: minimize z: sum{i in L} C[i]*x[i];
7: s.t.
8: r1{j in B}: sum{i in L} x[i]*A[i,j] >= 1;

```

2.15.2 Solución b. Modelo para maximizar la cobertura

Definición de conjuntos

- L : localización de estaciones {1, ..., 24}.
- B : barrios (comunas) {1, ..., 22}.

Definición de parámetros

- P_j : población del barrio $j \in B$ [habitantes].
- $A_{ij} : \begin{cases} 1, \text{ si la estación de bomberos ubicada en } i \in L \text{ puede} \\ \text{atender al barrio } j \in B \\ 0, \text{ en otro caso} \end{cases}$

Definición variables de decisión

- $x_i : \begin{cases} 1, \text{ si se decide asignar una estación de bomberos en la} \\ \text{localización } i \in L \\ 0, \text{ en otro caso} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in L} \sum_{j \in B} P_j \cdot A_{ij} \cdot x_i \quad (2.138)$$

Sujeto a:

$$\sum_{i \in L} x_i \leq 4 \quad (2.139)$$

$$x_i \in \{0, 1\}, \quad \forall i \in L \quad (2.140)$$

La expresión (2.139) es la función objetivo del problema, que representa la maximización de la población atendida por las estaciones de bomberos asignadas. La restricción (2.140) establece que el número de estaciones de bomberos asignadas debe ser menor o igual a 4. Finalmente, la expresión (2.138) indica que las variables son del tipo binarias.

El Algoritmo 16 contiene el código del modelo algebraico en AMPL. Según la solución obtenida con CPLEX, se debe construir una estación de bomberos en las localizaciones 11, 12, 15 y 17. Con esta asignación se logra atender a un máximo de 1 598 300 habitantes.

Algoritmo 16: archivo Cobertura2.mod

```

1: set L := {1..24};
2: set B := {1..22};
3: param P{B};
4: param A{L,B};
5: var x{L} binary;
6: maximize z: sum{i in L, j in B} A[i,j]*P[j]*x[i];
7: s.t.
8: r1: sum{i in L} x[i] <= 4;
```

Capítulo 3

Modelos de complejidad media

En este capítulo se proponen 15 ejercicios que se resuelven mediante modelos matemáticos en notación algebraica. Los casos seleccionados permiten visualizar y comprender la complejidad de enfrentarse a situaciones cotidianas y reales. Los casos incluyen: sistemas de producción, mezclas, secuenciamiento, inventarios, asignación, entre otros, que pueden ser generalizados y extendidos a problemas altamente complejos. Problemas que, a diario, enfrentan pequeñas, medianas y grandes empresas en su operación.

Los archivos correspondientes a la solución en AMPL de los problemas planteados en este capítulo se pueden encontrar en “Aprende Modelando - GitHub”. [Online]. Disponible en: <https://github.com/dmorill/AprendeModelandoMILP>.

3.1 Problema de suministro para la guerra (adaptado de [4])

Remontémonos a mediados de la década de los años 40, en la Segunda Guerra Mundial. Su comando tiene dos tipos de aviones disponibles: TIP1 y TIP2. Ambos deben ser usados para efectuar una misión de apoyo logístico de armamento y transporte terrestre a los aliados. Cada avión TIP1 tiene una capacidad de 20 toneladas de armamento, 4 vehículos y 3 jeeps; cada avión TIP2 tiene una capacidad de 15 toneladas de armamento, 1 vehículo y 2 jeeps.

Se recibe la orden de abastecer los puntos estratégicos A y B. El primero requiere, como mínimo, 570 toneladas de armamento, 25 vehículos y 18 jeeps. Para el punto estratégico B se requiere como mínimo 350 toneladas de armamento, y una cierta cantidad de vehículos y jeeps. Este punto estratégico tiene combustible solo para 120 vehículos o solo para 90 jeeps, o una combinación apropiada de ambos.

El comando ha determinado que esta misión se efectúe desde cierto aeropuerto clandestino y únicamente se dispone del tiempo suficiente para enviar solo 120 aviones TIP1 o solo 85 aviones TIP2, o cualquier combinación posible dentro de la capacidad aeroportuaria asignada.

En el aeropuerto clandestino se dispone de 2000 toneladas de armamento, 150 vehículos y 230 jeeps. Cada vuelo del avión TIP1 tiene un costo de 120 000 USD al punto A y 80 000 USD al punto B. Por su parte, cada vuelo del avión TIP2 tiene un costo de 90 000 USD al punto A y 60 000 USD al punto B. Asumiendo que cada avión siempre va lleno (es decir con un uso del 100 % de su capacidad de cada artículo) ¿cómo planificaría la misión para proporcionar la cobertura a mínimo costo?

3.1.1 Solución

Definición de conjuntos

- I : tipos de aviones de guerra $\{1, 2\}$.
- J : puntos estratégicos $\{1, 2\}$, que representan el punto A y el punto B, respectivamente.
- K : tipo de artículos a transportar $\{1, 2, 3\}$, que representan las toneladas de armamento, número de vehículos y de jeeps, respectivamente.

Definición de parámetros

- c_{ij} : costo por vuelo de avión tipo $i \in I$ al punto estratégico $j \in J$.
- cap_{ik} : capacidad de artículo $k \in K$ en avión tipo $i \in I$.
- dem_{A_k} : demanda de artículo $k \in K$ en punto estratégico A.

- $demB$: demanda de toneladas de armamento en punto estratégico B.
- $capAero_k$: disponibilidad de artículo $k \in K$ en aeropuerto.

Definición de las variables de decisión

- x_{ij} : número de aviones tipo $i \in I$ a enviar al punto estratégico $j \in J$.



Nótese que, a pesar de no conocer la cantidad de combustible disponible en el punto estratégico B, no se ha definido ningún parámetro ni variable al respecto. Esto se debe a que las condiciones y uso de combustible se garantizarán mediante restricciones, como se explicará a continuación.

Modelo algebraico de programación lineal

Píldora informativa 3.1: Capacidad porcentual

En ocasiones no se tiene estimada de forma numérica la capacidad de un recurso (por ejemplo, en este problema no se sabe la cantidad de combustible disponible en el punto estratégico B), pero **es posible saber su valor en función del número máximo de elementos que usan dicho recurso** (el combustible se agota si se usan 120 vehículos o 90 jeeps).

En estos casos, **se puede estimar la fracción de uso del recurso de cada elemento** (vehículo y jeep), al invertir su número máximo (equivalente a hacer una regla de tres). Por ejemplo, un vehículo consume $\frac{1}{120}$ % del combustible en B y un jeep consume $\frac{1}{90}$ % de dicho combustible.

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} c_{ij} \cdot x_{ij} \quad (3.1)$$

Sujeto a:

$$\sum_{i \in I} cap_{ik} \cdot x_{i1} \geq demA_k, \quad \forall k \in K \quad (3.2)$$

$$\sum_{i \in I} cap_{i1} \cdot x_{i2} \geq demB \quad (3.3)$$

$$\frac{1}{120} \cdot \sum_{i \in I} x_{i2} \cdot cap_{i2} + \frac{1}{90} \cdot \sum_{i \in I} x_{i2} \cdot cap_{i3} \leq 1 \quad (3.4)$$

$$\frac{1}{120} \cdot \sum_{j \in J} x_{1j} + \frac{1}{85} \cdot \sum_{j \in J} x_{2j} \leq 1 \quad (3.5)$$

$$\sum_{i \in I} \sum_{j \in J} cap_{ik} \cdot x_{ij} \leq capAero_k, \quad \forall k \in K \quad (3.6)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (3.7)$$

La expresión (3.1) es la función objetivo del problema, que busca minimizar el costo logístico de transporte, obtenido al realizar la distribución de vuelos para la misión.

Las restricciones (3.2) y (3.3) garantizan el cumplimiento de la demanda de artículos en cada punto estratégico. La restricción (3.4) garantiza no sobrepasar la capacidad de combustible en el punto estratégico B. Mientras que la restricción (3.5) asegura no sobrepasar la combinación de aviones posible dentro de la capacidad aeroportuaria. La restricción (3.6) garantiza no exceder los artículos disponibles en el aeropuerto. Por último, la expresión (3.7) expone la naturaleza de las variables.

El Algoritmo 17 contiene el código del modelo en AMPL. En conclusión, para minimizar el costo logístico de transporte, al punto estratégico A se deben enviar 21 aviones TIP1 y 10 TIP2; por otra parte, al punto estratégico B se debe enviar 10 aviones TIP1 y 10 TIP2. El costo de la misión asciende a USD 4820000.

Algoritmo 17: archivo GuerraUno.mod

```

1: set I := {1,2};
2: set J := {1,2};
3: set K := {1,2,3};
4: param c{I,J};
5: param cap{I,K};
6: param demA{K};
7: param demB;
8: param capAero{K};
9: var x{I,J} >= 0 integer;
10: minimize z: sum{i in I, j in J} c[i,j]*x[i,j];
11: s.t.
12: r1{k in K}: sum{i in I} cap[i,k]*x[i,1] >= demA[k];
13: r2: sum{i in I} cap[i,1]*x[i,2] >= demB;

```

```

14: r3:  $\sum\{i \text{ in } I\} x[i,2] \cdot \text{cap}[i,2] \cdot 1/120 + \sum\{i \text{ in } I\} x[i,2] \cdot$ 
     $\text{cap}[i,3] \cdot 1/90 \leq 1;$ 
15: r4:  $\sum\{j \text{ in } J\} x[1,j] \cdot 1/120 + \sum\{j \text{ in } J\} x[2,j] \cdot 1/85 \leq 1;$ 
16: r5{k in K}:  $\sum\{j \text{ in } J, i \text{ in } I\} \text{cap}[i,k] \cdot x[i,j] \leq \text{capAero}[k];$ 

```

3.1.2 Variación propuesta para el estudiante

¿Cuáles son los principales cambios que sufre el modelo si se considera que cada avión puede ir con cualquier fracción de la carga (sin exceder su capacidad)?

3.2 Problema de corte de madera (adaptado de [5])

En una pyme⁷ se fabrican marcos para cuadros. Estos se obtienen de cortar varillas de 300 cm de largo (materia prima) para elaborar boceses⁸. El departamento de ventas tiene pedidos para el mes siguiente de 175 cuadros de 119 cm $\times$ 90 cm (largo y ancho). Por cada cuadro se requieren dos boceses de cada dimensión ($175 \cdot 2 = 350$ boceses). Por lo tanto, el jefe de producción ordena cortar 350 boceses tanto de 119 cm como de 90 cm.

La pyme considera desperdicio todo material residual de la pieza original de 300 cm, luego de un corte del cual no se pueda obtener una pieza de 119 cm ni una de 90 cm. El costo de gestionar el desperdicio asciende a 1 USD por centímetro. Además, solo se tomarán en cuenta maneras eficientes de cortar la materia prima, es decir, sin dejar material para obtener una pieza útil (de 119 cm o 90 cm). El exceso de cada pieza incurre en un costo por manejo de inventario, costando 40 USD las piezas de 119 cm y 30 USD las de 90 cm.

Formule este problema mediante un modelo de programación lineal que minimice el desperdicio, el exceso de piezas y optimice la productividad.

⁷Pequeña y mediana empresa.

⁸Moldura convexa, cuya sección es un cuarto de círculo, se usan para enmarcar cuadros.

3.2.1 Solución

Píldora informativa 3.2: Definición de nuevas variables 2

Similar a la Píldora informativa 2.7, en este problema se requiere de nuevas variables, pero en esta ocasión, de estas variables se obtendrán los parámetros necesarios para el problema (como los desperdicios de materia prima obtenidos de cada tipo de corte). Para esto, primero se debe determinar todas las formas eficientes (combinaciones) de cortar la materia prima (Figura 3.1):

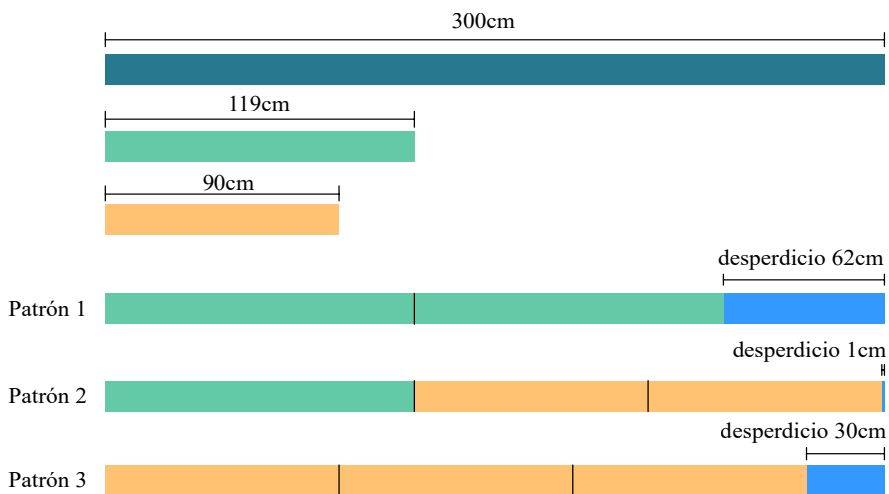


Figura 3.1: Patrones de corte eficientes.

Definición de conjuntos

- I : patrón de corte de varilla $\{1, 2, 3\}$ según la Figura 3.1.
- J : tipo de piezas demandadas $\{1, 2\}$ para 119 cm y 90 cm, respectivamente.

Definición de parámetros

- D_i : desperdicio en centímetros generado por el patrón de corte $i \in I$.

- B : número de bodeles ordenado por el jefe de producción por día (350 bodeles).
- C_j : costo de inventario de bodeles tipo $j \in J$.
- VP_{ij} : número de piezas de tipo $j \in J$ que contiene el patrón de corte $i \in I$.
- C_{des} : costo asociado al manejo de desperdicio (1 USD).

Definición de las variables de decisión

- x_i : número de varillas de 300 cm a cortar en patrón de corte $i \in I$.
- Inv_j : excedente de varillas tipo $j \in J$.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} D_i \cdot x_i \cdot C_{des} + \sum_{j \in J} Inv_j \cdot C_j \quad (3.8)$$

Sujeto a:

$$\sum_{i \in I} VP_{ij} \cdot x_i \geq B, \quad \forall j \in J \quad (3.9)$$

$$Inv_j = \sum_{i \in I} VP_{ij} \cdot x_i - B, \quad \forall j \in J \quad (3.10)$$

$$x_i \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I \quad (3.11)$$

$$Inv_j \in \mathbb{Z}^+ \cup \{0\}, \quad \forall j \in J \quad (3.12)$$

La expresión (3.8) corresponde a la función objetivo del problema y busca minimizar los costos del desperdicio total y del inventario excedente en la producción de los bodeles para los cuadros. La restricción (3.9) garantiza el cumplimiento de la demanda del número de bodeles. La restricción (3.10) define el número de piezas excedentes luego de cumplir con la demanda. Por último, las expresiones (3.11) y (3.12) definen la naturaleza de las variables.

El Algoritmo 18 contiene el código del modelo en AMPL. La solución obtenida por CPLEX en AMPL es cortar 87 varillas según el patrón 1, y 176 varillas según el patrón 2, generando un costo mínimo de 5630 USD.

Algoritmo 18: archivo Corte.mod

```

1: set I:={1,2,3};
2: set J:={1,2};
3: param D{I};
4: param C{J};
5: param B;
6: param Vp{I,J};
7: param Cdes;
8: var Inv{J} >= 0 integer;
9: var x{I} >= 0 integer;
10: minimize z: sum{i in I} x[i]*D[i]*Cdes + sum{j in J} Inv[j]*C[j];
11: s.t.
12: r1{j in J}: sum{i in I} x[i]*Vp[i,j] >= B;
13: r2{j in J}: Inv[j] = sum{i in I} x[i]*Vp[i,j]-B;

```

3.3 Problema del inventario de tapabocas

Una empresa distribuidora de tapabocas tiene la siguiente demanda pronosticada para los próximos cuatro meses: 2 800, 2 200, 3 200 y 2 500 unidades para los meses 1, 2, 3 y 4, respectivamente. La empresa tiene capacidad para producir 2700 unidades de tapabocas por mes, en sus turnos normales. Utilizando tiempo extra, es posible fabricar 300 unidades adicionales. La producción en tiempo extra tiene un sobre costo de 10 USD por unidad.

La administración ha estimado un costo de almacenamiento de 2 USD por unidad producida y que no sea vendida en un mes determinado, como se muestra a continuación:

$$\left[\frac{USD}{uni \cdot mes} \right]$$

¿Cuál es el programa óptimo de producción que minimice los costos de producción en tiempo extra y almacenamiento, cumpliendo la demanda de tapabocas cada mes? Asumir que la cantidad en existencia actual es 0 y se desea un inventario final del periodo también igual a 0.

3.3.1 Solución*Definición de conjuntos*

- T : meses de producción $\{1, \dots, 4\}$.

Definición de parámetros

- dem_t : demanda en mes $t \in T$.
- h : costo de inventario.
- ce : costo de fabricación de una unidad en tiempo extra.
- cap : capacidad de producción en horario normal.
- $capE$: capacidad de producción en horario extra.

Definición de las variables de decisión

- x_t : número de unidades a producir en mes $t \in T$ en tiempo normal.
- e_t : número de unidades a producir en mes $t \in T$ en tiempo extra.
- I_t : número de unidades en inventario en $t \in T \cup \{0\}$. Tener en cuenta que esta variable es la única que está definida en $t = 0$, debido a que existe un inventario inicial (0 en este caso). En el modelo matemático se debe establecer este inventario inicial.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{t \in T} (e_t \cdot ce + I_t \cdot h) \quad (3.13)$$

Sujeto a:

$$x_t \leq cap, \quad \forall t \in T \quad (3.14)$$

$$e_t \leq capE, \quad \forall t \in T \quad (3.15)$$

$$I_{t-1} + x_t + e_t = dem_t + I_t, \quad \forall t \in T \quad (3.16)$$

$$I_0 = 0 \quad (3.17)$$

$$I_4 = 0 \quad (3.18)$$

$$x_t, e_t \in \mathbb{Z}^+ \cup \{0\}, \quad \forall t \in T \quad (3.19)$$

$$I_t \in \mathbb{Z}^+ \cup \{0\}, \quad \forall t \in T \cup \{0\} \quad (3.20)$$

La expresión (3.13) representa la función objetivo, que busca minimizar el costo asociado a la contratación de horas extra e inventario.

La restricción (3.14) garantiza que la producción de tapabocas no exceda la capacidad de la empresa. La restricción (3.15) establece el límite máximo de unidades a producir en tiempo extra. La restricción (3.16) representa el balance de la producción. Las restricciones (3.17) y (3.18) establecen las condiciones iniciales y finales del inventario. Finalmente, las restricciones (3.19) y (3.20) describen la naturaleza de las variables.

Píldora informativa 3.3: Balance de inventarios

El balance de inventarios **consiste en igualar las cantidades de dos instantes en el tiempo t** , cada uno con sus propios elementos:

- Al iniciar t (**lo disponible**): se tiene el inventario del periodo pasado (I_{t-1}), la producción normal (x_t) y cualquier otra fuente de generación (como producción extra o unidades programadas).
- Al finalizar t (**lo sobrante**): se tiene la demanda de los artículos (o las ventas), el estado del inventario al finalizar t : inventario actual (I_t) en caso de un excedente, o bien una deuda en caso de unidades faltantes.

El Algoritmo 19 contiene el código del modelo en AMPL. La solución óptima del problema sugiere un plan de producción como se muestra en la Tabla 3.1. De esta forma se consigue un costo total mínimo de 2000 USD.

Algoritmo 19: archivo InvTapabocas.mod

```
1: set T := {1..4};
2: param dem{T};
3: param h;
```

```

4: param ce;
5: param cap;
6: param capE;
7: var x{T}>= 0 integer;
8: var e{T}>= 0 integer;
9: var I{T union {0}}>= 0 integer;
10: minimize z: sum{t in T}(e[t]*ce + I[t]*h);
11: s.t.
12: r1{t in T}: x[t] <= cap;
13: r2{t in T}: e[t] <= capE;
14: r3{t in T}: I[t-1] + x[t] + e[t] = dem[t] + I[t];
15: r4: I[0] = 0;
16: r5: I[4] = 0;

```

t [mes]	Demanda [und]	x_t [und]	e_t [und]	I_t [und]
0	-	-	-	0
1	2800	2700	100	0
2	2200	2700	0	500
3	3200	2700	0	0
4	2500	2500	0	0

Tabla 3.1: Plan de producción de tapabocas.

3.3.2 Variación propuesta para el estudiante

Reformular el problema considerando un segundo artículo. ¿Cómo cambia la restricción asociada al balance del inventario?

3.4 Problema de distribución en un avión (adaptado de [3])

Usted es el encargado de distribuir la carga al interior de un avión que tiene tres compartimentos. Estos están ubicados adelante, al centro y atrás de su fuselaje. Dichos compartimentos tienen límites para su almacenamiento y transporte, en cuanto al volumen y peso máximos, descritos en la Tabla 3.2.

Compartimentos	Capacidad máx. peso [ton]	Capacidad máx. volumen [m^3]
Adelante	16	200
Central	20	250
Atrás	14	150

Tabla 3.2: Capacidad de compartimentos.

Para el siguiente vuelo se ha determinado transportar la carga indicada en la Tabla 3.3. Si la carga excede la capacidad del avión, se puede decidir transportar solo una fracción de ella.

Clase de carga	Cantidad a transportar [ton]	Volumen [m^3/ton]	Utilidad [10^3 USD/ton]
Herramientas	20	1	250
Libros	15	2	280
Flores	8	10	500
Esculturas	10	6	360

Tabla 3.3: Carga disponible a transportar, su volumen y utilidad.

Por motivos de seguridad y para preservar el equilibrio del avión, es necesario que el peso transportado en cada compartimiento guarde la misma proporción con respecto a su capacidad. Por ejemplo, si el compartimento delantero se usa a un 60 % de su capacidad, los compartimentos central y trasero deberían usar la misma proporción (no cantidad).

¿Qué tipo de carga, en qué cantidad y en cuáles compartimentos se debe cargar el avión, de modo que el propietario maximice su utilidad y, a la vez, pueda viajar seguro en su avión?

3.4.1 Solución

Definición de conjuntos

- I : tipo de carga $\{1, \dots, 4\}$, para herramientas, libros, flores y esculturas, respectivamente.
- J : compartimiento $\{1, 2, 3\}$, para adelante, central y atrás, respectivamente.

Definición de parámetros

- u_i : utilidad generada al transportar carga de tipo $i \in I$ [10^3 USD].
- P_j : capacidad máxima de peso del compartimento $j \in J$ [ton].
- V_j : capacidad máxima de volumen del compartimento $j \in J$ [m^3].
- C_i : cantidad de carga de tipo $i \in I$ a transportar [ton].
- Vol_i : volumen máximo de la carga tipo $i \in I$ a transportar [m^3].

Definición de las variables de decisión

- x_{ij} : toneladas de carga tipo $i \in I$ a almacenar y transportar en el compartimento $j \in J$.

Píldora informativa 3.4: Variables de decisión auxiliares

En muchos casos se **definen variables de decisión auxiliares**. El problema no las menciona explícitamente, pero son de utilidad para el desarrollo del problema. En este caso, el porcentaje de utilización de los compartimientos no se establece con un valor, solo se solicita que sea el mismo. Aun así **el valor es realmente variable**, por esta razón se definirá como una variable de decisión (G), la misma para todos los compartimientos.

- G : porcentaje de utilización de los compartimientos [%].

Modelo algebraico de programación lineal

Píldora informativa 3.5: Nombre de la función objetivo

Es usual llamar a la función objetivo como z . Sin embargo, hay que recordar que dicha **función objetivo no es más que una variable que almacenará el valor del objetivo**, por tanto, **se puede usar cualquier nombre que se le asigne**. En este problema se usará la letra U para definirla.

$$\text{Maximizar } U = \sum_{i \in I} \sum_{j \in J} x_{ij} \cdot u_i \quad (3.21)$$

Sujeto a:

$$\sum_{i \in I} x_{ij} \leq P_j, \quad \forall j \in J \quad (3.22)$$

$$\sum_{i \in I} x_{ij} \cdot Vol_i \leq V_j, \quad \forall j \in J \quad (3.23)$$

$$\sum_{j \in J} x_{ij} \leq C_i, \quad \forall i \in I \quad (3.24)$$

$$\frac{\sum_{i \in I} x_{ij}}{P_j} = G, \quad \forall j \in J \quad (3.25)$$

$$x_{ij} \in \mathbb{R}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (3.26)$$

La expresión (3.21) representa la función objetivo, que busca maximizar las utilidades generadas al transportar los 4 tipos de carga.

Las restricciones (3.22) y (3.23) garantizan que la carga no supere el peso y volumen permitidos en cada compartimento, respectivamente. La restricción (3.24) limita la carga a la cantidad disponible de elementos a transportar. La restricción (3.25) garantiza que todos los compartimentos usen la misma proporción de peso, según su capacidad (peso usado/capacidad máxima). Finalmente, la restricción (3.26) expone la naturaleza de las variables.

El Algoritmo 20 contiene el código del modelo en AMPL. La solución obtenida por Gurobi logra alcanzar utilidades de 16050 USD. Para ello se deben transportar en cada compartimento las cargas como se muestra en la Figura 3.2.

Algoritmo 20: archivo AvionCarga.mod

```

1: set I := {1..4};
2: set J := {1..3};
3: param u{I};
4: param P{J};
5: param V{J};
6: param C{I};
7: param Vol{I};

```

```

8: var x{I,J} >= 0;
9: var G >= 0;
10: maximize z: sum{i in I,j in J} u[i]*x[i,j];
11: s.t.
12: r1{j in J}: sum{i in I} x[i,j] <= P[j];
13: r2{j in J}: sum{i in I} x[i,j]*Vol[i] <= V[j];
14: r3{i in I}: sum{j in J} x[i,j] <= C[i];
15: r4{j in J}: sum{i in I} x[i,j]/P[j] = G;

```

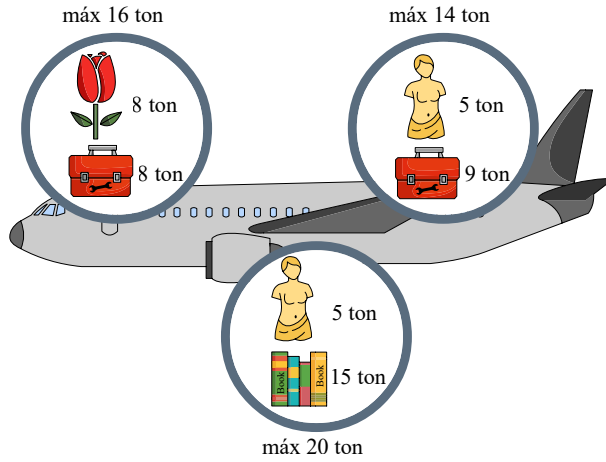


Figura 3.2: Distribución de peso en el avión.

3.5 Problema de mezcla de combustible (adaptado de [5])

Una compañía de petróleo produce tres tipos de gasolina: súper, normal y euro. Las gasolinas se obtienen mezclando tres tipos de crudo, que varían según su calidad (A, B y C). Los crudos, a su vez, contienen tres componentes (1, 2 y 3). El porcentaje de cada componente en los tres tipos de crudo se ilustra en la Tabla 3.4.

	Componente 1	Componente 2	Componente 3
Crudo A	80 %	10 %	5 %
Crudo B	45 %	30 %	20 %
Crudo C	30 %	40 %	25 %

Tabla 3.4: Composición de cada crudo.

La suma de los porcentajes de los tres componentes para cada crudo no constituyen el 100 %, por ejemplo, en el crudo A los componentes 1, 2, y 3 suman 95 % y el 5 % restante se puede considerar como desperdicio. De manera similar sucede con el crudo B y el crudo C. Por otra parte, debido a las especificaciones de cada tipo de gasolina, se debe cumplir con un porcentaje determinado para cada componente, como se muestra en la Tabla 3.5.

Tipo de gasolina	Componente 1	Componente 2	Componente 3
Súper	$\geq 60 \%$	$\leq 25 \%$	$\geq 10 \%$
Normal	$\geq 30 \%$	$\leq 20 \%$	$\leq 15 \%$
Euro	$\leq 40 \%$	$\geq 35 \%$	$\geq 20 \%$

Tabla 3.5: Especificaciones de componentes para cada tipo de gasolina.

Los costos por barril de crudo A, B y C son 250 USD, 120 USD y 200 USD, respectivamente. El presupuesto diario de compra es de 50000 USD. La disponibilidad diaria de los crudos B y C es limitada a 300 y 700 barriles, respectivamente. Ciertos acuerdos obligan a comprar al menos 10 barriles de crudo A.

Las demandas de gasolina súper y normal son de 60 y 70 barriles diarios, que deben satisfacerse. La compañía desea maximizar la producción de gasolina euro. Formule un modelo de programación lineal que dé respuesta al problema presente en la compañía.

3.5.1 Solución

Definición de conjuntos

- I : gasolinas tipo súper, normal y euro $\{1, 2, 3\}$.
- J : crudos A, B y C $\{1, 2, 3\}$.
- K : componentes $\{1, 2, 3\}$.

Definición de parámetros

- PC_{jk} : porcentaje de componente $k \in K$ en el crudo tipo $j \in J$.
- PG_{ik} : porcentaje de componente $k \in K$ requerido en la gasolina tipo $i \in I$.
- $CostoC_j$: costo por barril de crudo tipo $j \in J$.
- $Presu$: presupuesto diario de compra.
- $Disp_j$: barriles diarios disponibles de crudo tipo $j \in \{2, 3\}$.
- BA : barriles de crudo tipo A que deben ser comprados.
- DEM_i : demanda diaria de gasolina tipo $i \in \{1, 2\}$.

Definición de variables de decisión

- X_{ij} : barriles diarios de crudo tipo $j \in J$ comprados y destinados a la producción de gasolina tipo $i \in I$.

Modelo algebraico de programación lineal

Píldora informativa 3.6: Restricciones de mezcla

En los problemas donde varios productos o ingredientes (formados por una fracción o porcentaje de componentes útiles) se mezclan para generar un producto nuevo (producto final), se debe considerar que la **cantidad total** de dicho producto final debe estar relacionado con la **cantidad de ingredientes**. Así, en este problema, al mezclar 1 barril de cada crudo, se obtienen 3 barriles de producto final y sobre ese total se debe garantizar la concentración de cada componente. En la Figura 3.3 se muestra un ejemplo con el componente 1 al mezclar un barril de cada tipo de crudo.

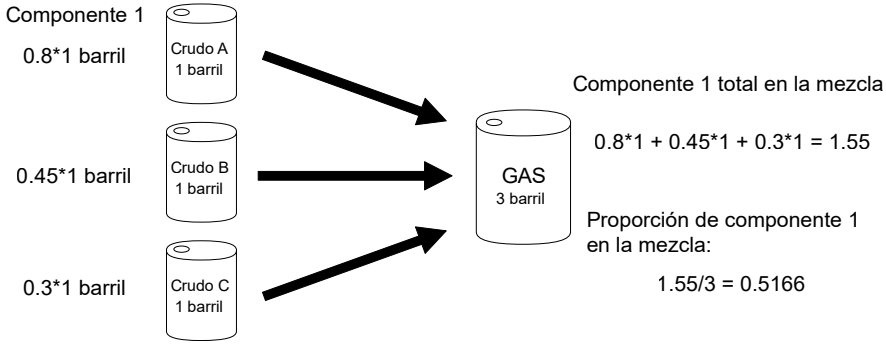


Figura 3.3: Mezcla de crudos para el componente 1.

$$\text{Maximizar } z = \sum_{j \in J} X_{3j} \quad (3.27)$$

Sujeto a:

$$\sum_{j \in J} PC_{jk} \cdot X_{1j} \geq PG_{1k} \cdot \sum_{j \in J} X_{1j}, \quad \forall k \in \{1, 3\} \quad (3.28)$$

$$\sum_{j \in J} PC_{j2} \cdot X_{1j} \leq PG_{12} \cdot \sum_{j \in J} X_{1j} \quad (3.29)$$

$$\sum_{j \in J} PC_{jk} \cdot X_{2j} \leq PG_{2k} \cdot \sum_{j \in J} X_{2j}, \quad \forall k \in \{2, 3\} \quad (3.30)$$

$$\sum_{j \in J} PC_{j1} \cdot X_{2j} \geq PG_{21} \cdot \sum_{j \in J} X_{2j} \quad (3.31)$$

$$\sum_{j \in J} PC_{jk} \cdot X_{3j} \geq PG_{3k} \cdot \sum_{j \in J} X_{3j}, \quad \forall k \in \{2, 3\} \quad (3.32)$$

$$\sum_{j \in J} PC_{j1} \cdot X_{3j} \leq PG_{31} \cdot \sum_{j \in J} X_{3j} \quad (3.33)$$

$$\sum_{i \in I} X_{ij} \leq Disp_j, \quad \forall j \in \{2, 3\} \quad (3.34)$$

$$\sum_{i \in I} \sum_{j \in J} X_{ij} \cdot CostoC_j \leq Presu \quad (3.35)$$

$$\sum_{i \in I} X_{i1} \geq BA \quad (3.36)$$

$$\sum_{j \in J} X_{ij} \geq DEM_i, \quad \forall i \in \{1, 2\} \quad (3.37)$$

$$X_{ij} \in \mathbb{R}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (3.38)$$

La expresión (3.27) es la función objetivo del problema, que busca maximizar solo la producción de barriles de gasolina tipo euro.

De las expresiones (3.28) a (3.33) se representa el cumplimiento de las proporciones de cada gasolina con respecto a los componentes. La expresión (3.34) restringe el número de barriles de crudo tipo B y C a utilizar. La restricción (3.35) establece no sobrepasarse con el presupuesto, mientras que la restricción (3.36) obliga el cumplimiento del número de barriles del crudo A que deben comprarse. La expresión (3.37) garantiza la satisfacción de la demanda de barriles de gasolina súper y normal. Finalmente, considerando que se puede comprar una cantidad decimal de barriles, la expresión (3.38) expone la naturaleza real de las variables.

El Algoritmo 21 contiene el código del modelo en AMPL. La solución obtenida por Gurobi en AMPL logra una producción de 165,67 barriles de gasolina euro. La distribución detallada de cada crudo, necesaria para su fabricación, corresponde a 25,7143 y 35 barriles de crudo A para producir gasolina súper y normal, respectivamente; 34,2857, 35 y 82,8348 barriles de crudo B para producir gasolina súper, normal, y euro, respectivamente; y 82,8348 barriles de crudo C solo para elaborar gasolina euro.

Algoritmo 21: archivo MezclaCombustible.mod

```

1: set I := {1..3};
2: set J := {1..3};
3: set K := {1..3};
4: param PC{J,K};
5: param PG{I,K};
6: param CostoC{J};
7: param Presu;
8: param Disp{{2,3}};
9: param BA;
10: param DEM{{1,2}};
11: var X{I,J} >= 0;
12: maximize z: sum{j in J} X[3,j];
13: s.t.
14: r1{k in {1,3}}: sum{j in J} PC[j,k]*X[1,j] >= PG[1,k]*sum{j in J}
    X[1,j];
15: r2: sum{j in J} PC[j,2]*X[1,j] <= PG[1,2]*sum{j in J} X[1,j];
16: r3{k in {2,3}}: sum{j in J} PC[j,k]*X[2,j] <= PG[2,k]*sum{j in J}
    X[2,j];
17: r4: sum{j in J} PC[j,1]*X[2,j] >= PG[2,1]*sum{j in J} X[2,j];

```

```

18: r5{k in {2,3}}: sum{j in J} PC[j,k]*X[3,j] >= PG[3,k]*sum{j in J}
    X[3,j];
19: r6: sum{j in J} PC[j,1]*X[3,j] <= PG[3,1]*sum{j in J} X[3,j];
20: r7: sum{i in I, j in J} X[i,j]*CostoC[j] <= Presu;
21: r8: sum{i in I} X[i,1] >= BA;
22: r9{i in {1,2}} : sum{j in J} X[i,j] >= DEM[i];

```

3.5.2 Variación propuesta para el estudiante

En este problema se asume que al mezclar los crudos, estos no pierden su volumen. ¿Cómo debe adaptarse el modelo si se asume que al mezclar los crudos se pierde un 5 % luego de un tratamiento dentro de la producción de gasolina?

3.6 Problema de localización de planta textil (adaptado de [5])

La empresa *Los Siete Enanitos* está dedicada al sector textil en Colombia. Actualmente tiene dos plantas de fabricación: una en Pereira y otra en Ibagué, con capacidades de 900 y 1500 unidades, respectivamente. Además, tiene cuatro almacenes regionales de distribución, que sirven a los clientes de sus respectivas zonas en: Neiva, Medellín, Cali y Bogotá, con demandas de 700, 800, 500 y 400 unidades, respectivamente.

En los próximos años, la empresa espera un crecimiento de la demanda del orden del 25 %. Por ello, la dirección requiere analizar la apertura de una nueva planta. A nivel gerencial se considera que existen dos alternativas: Pasto (alternativa 1) y Villavicencio (alternativa 2). La elegida será aquella que genere los menores costos totales de logística. Estos consisten en los costos de transporte entre las plantas y los almacenes, y en los costos fijos. Si la planta se localiza en Pasto, su costo fijo de instalación será de 1000 USD. Si se localiza en Villavicencio, el costo asciende a 900 USD. La Tabla 3.6 muestra los costos de transporte unitarios entre cada origen y destino.

¿Qué planta se debe abrir para cumplir la demanda al mínimo costo?

		Destinos			
		Neiva	Medellín	Cali	Bogotá
Orígenes	Pereira	6	4	2	6
	Ibagué	2	3	7	5
	Pasto	3	2	4	8
	Villavicencio	6	3	4	2

Tabla 3.6: Costos (USD) de transporte unitarios entre origen y su destino.

3.6.1 Solución

Píldora informativa 3.7: Supuestos en las restricciones

Se debe considerar que la capacidad de la nueva planta **no necesariamente debe ser igual** a la diferencia entre la demanda total con el 25 % adicional (3000) y, la capacidad actual de las dos plantas existentes (2400). De hecho, puede ser mayor, y con ello obtener una mejor solución, usando una menor capacidad actual. En este problema se analizan ambas opciones.

Definición de conjuntos

- I : plantas $\{1, 2, 3, 4\}$. Las plantas 1 y 2 son las actuales. Por otra parte, las plantas 3 y 4 son las dos opciones de apertura, Pasto y Villavicencio, respectivamente.
- J : almacenes $\{1, 2, 3, 4\}$, correspondientes a Neiva, Medellín, Cali y Bogotá, respectivamente. Estos corresponden a los nodos de demanda.

Definición de parámetros

- CF_i : costo fijo de instalar la planta $i \in \{3, 4\}$.
- CT_{ij} : costo de transportar una unidad desde la planta $i \in I$ hasta el almacén $j \in J$.
- DEM_j : demanda del almacén $j \in J$. Este valor debe incluir el incremento del 25 %.

- CAP_i : capacidad de la planta $i \in I$.

Definición de variables

- x_{ij} : unidades a transportar desde la planta $i \in I$ hasta el almacén $j \in J$.
- $y_i : \begin{cases} 1, & \text{si se decide instalar la planta } i \in \{3, 4\} \\ 0, & \text{en otro caso} \end{cases}$

Modelo algebraico de programación lineal

Píldora informativa 3.8: Relación entre variables binarias y continuas

Cuando se establecen variables binarias que están estrechamente relacionadas con variables continuas, **se debe garantizar dicha relación mediante restricciones**. Una forma usual es usar una desigualdad del tipo menor igual. Por ejemplo, al tomar las variables de este problema, es decir, y_i y x_{ij} , de **no** instalar una planta $i \in \{3, 4\}$ (variable y_i), no puede haber transporte de unidades desde esa planta a ningún cliente j (variable x_{ij}). Pues la planta en cuestión no existiría.

A continuación, se mostrará cómo crear esta relación entre variables. Sin pérdida de generalidad, se considerará solo la instalación 3 para el siguiente ejemplo. Si la capacidad de dicha planta es A , la restricción resultante sería:

- **Caso 1:** si $y_3 = 0$ cualquier valor de x_{3j} es obligado a ser cero.

$$\sum_{j \in J} x_{3j} \leq A \cdot y_3$$

- **Caso 2:** si $y_3 = 1$ cualquier valor de x_{3j} es permitido.

$$\sum_{j \in J} x_{3j} \leq A \cdot y_3$$

Píldora informativa 3.9: La gran M (*the big M*)

Si la capacidad de un recurso no es conocida, se suele usar la denominada gran M . De forma general, la gran M es **un parámetro conocido y constante**, igual a **un valor suficientemente grande** como para que ningún otro parámetro del problema pueda superarlo.

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} CT_{ij} \cdot x_{ij} + \sum_{i \in \{3,4\}} CF_i \cdot y_i \quad (3.39)$$

Sujeto a:

$$\sum_{i \in I} x_{ij} \geq DEM_j, \quad \forall j \in J \quad (3.40)$$

$$\sum_{j \in J} x_{ij} \leq CAP_i, \quad \forall i \in \{1, 2\} \quad (3.41)$$

$$\sum_{j \in J} x_{ij} \leq CAP_i \cdot y_i, \quad \forall i \in \{3, 4\} \quad (3.42)$$

$$\sum_{i \in \{3,4\}} y_i = 1 \quad (3.43)$$

$$y_i \in \{0, 1\}, \quad \forall i \in \{3, 4\} \quad (3.44)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (3.45)$$

La expresión (3.39) es la función objetivo del problema, que busca minimizar los costos logísticos de transporte más los costos fijos por instalación.

La restricción (3.40) establece la satisfacción de la demanda de los almacenes. La expresión (3.41) indica que las unidades enviadas desde las plantas existentes (1, 2) no pueden exceder su capacidad. La restricción (3.42) indica que las unidades enviadas desde la nueva planta, ya sea la 3 o la 4, no pueden exceder su capacidad, esto siempre y cuando se decida abrirla. Si no se desea limitar la capacidad se puede reemplazar por:

$$\sum_{j \in J} x_{ij} \leq M \cdot y_i, \quad \forall i \in \{3, 4\},$$

siendo M un valor muy grande. La expresión (3.43) restringe la apertura de solo una planta. Finalmente, las restricciones (3.44) y (3.45) indican la naturaleza de las variables.

El Algoritmo 22 contiene el código del modelo algebraico en AMPL. Si se limita la capacidad de la nueva planta a 600, se obtiene un costo mínimo de 8175 USD con la apertura de la planta 4. De no limitar la capacidad se obtiene un costo mínimo de 7900 USD. También con la apertura de la planta 4, pero una capacidad de 1500. La distribución de productos para este último caso se puede apreciar en la Tabla 3.7.

	Neiva	Medellín	Cali	Bogotá
Planta 1	0	0	625	0
Planta 2	875	0	0	0
Planta 4	0	1000	0	500

Tabla 3.7: Solución problema de localización.

Algoritmo 22: archivo LocalizacionPlanta.mod

```

1: set I := {1..4};
2: set J := {1..4};
3: param CF{{3,4}};
4: param CT{I,J};
5: param DEM{J};
6: param CAP{I};
7: var x{I,J} >= 0 integer;
8: var y{{3,4}} binary;
9: minimize z: sum{i in I, j in J} x[i,j]*CT[i,j] + sum{i in
    {3,4}} y[i]*CF[i];
10: s.t.
11: rDem{j in J}: sum{i in I} x[i,j] >= DEM[j];
12: rCap{i in {1,2}}: sum{j in J} x[i,j] <= CAP[i];
13: rCap2{i in {3,4}}: sum{j in J} x[i,j] <= CAP[i]*y[i];
14: rFabri: sum{i in {3..4}} y[i] = 1;

```

3.6.2 Variación propuesta para el estudiante

Teniendo en cuenta que la capacidad de la nueva planta es desconocida, definir una nueva variable que la represente, y reestructurar el modelo para incluirla.

3.7 Problema de producción minera

La empresa minera *Los Topos* puede extraer 5 tipos de minerales en 3 minas que actualmente explota en el país. Los períodos y capacidad de explotación son cuatrimestrales: enero-abril, mayo-agosto y septiembre-diciembre. La Tabla 3.8 muestra las capacidades de explotación, en toneladas, por período de cada mina.

	Cuatrimestre I	Cuatrimestre II	Cuatrimestre III
Mina 1	600	200	800
Mina 2	900	300	500
Mina 3	400	800	300

Tabla 3.8: Capacidad de explotación en toneladas.

Según el mineral extraído, se incurre en un costo de producción por tonelada procesada, detallado en la Tabla 3.9. Si en un período se extrae un mineral en cualquiera de las minas, se incurre en un costo fijo de procesamiento. Esto es debido a que cada tipo de mineral requiere una máquina especial para su extracción, que debe ser rentada a un proveedor, como se describe en la Tabla 3.10. Una vez que se renta una máquina, esta puede ser usada en diferentes cuatrimestres.

Mineral	Mina 1	Mina 2	Mina 3
Cobre (Cu)	250	125	200
Molibdeno (Mo)	125	300	350
Oro (Au)	500	450	600
Zinc (Zn)	100	200	125
Plata (Ag)	250	400	300

Tabla 3.9: Costos en dólares por tonelada extraída.

La explotación cuatrimestral de los mineral se realizará de acuerdo con los resultados de un estudio de estimación de la demanda mínima. En la Tabla 3.11 se detallan los pronósticos para cada mineral en toneladas.

Mineral	Mina 1	Mina 2	Mina 3
Cobre (Cu)	600	700	800
Molibdeno (Mo)	200	150	300
Oro (Au)	800	1000	900
Zinc (Zn)	300	120	250
Plata (Ag)	500	400	225

Tabla 3.10: Costos fijos de explotación en dólares.

Mineral	Cuatrimestre I	Cuatrimestre II	Cuatrimestre III
Cobre (Cu)	50	75	80
Molibdeno (Mo)	25	8	17
Oro (Au)	9	10	5
Zinc (Zn)	70	85	52
Plata (Ag)	60	26	35

Tabla 3.11: Pronóstico de demanda en toneladas por cuatrimestre.

Formular un modelo de programación lineal para determinar la cantidad óptima de mineral extraído de cada mina en cada cuatrimestre, para satisfacer la demanda al mínimo costo.

3.7.1 Solución

Definición de conjuntos

- I : minerales $\{1, \dots, 5\}$.
- J : minas $\{1, 2, 3\}$.
- T : cuatrimestres $\{1, 2, 3\}$.

Definición de parámetros

- C_{ij} : costo variable de extraer el mineral $i \in I$ de la mina $j \in J$.
- CF_{ij} : costo fijo de extraer el mineral $i \in I$ de la mina $j \in J$.

- Cap_{jt} : capacidad de extracción de la mina $j \in J$ en el cuatrimestre $t \in T$.
- Dem_{it} : demanda del mineral $i \in I$ en el cuatrimestre $t \in T$.
- M : valor muy grande en comparación con los parámetros del modelo.

Definición variables de decisión

- X_{ijt} : toneladas del mineral $i \in I$ a extraer en la mina $j \in J$ en el cuatrimestre $t \in T$.

Píldora informativa 3.10: Variables con diferentes subíndices

Cuando existen variables continuas y binarias estrechamente relacionadas, **la definición de sus subíndices es fundamental para representar el problema deseado**. En este caso la variable X_{ijt} tiene tres subíndices, porque se debe determinar el tipo de mineral, de qué mina procede y en qué cuatrimestre se efectuó la extracción. Sin embargo, la variable binaria asociada al uso, o no, de una mina para extraer cierto mineral, **no depende del cuatrimestre cuando se efectúa la extracción**. De hecho, una vez usada una mina para extraer un mineral, se puede volver a usar en otro cuatrimestre, porque aún se tiene la máquina rentada.

- $Y_{ij} : \begin{cases} 1, & \text{si el mineral } i \in I \text{ se extrae en la mina } j \in J \\ 0, & \text{en otro caso} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} \sum_{t \in T} C_{ij} \cdot X_{ijt} + \sum_{i \in I} \sum_{j \in J} CF_{ij} \cdot Y_{ij} \quad (3.46)$$

Sujeto a:

$$\sum_{i \in I} X_{ijt} \leq Cap_{jt}, \quad \forall j \in J, \forall t \in T \quad (3.47)$$

$$\sum_{j \in J} X_{ijt} \geq Dem_{it}, \quad \forall i \in I, \forall t \in T \quad (3.48)$$

$$\sum_{t \in T} X_{ijt} \leq M \cdot Y_{ij}, \quad \forall i \in I, \forall j \in J \quad (3.49)$$

$$X_{ijt} \in \mathbb{R}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J, \forall t \in T \quad (3.50)$$

$$Y_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J \quad (3.51)$$

La expresión (3.46) es la función objetivo del problema, que busca minimizar los costos variables por tonelada extraída (dependientes de la cantidad) y los costos fijos por extracción (no dependientes de la cantidad).

La restricción (3.47) limita la cantidad de toneladas de mineral a extraer en cada mina por cada cuatrimestre. La restricción (3.48) garantiza el cumplimiento de la demanda. La restricción (3.49) relaciona las variables de extracción (en cantidad) con las de uso de la mina (binaria). Finalmente, las restricciones (3.50) y (3.51) indican la naturaleza de las variables.

El Algoritmo 23 contiene el código del modelo algebraico en AMPL. El modelo matemático resuelto con Gurobi alcanzó un costo mínimo de 96 325 USD. Los resultados de las toneladas de mineral a extraer en cada cuatrimestre de las minas 1 y 2 se ilustran en la Tabla 3.12. La solución establece que no es conveniente la extracción de minerales en la mina 3.

Algoritmo 23: archivo ProduccionMinera.mod

```

1: set I:={1..5};
2: set J:={1..3};
3: set T:={1..3};
4: param C{I,J};
5: param CF{I,J};
6: param Cap{J,T};
7: param Dem{I,T};
8: param M := 10000000;
9: var X{I,J,T} >= 0;
10: var Y{I,J} binary;
11: minimize z: sum{i in I, t in T, j in J} X[i,j,t]*C[i,j] + sum{i
    in I, j in J} Y[i,j]*CF[i,j];
12: s.t.
13: r1{t in T, j in J}: sum{i in I} X[i,j,t] <= Cap[j,t];
14: r2{t in T, i in I}: sum{j in J} X[i,j,t] >= Dem[i,t];
15: r3{i in I, j in J}: sum{t in T} X[i,j,t] <= M*Y[i,j];

```

Extracción mina 1 [ton/cuatrimestre]			
Mineral	Cuatrimestre I	Cuatrimestre II	Cuatrimestre III
Cobre (Cu)	0	0	0
Molibdeno (Mo)	25	8	17
Oro (Au)	0	0	0
Zinc (Zn)	70	85	52
Plata (Ag)	60	26	35

Extracción mina 2 [ton/cuatrimestre]			
Mineral	Cuatrimestre I	Cuatrimestre II	Cuatrimestre III
Cobre (Cu)	50	75	80
Molibdeno (Mo)	0	0	0
Oro (Au)	9	10	5
Zinc (Zn)	0	0	0
Plata (Ag)	0	0	0

Tabla 3.12: Plan de extracción al mínimo costo.

3.8 Problema del escape de prisión (*shortest path*)

Suponga que usted es un prisionero en la famosa cárcel de Alcatraz. Luego de 10 años de planear su escape, ha logrado encontrar una ventana de tiempo donde los guardias están dormidos y/o distraídos. Para escapar, desde su celda usted debe atravesar el gigantesco patio antes de alcanzar la salida. Con sus habilidades de topografía, usted ha dibujado un plano con los posibles caminos y las puertas que existen. Además, ha estimado el tiempo para ir de puerta en puerta, así como el tiempo requerido para abrir, a la fuerza, dichas puertas (cada una cuenta con sistemas diferentes de seguridad). Como es de esperar, usted debe encontrar el camino para atravesar el patio desde su celda hasta la salida en el menor tiempo posible.

La Figura 3.4 muestra el esquema del patio, donde los nodos representan las puertas. El nodo 0 es su celda y el nodo 7 es la salida. En la Tabla 3.13 se muestran los minutos que tardará para viajar desde una puerta a otra (los valores de X corresponden a conexiones imposibles). Finalmente, el tiempo (en minutos) para sabotear las puerta es: 0, 6, 8, 8, 2, 7, 6 y 3 para las puertas 0, 1, 2, 3, 4, 5, 6 y 7, respectivamente. El tiempo de la puerta 0 es 0 segundos, puesto que usted ya la ha saboteado antes de emprender su huida.

Formular un modelo de programación lineal que represente este problema y permita obtener la ruta más rápida.

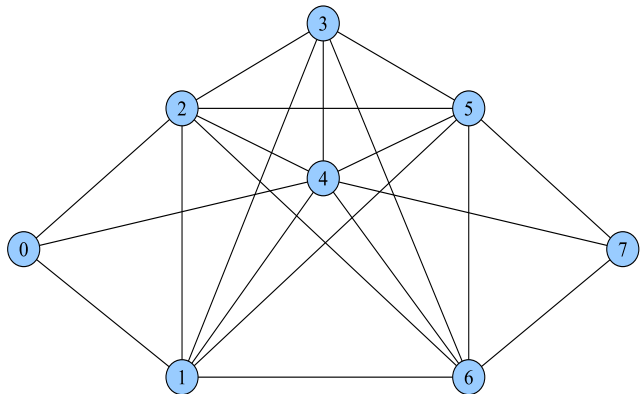


Figura 3.4: Esquema de patio de Alcatraz.

3.8.1 Solución

Definición de conjuntos

- I : puertas a atravesar $\{0, \dots, 7\}$.
- A : arcos posibles del grafo $\{(i \in I, j \in I), i \neq j\}$.

	0	1	2	3	4	5	6	7
0	.	3	6	X	10	X	X	X
1	8	.	10	2	1	9	3	X
2	7	6	.	1	5	3	5	X
3	X	8	8	.	3	9	10	X
4	4	7	7	4	.	2	5	10
5	X	4	2	9	1	.	2	8
6	X	6	2	7	10	2	.	3
7	X	X	X	X	1	6	6	.

Tabla 3.13: Tiempos en minutos para viajar de una puerta a otra.

Definición de parámetros

- c_{ij} : minutos requeridos para desplazarse desde la puerta $i \in I$ a la puerta $j \in I$.
- puerta_i : minutos requeridos para sabotear puerta $i \in I$.

Definición de las variables de decisión

- $x_{ij} : \begin{cases} 1, & \text{usar el arco } (i, j) \in A \text{ en la ruta de escape} \\ 0, & \text{en otro caso} \end{cases}$
- $y_i : \begin{cases} 1, & \text{si se ha visitado la puerta } i \in I \\ 0, & \text{en otro caso} \end{cases}$

*Modelo algebraico de programación lineal***Píldora informativa 3.11: Símbolo de diferencia de conjuntos ($\setminus$)**

En muchas ocasiones se necesita eliminar elementos de un conjunto, para esto se usa el símbolo $\setminus$. Formalmente, sean dos conjuntos A y B , $A \setminus B$ representa un conjunto formado por los elementos que quedan en A después de **sustraer** los elementos de B , es decir, elementos $i \in A$ e $i \notin B$:

$$\{1, 2, 3\} \setminus \{2, 3\} = \{1\}, \{1, 2, 3\} \setminus \{2\} = \{1, 3\}$$

Píldora informativa 3.12: Balance en grafos (o redes)

En optimización muchos problemas pueden ser representados por un grafo o red (en inglés *graph*). Formalmente, un grafo está definido por dos elementos: nodos V (*vertex*) y arcos E (*edges*). Un arco es el camino que conecta a dos nodos. La notación de un grafo G generalmente es $G = (V, E)$.

El balance en redes consiste en garantizar que la entrada y salida de arcos a un nodo sea igual. Para esto es usual hacer un intercambio entre subíndices. Por ejemplo, al tomar la variable de decisión de este problema x_{ij} y hacer el supuesto de necesitar balance en el nodo 5, se expresaría como:

$$\sum_{i=0}^7 x_{i5} = \sum_{i=0}^7 x_{5i}$$

El lado izquierdo de la restricción considera los posibles arcos que llegarán a 5 (desde cualquier i), mientras que el lado derecho considera todos los arcos que empiecen en 5 (hacia cualquier i).

Este intercambio se puede hacer siempre y cuando los conjuntos que recorren tanto el primer como el segundo subíndice sean los mismos. En este caso i y j están definidos en el conjunto de las puertas I .

$$\text{Minimizar } z = \sum_{(i,j) \in A} c_{ij} \cdot x_{ij} + \sum_{i \in I} y_i \cdot \text{puerta}_i \quad (3.52)$$

Sujeto a:

$$\sum_{j \in I \setminus \{0\}} x_{0j} = 1 \quad (3.53)$$

$$\sum_{i \in I \setminus \{7\}} x_{i7} = 1 \quad (3.54)$$

$$\sum_{i \in I, i \neq j} x_{ij} = \sum_{i \in I, i \neq j} x_{ji}, \quad \forall j \in I \setminus \{0, 7\} \quad (3.55)$$

$$y_i \geq \sum_{j \in I, j \neq i} x_{ji}, \quad \forall i \in I \quad (3.56)$$

$$x_{ij} \in \{0, 1\}, \quad \forall (i, j) \in A \quad (3.57)$$

$$y_i \in \{0, 1\}, \quad \forall i \in I \quad (3.58)$$

La expresión (3.52) representa la función objetivo, que busca minimizar los minutos usados en la ruta de escape (desplazándose), sumando los minutos que se requieren para sabotear las puertas encontradas en dicha ruta.

La restricción (3.53) establece el punto de salida inicial (la celda, nodo 0), garantizando que se saldrá desde dicho nodo. De forma similar, la restricción (3.54) establece que el prisionero debe llegar a la salida (nodo 7). La restricción (3.55) obliga el balance en el flujo de los nodos, es decir, el flujo de entrada a cada nodo debe ser igual al flujo de salida. La restricción (3.56) establece la relación entre las variables x_{ij} y y_i . Finalmente, las restricciones (3.57) y (3.58) exponen la naturaleza de las variables.

El Algoritmo 24 contiene el código del modelo en AMPL. La solución obtenida por Gurobi logra alcanzar la salida en 24 minutos. La ruta óptima es: 0 – 1 – 6 – 7 .



Es importante recordar que pueden haber múltiples óptimos en un problema. Cuando se resuelve, lo usual es encontrar uno de ellos.

Algoritmo 24: archivo Escape.mod

```

1: set I := {0..7};
2: set A := {i in I, j in I: i<>j};
3: param c{(i,j) in A};
4: param puerta{I};
5: var x{I,I} binary;
6: var y{I} binary;
7: minimize z: sum{i in I, j in I: i<>j} c[i,j]*x[i,j] + sum{i in I}
   y[i]* puerta[i];
8: s.t.
9: r1: sum{i in I diff {0}} x[0,i] = 1;
10: r2: sum{i in I diff {7}} x[i,7] = 1;
11: r3{j in I diff {0,7}}: sum{i in I: i<>j} x[i,j] = sum{i in I:
   i <>j} x[j,i];
12: r4{i in I}: y[i] >= sum{j in I: j<>i} x[j,i];

```

3.9 Problema de planeación de producción de puertas (adaptado de [3])

La empresa *Madera Verde* necesita cumplir un contrato de suministro de 1200 puertas dentro de un plazo de 5 semanas. Específicamente, este programa de entregas consiste de 100, 200, 300, 400 y 200 puertas para las semanas 1, 2, 3, 4 y 5, respectivamente. Sin embargo, este plan

puede no cumplirse rigurosamente, es decir, se permite producir más artículos para un uso futuro (inventario), o bien quedar con deudas de artículos pendientes para entregarlas luego (faltantes). Lo importante es que al finalizar las 5 semanas se debe haber entregado el total de las 1 200 puertas.

Madera Verde cuenta con unas políticas dentro de su fábrica, que deben ser cumplidas. Entre las que destacan:

- Se permiten tener pedidos atrasados en cualquier periodo, con excepción del último. El costo por semana de retraso y por unidad no entregada es de 30 USD.
- Se permite el almacenamiento de producción. El costo por almacenar una unidad durante una semana alcanza los 10 USD.
- Actualmente la empresa cuenta con 20 operarios expertos y 10 puertas como inventario inicial. Cada operario fabrica 8 puertas por semana.
- Se puede contratar nuevo personal. Sin embargo, las personas contratadas se considerarán novatas durante una semana, luego de la cual ya serán considerados operarios expertos. Para esto, los novatos deberán ser capacitados por un operario experto (llamado instructor o profesor).
- Cada operario experto solo puede capacitar a 5 novatos en una semana, y mientras dure esta capacitación no puede fabricar puertas.
- El salario de un operario, fabrique o no fabrique puertas, novato o experto, es de 100 USD por semana.
- Es permitido despedir operarios, pero esto se hace efectivo una semana después de la notificación. El costo de despedir a un operario es de 100 USD.

Formule un modelo de programación lineal que permita planear la producción al mínimo costo considerando todas las condiciones del problema.

3.9.1 Solución

Definición de conjuntos

- T : horizonte de planeación en semanas $\{1, 2, 3, 4, 5\}$.

Definición de parámetros

- D_t : cantidad de puertas a entregar en la semana $t \in T$ [puerta].
- Cf : costo por unidad no entregada durante una semana $\left[\frac{USD}{puerta \cdot sem} \right]$.
- Ch : costo por almacenar una puerta en inventario durante una semana $\left[\frac{USD}{puerta \cdot sem} \right]$.
- Num : cantidad de puertas que puede fabricar un operario en una semana.
- $C1$: salario de un operario por semana $\left[\frac{USD}{operario} \right]$.
- $C2$: costos de despedir a un operario $\left[\frac{USD}{operario} \right]$.

En este caso $C1$ y $C2$ son iguales, pero en la formulación se usarán parámetros diferentes para mostrar el caso más general.

Definición de variables de decisión

- x_t : cantidad de puertas no entregadas en la semana $t \in T \cup \{0\}$, pero que se entregarán en un futuro (faltantes o pedidos atrasados).
- I_t : cantidad de puertas almacenadas en la semana (inventario) $t \in T \cup \{0\}$.
- y_t : número de operarios expertos que producen puertas en la semana $t \in T$. Nótese que el número de puertas a producir depende directamente de y_t .

- z_t : número de operarios novatos en capacitación en la semana $t \in T \cup \{0\}$. El número de operarios que capacitan (instructores) se puede calcular como $z_t/5$, al asumir que siempre cada instructor capacita a 5 novatos.



De no considerarse que cada instructor siempre capacitan a 5 novatos, el modelo cambiaría significativamente, pues el número de personas a capacitar sería una variable nueva. Además, dicho número también estaría asignado a un instructor específico, obligando al modelo a considerar otra variable binaria de asignación, asociada a los instructores.

- m_t : número de operarios despedidos en la semana $t \in T \cup \{0\}$.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{t \in T} C1 \cdot \left(y_t + z_t + \frac{z_t}{5} \right) + C2 \cdot m_t + Ch \cdot I_t + Cf \cdot x_t \quad (3.59)$$

Sujeto a:

$$I_{t-1} + y_t \cdot Num = D_t + x_{t-1} + I_t - x_t, \quad \forall t \in T \quad (3.60)$$

$$y_t = y_{t-1} - m_{t-1} + z_{t-1} - \frac{z_t}{5} + \frac{z_{t-1}}{5}, \quad \forall t \in T \quad (3.61)$$

$$x_0 = 0 \quad (3.62)$$

$$I_0 = 10 \quad (3.63)$$

$$y_0 = 20 \quad (3.64)$$

$$z_0 = 0 \quad (3.65)$$

$$m_0 = 0 \quad (3.66)$$

$$x_t, I_t, y_t, z_t, m_t \in \mathbb{Z}^+ \cup \{0\}, \quad \forall t \in T \quad (3.67)$$

La expresión (3.59) es la función objetivo del problema, que busca minimizar los costos totales de la planeación de producción.

La expresión (3.60) corresponde al balance de puertas, similar a las ecuaciones planteadas para el problema 3.3.

Píldora informativa 3.13: Balance de inventario 2

En este caso, el balance de inventario se modelará en su forma más general. Se establece que las siguientes dos cantidades deben ser iguales:

- **Al inicio de t se cuenta con:** el inventario pasado (I_{t-1}), más la producción en t ($y_t \cdot Num$).
- **Al final de t se cuenta con:** la demanda, más la deuda pasada en t ($D_t + x_{t-1}$), más el inventario en t (en caso de sobrar productos) (I_t), menos la deuda en t (en caso de generar) (x_t).

Para comprender mejor el funcionamiento de este tipo de restricciones, se analizarán los dos escenarios posibles. Para esto, sin pérdida de generalidad, se supondrá un $t = 8$, sin inventario en $t = 7$, una demanda de 80 en $t = 8$ con una deuda de $x_7 = 20$, y la producción de puertas por operario experto se mantendrá en $Num = 8$.

1. Si la producción es superior a la demanda ($y_8 = 15$):

$$\underbrace{I_7}_{0} + \underbrace{y_8 \cdot Num}_{120} = \underbrace{D_8 + x_7}_{100} + \underbrace{I_8}_{?} - \underbrace{x_8}_{?}$$

Para cumplir la igualdad, la única variable que puede tomar un valor positivo es I_8 con 20, representando el inventario de este mes.

2. Si la producción es inferior a la demanda ($y_8 = 10$):

$$\underbrace{I_7}_{0} + \underbrace{y_8 \cdot Num}_{80} = \underbrace{D_8 + x_7}_{100} + \underbrace{I_8}_{?} - \underbrace{x_8}_{?}$$

En este caso, para cumplir la igualdad la única variable que puede tomar un valor positivo es x_8 con 20, representando la deuda generada este mes.

La expresión (3.61) hace referencia al balance de operarios de cada semana t , explicada en la Figura 3.5.

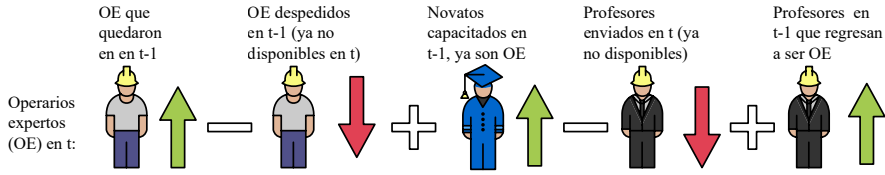


Figura 3.5: Balance de operarios en una semana t .

Las expresiones (3.62) a (3.66) indican los estados iniciales, entendiéndose la semana cero como el estado inicial. Finalmente, la expresión (3.67) establece la naturaleza de las variables.

El Algoritmo 25 contiene el código del modelo en AMPL. En la Tabla 3.14 se presenta la solución obtenida por Gurobi, que alcanza un costo mínimo de 20700 USD.

Algoritmo 25: archivo Puertas.mod

```

1: set T := {1..5};
2: param D{T};
3: param Ch;
4: param Cf;
5: param C1;
6: param C2;
7: param Num;
8: var x{T union {0}} >=0 integer;
9: var I{T union {0}} >=0 integer;
10: var y{T union {0}} >=0 integer;
11: var z{T union {0}} >=0 integer;
12: var m{T union {0}} >=0 integer;
13: minimize costo: sum{t in T} (C1*(y[t]+z[t]+(z[t]/5)) + C2*m[t]+
    Ch*I[t] + Cf*x[t]);
14: s.t.
15: r1{t in T}: I[t-1] + y[t]*Num = D[t] + x[t-1] + I[t] - x[t];
16: r2{t in T}: y[t] = y[t-1] - m[t-1] + z[t-1] - z[t]/5 + z[t-1]/5;
17: r3: x[0] = 0;
18: r4: I[0] = 10;
19: r5: y[0] = 20;
20: r6: z[0] = 0;
21: r7: m[0] = 0;

```

	$t = 0$	$t = 1$	$t = 2$	$t = 3$	$t = 4$	$t = 5$
Puertas no entregadas (x_t)	0	0	0	0	54	0
Puertas en inventario (I_t)	10	54	86	66	0	2
Expertos en producción (y_t)	20	18	29	35	35	32
Novatos en capacitación (z_t)	0	10	5	0	0	0
Operarios despedidos (m_t)	0	0	0	0	3	0

Tabla 3.14: Planeación óptima de producción de puertas.

3.9.2 Variación propuesta para el estudiante

¿Cómo se afecta el modelo si se considera que la capacitación tarda 2 periodos?

3.10 Problema de mezcla de aleaciones

La empresa *Los Mineros de San Juan* produce 2 aleaciones: acero y cobre. Las aleaciones están compuestas por 4 metales diferentes: M1, M2, M3 y M4, cuyas composiciones deben obedecer a las siguientes especificaciones:

1. Para el **Acero**: se requiere menos del 40 % de M1, como máximo el 75 % de M2 y al menos el 30 % de M4.
2. Para el **Cobre**: se requiere entre el 25 % y el 80 % de M2, como mínimo el 50 % de M3 y máximo el 66 % de M4.

Para producir las aleaciones la empresa debe extraer los metales necesarios de tres minas cercanas: Río Azul, Macadamia y Cerro Rojo. En la [Tabla 3.15](#) se muestra la capacidad máxima de extracción, el porcentaje de metal que puede ser extraído por tonelada y el costo de extracción de cada mina.

Mina	Cap. máx. (ton)	M1	M2	M3	M4	Otros	Costo (\$/ton)
Río Azul	5000	15 %	20 %	40 %	25 %	0 %	70
Macadamia	3000	10 %	30 %	10 %	40 %	10 %	50
Cerro Rojo	4000	5 %	10 %	70 %	10 %	5 %	65

Tabla 3.15: Características de las minas, metales y costos.

Debido a la lejanía en la ubicación de las minas Río Azul y Macadamia, y también al horario laboral de los trabajadores, no es posible extraer en ambas minas. Además, a causa de los contratos con el administrador de la mina Cerro Rojo, solo se podrán extraer en dicha mina metales para la elaboración de 1 de los 2 tipos de aleación. Los precios de venta para las aleaciones son \$250/ton y \$170/ton para el acero y el cobre, respectivamente.

Plantear un modelo de programación lineal para maximizar las ganancias de la empresa *Los Mineros de San Juan*.

3.10.1 Solución

Definición de conjuntos

- I : aleaciones $\{1, 2\}$ para acero y cobre, respectivamente.
- J : metales $\{1, 2, 3, 4\}$, que corresponden a M1, M2, M3 y M4, respectivamente.
- K : minas $\{1, 2, 3\}$ para Río Azul, Macadamia y Cerro Rojo.

Definición de parámetros

- $Cmax_k$: capacidad máxima de mineral que se puede extraer en la mina $k \in K$ [ton].
- Por_{jk} : porcentaje de metal $j \in J$ obtenido por tonelada extraída en la mina $k \in K$.
- CT_k : costo por tonelada extraída en la mina $k \in K$.
- PV_i : precio de venta de la aleación $i \in I$.

Definición de variables de decisión

Píldora informativa 3.14: Alcance de una variable de decisión

Al realizar la definición de las variables de decisión se debe tener en cuenta el alcance de dichas variables. Este alcance hace referencia a “**cuánto puede controlar**” la variable. Para ejemplificar esto, se analizará el porqué la definición de la siguiente variable **no** es adecuada para este problema:

X_{jki} : cantidad de toneladas de metal $j \in J$ a extraer en mina $k \in K$, para producir aleación $i \in I$.

En este caso **la extracción de una tonelada de mineral en cualquier mina implica la obtención de todos los metales M1, M2, M3 y M4** (en diferentes porcentajes). Esto quiere decir que **no es posible elegir qué metal extraer**. Por esta razón, la variable X_{jki} no es adecuada, porque involucra que pueden existir valores como: $X_{(M1),(1),(2)} = 10$ y $X_{(M2),(1),(2)} = 0$, que implican la extracción en la mina 1 para la aleación de 2 con cantidades independientes de los metales M1 y M2; valores incorrectos, porque no se puede extraer solo un metal, ya que todos vienen combinados con porcentajes definidos (Tabla 3.15).

- X_{ki} : cantidad de toneladas de mineral a extraer en la mina $k \in K$, para producir aleación $i \in I$. El mineral contiene los metales según las cantidades acordes a Tabla 3.15.
- $Y_k : \begin{cases} 1, & \text{si se extrae en la mina } k \in K \setminus \{3\} \\ 0, & \text{en otro caso} \end{cases}$

Píldora informativa 3.15: Variables binarias con más información

La definición de variables binarias se realizó en la Píldora informativa 2.10, pero en este caso se mostrará cómo el estado **false** no necesariamente implica la no realización de una decisión. En este problema en particular se pueden definir dos variables binarias: una para la elaboración (*true*) o no elaboración (*false*) de acero en Cerro Rojo, y la otra para la elaboración (*true*) o no elaboración (*false*) de cobre, también en Cerro Rojo. Pero cuando solo **se tienen dos alternativas excluyentes** (se hace acero o se hace cobre), se puede usar el estado **false** para una definición con más información. En este ejemplo, *true* para fabricar acero y *false* para fabricar cobre. Sin embargo, en las restricciones se suele requerir el valor de 1 para efectuar las multiplicaciones (el estado *false* es un 0). En este caso, si la variable binaria fuese Z , la multiplicación se puede hacer como $(1 - Z)$.

$$\blacksquare \quad Z : \begin{cases} 1, & \text{si se extrae en la mina 3 (Cerro Rojo) para la elaboración} \\ & \text{de acero} \\ 0, & \text{si se extrae en la mina 3 para la elaboración de cobre} \end{cases}$$

No confundir esta variable con la función objetivo, pues esta se suele representar con una zeta minúscula (z).

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in I} \sum_{k \in K} PV_i \cdot X_{ki} - \sum_{i \in I} \sum_{k \in K} CT_k \cdot X_{ki} \quad (3.68)$$

Sujeto a:

$$\sum_{i \in I} X_{ki} \leq Cmax_k \cdot Y_k, \quad \forall k \in \{1, 2\} \quad (3.69)$$

$$\sum_{i \in I} X_{ki} \leq Cmax_k, \quad \forall k \in \{3\} \quad (3.70)$$

$$\sum_{k \in K} X_{k1} \cdot Por_{1k} \leq \sum_{k \in K} X_{k1} \cdot 0,4 \quad (3.71)$$

$$\sum_{k \in K} X_{k1} \cdot Por_{2k} \leq \sum_{k \in K} X_{k1} \cdot 0,75 \quad (3.72)$$

$$\sum_{k \in K} X_{k1} \cdot Por_{4k} \geq \sum_{k \in K} X_{k1} \cdot 0,3 \quad (3.73)$$

$$\sum_{k \in K} X_{k2} \cdot Por_{2k} \geq \sum_{k \in K} X_{k2} \cdot 0,25 \quad (3.74)$$

$$\sum_{k \in K} X_{k2} \cdot Por_{2k} \leq \sum_{k \in K} X_{k2} \cdot 0,8 \quad (3.75)$$

$$\sum_{k \in K} X_{k2} \cdot Por_{3k} \geq \sum_{k \in K} X_{k2} \cdot 0,5 \quad (3.76)$$

$$\sum_{k \in K} X_{k2} \cdot Por_{4k} \leq \sum_{k \in K} X_{k2} \cdot 0,66 \quad (3.77)$$

$$Y_1 = (1 - Y_2) \quad (3.78)$$

$$X_{31} \leq Z \cdot M \quad (3.79)$$

$$X_{32} \leq (1 - Z) \cdot M \quad (3.80)$$

$$X_{ki} \in \mathbb{R}^+ \cup \{0\}, \quad \forall i \in I, \forall k \in K \quad (3.81)$$

$$Y_i \in \{0, 1\}, \quad \forall i \in \{1, 2\} \quad (3.82)$$

$$Z \in \{0, 1\} \quad (3.83)$$

La expresión (3.68) representa la función objetivo, que indica la utilidad generada por el total de toneladas de aleación vendidas.

Las restricciones (3.69) y (3.70) determinan que la cantidad de minerales a extraer no puede exceder la capacidad de extracción de cada mina. En el caso de la restricción (3.69), solo se dispondrá de las capacidades de las minas, en caso de elegir extraer en la 1 o 2. Las restricciones (3.71) a (3.77) garantizan la composición de las aleaciones, considerando que en cada mina existen determinados porcentajes de cada metal.

La restricción (3.78) expresa la incompatibilidad de extracción entre las minas 1 y 2, mientras que las restricciones (3.79) y (3.80) representan la incompatibilidad de elaboración de acero y cobre en la mina 3. Finalmente, las restricciones (3.81) a (3.83) exponen la naturaleza real y binaria de las variables.

El Algoritmo 26 contiene el código del modelo algebraico en AMPL. Según la solución obtenida por CPLEX, las ganancias máximas son de \$877 500, extrayendo 3000 toneladas de mineral en la mina Macadamia para la elaboración de acero y 1500 toneladas en la mina Cerro Rojo, también para la elaboración de acero.

Algoritmo 26: archivo MezclaAleaciones.mod

```

1: set I := {1,2};
2: set J := {1..4};
3: set K := {1..3};
4: param M;
5: param Cmax{K};
6: param Por{J,K};
7: param CT{K};
8: param PV{I};
9: var X{K,I} >=0;
10: var Y{K diff {3}} binary;
11: var Z binary;
12: maximize z: sum{i in I, k in K} PV[i]*X[k,i] - sum{i in I, k in K}
    CT[k]*X[k,i];
13: s.t.
14: r1{k in {1,2}}: sum{i in I} X[k,i] <= Cmax[k]*Y[k];
15: r2{k in {3}}: sum{i in I} X[k,i] <= Cmax[k];
16: r3:sum{k in K} X[k,1]*Por[1,k] <= sum{k in K} X[k,1]*0.4;
17: r4:sum{k in K} X[k,1]*Por[2,k] <= sum{k in K} X[k,1]*0.75;
18: r5:sum{k in K} X[k,1]*Por[4,k] >= sum{k in K} X[k,1]*0.3;
19: r6:sum{k in K} X[k,2]*Por[2,k] >= sum{k in K} X[k,2]*0.25;
20: r7:sum{k in K} X[k,2]*Por[2,k] <= sum{k in K} X[k,2]*0.8;
21: r8:sum{k in K} X[k,2]*Por[3,k] >= sum{k in K} X[k,2]*0.5;
22: r9:sum{k in K} X[k,2]*Por[4,k] <= sum{k in K} X[k,2]*0.66;
23: r10:Y[1] = 1-Y[2];
24: r11:X[3,1] <= Z*M;
25: r12:X[3,2] <= (1-Z)*M;

```

3.11 Problema de la repostería

La empresa *Bananitos Company* es conocida por su gran desempeño en el sector de la repostería. Sus dos productos estrella son los pasteles de banano-uva (BV) y los pasteles de banano-chocolate (BC). Cada pastel está compuesto por tres elementos: una base de pastel (que es igual para

ambos), una crema y el ingrediente especial. Estos últimos dos elementos son diferentes, dependiendo del tipo de pastel (hay una crema para BV y otra para BC, igualmente con el ingrediente especial).

Todos los elementos pueden ser elaborados por la empresa o externalizados. En la Tabla 3.16 se detallan los costos de producción y compra. La empresa ha pronosticado una demanda de 3000 pasteles BV y 2000 pasteles BC. La empresa quiere cumplir con la demanda para aumentar sus ingresos, pero tiene una limitada capacidad de producción, porque solo cuenta con 200 horas de trabajo (hora regular) y puede contratar hasta 50 horas adicionales (hora extra), estas últimas a un costo de \$9 por hora extra. La Tabla 3.17 muestra los tiempos de manufactura de cada elemento.

¿Cuántos pasteles BV y BC (elementos de cada uno) debe producir y/o comprar la empresa para cumplir con la demanda al mínimo costo?

Componente	Costos de manufactura [\$/ <i>und</i>]	Costo de compra [\$/ <i>und</i>]
Base de pastel	0,50	0,60
Crema BV	3,75	4,00
Crema BC	3,30	3,90
Ingrediente especial BV	0,60	0,65
Ingrediente especial BC	0,75	0,78

Tabla 3.16: Costos de los elementos manufacturados o comprados.

Componente	Tiempo de manufactura [<i>min</i>]
Base de pastel	1,0
Crema BV	3,0
Crema BC	2,5
Ingrediente especial BV	1,0
Ingrediente especial BC	1,5

Tabla 3.17: Tiempo de manufactura de los elementos.

3.11.1 Solución

Definición de conjuntos

- I : tipo de pasteles $\{1, 2\}$, siendo 1 banano-uva y 2 banano-chocolate.
- J : componentes $\{1, 2, 3\}$, para base de pastel, crema e ingrediente especial, respectivamente. Es importante notar que no hace falta diferenciar para qué tipo de pastel es el componente, pues esto lo hará el subíndice $i \in I$ de la variable.
- K : modo de adquisición $\{1, 2\}$, donde 1 es manufactura y 2 es externalizado.

Definición de parámetros

- Dem_i : demanda de los pasteles de sabor (tipo) $i \in I$.
- C_{ijk} : costo de adquirir el componente $j \in J$ para pastel de sabor (tipo) $i \in I$ en el modo $k \in K$.
- $CapH$: capacidad de producción en horas de trabajo regular.
- $CapHE$: capacidad de contratación de horas extras.
- CHE : costo de contratar horas extras.
- T_{ij} : tiempo en minutos para la producción del componente $j \in J$ para un pastel de sabor (o tipo) $i \in I$.

Definición de variables de decisión

- X_{ijk} : cantidad de componentes $j \in J$ para pasteles $i \in I$, a adquirir en el modo $k \in K$.
- H : cantidad de horas extra a contratar.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} \sum_{k \in K} C_{ijk} \cdot X_{ijk} + CHE \cdot H \quad (3.84)$$

Sujeto a:

$$\sum_{k \in K} X_{ijk} \geq Dem_i, \quad \forall i \in I, \forall j \in J \quad (3.85)$$

$$H \leq CapHE \quad (3.86)$$

$$\sum_{i \in I} \sum_{j \in J} \frac{T_{ij}}{60} \cdot X_{ij1} \leq CapH + H \quad (3.87)$$

$$X_{ijk} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J, \forall k \in K \quad (3.88)$$

$$H \in \mathbb{R}^+ \cup \{0\} \quad (3.89)$$

La expresión (3.84) es la función objetivo del problema, que busca minimizar el costo asociado a la adquisición de los componentes para elaborar pasteles, más los costos generados por contratar horas extras que permitan satisfacer la demanda.

La restricción (3.85) garantiza cubrir la totalidad de la demanda; esta considera cada componente, así se garantiza disponer de todos los componentes para la elaboración de pasteles. La restricción (3.86) establece la cantidad máxima de horas extras factibles de contratar. Mientras que la inecuación (3.87) indica que el tiempo, en horas, empleado en la manufactura de los componentes para pastel no puede superar el tiempo total disponible (horas regulares más las horas extras contratadas). No incluye la variable X_{ij2} , pues esta hace referencia a la compra externa y no hay límite. Finalmente, la expresiones (3.88) y (3.89) exponen la naturaleza de las variables.

El Algoritmo 27 contiene el código del modelo en AMPL. La Tabla 3.18 muestra el plan de producción y compras que debe seguirse para obtener un costo mínimo de \$24443,3. En este plan de producción se toma la decisión de no contratar horas extras.

Pastel	Componente	Producción	Compra	Total
BV	Base	3000	0	3000
	Crema	667	2333	3000
	Especial	0	3000	3000
BC	Base	1999	1	2000
	Crema	2000	0	2000
	Especial	0	2000	2000

Tabla 3.18: Plan de producción y compras para los componentes de los pasteles.

Algoritmo 27: archivo Reposteria.mod

```

1: set I:={1,2};
2: set J:={1..3};
3: set K:={1,2};
4: param Dem{I};
5: param C{I,J,K};
6: param CapH;
7: param CapHE;
8: param CHE;
9: param T{I,J};
10: var X{I,J,K} >=0 integer;
11: var H >=0;
12: minimize z: sum{i in I, j in J, k in K} C[i,j,k]*X[i,j,k] + CHE*H;
13: s.t.
14: Demanda{i in I, j in J}: sum{k in K} X[i,j,k] >= Dem[i];
15: Extra: H <= CapHE;
16: Horas: sum{i in I, j in J} (T[i,j]/60)*X[i,j,1] <= H + CapH;

```

3.12 Problema de asignación de enfermeros 1

El hospital *Ve Hacia la Luz* cuenta con un servicio de urgencias muy concurrido en la ciudad de Cali, debido a la alta accidentabilidad por parte de las motocicletas. Por esta razón, la gerente, la Dra. Cecilia Grierson, ha realizado un estudio para determinar la asignación de los enfermeros a los turnos de trabajo, para cumplir con la demanda estimada de pacientes. Sin embargo, la asignación es más complicada los fines de semana (sábados y domingos), ya que cada enfermero es libre de determinar su precio de contratación por turno y, además, existen normativas específicas para dichos días.

La Dra. Grierson está pensando en contratar a 5 nuevos enfermeros para cubrir los turnos del fin de semana. Sus nombres y costos de contratación por turno de 4 horas se detallan a continuación: María (\$50/turno), Pedro (\$30/turno), Raúl (\$35/turno), Valentina (\$60/turno) y José (\$40/turno).

La demanda de enfermeros es diferente los días sábados y domingos, ya que esta se ha estimado en función de los 4 turnos disponibles, detallados en la Tabla 3.19.

Turnos de 4 horas	Demanda los sábados (enfermeros por turno)	Demanda los domingos (enfermeros por turno)
Turno 1 → 4:00-8:00	3	3
Turno 2 → 8:00-12:00	2	2
Turno 3 → 12:00-16:00	2	2
Turno 4 → 16:00-20:00	4	3

Tabla 3.19: Definición de turnos y demanda de enfermeros por turno.

La normativa para el trabajo en estos días indica:

1. Si alguien es asignado al turno 1, también debe ser asignado al turno 4 del mismo día.
2. No se puede asignar a alguien en el turno 2 y en el turno 3 el mismo día.
3. No se pueden asignar 3 turnos seguidos a ningún enfermero.

La Dra. Grierson le pide formular esta situación como un problema de programación lineal, para asignar los enfermeros al mínimo costo.

3.12.1 Solución

Definición de conjuntos

- I : enfermeros $\{1, \dots, 5\}$, para María, Pedro, Raúl, Valentina y José, respectivamente.
- J : turnos $\{1, \dots, 4\}$.

- K : días $\{1, 2\}$, para sábado y domingo, respectivamente.

Definición de parámetros

- C_i : costo de contratación por turno del enfermero $i \in I$.
- DEM_{jk} : demanda del turno $j \in J$ en el día $k \in K$.

Definición de las variables de decisión

- $x_{ijk} : \begin{cases} 1, & \text{si el enfermero } i \in I \text{ es asignado a turno } j \in J \text{ del día } k \in K \\ 0, & \text{en otro caso} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} \sum_{k \in K} C_i \cdot x_{ijk} \quad (3.90)$$

Sujeto a:

$$\sum_{i \in I} x_{ijk} \geq DEM_{jk}, \quad \forall j \in J, \forall k \in K \quad (3.91)$$

$$x_{i4k} \geq x_{i1k}, \quad \forall i \in I, \forall k \in K \quad (3.92)$$

$$x_{i2k} + x_{i3k} \leq 1, \quad \forall i \in I, \forall k \in K \quad (3.93)$$

$$\sum_{j \in \{1,2,3\}} x_{ijk} \leq 2, \quad \forall i \in I, \forall k \in K \quad (3.94)$$

$$\sum_{j \in \{2,3,4\}} x_{ijk} \leq 2, \quad \forall i \in I, \forall k \in K \quad (3.95)$$

$$x_{ijk} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J, \forall k \in K \quad (3.96)$$

La expresión (3.90) representa la función objetivo, que busca minimizar los costos de contratación de los enfermeros.

La restricción (3.91) garantiza que la demanda de enfermeros se cumplirá para los dos días de planeación. La expresión (3.92) establece que si algún enfermero es asignado al turno 1, también se debe asignar al turno 4.

La restricción (3.93) garantiza que los turnos 2 y 3 no sean asignados simultáneamente. Las restricciones (3.94) y (3.95) obligan a no asignar 3 turnos seguidos. Finalmente, la restricción (3.96) muestra la naturaleza de las variables.

El Algoritmo 28 contiene el código del modelo en AMPL. En la Tabla 3.20 se presenta la solución obtenida por Gurobi. La solución encontrada logra satisfacer la demanda, con un costo de asignación de enfermeros de \$780.

Algoritmo 28: archivo AsignacionEnfer.mod

```

1: set I := {1..5};
2: set J := {1..4};
3: set K := {1..2};
4: param C{I};
5: param DEM{J};
6: var x{I,J,K} binary;
7: minimize z: sum{i in I, j in J, k in K} C[i] * x[i,j,k];
8: s.t.
9: r1{j in J, k in K}: sum{i in I} x[i,j,k] >= DEM[j,k];
10: r2{i in I, k in K}: x[i,4,k] >= x[i,1,k];
11: r3{i in I, k in K}: x[i,2,k] + x[i,3,k] <= 1;
12: r4{i in I, k in K}: sum{j in {1..3}} x[i,j,k] <= 2;
13: r5{i in I, k in K}: sum{j in {2..4}} x[i,j,k] <= 2;

```

Sábado	Domingo
Enfermero 1: turno 2 y 4	Enfermero 1: turno 2
Enfermero 2: turno 1, 3 y 4	Enfermero 2: turno 1, 3 y 4
Enfermero 3: turno 1, 2 y 4	Enfermero 3: turno 1, 2 y 4
Enfermero 4: no fue asignado	Enfermero 4: no fue asignado
Enfermero 5: turno 1, 3 y 4	Enfermero 5: turno 1, 3 y 4

Tabla 3.20: Solución del problema de asignación de enfermeros.

3.12.2 Variación propuesta para el estudiante

Reformular el problema considerando que los enfermeros se deben contratar un mismo número de veces. Es decir, si en total el enfermero 1 se ha contratado 2 veces, todos los demás también se deben contratar 2 veces.

3.13 Problema de los libros

La empresa *El Libro Balbuciente* se dedica a la producción y distribución de libros. La gerente, la Ingeniera Matilda, lo contrata a usted para gestionar el envío de cajas de libros. Ella le comenta que, principalmente, la empresa tiene tres clientes: dos colegios (C_1 , C_2) y una universidad (U_1). Las demandas estimadas para este mes son de 100, 120 y 80 cajas de libros para cada cliente C_1 , C_2 y U_1 , respectivamente. La empresa cuenta con 4 plantas de fabricación para satisfacer la demanda. Cada planta tiene una capacidad de 75 cajas de libros por mes. Sin embargo, la capacidad es variable, es decir, se puede transferir capacidad para fabricar cajas de libros de una planta a otra (asuma que se pueden enviar máquinas de imprenta y suministros). Esta capacidad será medida en cajas de libros y la cantidad máxima a transferir es de 75 cajas, pero se debe garantizar que cada planta tenga al menos la capacidad de producción de 10 cajas para justificar el funcionamiento de las instalaciones.

La gerente Matilda indica que el proceso de distribución de libros debe estar basado en la minimización de los costos de transporte de cada planta a cada cliente. Estos costos por caja se muestran en la Tabla 3.21. La gerente le encarga que diseñe un modelo de programación lineal para determinar el número de cajas que cada planta debe producir y enviar a los destinos. El modelo también debe establecer qué capacidades deben ser transferidas.

[\$/caja]	C_1	C_2	U_1
Planta 1	18	48	17
Planta 2	47	10	48
Planta 3	38	10	12
Planta 4	20	50	35

Tabla 3.21: Costos de transporte de enviar una caja desde cada planta a cada cliente.

3.13.1 Solución

Definición de conjuntos

- I : plantas de producción $\{1, \dots, 4\}$.

- J : clientes $\{1, 2, 3\}$, representando a C_1 , C_2 y U_1 , respectivamente.

Definición de parámetros

- c_{ij} : costo unitario de envío de una caja de la planta $i \in I$ al cliente $j \in J$.
- of_i : capacidad inicial de fabricación de cajas en la planta $i \in I$; en este caso particular 75 cajas para todas las plantas.
- dem_j : demanda de cajas del cliente $j \in J$.

Definición de variables de decisión

- x_{ij} : cantidad de cajas enviadas de la planta $i \in I$ al cliente $j \in J$.
- H_{ik} : capacidad trasladada de la planta $i \in I$ a la planta $k \in I$. Cuando H_{ik} cumple que $i = k$, significa que la planta i no transfiere capacidad a ninguna otra.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} c_{ij} \cdot x_{ij} \quad (3.97)$$

Sujeto a:

$$\sum_{i \in I} x_{ij} \geq dem_j, \quad \forall j \in J \quad (3.98)$$

$$\sum_{j \in J} x_{ij} \leq \sum_{k \in I} H_{ki}, \quad \forall i \in I \quad (3.99)$$

$$\sum_{k \in I} H_{ik} \leq of_i, \quad \forall i \in I \quad (3.100)$$

$$\sum_{k \in I} H_{ki} \geq 10, \quad \forall i \in I \quad (3.101)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (3.102)$$

$$H_{ik} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall k \in I \quad (3.103)$$

La expresión (3.97) es la función objetivo del problema, que busca minimizar los costos de envío de cajas de libros.

La expresión (3.98) garantiza el cumplimiento de la demanda de cada cliente. La expresión (3.99) establece que el envío de cajas de cada planta no supere la capacidad disponible, incluyendo la transferida recibida de cualquier k . Las restricciones (3.100) y (3.101) establecen la cantidad máxima de cajas transferidas entre plantas y su producción mínima, respectivamente. Finalmente, las restricciones (3.102) y (3.103) exponen la naturaleza de las variables.

El Algoritmo 29 contiene el código del modelo en AMPL. La solución obtenida por Gurobi indica que se debe enviar: $x_{11} = 90$, $x_{22} = 60$, $x_{32} = 60$, $x_{33} = 80$ y $x_{41} = 10$. las transferencias de capacidad son: $H_{13} = 65$, $H_{14} = 10$, $H_{21} = 15$, $H_{22} = 60$, $H_{33} = 75$ y $H_{41} = 75$. La solución alcanza un mínimo de \$ 3980.

Algoritmo 29: archivo Libros.mod

```

1: set I = {1..4};
2: set J = {1..3};
3: param c{I,J};
4: param of{I};
5: param dem{J};
6: var x{I,J} >= 0;
7: var H{I,I} >= 0;
8: minimize z: sum{i in I, j in J} c[i,j]*x[i,j];
9: s.t.
10: rDem{j in J}: sum{i in I} x[i,j] >= dem[j];
11: rOf{i in I}: sum{j in J} x[i,j] <= sum{k in I} H[k,i];
12: pasar1{i in I}: sum{k in I} H[i,k] <= of[i];
13: pasar2{i in I}: sum{k in I} H[k,i] >= 10;

```

3.13.2 Variación propuesta para el estudiante

- a. Incluir la construcción de un centro de distribución (CD) con dos posibles ubicaciones. Este CD se comunicará entre las plantas y clientes. Generar los datos necesarios (como el costo de construcción y el de transporte entre CD y clientes o la capacidad de este) y reformular el problema.

- b. Considerar el caso anterior pero permitiendo que los CDs tengan modos de apertura, cuya principal característica sean las diferentes capacidades con su respectivo cambio en el costo de construcción (cuanto más pequeño, más barato).

3.14 Problema de espadas y hachas

Usted se encuentra en la Edad Media y está a cargo de la pomposa herrería *La Espada de Damocles*, que se especializa en la elaboración de espadas y hachas de gran calidad. El gerente de la herrería, Thor, le comenta que quiere maximizar las utilidades de sus ventas.

La capacidad máxima de producción mensual se traduce en una producción de solo 200 espadas o en una producción de solo 220 hachas, por tanto, cualquier combinación factible entre dichos límites es permitida. Las espadas se pueden endurecer si se aplica un proceso de templado, pero dicho proceso solo puede elaborar, como máximo, 25 espadas por mes. Las hachas no requieren este proceso.

Para la fabricación de las armas se tienen dos máquinas ($m1$ y $m2$). Una hacha requiere 3 horas en $m1$ y 2 horas en $m2$. Las espadas se pueden hacer completamente en $m1$ con una duración de 6 horas, o bien, se pueden hacer en las dos máquinas, usando 2 horas en $m1$ y 3 horas en $m2$. Cada máquina puede trabajar como máximo 480 horas por mes.

Finalmente, las armas están hechas con dos metales: plata y oro. Una espada requiere 1 kg de plata y 2 kg de oro, un hacha requiere 2 kg de plata y 1 kg de oro. Las reservas de estos materiales son de 600 kg de plata y 500 kg de oro. Cada kg de oro procesado le cuesta a la empresa \$15/kg y cada kg de plata le cuesta \$25/kg.

Una espada templada se puede vender a \$200, una espada sin templado a \$150, y una hacha a \$180. Formule un modelo de programación lineal para determinar el plan de producción que permita maximizar la utilidad mensual de la herrería.

3.14.1 Solución

Definición de conjuntos

Píldora informativa 3.16: Evitar conjuntos innecesarios

En varios problemas donde existen productos se ha definido un conjunto que los representa. Sin embargo, esto no siempre puede ser conveniente. Por ejemplo, en este problema las espadas tienen **tipos** de productos (con o sin templado) y, además, tienen **modos** de fabricación (solo $m1$, o $m1$ y $m2$), por tanto, la variable de decisión que represente el número de espadas a fabricar **debe diferenciar de qué modo se hace y qué máquinas usa**, mediante subíndices. Pero las hachas no tienen ninguna de estas características, por eso, definir una variable que incluya hachas y espadas en un mismo conjunto no es conveniente.

Por ejemplo, al definir la variable x_{bij} , que representa el tipo de producto $b \in \{e, h\}$ (espadas o hachas) de la forma $i \in \{1, 2\}$ (con o sin templado) y fabricado del modo $j \in \{1, 2\}$ (una o dos máquinas). Cuando b toma el valor e (espadas), la variable con todos los subíndices es correcta. Sin embargo, cuando toma el valor h (hachas) representaría variables imposibles como: x_{h12} , que representa hachas templadas, o x_{h21} , que representa hachas fabricadas en una única máquina.

- I : tipos de espadas $\{1, 2\}$, para templadas y sin templar, respectivamente.
- J : modo de procesamiento de las espadas $\{1, 2\}$, donde el modo 1 emplea únicamente la máquina 1, mientras que el modo 2 emplea ambas máquinas.
- K : máquinas $\{1, 2\}$.
- M : materia prima $\{1, 2\}$, para plata y oro, respectivamente.

Definición de parámetros

- Pe_i : precio de venta de las espadas tipo $i \in I$.
- Ph : precio de venta de las hachas.

- C_m : costo de la materia prima por kg tipo $m \in M$.
- Me_m : requerimiento en kg de materia prima tipo $m \in M$ para producir cada espada.
- Mh_m : requerimiento en kg de materia prima tipo $m \in M$ para producir cada hacha.
- $CapE$: capacidad máxima de producción si se fabrican solo espadas.
- $CapH$: capacidad máxima de producción si se fabrican solo hachas.
- $CapT$: capacidad máxima de producción para espadas templadas.
- Te_{kj} : horas requeridas en la máquina $k \in K$ para producir cada espada de la modalidad $j \in J$. Cabe resaltar que $Te_{21} = 0$, puesto que en la modalidad 1 de procesamiento no se requiere la máquina 2.
- Th_k : horas requeridas en la máquina $k \in K$ para producir cada hacha.
- $Horas$: tiempo disponible para la producción en cada máquina.
- Res_m : cantidad de metal tipo $m \in M$ en reserva.

Definición de variables de decisión

- E_{ij} : cantidad de espadas a producir del tipo $i \in I$ en la modalidad $j \in J$.
- H : cantidad de hachas a producir.

Modelo algebraico de programación lineal

$$\begin{aligned} \text{Maximizar } z = & \sum_{i \in I} \sum_{j \in J} Pe_i \cdot E_{ij} + Ph \cdot H \\ & - \sum_{m \in M} C_m \cdot \left(Me_m \cdot \sum_{i \in I} \sum_{j \in J} E_{ij} + Mh_m \cdot H \right) \end{aligned} \tag{3.104}$$

Sujeto a:

$$\frac{1}{CapE} \cdot \sum_{i \in I} \sum_{j \in J} E_{ij} + \frac{1}{CapH} \cdot H \leq 1 \quad (3.105)$$

$$\sum_{j \in J} E_{1j} \leq CapT \quad (3.106)$$

$$\sum_{j \in J} \sum_{i \in I} Te_{kj} \cdot E_{ij} + Th_k \cdot H \leq Horas, \quad \forall k \in K \quad (3.107)$$

$$Me_m \cdot \sum_{i \in I} \sum_{j \in J} E_{ij} + Mh_m \cdot H \leq Res_m, \quad \forall m \in M \quad (3.108)$$

$$E_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (3.109)$$

$$H \in \mathbb{Z}^+ \cup \{0\} \quad (3.110)$$

La expresión (3.104) es la función objetivo, que busca maximizar la utilidad del plan de producción de espadas y hachas. Esta función se compone principalmente de dos partes: los ingresos al vender las unidades producidas y los costos asociados a la materia prima.

La inecuación (3.105) es la restricción asociada a la capacidad de producción. Como $CapE$ es el total de espadas que se pueden producir, entonces $\frac{1}{CapE}$ es la proporción de la capacidad que consume elaborar una espada. Así mismo, $\frac{1}{CapH}$ es la proporción de la capacidad que se consume por elaborar una hacha; por supuesto, la capacidad consumida no puede superar el 100 %. Las restricciones (3.106) y (3.107) representan la dinámica de la capacidad de producción de espadas templadas y de procesamiento (en horas) en cada una de las máquinas, respectivamente. Mientras que la expresión (3.108) garantiza que el consumo de materia prima no supere las reservas disponibles. Finalmente, las expresiones (3.109) y (3.110) exponen la naturaleza entera de las variables.

El Algoritmo 30 contiene el código del modelo en AMPL. El plan de producción obtenido por CPLEX indica que se deben producir 96 hachas, 25 espadas templadas y 71 espadas sin temple, ambos tipos de espadas en el modo de procesamiento 2, es decir, haciendo uso de las dos máquinas, con el fin de alcanzar una utilidad mensual de \$21 410.

Algoritmo 30: archivo EspHac.mod

```

1: set I = {1..2};
2: set J = {1..2};
3: set K = {1..2};
4: set M = {1..2};
5: param Pe{I}; param Ph; param C{M};
6: param Me{M}; param Mh{M}; param CapE;
7: param CapH; param CapT; param Te{K,J};
8: param Th{K}; param Horas; param Res{M};
9: var E{I,J} >=0;
10: var H >=0;
11: maximize z: sum{i in I, j in J} (Pe[i]*E[i,j]) + Ph*H - sum{m
    in M} C[m]*(Me[m]*sum{i in I, j in J} E[i,j] + Mh[m]*H);
12: s.t.
13: Capacidad: (1/CapE)*sum{i in I, j in J} E[i,j] + (1/CapH)*H <= 1;
14: Templado: sum{j in J} E[1,j] <= CapT;
15: HorasMaq{k in K}: sum{j in J, i in I} Te[k,j]*E[i,j] + Th[k]*H <=
    Horas;
16: CapacidadMP{m in M}: Me[m]*sum{i in I, j in J} E[i,j] + Mh[m]
    *H <= Res[m];

```

3.15 Problema de la construcción del barco

La empresa constructora de barcos *Monkey Island* está frente a un problema de producción. Por eso, el gerente LeChuck lo contrata a usted y le comenta lo siguiente: la fabricación de un nuevo barco consiste en la realización de 9 tareas. Cada una se puede realizar de dos formas diferentes: tradicional (tiempo normal) y rápida (tiempo reducido).

Cada vez que una actividad se realiza de forma rápida se incurre en un costo. Adicionalmente, existen relaciones de precedencia entre tareas, es decir, si una tarea i es predecesora de la tarea j , implica que j solo se puede hacer después de finalizada la tarea i .

LeChuck le comenta que deben terminar todas las tareas antes de 15 días, y sabe que no puede cumplir con ese plazo, a menos que algunas tareas se hagan de forma rápida. Los tiempos y precedencias se muestran en la Tabla 3.22. LeChuck le pregunta: ¿qué actividades debe hacer de forma rápida para cumplir con el plazo al mínimo costo?

Nota: la capacidad de la empresa es ilimitada, es decir, se pueden realizar cuántas tareas se deseen de forma simultánea, siempre y cuando se respeten las normas de precedencia.

3.15.1 Solución

Definición de conjuntos

- I : tareas $\{A, B, C, D, E, F, G, H, I\}$.
- J : forma de realizar las tareas $\{1, 2\}$, para normal y rápido, respectivamente.

Tarea	Duración reducida (días)	Duración normal (días)	Costo para ejecución rápida (\$)	Predecesores inmediatos
A	1	3	4	Ninguno
B	2	4	1	Ninguno
C	0,5	2	1	A
D	2	5	1	A
E	1	6	3	B,C
F	1	2	7	E
G	3	4	9	D
H	2	3	5	F
I	4	5	8	G

Tabla 3.22: Tiempos, costos y precedencias de las actividades.

Píldora informativa 3.17: Conjuntos auxiliares

En algunos problemas puede ser de gran utilidad definir conjuntos auxiliares que contienen información agrupada. En este caso, las precedencias de las tareas se pueden agrupar en una tupla (secuencia de valores agrupados) de dos elementos (n, m) , de forma que la tarea n se debe hacer antes que la m .

- $Pred$: relaciones de precedencia entre tareas $\{(A, C), (A, D), (B, E), (C, E), (E, F), (D, G), (F, H), (G, I)\}$.

Definición de parámetros

- $DueDate$: fecha de terminación de la construcción del barco (15 días).
- P_{ij} : duración de la actividad $i \in I$ al ser realizada de la forma $j \in J$.
- $Costo_i$: costo por realizar la tarea i de la forma 2 (rápida).

Definición variables de decisión

- x_i : tiempo de inicio de la tarea $i \in I$.
- $y_i : \begin{cases} 1, & \text{si la tarea } i \in I \text{ se hace de la forma 2 (rápida)} \\ 0, & \text{si la tarea } i \in I \text{ se hace de la forma 1 (tradicional)} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} Costo_i \cdot y_i \quad (3.111)$$

Sujeto a:

$$x_i + P_{i1} \cdot (1 - y_i) + P_{i2} \cdot y_i \leq DueDate, \quad \forall i \in I \quad (3.112)$$

$$x_k \geq x_i + P_{i1} \cdot (1 - y_i) + P_{i2} \cdot y_i, \quad \forall (i, k) \in Pred \quad (3.113)$$

$$x_i \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I \quad (3.114)$$

$$y_i \in \{0, 1\}, \quad \forall i \in I \quad (3.115)$$

La expresión (3.111) representa la función objetivo, que busca minimizar el costo de la ejecución rápida de las actividades.

La restricción (3.112) garantiza que la ejecución de cada actividad, en cualquiera de las dos formas elegidas, más su duración respectiva, sea menor o igual que el límite de tiempo. La restricción (3.113) obliga a que cada actividad solo puede iniciar después de que se hayan finalizado todas sus actividades predecesoras. Finalmente, las restricciones (3.114) y (3.115) muestran la naturaleza de las variables.

Píldora informativa 3.18: Subíndices nuevos

Nótese que en la expresión (3.113) se usa la variable x_k , a pesar que, hasta ese momento, no se había definido el subíndice k . Esto se puede hacer **siempre y cuando se cumplan** dos condiciones:

1. Si se define el nuevo subíndice (por ejemplo k) en la misma restricción. En este caso, se define $\forall(i, k) \in Pred$.
2. Que dicho subíndice solo pueda tomar valores compatibles con la definición de la variable. En este caso x_i está definida en el conjunto de tareas I , y cada pareja (i, k) está definida en $Pred$, que a su vez solo contiene tareas I .

El Algoritmo 31 contiene el código del modelo algebraico en AMPL. La solución obtenida por Gurobi indica que las tareas C y D se deben realizar de forma rápida, cumpliendo con el tiempo de entrega y minimizando los costos asociados a la realización de las actividades. El costo obtenido fue \$2. Los tiempos de inicio y finalización de cada actividad se pueden apreciar en la Figura 3.6. En ella se demuestra que la ejecución de tareas no excede el tiempo límite de finalización.

Algoritmo 31: Archivo monkeyIsland.mod

```

1: set I := {'A','B','C','D','E','F','G','H','I'};
2: set J := {1,2};
3: set Pred := {('A','C'),('A','D'),('B','E'),('C','E'),('E','F'),('D','G'),
  ('F','H'),('G','I')};
4: param dueDate;
5: param P{I,J};
6: param costo{I};
7: var x{I} >= 0 integer;
8: var y{I} binary;
9: minimize z: sum{i in I} costo[i]*y[i];
10: s.t.
11: r2{i in I}: x[i] + P[i,1]*(1-y[i]) + P[i,2]*y[i] <= dueDate;
12: r3{(i,k) in Pred}: x[k] >= x[i] + P[i,1]*(1-y[i]) + P[i,2]*y[i];

```

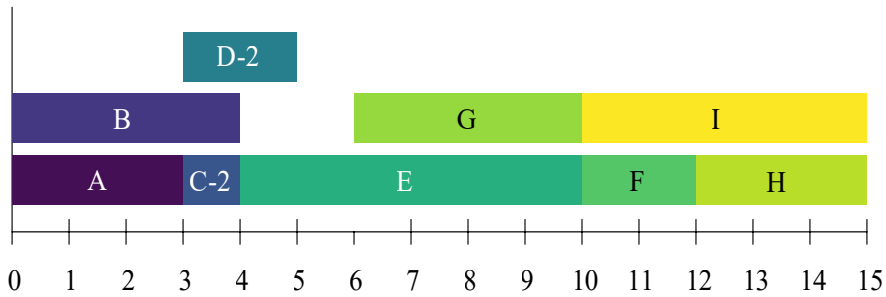


Figura 3.6: Diagrama de *Gantt* de la solución encontrada.

Recordar que AMPL usa comillas simples (') o dobles (") para referirse a caracteres de texto. No confundir con el carácter (•). Para diferenciarlos puede usar la codificación ASCII. Símbolos válidos: char 39 = ', char 34 = ", inválido char 96 = `.

3.15.2 Variación propuesta para el estudiante

Considerando que el nuevo objetivo es minimizar la duración de todo el proyecto, elabore varios presupuestos en los que especifique el mejor tiempo que se puede obtener con cada uno.

Capítulo 4

Modelos de complejidad alta

Finalmente, en este capítulo se proponen 15 ejercicios que se resuelven mediante modelos matemáticos en notación algebraica. El propósito de este capítulo es mostrar conceptos más avanzados de la programación matemática, como la propagación de restricciones en el tiempo, parámetros incompletos, linealización de expresiones, componentes multi-objetivo básicos, dinámica de ruteo simple, secuenciación de tareas, entre otros. Estos elementos permiten la formulación y resolución de problemas más realistas y complejos.

Los archivos correspondientes a la solución en AMPL de los problemas planteados en este capítulo se pueden encontrar en “Aprende Modelando - GitHub”. [Online]. Disponible en: <https://github.com/dmorill/AprendeModelandoMILP>.

4.1 Problema de los bonos

Luego de algunos años de trabajo, usted ha ahorrado una suma de 10000000 COP. Como actualmente cuenta con una estabilidad económica, usted desea usar estos ahorros para invertir en el mundo de valores del mercado de renta fija para incrementar su riqueza. Pero como no tiene muy claro en qué consisten realmente estas inversiones, le consulta a Logan, su asesor financiero de confianza. Usted ha establecido que estas inversiones quiere hacerlas en un horizonte de 4 meses.

Logan le comenta que este mercado funciona comprando bonos. Al precio de compra se le llama *face value*. Estos bonos tienen dos características

principales: la *tasa cupón* (porcentaje de ganancias que generan los bonos cada mes en función del *face value*) y la *duración* (el tiempo de vida del bono). Por supuesto, cualquier beneficio obtenido en un mes se puede usar para volver a invertirlo en el siguiente mes o bien reservarlo.

Una vez termine el bono se retorna por completo el *face value* inicial (junto con el último beneficio generado). Finalmente, los bonos tienen una fecha en la que se puede hacer la compra, a esto se le llama *fecha de disponibilidad*. Estas fechas solo indican la posibilidad de hacer la compra del bono, por tanto, no afectan su duración ni los beneficios que generan.

Los posibles bonos se muestran a continuación, en la Tabla 4.1.

Bono	1	2	3
<i>Face value</i> (COP)	120 000	200 000	1 000 000
<i>Tasa cupón</i> (%)	5 %	3,5 %	12 %
<i>Duración</i> (meses)	3	2	1
<i>Fecha de disponibilidad</i> (mes)	1 y/o 2	2 y/o 3	3 y/o 4

Tabla 4.1: Datos de los bonos disponibles.

Como ejemplo, si compramos 2 bonos tipo 1 en el mes 2 (permitido por su disponibilidad) por un valor total de $(120\,000 \text{ COP} \cdot 2 \text{ bonos})$. Basándonos en su duración de 3 meses, los bonos generarán $(120\,000 \text{ COP} \cdot 2 \text{ bonos} \cdot 0,05)$ en el mes 2 y en el mes 3, por su parte, en el mes 4 se generarán $(120\,000 \text{ COP} \cdot 2 \text{ bonos} \cdot 0,05)$ más el face value invertido $(120\,000 \text{ COP} \cdot 2 \text{ bonos})$.

Formular este problema mediante programación lineal para encontrar la forma óptima de inversión, de modo que se maximice el dinero obtenido al final del mes 4.

4.1.1 Solución

Definición de conjuntos

- T : meses $\{1, \dots, 4\}$.
- I : tipos de inversiones $\{1, 2, 3\}$.

Definición de parámetros

- C_i : *face value* del bono $i \in I$.
- TC_i : *tasa cupón* del bono $i \in I$.

Definición de variables de decisión

- $Tengo_t$: dinero disponible en COP al inicio del mes $t \in T \cup \{0\}$.
- $Sobra_t$: dinero que sobra en COP luego de haber realizado las compras de bonos respectivas en el mes $t \in T$.
- x_{it} : cantidad de bonos tipo $i \in I$ comprados en el mes $t \in T$.

*Modelo algebraico de programación lineal***Píldora informativa 4.1: Variables con consecuencias en el tiempo**

En algunos problemas **las variables de decisión tienen consecuencias a lo largo del tiempo** (restricciones que se propagan en el tiempo). Por ejemplo, si se invierte en 2 bonos tipo 1 en el segundo mes, generaría un beneficio en el mes 2 y en los dos siguientes (mes 3 y mes 4). Es decir, la inversión en el mes 2 ocasionaría un beneficio para 3 mese en total.

Mes	Ganancias	
2	$2 \cdot 0,05 \cdot 120\,000$	= 12000
3	$2 \cdot 0,05 \cdot 120\,000$	= 12000
4	$2 \cdot (1 + 0,05) \cdot 120\,000$	= 252000
Total		= 276000

En estos casos **se debe garantizar este comportamiento en las restricciones asociadas al tiempo**. En este problema serían restricciones mensuales. La dinámica solo para el ejemplo anterior ($x_{11} = 2$), se muestra a continuación:

Restricción	Ganancias
Mes 2:	$\dots + (x_{11} \cdot 0,05 \cdot 120\,000) + \dots$
Mes 3:	$\dots + (x_{11} \cdot 0,05 \cdot 120\,000) + \dots$
Mes 4:	$\dots + [x_{11} \cdot (1 + 0,05) \cdot 120\,000] + \dots$

A estas restricciones se les debe añadir los beneficios generados por la compra de otros bonos en el mes correspondiente. Por ejemplo, si compramos 5 bonos tipo 2 en el mes 3 ($x_{23} = 5$), teniendo en cuenta que dichos bonos tienen una duración de 2 meses, en la restricción del mes 3 se debería sumar $(x_{23} \cdot 0,035 \cdot 200\,000)$, y en la ecuación del mes 4 se debería sumar $[x_{23} \cdot (1 + 0,035) \cdot 200\,000]$. Al tener valores que actualizar cada mes, estas expresiones no son tan fáciles de representar en la notación algebraica.

$$\text{Maximizar } z = \text{Tengo}_4 \quad (4.1)$$

Sujeto a:

$$\text{Sobra}_i = \text{Tengo}_{i-1} - \sum_{i \in I} x_{it} \cdot C_i, \quad \forall t \in T \quad (4.2)$$

$$\text{Tengo}_0 = 10\,000\,000 \quad (4.3)$$

$$\text{Tengo}_1 = \text{Sobra}_1 + (x_{11} \cdot TC_1 \cdot C_1) \quad (4.4)$$

$$\text{Tengo}_2 = \text{Sobra}_2 + (x_{11} \cdot TC_1 \cdot C_1) + (x_{12} \cdot TC_1 \cdot C_1) + (x_{22} \cdot TC_2 \cdot C_2) \quad (4.5)$$

$$\begin{aligned} \text{Tengo}_3 = \text{Sobra}_3 &+ [x_{11} \cdot (1 + TC_1) \cdot C_1] + (x_{12} \cdot TC_1 \cdot C_1) + [x_{22} \cdot (1 + TC_2) \cdot C_2] + (x_{23} \cdot \\ &TC_2 \cdot C_2) + [x_{33} \cdot (1 + TC_3) \cdot C_3] \end{aligned} \quad (4.6)$$

$$\begin{aligned} \text{Tengo}_4 = \text{Sobra}_4 &+ [x_{12} \cdot (1 + TC_1) \cdot C_1] + [x_{23} \cdot (1 + TC_2) \cdot C_2] + [x_{34} \cdot (1 + TC_3) \cdot C_3] \end{aligned} \quad (4.7)$$

$$\text{Tengo}_t \in \mathbb{Z}^+ \cup \{0\}, \quad \forall t \in T \cup \{0\} \quad (4.8)$$

$$\text{Sobra}_i \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I \quad (4.9)$$

$$x_{it} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall t \in T \quad (4.10)$$

La expresión (4.1) es la función objetivo del problema, que busca maximizar el dinero generado al final del mes 4.

La restricción (4.2) define la cantidad de dinero que sobra cada mes, luego de hacer las compras correspondientes. La restricción (4.3) establece el valor inicial de dinero disponible. Las ecuaciones (4.4) a (4.7) definen el dinero disponible en cada mes, teniendo en cuenta el valor de beneficios generados. Finalmente, las expresiones (4.8) a (4.10) establecen la naturaleza de las variables.

El Algoritmo 32 contiene el código del modelo en AMPL. El plan de inversión óptimo se muestra en la Tabla 4.2. Con él se obtienen 12934000 COP al final del cuarto mes.

Algoritmo 32: archivo Bonos.mod

```

1: set T := {1..4};
2: set I := {1..3};
3: param C{I};
4: param TC{I};
5: var Tengo{T union {0}} >= 0;
6: var Sobra{T} >= 0;
7: var x{I,T} integer >= 0;
8: maximize z: Tengo[4];
9: s.t.
10: rPres{t in T}: Sobra[t] = Tengo[t-1] - sum{i in I} x[i,t]*C[i];
11: rini1: Tengo[0] = 10000000;
12: r1: Tengo[1] = Sobra[1] + x[1,1]*TC[1]*C[1];
13: r2: Tengo[2] = Sobra[2] + x[1,1]*TC[1]*C[1] + x[1,2]*TC[1]*C[1]
    + x[2,2]*TC[2]*C[2];
14: r3: Tengo[3] = Sobra[3] + x[1,1]*(1+TC[1])*C[1] + x[1,2]*TC [1]
    *C[1] + x[2,2]*(1+TC[2])*C[2] + x[2,3]*TC[2]*C[2] + x[3,3]*(1+
    TC[3])*C[3];
15: r4: Tengo[4]=Sobra[4]+x[1,2]*(1+TC[1])*C[1]+x[2,3]*(1+TC[2])*C[2]
    + x[3,4]*(1+TC[3])*C[3];

```

Mes	— $x_{11} = 82$	+ $x_{11} = 82$	— $x_{12} = 1$	+ $x_{12} = 1$	— $x_{33} = 1$	+ $x_{33} = 1$	— $x_{34} = 11$	+ $x_{34} = 11$	Balance
0	—	—	—	—	—	—	—	—	10 000 000
1	9 840 000	492 000	120 000	6 000	—	—	—	—	538 000
2	—	492 000	—	6 000	—	—	—	—	1 036 000
3	—	10 332 000	—	126 000	1 000 000	1 120 000	—	—	11 614 000
4	—	—	—	—	—	—	11 000 000	12 320 000	12 934 000

*—: compra, +: beneficio.

Tabla 4.2: Plan de inversión óptimo.

4.1.2 Variación propuesta para el estudiante

Reformule el problema asumiendo que todos los bonos están disponibles desde el primer mes.

4.2 Problema de las metas (adaptado de [3])

Una compañía tecnológica planea introducir al mercado tres nuevos productos, dada su próxima obsolescencia. El interés de la gerencia es determinar las tasas de producción de dichos productos, según tres objetivos fundamentales:

- **Utilidad a largo plazo (objetivo 1):** lograr un valor presente neto (VPN) mínimo de 1 000 millones de pesos.
- **Nivel de empleo (objetivo 2):** mantener el recurso laboral actual de 1 000 empleados.
- **Inversión inicial (objetivo 3):** sostener la inversión de capital en el nuevo equipo de 400 millones de pesos.

Estos objetivos son incompatibles unos con otros. Por ejemplo, para generar más ingresos se puede necesitar más personal, pero se busca no modificar el número de empleados. Por esta razón, el gerente ha establecido un “puntaje de penalización” para cada objetivo, en caso de no cumplirse a cabalidad, así:

- **Objetivo 1:** se penalizan 6 puntos por cada 10 millones que se obtengan en déficit en relación a la meta.
- **Objetivo 2:** se penalizan 3 puntos por cada 10 empleados que se deban contratar en exceso y 4 puntos por cada 10 empleados en déficit no contratado.
- **Objetivo 3:** se penalizan 4 puntos por cada 10 millones de exceso en el presupuesto.

Además, se ha determinado que la contribución de cada producto, en decenas⁹, a cada objetivo, es proporcional a su nivel de producción. Es decir, cuanto más se produce, más se aumenta la contribución a los objetivos (simultáneamente), según los valores descritos en la Tabla 4.3. ¿Cuáles deben ser las tasas de producción de cada artículo para cumplir con los objetivos de la mejor forma posible?

Objetivo	Producto 1	Producto 2	Producto 3	Decenas
Objetivo 1	\$ 15	\$ 8	\$ 12	Millones
Objetivo 2	3	2	1	Empleados
Objetivo 3	\$ 4	\$ 5	\$ 7	Millones

Tabla 4.3: Contribución de los productos al cumplimiento de los objetivos.

4.2.1 Solución

Definición de conjuntos

- I : objetivos $\{1, 2, 3\}$.
- J : productos $\{1, 2, 3\}$.

Definición de parámetros

- a_{ij} : decenas que contribuye el producto j al cumplimiento del objetivo i .
- $meta_i$: decenas a alcanzar del objetivo i .

⁹Conjunto de 10 elementos.

En la formulación se usará la meta de cada uno de los objetivos, en decenas. Por lo tanto, los valores de referencia para los objetivos 1, 2 y 3 son 100, 100 y 40, respectivamente.

Definición de variables de decisión

Píldora informativa 4.2: Variables de desviación

En ocasiones puede que no se busque maximizar o minimizar un valor, sino **alcanzar un valor lo más cercano a una meta**. Se puede dar el escenario en el que sobrepasar ese valor sea negativo (por ejemplo, un presupuesto) o positivo (en el caso de ingresos esperados). De forma similar, el no alcanzar dicha meta puede ser positivo o negativo para una empresa.

Por esta razón, se **requiere calcular las desviaciones por encima y por debajo con respecto a esas metas**, es decir, qué tanto se ha desviado de lo buscado (o esperado). Para esto se usan las **variables de desviación**, que tienen las mismas características que cualquier otra variable de decisión. Sin embargo, se definen en las restricciones, bajo la certeza que **cualquier valor (desviación positiva o negativa) puede ser representado mediante la resta de dos variables positivas**.

Como ejemplo, si se quiere obtener una utilidad de 500 USD y para conseguirlo se venden 2 productos (x_1 , x_2) a un precio de p_1 y p_2 , respectivamente; las variables de desviación se definen como y^+ y y^- , ambas mayores o iguales a cero. y^+ representa la desviación positiva (lo sobrante una vez alcanzada la meta) y y^- representa la desviación negativa (lo faltante para alcanzar la meta). Es importante resaltar que **el uso de los símbolos + y - como exponente solo es estético**, por tanto, se podrían definir dos variables diferentes para representar lo mismo.

La relación para que estas variables cumplan su objetivo es:

$$\underbrace{\sum_{i=1}^2 x_i * p_i}_{\text{producido}} - \underbrace{500}_{\text{meta}} = \underbrace{y^+}_{\text{sobra}} - \underbrace{y^-}_{\text{falta}}$$

La parte izquierda representa la diferencia entre lo producido y la meta, es decir la desviación. El lado derecho obliga a igualar la desviación a la resta de las dos variables positivas. Bajo este contexto, pueden ocurrir dos posibilidades:

1. Si se produce más de la meta:

$$\underbrace{800}_{\text{prod.}} - \underbrace{500}_{\text{meta}} = \underbrace{y^+}_{300} - \underbrace{y^-}_0$$

Para cumplir la igualdad se obliga a y^+ a tomar el valor de 300, es decir, el excedente de la meta, y y^- debe tomar el valor de 0.

2. Si se produce menos de la meta:

$$\underbrace{200}_{\text{prod.}} - \underbrace{500}_{\text{meta}} = \underbrace{y^+}_0 - \underbrace{y^-}_{300}$$

Para cumplir la igualdad se obliga a y^- a tomar el valor de 300, es decir, lo faltante para alcanzar la meta, y y^+ debe tomar el valor de 0.

Finalmente, es importante notar que, en cualquier caso, la igualdad se puede alcanzar con infinitos valores, por ejemplo, en el caso 1, el valor 300 positivo se puede alcanzar con $1\,300 - 1\,000$ o $5\,500 - 5\,200$, o cualquier otra combinación cuya resta sea 300. Para obligar a que una de las variables (y^+ o y^-) tome el valor de 0, en el caso de que la función objetivo (FO) sea de maximización, se suele asignar un coeficiente negativo a estas variables en la FO. En el caso de que la FO sea de minimización, se asigna un coeficiente positivo a estas variables en la FO. Esto se denomina **penalizar la variable en la función objetivo**.

- x_j : cantidad de producto tipo j a producir.
- y_i^+ : decenas excedentes obtenidas, respecto al valor de referencia del objetivo i .

- y_i^- : decenas por déficit obtenidas, respecto al valor de referencia del objetivo i .

Modelo algebraico de programación lineal

$$\text{Minimizar } z = 6 \cdot y_1^- + 3 \cdot y_2^+ + 4 \cdot y_2^- + 4 \cdot y_3^+ \quad (4.11)$$

Sujeto a:

$$\sum_{j \in J} a_{ij} \cdot x_j - \text{meta}_i = y_i^+ - y_i^-, \quad \forall i \in I \quad (4.12)$$

$$x_j \in \mathbb{Z}^+ \cup \{0\}, \quad \forall j \in J \quad (4.13)$$

$$y_i^+, y_i^- \in \mathbb{R}^+ \cup \{0\}, \quad \forall i \in I \quad (4.14)$$

La expresión (4.11) es la función objetivo del problema, que busca minimizar las penalizaciones obtenidas por las desviaciones con respecto a los objetivos.

La restricción (4.12) establece que la diferencia entre el objetivo y el valor real es igual a la desviación, ya sea por exceso o por déficit, con respecto al valor de referencia. Finalmente, las expresiones (4.13) y (4.14) representan la no negatividad de las variables.

El Algoritmo 33 contiene el código del modelo en AMPL. El plan de producción obtenido por Gurobi en AMPL indica producir solo 10 unidades del producto 1 para alcanzar una penalización mínima de 280 puntos. Además, las desviaciones para cada uno de los objetivos se presentan en la Tabla 4.4, donde, para la primera restricción, el VPN tiene 500 millones de pesos en exceso respecto a la meta; para el segundo objetivo se tienen 700 empleados en déficit respecto a la meta, mientras que el último objetivo se cumplió a cabalidad.

Objetivo	y_i^-	y_i^+
Objetivo 1	\$ 0	\$ 50
Objetivo 2	70	0
Objetivo 3	\$ 0	\$ 0

Tabla 4.4: Contribución de los productos al cumplimiento de los objetivos.

Algoritmo 33: archivo Metas.mod

```

1: set I := {1..3};
2: set J := {1..3};
3: param a{I,J};
4: param meta{I};
5: var x{J} >=0 integer;
6: var yma{I} >=0;
7: var yme{I} >=0;
8: minimize z: yme[1]*6 + 3*yma[2] + 4*yme[2] + 4*yma[3];
9: s.t.
10: rMetas{i in I}: sum{j in J} x[j]*a[i,j] - meta[i] = yma[i] - yme[i];

```

4.3 Problema del centro de cómputo

El centro de cómputo *Masmelos en la Nube* se encarga de procesar trabajos que exigen grandes capacidades de procesamiento de datos, por ejemplo, algoritmos de optimización, inteligencia artificial y *big data*. Actualmente cuenta con dos supercomputadoras para realizar sus trabajos: CPU 1 y CPU 2.

La secuencia de los trabajos es la clave para esta empresa, pues el costo de operación de cada trabajo depende de la posición en la secuencia y del computador donde se realice dicho trabajo. Por secuencia se entiende el orden cómo los trabajos son procesados por cada máquina (existen dos secuencias, una por cada computador). En las Tablas 4.5 y 4.6 se muestran los costos en dólares para los siguientes 6 trabajos en función de la secuencia (posición). Ejemplo, si el trabajo 4 se hace en la primera posición del computador 1, cuesta 30 USD.

		Posición					
		1	2	3	4	5	6
Trabajos	1	29	22	7	33	14	34
	2	8	32	14	30	15	27
	3	31	20	18	7	22	29
	4	30	29	25	12	29	7
	5	26	17	12	27	7	9
	6	20	29	12	6	23	32

Tabla 4.5: Costos de operación (USD) de los trabajos en la CPU 1 según la posición.

		Posición					
		1	2	3	4	5	6
Trabajos	1	6	27	35	31	24	19
	2	7	10	7	28	32	29
	3	2	9	17	31	17	27
	4	20	5	19	15	33	33
	5	3	21	7	33	10	28
	6	15	7	19	25	12	21

Tabla 4.6: Costos de operación (USD) de los trabajos en la CPU 2 según la posición.

El gerente de la empresa requiere asignar la secuencia de trabajos en cada computador, de tal manera que se minimice el costo de la realización de todos los trabajos. Formular un modelo de programación lineal que dé respuesta a esta problemática. Asuma que:

- Cualquier computador puede ejecutar todos los trabajos, pero también es válido que unos trabajos se hagan en el CPU 1 y otros en el CPU 2.
- Una secuencia válida es aquella donde si existe una posición x , entonces existe una $x - 1$; por ejemplo, si existe una posición tercera debe existir una segunda.

4.3.1 Solución

Definición de conjuntos

- T : trabajos $\{1, ..., 6\}$.
- M : CPUs $\{1, 2\}$.

Píldora informativa 4.3: Conjuntos reutilizables

Nótese la **exclusión de un conjunto para las posiciones de la secuencia**. Esto se debe a que existen tantas posiciones como trabajos, por ello **se hará uso del mismo conjunto T** para hacer referencia a la secuencia.

De esta forma se pueden usar diferentes subíndices con el mismo conjunto, por ejemplo, al suponer que existe una variable y_{ij} se podría definir una expresión de esta forma:

$$\sum_{i \in T} \sum_{j \in T} y_{ij} = 8$$

En estos casos, **los subíndices siempre son independientes**, aunque comparten el conjunto que recorren. La sumatoria se resuelve como es usual.

Definición de parámetros

- C_{ijm} : costo de procesar el trabajo $i \in T$, en la posición $j \in T$, en la CPU $m \in M$.

Definición de variables de decisión

- $X_{ijm} : \begin{cases} 1, \text{ si el trabajo } i \in T \text{ se procesa en la posición } j \in T \text{ en la} \\ \text{CPU } m \in M \\ 0, \text{ en otro caso} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in T} \sum_{j \in T} \sum_{m \in M} C_{ijm} \cdot X_{ijm} \quad (4.15)$$

Sujeto a:

$$\sum_{i \in T} X_{ijm} \leq 1, \quad \forall j \in T, \forall m \in M \quad (4.16)$$

$$\sum_{j \in T} \sum_{m \in M} X_{ijm} = 1, \quad \forall i \in T \quad (4.17)$$

$$\sum_{i \in T} X_{ijm} \leq \sum_{i \in T} X_{i,j-1,m}, \quad \forall j \in T \setminus \{1\}, \forall m \in M \quad (4.18)$$

$$X_{ijm} \in \{0, 1\}, \quad \forall i, j \in T, \forall m \in M \quad (4.19)$$

La expresión (4.15) es la función objetivo del problema, que busca minimizar el costo asociado al procesamiento de los seis trabajos.

La restricción (4.16) garantiza que por cada posición en cada una de las máquinas, se ejecute como máximo un trabajo a la vez. La restricción (4.17) establece que cada uno de los trabajos debe ejecutarse exactamente una vez (en alguna máquina y en alguna posición). La ecuación (4.18) establece que, de procesar un trabajo en la posición j , debe existir otro trabajo en la posición $j - 1$ (se cumple en todo j excepto en $j = 1$). Finalmente, la expresión (4.19) define la naturaleza de las variables.

El Algoritmo 34 contiene el código del modelo en AMPL. El plan de procesamiento indica que la CPU 1 debe procesar los trabajos 2, 5, 1 y 6, en ese orden. Mientras que, la CPU 2 debe procesar primero el trabajo 3, y luego el 4. Si este plan se sigue, se obtendrá un costo mínimo de 45 USD.

Algoritmo 34: archivo Computo.mod

```

1: set T := {1..6};
2: set M := {1..2};
3: param C{T, T, M};
4: var X{T, T, M} >=0 binary;
5: minimize z: sum{i in T, j in T, m in M} X[i,j,m]*C[i,j,m];
6: s.t.
7: r1{j in T, m in M}: sum{i in T} X[i,j,m] <= 1;
8: r2{i in T}: sum{j in T, m in M} X[i,j,m] = 1;
9: r3{j in T diff {1}, m in M}: sum{i in T} X[i,j,m] <= sum{i in T}
    X[i,j-1,m];

```

4.3.2 Variación propuesta para el estudiante

Reformule el problema considerando dos problemas independientes, idénticos al original, pero cada uno con una CPU diferente.

4.4 Problema de los bombarderos (adaptado de [3])

En un futuro lejano, usted está al mando de la armada *Mitras* que lucha contra el reino de “Ymir”, quien ha instalado una fábrica de tanques en un territorio vulnerable. Aprovechando esta oportunidad, usted debe realizar una estrategia para el ataque. La fábrica consta de 4 plantas, 3 de ellas para

la efectiva creación de tanques, y la restante es ficticia (solo creada para confundir al enemigo).

Para detener la elaboración de tanques, basta con destruir solo una de las 3 plantas, el problema es que se desconoce cuál es la ficticia.

El jefe de exploración le facilita los datos asociados a la distancia desde su base central a cada una de las plantas enemigas (Tabla 4.7). Para hacer el ataque, usted cuenta con un total de 51 000 galones de combustible y dos tipos de bombarderos: 48 pesados y 35 ligeros. El rendimiento del combustible es de 3,0 y 4,5 kilómetros por galón para los bombarderos pesado y liviano, respectivamente. En cada viaje se debe llevar el combustible necesario para ir al destino, regresar y una reserva de 150 galones adicionales.

Planta	Distancia desde la base [km]
1	675
2	720
3	810
4	900

Tabla 4.7: Distancia desde la base central a las plantas enemigas.

El éxito de un ataque depende de las condiciones meteorológicas, localización, velocidad, presión y tipo de bombas. Por tanto, se desconoce el resultado, pero se ha estimado la probabilidad de éxito dependiendo de la planta y el tipo de bombardero. Los datos se muestran en la Tabla 4.8.

Planta	Bombardero pesado	Bombardero liviano
1	0,10	0,08
2	0,20	0,16
3	0,15	0,12
4	0,25	0,20

Tabla 4.8: Probabilidad de éxito en el ataque.

Para asegurar la destrucción de la planta, usted ha decidido enviar bombarderos como mínimo a dos plantas. ¿Cuántos bombarderos, de qué tipo y a qué plantas (ya sabe que serán al menos dos) debe enviar para maximizar la probabilidad de destrucción de la fábrica de tanques?

4.4.1 Solución

Píldora informativa 4.4: Probabilidad en modelos deterministas lineales

En los problemas de programación lineal tradicionales, los parámetros son constantes conocidas, y no se incluyen variables aleatorias en los modelos. Sin embargo, esto no excluye la posibilidad de **usar probabilidades como parámetros conocidos y constantes**, y así calcular la probabilidad de eventos.

En este caso, el cálculo de la probabilidad de éxito se puede realizar suponiendo que **cada ataque (con éxito o fracaso) es independiente de los demás**. De esta forma, si se envían 4 bombarderos pesados a la planta 1, sin importar si el bombardero número 1 acierta o falla, se enviarán la totalidad de aviones planeados (4 en este ejemplo). Esto se puede justificar bajo el supuesto que, cuando un avión hace el bombardeo, el piloto desconoce si tendrá éxito o no (los otros pilotos tampoco pueden saberlo), y solo lo descubre cuando regresa a la base central.

Para el cálculo de la probabilidad de éxito ($P(E)$), se debe tener en cuenta que con solo un acierto de un bombardero, la planta atacada se destruirá completamente. Por este motivo, se deben considerar **todas las posibilidades** donde se tenga **al menos un acierto** (espacio muestral), y sumar sus probabilidades. Para mostrar un ejemplo, en el Figura 4.1 se observan todas las posibilidades para destruir la planta 1 si se envían 3 bombarderos pesados, donde el símbolo ($\checkmark$) representa la probabilidad de éxito del bombardero (0,1) y el símbolo ($\times$) representa la respectiva probabilidad de fracaso (0,9). En dicha figura, también se muestra la única posibilidad de no destruir la planta con los 3 bombarderos.

Como se observa en la Figura 4.1, el cálculo de $P(E)$ es más complejo que el cálculo de la probabilidad de fracaso ($P(F)$). Mientras $P(F)$ se puede calcular multiplicando la probabilidad de fracaso del bombardero tantas veces como el número de aviones enviados (en el ejemplo: $0,9^3$), para $P(E)$ se deben tener en cuenta las múltiples combinaciones posibles. Sin embargo, $P(E)$ se puede estimar fácilmente con el complemento:

$$P(E) = 1 - P(F), \text{ para el ejemplo } \rightarrow 1 - 0,9^3 = 0,271$$

Por esta razón, en este tipo de problemas se sugiere minimizar la probabilidad de fracaso, que es equivalente a maximizar la probabilidad de éxito.

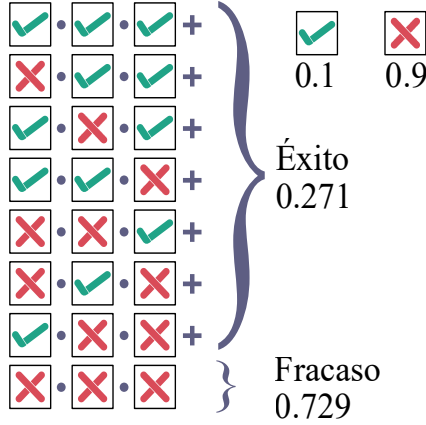


Figura 4.1: Probabilidades de éxito y fracaso en el ataque con 3 bombarderos.

Definición de conjuntos

- I : tipo de bombarderos $\{1, 2\}$, para pesados y livianos, respectivamente.
- J : plantas de la fábrica de tanques $\{1, \dots, 4\}$.

Definición de parámetros

- Gas : combustible total disponible para la misión.
- $kmGalon_i$: kilómetros recorridos por galón de combustible del bombardero tipo $i \in I$.
- Cap_i : número máximo de bombarderos tipo $i \in I$.
- Dis_j : distancia, en kilómetros, desde la base central a la planta $j \in J$.
- pF_{ij} : probabilidad de fracaso en el ataque de la planta $j \in J$ al enviar un bombardero tipo $i \in I$.

- M : un valor muy grande en comparación con los demás parámetros.

Definición de variables de decisión

- x_{ij} : número de bombarderos tipo $i \in I$ enviados a atacar la planta $j \in J$.
- $y_j: \begin{cases} 1, & \text{si se ha enviado algún bombardero a la planta } j \in J \\ 0, & \text{en otro caso} \end{cases}$

Modelo algebraico de programación lineal

Píldora informativa 4.5: Linealizar funciones mediante el logaritmo natural

La generalización del cálculo de la probabilidad de fracaso para un bombardero tipo i en una planta j es:

$$(pF_{ij})^{x_{ij}}$$

De esta forma la probabilidad total de fracaso se puede estimar al multiplicar la probabilidad de fracaso de cada tipo de avión y planta (asumiendo independencia entre eventos):

$$z = \prod_{i \in I} \prod_{j \in J} (pF_{ij})^{x_{ij}}$$

Esta función es no lineal, pero se puede linealizar mediante el uso del logaritmo natural y sus propiedades con las funciones exponenciales:

$$\text{Ln}(a^x) = x \cdot \text{Ln}(a) \text{ y } \text{Ln}(a \cdot b) = \text{Ln}(a) + \text{Ln}(b)$$

De esta forma, se minimizaría $\text{Ln}(z)$ en lugar de z . Esto es posible, puesto que el logaritmo de una función no afecta su forma, dejando los puntos máximos y mínimos en el mismo lugar, afectando solo la escala, como se muestra en la Figura 4.2. En ella se presenta la función polinómica $F(x) = (x-6)^4 + 3 \cdot (x-6)^3 - 9 \cdot (x-6)^2 - 23 \cdot (x-6) + 60$ (con el eje vertical a la izquierda) y su logaritmo ($\text{Ln}(F(x))$) (con el eje vertical a la derecha). En ambos casos el mínimo se encuentra en el punto $x = 7,853$.

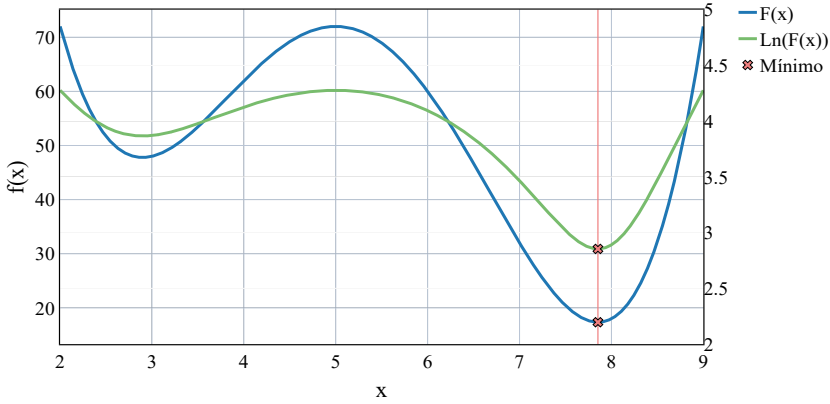


Figura 4.2: Efecto del logaritmo natural en una función polinómica.

$$\text{Minimizar } \ln(z) = \sum_{i \in I} \sum_{j \in J} x_{ij} \cdot \ln(pF_{ij}) \quad (4.20)$$

Sujeto a:

$$\sum_{j \in J} x_{ij} \leq Cap_i, \quad \forall i \in I \quad (4.21)$$

$$\sum_{i \in I} \sum_{j \in J} \left(\frac{2 \cdot Dis_j}{kmGalon_i} + 150 \right) \cdot x_{ij} \leq Gas \quad (4.22)$$

$$\sum_{i \in I} x_{ij} \leq y_j \cdot M, \quad \forall j \in J \quad (4.23)$$

$$\sum_{i \in I} x_{ij} \geq y_j, \quad \forall j \in J \quad (4.24)$$

$$\sum_{j \in J} y_j \geq 2 \quad (4.25)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (4.26)$$

$$y_j \in \{0, 1\}, \quad \forall j \in J \quad (4.27)$$

La expresión (4.20) es la función objetivo del problema, que busca minimizar el logaritmo de la probabilidad de fracaso de la misión.

La restricción (4.21) asegura que no se usen más bombarderos de los que hay disponibles de cada tipo. La restricción (4.22) garantiza que el uso de combustible en los bombardeos sea menor o igual que el disponible en

la base central (teniendo en cuenta el combustible de ida y regreso). Las restricciones (4.23) y (4.24) establecen la relación entre x_{ij} y y_j . La restricción (4.23) asegura que, de existir un valor positivo para x_{ij} , la variable y_j tomará el valor de 1. La restricción (4.24) asegura que de no existir un valor para x_{ij} , la variable y_j tome un valor de 0. La restricción (4.25) obliga atacar a mínimo 2 plantas enemigas. Finalmente, las expresiones (4.26) y (4.27) definen la naturaleza de las variables.

El Algoritmo 35 contiene el código del modelo en AMPL. La solución óptima sugiere enviar 44 bombarderos pesados y 31 livianos a atacar la planta 4, y enviar 2 aviones livianos a la planta 2. Con esto se obtiene un valor de $z = -19,9242$ en la función objetivo, que se traduce en una probabilidad de fracaso de $e^z = 2,2235 \times 10^{-9}$.

Algoritmo 35: archivo Guerra2.mod

```

1: set I := {1,2};
2: set J := {1..4};
3: param Gas;
4: param kmGalon{I};
5: param Cap{I};
6: param Dis{J};
7: param pF{I,J};
8: param M;
9: var x{I,J} >=0 integer;
10: var y{J} binary;
11: minimize z: sum{i in I, j in J} log(pF[i,j])*x[i,j];
12: s.t.
13: cap{i in I}: sum{j in J} x[i,j] <= Cap[i];
14: Gaso: sum{i in I, j in J} (2*Dis[j]/kmGalon[i]+150)*x[i,j] <= Gas;
15: rela1{j in J}: sum{i in I} x[i,j] <= y[j]*M;
16: rela2{j in J}: sum{i in I} x[i,j] >= y[j];
17: min2: sum{j in J} y[j] >= 2;

```

4.4.2 Variación propuesta para el estudiante

La solución obtenida es óptima pero quizá muy riesgosa. Es decir, se envían 75 bombarderos a la planta 4 y solo 2 bombarderos a atacar la planta 2. Reformule este problema balanceando el número de bombarderos que se envían a las plantas elegidas. ¿Qué tipo de restricciones de podrían usar para este fin?

4.5 Problema del plan agregado

La empresa japonesa *Arigato* se encuentra en un gran dilema a nivel corporativo, puesto que a cargo se encuentran el gerente general Lee y el accionista Shoung, ambos con pensamientos operativos muy diferentes. Dada la experiencia de esta empresa, Lee piensa que *Arigato* debe centrarse en la fabricación de artículos para *gamers* (piezas de computador de alto rendimiento y accesorios). Por su parte, Shoung está convencido de que el futuro son los equipos de oficina. Dos familias de productos muy diferentes.

La compañía tiene 6 productos disponibles en su portafolio de servicio (3 para el mercado *gamer* y 3 para el mercado de *oficina*). Las utilidades en USD pueden verse en la Tabla 4.9.

1. Silla <i>gamer</i> (\$27)	2. Silla <i>oficina</i> (\$38)
3. Mouse <i>gamer</i> (\$36)	4. Mouse <i>oficina</i> (\$28)
5. Teclado <i>gamer</i> (\$19)	6. Teclado <i>oficina</i> (\$39)

Tabla 4.9: Productos disponibles para producir y sus utilidades.

La fabricación de estos artículos se hace en 8 estaciones de trabajo. Por supuesto, no todos los artículos necesitan de todas las estaciones. Los tiempos de producción (en horas) se muestran la Tabla 4.10 (un tiempo de cero horas indicaría que el producto no requiere del uso de esa estación de trabajo).

	[h]	Estaciones de trabajo							
		1	2	3	4	5	6	7	8
Producto	1	1	1	2	3	2	3	1	1
	2	1	1	1	0	2	0	1	2
	3	1	1	2	3	1	1	3	0
	4	3	2	2	0	1	3	3	0
	5	0	0	1	2	0	1	0	1
	6	1	0	3	3	2	2	3	3

Tabla 4.10: Horas requeridas por estación de trabajo para la fabricación de los productos.

Las horas disponibles por mes de trabajo dependen del número de empleados y de los días laborables (cada uno trabaja 8 horas día). Por políticas internas, en los próximos meses no se contratará ni se despedirá a nadie. Actualmente, la empresa cuenta con 10 empleados. La producción de todos los artículos no debe superar la capacidad disponible. Además, existe la posibilidad de guardar la producción sobrante de un mes en inventario, para un uso futuro, pero a un costo mensual variable, como se presenta en la Tabla 4.11.

	Días laborables por mes	Costo de inventario $\left[\frac{\$}{und \cdot mes}\right]$
Mes	1	27
	2	24
	3	23
	4	24
	5	26
	6	20

Tabla 4.11: Días laborables y costo de inventario por mes.

Respecto a la demanda, la empresa *Arigato* ha contratado a una prestigiosa empresa para realizar pronósticos de la demanda máxima de cada tipo de producto para los próximos 6 meses. Los resultados se muestran en la Tabla 4.12.

		Mes					
	[und]	1	2	3	4	5	6
Producto	1	34	246	40	375	81	255
	2	87	107	57	157	44	398
	3	52	128	87	400	40	469
	4	32	406	39	333	34	418
	5	92	144	37	207	31	440
	6	33	284	73	448	57	182

Tabla 4.12: Demanda máxima de cada producto dependiente del mes.

El problema radica en que Lee y Shoung no están de acuerdo en qué artículos fabricar. Producir ambos artículos no es posible, ya que el presupuesto para

publicidad solo permitiría enfocarse en un mercado: *gamer* u oficina. Por tanto, solo se puede fabricar una familia de artículos. Para garantizar las ventas de la familia de productos seleccionada, todos los artículos de una misma familia se deben producir (y vender) al menos 10 productos.

Al no considerar la política de pedidos atrasados¹⁰, ¿qué familia de artículos y qué cantidad de ellos se deben fabricar, almacenar y vender cada mes para maximizar las utilidades de la empresa? Formular un modelo de programación lineal que dé solución a este problema.

4.5.1 Solución

Definición de conjuntos

- A : productos completos $\{1, \dots, 6\}$.
- T : meses $\{1, \dots, 6\}$.
- J : estaciones de trabajo $\{1, \dots, 8\}$.
- AA : solo productos de oficina (pares) $\{2, 4, 6\}$.

Definición de parámetros

- Dem_{it} : demanda máxima del producto $i \in A$ en el mes $t \in T$.
- Cap : capacidad diaria en horas disponibles. Se puede calcular como $8 * 10$, donde 8 son las horas disponibles al día, y 10 los empleados $[h/d]$.
- a_{ij} : horas requeridas de la estación de trabajo $j \in J$ para fabricar el producto $i \in A$.
- h_i : costo por mantener en inventario una unidad del producto $i \in A$ durante un mes.
- r_i : utilidad asociada a la venta del producto $i \in A$.
- Dia_t : días laborables del mes $t \in T$.

¹⁰Consiste en hacer ventas sin tener el artículo, y entregarlos en un futuro.

Definición de variables de decisión

- x_{it} : cantidad de productos tipo $i \in A$ producidos en el mes $t \in T$ [und].
- I_{it} : cantidad de productos tipo $i \in A$ almacenado en el mes $t \in T$ [und]. Tener en cuenta que el inventario inicial ($t = 0$) de cualquier producto es cero.
- s_{it} : cantidad de productos tipo $i \in A$ vendidos en el mes $t \in T$ [und].
- $y : \begin{cases} 1, & \text{si se produce solo la familia } \textit{gamer} \\ 0, & \text{si se produce solo la familia } \textit{oficina} \end{cases}$

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in A} \sum_{t \in T} (s_{it} \cdot r_i - h_i \cdot I_{it}) \quad (4.28)$$

Sujeto a:

$$s_{it} \leq Dem_{it}, \quad \forall i \in A, t \in T \quad (4.29)$$

$$\sum_{j \in J} \sum_{i \in AA} x_{it} \cdot a_{ij} \leq Cap \cdot Dia_t \cdot y, \quad \forall t \in T \quad (4.30)$$

$$\sum_{j \in J} \sum_{i \in A \setminus AA} x_{it} \cdot a_{ij} \leq Cap \cdot Dia_t \cdot (1 - y), \quad \forall t \in T \quad (4.31)$$

$$x_{it} \geq 10 \cdot y, \quad \forall t \in T, \forall i \in AA \quad (4.32)$$

$$x_{it} \geq 10 \cdot (1 - y), \quad \forall t \in T, \forall i \in A \setminus AA \quad (4.33)$$

$$I_{i,(t-1)} + x_{it} = s_{it} + I_{it}, \quad \forall t \in T, \forall i \in A \quad (4.34)$$

$$I_{i0} = 0, \quad \forall i \in A \quad (4.35)$$

$$x_{it}, I_{it}, s_{it} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in A, \forall t \in T \quad (4.36)$$

$$y \in \{0, 1\} \quad (4.37)$$

La expresión (4.28) es la función objetivo del problema, que busca maximizar las utilidades generadas por las ventas.

La expresión (4.29) garantiza no sobrepasar la demanda máxima de cada

producto en cada mes. Las expresiones (4.30) y (4.31) establecen la capacidad máxima de las horas disponibles para la producción de cada mes (teniendo en cuenta el uso de las estaciones de trabajo de cada producto). Las restricciones (4.32) y (4.33) obligan la producción de mínimo 10 unidades de la familia de productos seleccionada.

La restricción (4.34) establece el balance del inventario, es decir, que la producción excedente de cada producto en cada mes se almacene para un uso posterior. La restricción (4.35) fija el valor del inventario inicial de cada producto en cero. Finalmente, las restricciones (4.36) y (4.37) exponen la naturaleza de las variables.

El Algoritmo 36 contiene el código del modelo en AMPL. La solución obtenida por Gurobi en AMPL se puede apreciar en la Tabla 4.13. Con un valor de utilidad máximo de 42092 USD.

Algoritmo 36: archivo Arigato.mod

```

1: set A := {1..6};
2: set T := {1..6};
3: set J := {1..8};
4: set AA := {1..6 by 2};
5: param Dem{A,T};
6: param Cap = 8*10;
7: param a{A,J};
8: param h{A};
9: param r{A};
10: param Dia {T};
11: var x{A,T} >= 0 integer;
12: var I{A,{0} union T} >= 0 integer;
13: var s{A,T} >= 0 integer;
14: var y binary;
15: maximize z: sum{i in A, t in T} (s[i,t]*r[i] - h[i]*I[i,t]);
16: s.t.
17: r1{i in A, t in T}: s[i,t] <= Dem[i,t];
18: r2_1{t in T}: sum{j in J, i in AA} x[i,t]*a[i,j] <= Cap*Dia[t]*y;
19: r2_2{t in T}: sum{j in J, i in A diff AA} x[i,t]*a[i,j] <=
    Cap*Dia[t]*(1-y);
20: r3_1{t in T, i in AA}: x[i,t] >= 10*y;
21: r3_2{t in T, i in A diff AA}: x[i,t] >= 10*(1-y);
22: r4{t in T, i in A}: I[i,t-1] + x[i,t] = s[i,t] + I[i,t];
23: r5{i in A}: I[i,0] = 0;

```

Producto (i)	Mes (t)	Cantidad (x_{it})	Inventario (I_{it})	Ventas (s_{it})
2	1	91	4	87
2	2	106	3	107
2	3	70	16	57
2	4	201	60	157
2	5	221	237	44
2	6	161	0	398
4	1	10	0	10
4	2	11	0	11
4	3	10	0	10
4	4	10	0	10
4	5	10	0	10
4	6	10	0	10
6	1	76	43	33
6	2	54	0	97
6	3	67	0	67
6	4	10	0	10
6	5	10	0	10
6	6	10	0	10

Tabla 4.13: Plan de producción óptimo para los siguientes 6 meses.

4.6 Problema del arancel (adaptado de [3])

Tres inversionistas disponen de un presupuesto de 4, 8 y 3 millones de pesos para comprar productos en Estados Unidos para vender en Colombia. Según sus investigaciones de mercado, existen tres alternativas de inversión: comprar televisores, equipos de sonido y hornos microondas; sus costos unitarios son: 380 000 COP, 195 000 COP y 47 000 COP, respectivamente. Estos artículos se pueden vender a: 460 000 COP, 260 000 COP y 70 000 COP, respectivamente. Como información adicional para el viaje, se sabe que cada televisor pesa 40 kg, cada equipo de sonido 28 kg, y cada horno 10 kg.

Las aerolíneas solo permiten el transporte de 50 kg sin recargo. Sin embargo, se puede pagar una tarifa de 2000 COP por cada kilo adicional. Además, la aduana cobra un arancel del 15 % sobre el precio total de compra, siempre y cuando dicho valor supere los 3000000 COP. Formular un modelo de programación lineal para determinar el dinero a invertir y los artículos a comprar por cada inversionista para maximizar las utilidades netas.

4.6.1 Solución

Definición de conjuntos

- I : inversionista $\{1,2,3\}$.
- J : mercancía $\{1,2,3\}$, para televisores, equipos de sonido y hornos microondas.

Como en otros casos, se podría usar un mismo conjunto para representar tanto a los inversionistas como a los productos, porque numéricamente son iguales. Sin embargo, se tomarán de forma separada para centrar la atención del lector en la restricción de la aduana.

Definición de parámetros

- C_j : costo unitario del producto $j \in J$.
- V_j : precio de venta del producto $j \in J$.
- PE_j : peso del producto $j \in J$.
- $PRES_i$: presupuesto excedente de 3 millones del inversionista $i \in I$. Este parámetro solo almacena el excedente de 3 millones para cada inversionista i ; así: $PRES_1 = 4000000 - 3000000 = 1000000$, $PRES_2 = 5000000$ y $PRES_3 = 0$.
- M : valor muy grande en comparación con los parámetros del modelo.

Definición variables de decisión

- x_{ij} : cantidad de producto $j \in J$ comprado por el inversionista $i \in I$.
- y_i : $\begin{cases} 1, & \text{si el inversionista } i \in I \text{ invierte más de 3 millones} \\ 0, & \text{en otro caso} \end{cases}$
- A_i : impuesto a pagar por el inversionista $i \in I$ cuando se excede de los 3000000 COP. El porqué de su creación se detalla en las píldoras informativas 4.6 y 4.7.
- SP_i : excedente de peso del inversionista $i \in I$.

Píldora informativa 4.6: Linealizar restricciones

En el este problema, el costo del arancel para cada inversionista i se puede estimar mediante la siguiente ecuación:

$$0,15 \cdot \sum_{j \in J} x_{ij} \cdot C_j, \quad \forall i \in I$$

Esta expresión depende directamente de la variable de decisión x_{ij} , pero solo se debe cobrar en caso de gastar más de 3000000 COP. Para esto se definió la variable binaria y_i , que toma el valor de 1 únicamente cuando se supera dicho valor y es 0 en caso contrario. Así, el impuesto total a considerar en la función objetivo debería estar multiplicado por la variable binaria:

$$0,15 \cdot \sum_{j \in J} x_{ij} \cdot C_j \cdot y_i, \quad \forall i \in I$$

Sin embargo, **no es una ecuación lineal**. Por este motivo, se ha definido la variable A_i , para linealizar dicha expresión con ayuda del método de la gran M .

Píldora informativa 4.7: Linealizar la multiplicación de una variable continua y una variable binaria

Si un modelo tiene la siguiente restricción **no lineal**:

$$F = x \cdot y$$

Donde F , $x \geq 0$ son variables continuas y y es una variable binaria. Además, se asume que x tiene una cota superior (valor máximo) K , por tanto, todo x es menor o igual a K . **F se puede linealizar de dos formas:**

1. Definir tres restricciones:

$$r1 : F \geq x - (1 - y) \cdot K,$$

$$r2 : F \leq x,$$

$$r3 : F \leq K \cdot y$$

Luego, las dos posibilidades son:

- Cuando $y = 1$

$$r1 : F \geq x - 0,$$

$$r2 : F \leq x,$$

$$r3 : F \leq K \cdot 1$$

Por tanto, $F = x$

- Cuando $y = 0$

$$r1 : F \geq x - K,$$

$$r2 : F \leq x,$$

$$r3 : F \leq 0$$

Por tanto, $F = 0$

2. Definir una restricción y modificar la función objetivo:

$$\text{Maximizar } z = (\dots) - F$$

Sujeto a:

$$r1 : F \geq x - (1 - y) \cdot K$$

Luego, las dos posibilidades son:

- Cuando $y = 1$

Maximizar $z = (\dots) - F$

Sujeto a:

$$r1 : F \geq x - 0$$

Como F está restando en la función objetivo, el modelo tratará de tomar el menor valor posible. **Por tanto**, $F = x$.

- Cuando $y = 0$

Maximizar $z = (\dots) - F$

Sujeto a:

$$r1 : F \geq x - K$$

Como F está restando en la función objetivo, el modelo tratará de tomar el menor valor posible, y teniendo en cuenta que $x - K \leq 0$, $r1$ no estaría activa. **Por tanto**, $F = 0$.

Cuando el objetivo es minimizar, el procedimiento es similar, pero en la función objetivo se suma F , en lugar de restarla.

En este problema se usará el segundo enfoque.

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in I} \sum_{j \in J} x_{ij} \cdot (V_j - C_j) - \left(\sum_{i \in I} 2\,000 \cdot SP_i + \sum_{i \in I} A_i \right) \quad (4.38)$$

Sujeto a:

$$\sum_{j \in J} x_{ij} \cdot PE_j \leq 50 + SP_i, \quad \forall i \in I \quad (4.39)$$

$$\sum_{j \in J} x_{ij} \cdot C_j \leq 3\,000\,000 + y_i \cdot PRES_i, \quad \forall i \in I \quad (4.40)$$

$$A_i \geq 0,15 \cdot \sum_{j \in J} x_{ij} \cdot C_j - (1 - y_i) \cdot M, \quad \forall i \in I \quad (4.41)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (4.42)$$

$$y_i \in \{0, 1\}, \quad \forall i \in I \quad (4.43)$$

La expresión (4.38) representa la función objetivo, que busca maximizar las utilidades de las ventas de los productos.

La restricción (4.39) limita el peso que puede llevar cada inversionista en los vuelos sin un cobro adicional y cuantifica el peso extra en caso de existir. La expresión (4.40) tiene dos funciones: limitar las compras al presupuesto disponible y garantizar la dinámica de la variable binaria y_i . La expresión (4.41) garantiza que, si $y_i = 1$, se cobre el impuesto correspondiente, y en caso de que $y_i = 0$, el cobro sea 0. Finalmente, las restricciones (4.42) y (4.43) muestran la naturaleza de las variables.

En la restricción (4.40) se podría pensar que la variable y_i podría valer 1 sin necesidad de usar más de los 3000000 COP. Sin embargo, el modelo tratará de buscar que las variables y_i sean cero, puesto que esta es la única forma de eliminar el costo de arancel A_i , el que, además, se resta en la función objetivo para maximizar la utilidad.

El Algoritmo 37 contiene el código del modelo algebraico en AMPL. Según la solución obtenida por Gurobi, cada inversionista debe adquirir 63 hornos microondas. De esta forma, cada inversionista invierte 2961000 COP, maximizando las ganancias con un valor de 867000 COP y sin tener que pagar el impuesto del arancel.

Algoritmo 37: archivo Arancel.mod

```

1: set I := {1..3};
2: set J := {1..3};
3: param C{J};
4: param V{J};
5: param PE{J};
6: param PRESU{I};
7: param M = 10000000;
8: var x{I,J} >= 0 integer;
9: var y{I} binary;
10: var A{I} >= 0;
11: var SP{I} >= 0;
12: maximize z: sum{i in I, j in J} x[i,j]*(V[j]-C[j]) - (sum
    {i in I} 2000*SP[i] + sum{i in I} A[i]);
13: s.t.
14: rPeso{i in I}: sum{j in J} x[i,j]*PE[j] <= 50 + SP[i];
15: presu{i in I}: sum{j in J} x[i,j]*C[j] <= 3000000 + y[i]
    *PRESU[i];
16: impue{i in I}: A[i] >= sum{j in I} x[i,j]*C[j]*0.15 -
    (1-y[i])*M;

```

4.6.2 Variación propuesta para el estudiante

Si se considera el costo del arancel como un valor constante, ¿cómo se debe reformular el problema?, ¿cambia la definición de las variables de decisión?

4.7 Problema de programación de actividades (*job shop problem*)

Una empresa de desarrollo de software cuenta con 5 programadores especializados en cinco áreas de diseño: 1. seguridad (E_1), 2. páginas web (E_2), 3. bases de datos (E_3), 4. interfaces (E_4), y 5. motor de procesamiento (E_5). Dentro de su portafolio de servicio, la empresa ofrece un diseño e implementación de proyectos personalizados para las necesidades de cada cliente. Cada proyecto debe pasar por cada programador especializado, pero debido a flexibilidad de los pedidos, cada proyecto puede tener una secuencia diferente.

Actualmente, la empresa tiene que planificar el desarrollo de 6 proyectos. En la Tabla 4.14 se muestran las secuencias específicas que los proyectos deben seguir para su desarrollo. Estas secuencias pueden ser inmediatas o no; por ejemplo, si un proyecto debe pasar por los programadores 1 y 2, el segundo programador puede iniciar el procesamiento del proyecto en cualquier momento, luego del tiempo de finalización del primer programador.

Adicionalmente, el tiempo de procesamiento de cada proyecto, con relación a cada programador especialista, puede ser diferente. Estos datos, en horas, se detallan en la Tabla 4.15.

		Proyecto					
		1	2	3	4	5	6
Orden	1.°	E_1	E_2	E_3	E_3	E_4	E_5
	2.°	E_3	E_3	E_1	E_2	E_3	E_1
	3.°	E_5	E_5	E_5	E_4	E_1	E_4
	4.°	E_2	E_4	E_4	E_1	E_2	E_3
	5.°	E_4	E_1	E_2	E_5	E_5	E_2

Tabla 4.14: Secuencias para el desarrollo de los proyectos.

		Proyecto					
		1	2	3	4	5	6
Especialistas	E_1	6h	1h	10h	2h	4h	2h
	E_2	6h	1h	2h	6h	9h	6h
	E_3	1h	4h	3h	3h	8h	4h
	E_4	10h	4h	10h	8h	6h	4h
	E_5	1h	2h	9h	6h	10h	5h

Tabla 4.15: Tiempos de procesamiento de los proyectos dependientes del programador.

¿Cómo se deben secuenciar los proyectos en cada especialista teniendo en cuenta sus requerimientos particulares, para lograr finalizarlos todos lo antes posible?

4.7.1 Solución

Definición de conjuntos

- I : especialistas $\{1, \dots, m\}$, $m = 5$.
- J : proyectos $\{1, \dots, n\}$, $n = 6$.

Definición de parámetros

- P_{ij} : tiempo de procesamiento del proyecto $j \in J$ con el especialista $i \in I$ [h].

Píldora informativa 4.8: Posición de los subíndices como notación

En algunos modelos de programación lineal se pueden encontrar variables o parámetros cuyos **subíndices están como exponentes** (superíndices), pero tienen **la misma función que un subíndice** tradicional. Por ejemplo, se podría definir una variable así:

- x_{ij}^k : número de aviones que viajan de la ciudad i a la ciudad j , usando el combustible tipo k .

El propósito de estos superíndices es **estético**. Se suelen agrupar subíndices estrechamente relacionados en la parte superior o inferior de las variables.

- θ_h^j : secuencia del proyecto $j \in J$ según el orden $h \in I$. El orden hace referencia a **la posición en la secuencia de un especialista**. En este parámetro se almacenará la información de la Tabla 4.14. Nótese que el subíndice de orden h también pertenece al conjunto de especialistas I , porque existen tantas posiciones (en el orden de procesamiento) como especialistas.

Por ejemplo, como el proyecto 1 tiene el siguiente orden de especialistas 1, 3, 5, 2 y 4, el valor de este parámetro es: $\theta_1^1 = 1$, $\theta_2^1 = 3$, $\theta_3^1 = 5$, $\theta_4^1 = 2$, y $\theta_5^1 = 4$.

- M : valor de penalidad muy grande en comparación con el valor de los demás parámetros.

Definición de variables de decisión

- x_{ij} : tiempo de inicio del proyecto $j \in J$ con el especialista $i \in I$ $[h]$.
- $z_{ijk} : \begin{cases} 1, \text{ si el proyecto } j \in J \text{ es predecesor de } k \in J \text{ con el especialista} \\ i \in I, j \neq k \\ 0, \text{ en otro caso} \end{cases}$

Por ejemplo, la variable $z_{321} = 1$ implica que el proyecto 2 inicia y finaliza antes del proyecto 1 (es predecesor), en el orden de trabajos con el especialista 3.

- $C_{\text{máx}}$: tiempo máximo de finalización de los proyectos (*makespan*).

Modelo algebraico de programación lineal

$$\text{Minimizar } z = C_{\text{máx}} \quad (4.44)$$

Sujeto a:

$$x_{(\theta_h^j), (j)} \geq x_{(\theta_{h-1}^j), (j)} + P_{(\theta_{h-1}^j), (j)}, \quad \forall j \in J, \forall h \in \{2, \dots, m\} \quad (4.45)$$

$$x_{ij} \geq x_{ik} + P_{ik} - M \cdot z_{ijk}, \quad \forall j, k \in J : j < k, \forall i \in I \quad (4.46)$$

$$x_{ik} \geq x_{ij} + P_{ij} - M \cdot (1 - z_{ijk}), \quad \forall j, k \in J : j < k, \forall i \in I \quad (4.47)$$

$$C_{\text{máx}} \geq x_{(\theta_m^j), (j)} + P_{(\theta_m^j), (j)}, \quad \forall j \in J \quad (4.48)$$

$$C_{\text{máx}} \in \mathbb{Z}^+ \cup \{0\} \quad (4.49)$$

$$x_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (4.50)$$

$$z_{ijk} \in \{0, 1\}, \quad \forall i \in I, \forall j, k \in J \quad (4.51)$$

La expresión (4.44) es la función objetivo del problema, que busca minimizar el tiempo de finalización de todos los proyectos (también conocido como *makespan*).

La restricción (4.45) establece que un proyecto solo se puede ejecutar en un tiempo mayor o igual al tiempo de inicio del proyecto predecesor correspondiente, más su tiempo de procesamiento, es decir, cuando se termina de procesar. Esto aplica a cada orden de trabajo de los especialistas.

Las expresiones (4.46) y (4.47) impiden que más de un proyecto se pueda ejecutar simultáneamente, usando *restricciones disyuntivas*¹¹ (para mayor detalles consultar la Píldora informativa 4.9), que se activan y desactivan mediante el uso de la gran M .

Píldora informativa 4.9: Restricciones disyuntivas

Son restricciones que representan **dos situaciones diferentes pero que son incompatibles** entre sí. Por ejemplo, si un proyecto A inicia antes que un proyecto B , no es posible que B inicie antes de A . Además, existen unas condiciones cuando A precede a B y otras diferentes cuando B precede a A . Las restricciones disyuntivas permiten modelar estas situaciones.

Particularmente, en este problema las restricciones (4.46) y (4.47) son disyuntivas y consideran estas dos situaciones:

¹¹Situación donde se debe elegir entre dos elementos o soluciones diferentes.

1. Cuando $z_{ijk} = 1$, es decir, cuando un proyecto j es predecesor de uno k con un programador especialista i , la restricción (4.46) se desactiva, mientras que la restricción (4.47) queda activa:

$$x_{ij} \geq x_{ik} + P_{ik} - M \cdot 1 \quad \rightarrow \quad x_{ij} \geq -M \quad \text{Inactiva}$$

$$x_{ik} \geq x_{ij} + P_{ij} - M \cdot (1 - 1) \quad \rightarrow \quad x_{ik} \geq x_{ij} + P_{ij} \quad \text{Activa}$$

La restricción activa obliga a que el proyecto k inicie luego del proyecto j , pues j es predecesor de k .

2. Cuando $z_{ijk} = 0$, es decir, cuando un proyecto j no es predecesor de uno k con un programador especialista i , la restricción (4.46) queda activa, mientras que la restricción (4.47) se desactiva:

$$x_{ij} \geq x_{ik} + P_{ik} - M \cdot 0 \quad \rightarrow \quad x_{ij} \geq x_{ik} + P_{ik} \quad \text{Activa}$$

$$x_{ik} \geq x_{ij} + P_{ij} - M \cdot (1 - 0) \quad \rightarrow \quad x_{ik} \geq -M \quad \text{Inactiva}$$

La restricción activa obliga a que el proyecto j inicie luego del proyecto k , pues en este caso j **no** precede a i .

Por otra parte, estas restricciones están definidas con $j \leq k$, ambas pertenecientes a J . Esto se debe a que solo una combinación de j y k proporciona toda la información necesaria tanto de j , k y j . Por ejemplo, $z_{ijk} = 0$ implica que j **no** precede a k y, por tanto, es k quien precede a j . De forma análoga ocurre con $z_{ijk} = 1$, que implica que j sí precede a k entonces, k no precede a j .

La restricción (4.48) define el denominado *makespan* como la mayor duración del último especialista que debe recorrer cada proyecto. Finalmente, las expresiones (4.49) a (4.51) exponen la naturaleza de las variables de decisión.

El Algoritmo 38 contiene el código del modelo en AMPL. La solución obtenida por Gurobi en AMPL se puede apreciar en la Figura 4.3. En ella se pueden identificar los tiempos de inicio y finalización de todos los proyectos con cada programador especialista, alcanzando un *makespan* de 45 horas.

Algoritmo 38: archivo JobShop.mod

```

1: param m = 5;
2: param n = 6;
3: param M = 10000;
4: set I := {1..m};
5: set J := {1..n};
6: param P{I,J};
7: param theta{I,J};
8: var x{I,J} >= 0;
9: var z{I,J,J} binary;
10: var Cmax >= 0;
11: minimize makespan: Cmax;
12: s.t.
13: Prec{j in J, h in {2..m}}: x[theta[h,j],j] >= x[theta[h-1,j],j] + P[theta[h-1,j],j];
14: Pj1{j in J, k in J, i in I: j < k}: x[i,j] >= x[i,k] + P[i,k] - M*z[i,j,k];
15: Pj2{j in J, k in J, i in I: j < k}: x[i,k] >= x[i,j] + P[i,j] - M*(1-z[i,j,k]);
16: FO{j in J}: Cmax >= x[theta[m,j],j] + P[theta[m,j],j];

```

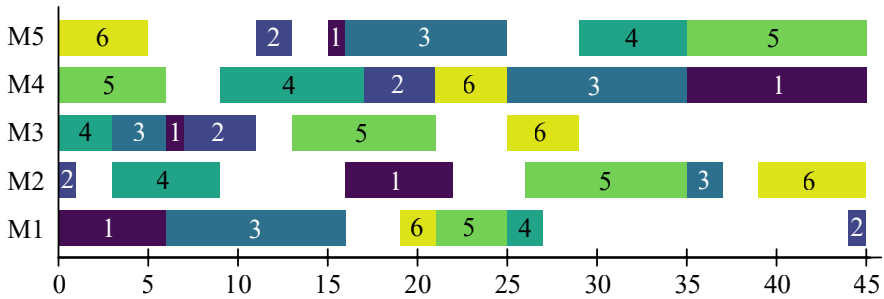


Figura 4.3: Diagrama de *Gantt* de la solución encontrada.

4.8 Problema de las divisas

Usted se aventura en el extraordinario mundo del mercado bursátil (la bolsa de valores); específicamente quiere participar en el intercambio de divisas¹². Para esto usted crea la empresa *La Divisa con Prisa*, que tiene como propósito asesorar a sus clientes en la compra y ventas de

¹²Moneda de cambio.

dólares (USD), pesos colombianos (COP) y euros (EUR). Su primer cliente dispone de un capital inicial de 100 000 COP, y quiere realizar una inversión para comprar EUR. El tiempo que tiene planeado para hacer las inversiones es de 4 días. Dicha inversión consiste en comprar COP, EUR o USD cada día a la tasa de cambio correspondiente (cambia cada día), con la finalidad de obtener la mayor cantidad de EUR al finalizar el cuarto día.

Una condición especial del mercado establece que si un día no se invierte en ninguna divisa, se le multa con un 5 % del total de capital no invertido (en la moneda correspondiente). En la Tabla 4.16 se aprecian las tasas de cambio de los siguientes 4 días.

¿De qué forma debe invertir el dinero disponible cada día para maximizar la cantidad de euros al finalizar el cuarto día?

Día 1	COP	USD	EUR
COP	*	0,00032	0,00028
USD	3 123	*	0,88
EUR	3 550	1,1363	*

Día 2	COP	USD	EUR
COP	*	0,00032	0,00028
USD	3 200	*	0,8
EUR	3 541	1,40	*

Día 3	COP	USD	EUR
COP	*	0,00058	0,00045
USD	3 123	*	0,9
EUR	3 541	1,140	*

Día 4	COP	USD	EUR
COP	*	0,00032	0,00028
USD	3 250	*	0,75
EUR	3 400	1,1363	*

Tabla 4.16: Tasas de cambio entre divisas.

4.8.1 Solución

Definición de conjuntos

- I : tipos de divisa $\{1, 2, 3\}$, para COP, USD y EUR, respectivamente.
- T : días $\{1, \dots, 5\}$. Se incluye el día 5 como finalización del día 4.

Definición de parámetros

- r_{ijt} : factor de conversión de la divisa $i \in I$ a la divisa $j \in I$ en el día $t \in T$.

Píldora informativa 4.10: Completar parámetros

A veces, los parámetros mostrados en tablas o figuras no están completos, pero con la información del enunciado se pueden complementar. Por ejemplo, en este problema, para modelar la multa del 5 % en caso de no invertir en un día, se tomará como factor de conversión a r_{ijt} con $i = j$, por tanto, se reemplazarán los asteriscos de la Tabla 4.16 por 0,95. Así, $r_{221} = 0,95$ indica que no invertir dólares el primer día ocasiona una pérdida del 5 %.

Definición de variables de decisión

- x_{ijt} : cantidad de divisa $i \in I$ para comprar divisa $j \in I$ en el día $t \in T$.

Modelo algebraico de programación lineal

$$\text{Maximizar } z = \sum_{i \in I} x_{3i5} \quad (4.52)$$

Sujeto a:

$$\sum_{i \in I} x_{1i1} \leq 100\,000 \quad (4.53)$$

$$\sum_{i \in I} x_{ji1} = 0, \quad \forall j \in \{2, 3\} \quad (4.54)$$

$$\sum_{m \in I} x_{mjt} \cdot r_{mjt} = \sum_{m \in I} x_{(j),(m),(t+1)}, \quad \forall j \in I, \forall t \in T \setminus \{5\} \quad (4.55)$$

$$x_{ijt} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in I, \forall t \in T \quad (4.56)$$

La expresión (4.52) es la función objetivo del problema, que busca maximizar los EUR que se podrían gastar al inicio del día 5. Nótese que sería equivalente a maximizar la cantidad de EUR que se obtendrán al final del día 4 ($\sum_{m \in I} x_{m34} \cdot r_{m34}$).

Las expresiones (4.53) y (4.54) establecen que al principio del día 1 se cuenta

solo con 100 000 COP. La restricción (4.55) garantiza que el dinero gastado de la divisa j en el periodo $t + 1$ es igual a toda la divisa j obtenida desde una divisa m en el periodo t . Finalmente, la restricción (4.56) representa la no negatividad de las variables.

El Algoritmo 39 contiene el código del modelo en AMPL. La solución obtenida por Gurobi en AMPL consiste en usar los 100 000 COP para comprar 32 USD el día 1. El día 2 usar los USD para comprar 102 400 COP. El día 3 usar todos los COP para comprar 59,392 USD. Finalmente, el día 4 usar todos los USD para comprar 44,544 EUR al final del día 4.

Algoritmo 39: archivo Divisas.mod

```

1: set I := {1..3};
2: set T := {1..5};
3: param r{I,I,T};
4: var x{I,I,T} >= 0;
5: maximize z: sum{i in I} x[3,i,5];
6: s.t.
7: r0: sum{i in I} x[1,i,1] <= 100000;
8: r1{j in {2,3}}: sum{i in I} x[j,i,1] = 0;
9: r2{j in I, t in T diff {5}}: sum{m in I} x[m,j,t]*r[m,j,t] =
    sum{m in I} x[j,m,t+1];

```

4.8.2 Variación propuesta para el estudiante

Asuma que las inversiones solo se pueden hacer con el monto total disponible. Es decir, que si se tienen 100 USD, se deben usar todos los 100 en caso de querer invertir. Adapte el modelo bajo esta nueva condición.

4.9 Problema de los muelles

Una compañía que presta servicio de almacenamiento a distribuidores multinacionales está enfrentando la problemática de ubicación de productos. La bodega principal de la compañía tiene una puerta de recepción con 6 muelles, donde se deben almacenar sus 10 productos principales. La compañía ha calculado el costo de manejo e inventario mensual en USD por cada 100 unidades de producto en cada muelle (Tabla 4.17).

Cada muelle tiene una capacidad de almacenamiento según el tipo de producto. Por otra parte, la empresa ha firmado un contrato con unas distribuidoras y debe cumplir la demanda mensual especificada. Esta información se expone en la Tabla 4.18.

Con base en la experiencia de la gerencia, se sabe que el jabón y la lejía no pueden estar almacenados junto con la leche y los jugos, porque puede existir contaminación cruzada. Además, normalmente se recolectan las linternas junto con las baterías, de esta manera se busca que estos productos queden juntos y en igual cantidad para facilitar la recolección. Finalmente, es deseable (pero no obligatorio) que las papas queden junto al aceite en mismas cantidades, dado que son productos complementarios.

Productos	Muelle 1	Muelle 2	Muelle 3	Muelle 4	Muelle 5	Muelle 6
1 Leche	6,84	343,14	346,04	286,73	244,52	284,39
2 Jabón	281,12	168,57	274,48	484,27	334,76	440,89
3 Papa	141,51	0,99	196,41	397,35	495,84	281,01
4 Aceite	317,79	107,09	279,88	273,22	395,05	164,32
5 Batería	390,45	87,1	375,14	385,99	160,09	39,73
6 Linterna	231,25	112,49	114,51	53,26	420,11	225,61
7 Arroz	7,73	211	16,41	458,01	166,22	470,88
8 Jugo	34,34	141,58	346,89	472,33	88,61	152,78
9 Lejía	32,76	91,99	472,78	172,28	1,19	368,29
10 Bolsa	156,86	257,41	430,69	338,64	257,7	276,47

Tabla 4.17: Costos en USD de manejo e inventario por cada 100 unidades de cada producto dependiendo del muelle.

Producto	Capacidad	Demanda
1. Leche	40	60
2. Jabón	40	80
3. Papa	60	120
4. Aceite	30	45
5. Batería	60	120
6. Linterna	60	90
7. Arroz	60	90

Producto	Capacidad	Demanda
8. Jugo	60	120
9. Lejía	40	60
10. Bolsa	50	75

Tabla 4.18: Capacidad y demanda en cientos estimada de cada producto.

Formule un modelo que le permita solucionar el problema de asignación de productos a los muelles, respetando las restricciones de los productos.

4.9.1 Solución

Dado que todos los datos para este problema están en cientos, para este modelo se considerará que una unidad de producto hace referencia realmente a 100 unidades (cambio de variable).

Definición de conjuntos

- I : muelles de almacenamiento $\{1, \dots, 6\}$.
- J : productos $\{1, \dots, 10\}$, en el siguiente orden: leche (1), jabón (2), papas (3), aceite (4), baterías (5), linternas (6), arroz (7), jugos (8), desinfectante (9) y bolsas (10).
- A : parejas de productos incompatibles, es decir $\{(1, 2), (1, 9), (2, 8), (8, 9)\}$.

Definición de parámetros

- Cap_j : capacidad máxima de producto $j \in J$, para almacenar en cada muelle. Nótese que este parámetro no distingue el muelle, esto se debe a que la capacidad es la misma en todos ellos.
- Dem_j : demanda a almacenar de producto $j \in J$.
- C_{ij} : costo unitario de manejo e inventario del producto $j \in J$ en el muelle $i \in I$.
- M : valor de penalidad muy grande en comparación con los demás parámetros.

Definición de variables de decisión

- x_{ij} : cantidad de producto $j \in J$ a almacenar en el muelle $i \in I$.
- $Y_{ij} : \begin{cases} 1, & \text{si el producto } j \in J \text{ es almacenado en el muelle } i \in I \\ 0, & \text{en otro caso} \end{cases}$
- X_i^+ : variación positiva entre las cantidades de papas (3) y aceite (4) en el muelle $i \in I$.
- X_i^- : variación negativa entre las cantidades de papas (3) y aceite (4) en el muelle $i \in I$.
- $CostoReal$: costo asociado al manejo e inventario del sistema.

*Modelo algebraico de programación lineal***Píldora informativa 4.11: Varios objetivos**

En este problema se buscan **dos objetivos**: el primero, asociado a **minimizar el costo** de manejo e inventario, y el segundo, que busca **minimizar la desviación^a** respecto a la cantidad de papas (3) y aceite (4) en cada muelle. La expresión matemática que representa esta función objetivo es:

$$\text{Minimizar } \underbrace{CostoReal}_{\text{obj.1}} + \overbrace{\sum_{i \in I} (X_i^+ + X_i^-)}^{\text{obj.2}}$$

El problema es que tanto el objetivo 1, como el objetivo 2 tienen unidades y magnitudes diferentes. El primero está en dólares (que puede medirse en miles o millones) y el segundo en unidades de diferencia de productos (que al ser de desviación, su valor estará alrededor del cero).

^aConcepto desarrollado en la Píldora Informativa 4.2.

Cuando esto sucede se puede optar por uno de los siguientes enfoques^b:

1. **Normalizar la función objetivo.** Es decir, encontrar un factor para transformar los dos objetivos a una escala de 0 a 1. Un ejemplo de ello sería saber el valor máximo del costo y el valor máximo de desviación y dividir por estos valores el objetivo correspondiente.
2. **Transformar un objetivo en las unidades del otro.** En este caso, se suele usar un factor de penalidad en las mismas unidades del objetivo seleccionado. Este será el enfoque empleado en este problema, y se definirá una penalidad de 1 USD por cada producto de desviación.

^bLas alternativas planteadas son enfoques simples y rápidos para este tipo de problemas, sin tener que usar la teoría completa de la optimización multiobjetivo.

$$\text{Minimizar } CostoTotal = CostoReal + 1 \cdot \sum_{i \in I} (X_i^+ + X_i^-) \quad (4.57)$$

Sujeto a:

$$x_{ij} \leq Cap_j, \quad \forall i \in I, \forall j \in J \quad (4.58)$$

$$\sum_{i \in I} x_{ij} \geq Dem_j, \quad \forall j \in J \quad (4.59)$$

$$x_{ij} \leq Y_{ij} \cdot M, \quad \forall i \in I, \forall j \in J \quad (4.60)$$

$$x_{ij} \geq Y_{ij}, \quad \forall i \in I, \forall j \in J \quad (4.61)$$

$$Y_{ik} \leq 1 - Y_{im}, \quad \forall i \in I, \forall (k, m) \in A \quad (4.62)$$

$$x_{i5} = x_{i6}, \quad \forall i \in I \quad (4.63)$$

$$x_{i3} - x_{i4} = X_i^+ - X_i^-, \quad \forall i \in I \quad (4.64)$$

$$CostoReal = \sum_{i \in I} \sum_{j \in J} C_{ij} \cdot x_{ij} \quad (4.65)$$

$$x_{ij} \geq 0, \quad \forall i \in I, \forall j \in J \quad (4.66)$$

$$X_i^+, X_i^- \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I \quad (4.67)$$

$$CostoReal \in \mathbb{Z}^+ \cup \{0\} \quad (4.68)$$

La expresión (4.57) es la función objetivo, formada por dos componentes: el primero, *CostoReal*, que representa el costo de manejo e inventarios de los productos en los muelles; el segundo, $(X_i^+ + X_i^-)$, representa las

variaciones en la cantidad de papas y aceite juntos en cada muelle i . Esta es la restricción deseable pero no obligatoria. El objetivo es minimizar la suma de ambos elementos.

La expresión (4.58) garantiza que no se sobrepase la capacidad de almacenamiento de cada muelle. La expresión (4.59) establece el cumplimiento de la demanda de productos. Las restricciones (4.60) y (4.61) relacionan las variables x_{ij} y Y_{ij} , de forma que la variable x_{ij} solo pueda tomar valores siempre y cuando $Y_{ij} = 1$. Las restricciones (4.62) y (4.63) garantizan las restricciones de incompatibilidades e igual cantidad de artículos compatibles, respectivamente.

La ecuación (4.64) expresa la diferencia entre cantidades de papas (3) y aceite (4), almacenando este valor en las variables X_i^+ , en caso de tener una diferencia positiva (más papas que aceite), y en X_i^- , en caso de ser negativa (más aceite que papas). La expresión (4.65) calcula los costos de manejo e inventarios de los muelles. Finalmente, las restricciones (4.66) a (4.68) exponen la naturaleza de las variables.

El Algoritmo 40 contiene el código del modelo en AMPL. La solución obtenida por Gurobi en AMPL alcanza un valor de $CostoTotal = 94260,7$ USD, compuesto por $CostoReal = 94155,7$ y $X_1^+ = 60$, $X_2^+ = 30$ y $X_6^- = 15$. La distribución de productos en los muelles se puede apreciar en la Figura 4.4.

Algoritmo 40: archivo Muelles.mod

```
1: set I := {1..6};
2: set J := {1..10};
3: set A := {(1,2), (1,9), (2,8), (8,9)};
4: param Cap{J};
5: param Dem{J};
6: param C{I,J};
7: param M = 100000;
8: var X{I,J} >= 0;
9: var Y{I,J} binary;
10: var CONmas{I} >= 0;
11: var CONmen{I} >= 0;
12: var CostoReal >= 0;
13: CostoTotal: CostoReal + sum{i in I}(CONmas[i] + CONmen[i]);
```

```
14: s.t.  
15: r1{i in I, j in J}: X[i,j] <= Cap[j];  
16: r2{j in J}: sum{i in I} X[i,j] >= Dem[j];  
17: r3{i in I, j in J}: X[i,j] <= Y[i,j]*M;  
18: r4{i in I, j in J}: X[i,j] >= Y[i,j];  
19: r5{i in I, (k,m) in A}: Y[i,k] <= 1- Y[i,m];  
20: r6{i in I}: X[i,5] = X[i,6];  
21: r7{i in I}: X[i,3] - X[i,4] = CONmas[i] - CONmen[i];  
22: r8: CostoReal = sum{i in I, j in J} C[i,j]*X[i,j];
```

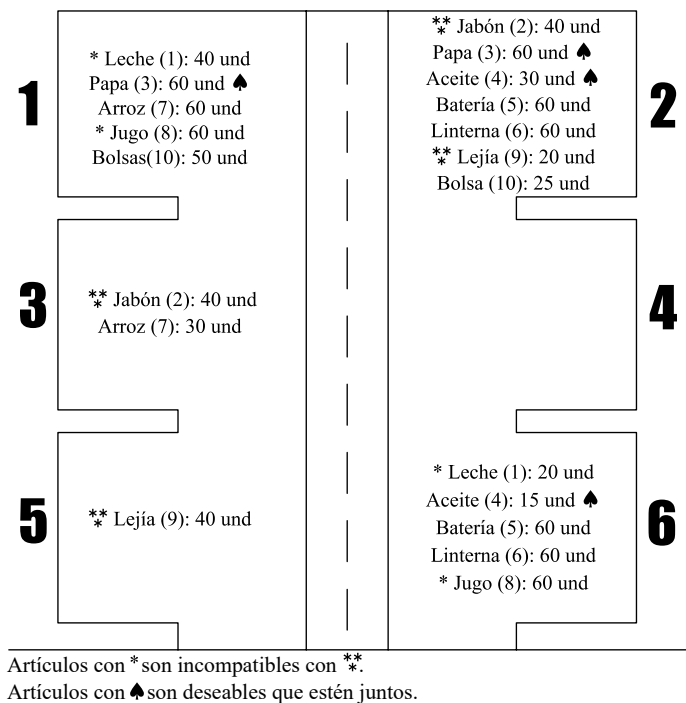


Figura 4.4: Distribución óptima de productos en los muelles.

4.10 Problema del plan de extracción de una mina

Una empresa minera dedicada a la extracción y venta de vetas de oro no procesadas, debe diseñar un plan de extracción de las 9 vetas de oro de su nueva adquisición: la mina *Mano de Midas*. Para esto cuenta solo con 5 semanas. El gerente lo contrata a usted como asesor; él le comenta que el problema radica en que el mercado de vetas de oro es muy variante

en función de la calidad y la estructura de la veta, por este motivo, el precio de venta de cada veta cambia con el tiempo, como se muestra en la Tabla 4.19.

La semana que se extrae una veta de oro (sea al principio o al final), debe ser la misma en la que se vende, al precio respectivo. Por tanto se puede asumir que tanto la extracción como la venta se realizan en una sola semana.

El gerente le presenta un esquema de las vetas de oro de la mina *Mano de Midas* (Figura 4.5). Como es lógico, agrega el gerente, no se pueden extraer las vetas que están en el interior, sin extraer antes las vetas que están en el exterior, es decir, para extraer, por ejemplo, la veta número 0, se deben extraer antes las vetas 2 y 3; y a su vez, también se debe extraer las vetas 5, 6 y 7, pues las vetas no pueden ser extraídas sin antes extraer todas aquellas que están encima de ellas (en semanas previas o en la misma semana). Esto obliga a vender estas vetas antes o en la misma semana de la extracción de la veta 0.

Formular un modelo de programación lineal para realizar el plan de extracción de las vetas, cuyo objetivo sea maximizar las ganancias de las ventas de las vetas de oro. Considerar que en una semana se pueden extraer todas la vetas de ser necesario.

		Semana				
		1	2	3	4	5
Veta de oro	0	7	10	8	11	13
	1	11	9	10	13	10
	2	9	7	10	12	8
	3	12	9	12	9	8
	4	9	13	7	11	10
	5	11	7	12	13	8
	6	9	13	8	11	9
	7	8	10	12	9	10
	8	10	12	13	8	11

Tabla 4.19: Precio de venta en dólares de las vetas de oro en cada semana.

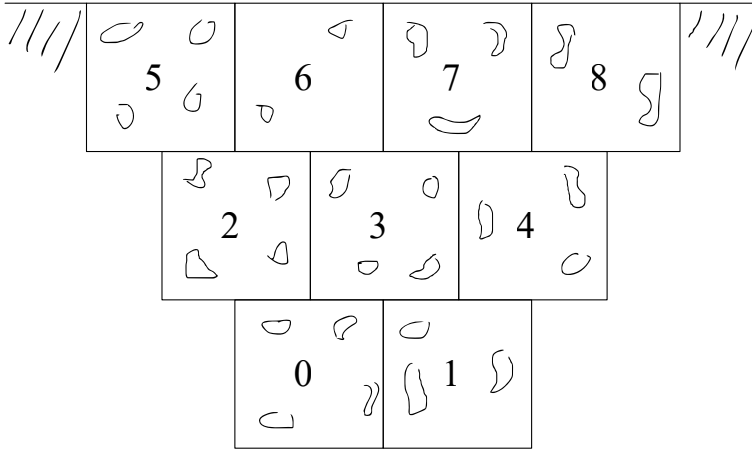


Figura 4.5: Esquema de la mina *Mano de Midas* y sus vetas de oro.

4.10.1 Solución

Definición de conjuntos

- T : semanas disponibles $\{1, \dots, 5\}$.
- P : vetas de oro $\{0, \dots, 8\}$.
- SUC : parejas (i, j) de vetas, donde la extracción de la veta i requiere la extracción previa de la veta j , $\{(0, 2), (0, 3), (1, 3), (1, 4), (2, 5), (2, 6), (3, 6), (3, 7), (4, 7), (4, 8)\}$.

Definición de parámetros

- C_{pt} : precio de venta de la veta $p \in P$ que se vende en la semana $t \in T$.

Definición de variables de decisión

- $x_{pt} : \begin{cases} 1, & \text{si la veta } p \in P \text{ es extraída (y vendida) en la semana } t \in T \\ 0, & \text{en otro caso} \end{cases}$

Modelo algebraico de programación lineal

Píldora informativa 4.12: Multiplicar por subíndices

En algunos problemas puede ser conveniente **multiplicar el valor numérico de los subíndices**. Por ejemplo, en este problema, las restricciones de precedencia podrían ser definidas mediante la siguiente relación (sin pérdida de generalidad, se tomará la relación de la veta 0 y la veta 2):

Día de extracción de la veta 2 $\leq$ día de extracción de la veta 0

De esta forma, si la veta 0 se extrae en el periodo 3, la veta 2 debe haber sido extraída en el periodo 3, 2 o 1. Sin embargo, la variable de decisión x_{pt} , al ser binaria, solo nos puede retornar un 0 o un 1. ¿Cómo se puede recuperar el valor del día de la variable x_{pt} ? Multiplicando por el subíndice t correspondiente:

Día de extracción de la veta 2 = $(1 \cdot x_{21}) + (2 \cdot x_{22}) + (3 \cdot x_{23}) + (4 \cdot x_{24}) + (5 \cdot x_{25})$

Día de extracción de la veta 0 = $(1 \cdot x_{01}) + (2 \cdot x_{02}) + (3 \cdot x_{03}) + (4 \cdot x_{04}) + (5 \cdot x_{05})$

$$\text{Maximizar } z = \sum_{t \in T} \sum_{p \in P} x_{pt} \cdot C_{pt} \quad (4.69)$$

Sujeto a:

$$\sum_{t \in T} x_{pt} = 1, \quad \forall p \in P \quad (4.70)$$

$$\sum_{t \in T} t \cdot x_{jt} \leq \sum_{t \in T} t \cdot x_{it}, \quad \forall (i, j) \in SUC \quad (4.71)$$

$$x_{pt} \in \{0, 1\}, \quad \forall p \in P, \forall t \in T \quad (4.72)$$

La expresión (4.69) es la función objetivo, que busca maximizar las utilidades obtenidas al vender las vetas de oro.

La expresión (4.70) garantiza que todas las minas sean extraídas en algún momento de las 5 semanas. La expresión (4.71) garantiza la extracción de las vetas que tienen precedencias. Finalmente, la restricción (4.72) expone la naturaleza de las variables.

El Algoritmo 41 contiene el código del modelo en AMPL. La solución obtenida por Gurobi en AMPL alcanza un valor de 112 USD. El plan de extracción es:

- Semana 1: no hay extracción.
- Semana 2: extracción de la veta 6.
- Semana 3: extracción de las vetas 3, 7 y 8.
- Semana 4: extracción de las vetas 1, 2, 4 y 5.
- Semana 5: extracción de la veta 0.

Algoritmo 41: archivo MinasOro.mod

```

1: set T := {1..5};
2: set P := {0..8};
3: set SUC := {(0,2), (0,3), (1,3), (1,4), (2,5), (2,6), (3,6),
    (3,7), (4,7), (4,8)};
4: param C{P,T};
5: var x{P,T} binary;
6: maximize z: sum{p in P, t in T} C[p,t]*x[p,t];
7: s.t.
8: r{p in P}: sum{t in T} x[p,t] = 1;
9: Suce{(i,j) in SUC}: sum{t in T} t*x[j,t] <= sum{t in T} t*x[i,t];

```

4.10.2 Variación propuesta para el estudiante

Considere que usted está limitado a extraer solo 7 vetas de oro. Reformule el modelo teniendo en cuenta esta nueva condición.

4.11 Problema de las rutas de pizza

Debido a que usted desea un ingreso extra para sus vacaciones, ha decidido trabajar en la pizzería local llamada *Pizzeria da Baffetto*. Ahí, el gerente le comenta que la única vacante es de repartidor. Usted acepta gustoso el contrato.

Su función, el día de hoy, consiste en llevar 50 pizzas a 4 clientes. A usted le entregan una moto para los despachos. Sin embargo, el costo del combustible lo tiene que asumir usted. Como es lógico, dicho consumo está asociado a la distancia y al peso (pizzas) que transporte. Esto se debe a que no es igual viajar 20 kilómetros llevando 1 pizza, que hacerlo con 50.

Usted ha dibujado un mapa para aclarar sus ideas (Figura 4.6). Las demandas de pizzas se pueden ver al lado de cada nodo, que representa a un cliente (2, 3, 4 y 5). Por ejemplo, el nodo 2 tiene una demanda de 10 pizzas. El nodo 1 representa la pizzería.

Usted debe llevar las pizzas correspondientes a cada cliente. El costo de combustible de ir de un nodo i a un nodo j lo ha estimado como C_{ij} por pizza transportada [$USD/pizza$].

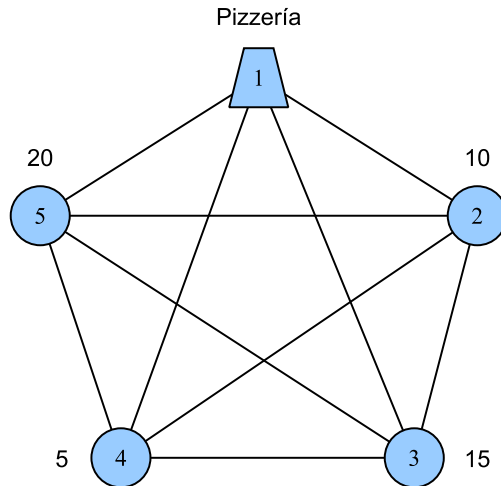


Figura 4.6: Diagrama de los clientes para el envío.

C_{ij}	1	2	3	4	5
1	-	5	3	2	1
2	2	-	4	1	2
3	3	5	-	2	2
4	1	3	4	-	1
5	1	2	2	2	-

Tabla 4.20: Costo en dólares por pizza transportada entre nodos.

4.11.1 Solución

Definición de conjuntos

- I : conjunto de nodos $\{1, \dots, 5\}$.

Definición de parámetros

- C_{ij} : costo por pizza transportada del nodo $i \in I$ al nodo $j \in I$.
- D_i : demanda de pizzas en el nodo $i \in I$.
- M : parámetro constante muy grande con comparación con los demás.

Definición de variables de decisión

- $x_{ij} : \begin{cases} 1, & \text{si viajas del nodo } i \in I \text{ al nodo } j \in I \\ 0, & \text{en otro caso} \end{cases}$
- y_{ij} : cantidad de pizzas transportadas del nodo $i \in I$ al nodo $j \in I$.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in I: i \neq j} y_{ij} \cdot C_{ij} \quad (4.73)$$

Sujeto a:

$$\sum_{j \in I: j \neq i} x_{ij} = 1, \quad \forall i \in I \quad (4.74)$$

$$\sum_{i \in I: i \neq j} x_{ij} = 1, \quad \forall j \in I \quad (4.75)$$

$$y_{ij} \leq M \cdot x_{ij}, \quad \forall i \in I, \forall j \in I: i \neq j \quad (4.76)$$

$$\sum_{i \in I: i \neq j} y_{ij} = d_j + \sum_{i \in I: i \neq j} y_{ji}, \quad \forall j \in I \setminus \{1\} \quad (4.77)$$

$$\sum_{i \in I: i \neq 1} y_{1i} = 50 \quad (4.78)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J \quad (4.79)$$

$$y_{ij} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (4.80)$$

La expresión (4.73) es la función objetivo, que busca minimizar los costos de transporte de pizzas entre nodos.

Las expresiones (4.74) y (4.75) garantizan que de cada nodo se salga una vez y se llegue a él una vez, respectivamente. La restricción (4.76) establece que si no se visita una pareja de nodos (i, j) , no se puede transportar unidades de pizzas entre dichos nodos.

La expresión (4.77) asegura que en cada nodo destino, el número de pizzas sea igual a la demanda de dicho nodo más las pizzas restantes que salen de él.

Píldora informativa 4.13: Restricciones redundantes

Algunas veces, en los modelos matemáticos **las restricciones cumplen más de un papel**. Por ejemplo, la restricción (4.77) no solo garantiza el balance de las pizzas (y_{ij}) entre los nodos, es decir, que toda pizza que ingrese al nodo debe abastecer la demanda y salir al siguiente nodo, sino que también, de forma simultánea, garantiza el balance de las variables de asignación de arcos a una ruta (x_{ij}), es decir garantizar que si se viaja del nodo i al nodo j , se debe salir del nodo j más adelante en la ruta.

Esto se logra debido a la relación establecida en la restricción (4.76) entre las variables y_{ij} y x_{ij} . Por tanto, no es necesario establecer el balance de forma explícita solo con las variables x_{ij} :

$$\sum_{j \in I: i \neq j} x_{ji} = \sum_{j \in I: i \neq j} x_{ij}, \quad \forall i \in I$$

La ecuación (4.78) establece el número de pizzas iniciales que deben transportarse a algún nodo. Finalmente, las restricciones (4.79) a (4.80) exponen la naturaleza de las variables.

El Algoritmo 42 contiene el código del modelo en AMPL. La solución obtenida por CPLEX en AMPL alcanza un valor de 170 USD. La ruta óptima es $1 \rightarrow 5 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow 1$.

Algoritmo 42: archivo Pizza.mod

```

1: set I := {1..5};
2: param c{I,I};
3: param d{I};
4: param M;
5: var x{i in I, j in I: j <> i} binary;
6: var y{i in I, j in I: j <> i} >= 0;
7: minimize z: sum{i in I, j in I: i <> j} y[i,j]*c[i,j];
8: s.t.
9: rsalid{i in I}: sum{j in I: j <> i} x[i,j] = 1;
10: rllega{j in I}: sum{i in I: i <> j} x[i,j] = 1;
11: rela{i in I, j in I: i <> j}: y[i,j] <= M*x[i,j];
12: rInv{j in I diff {1}}: sum{i in I: i <> j} y[i,j] = d[j] + sum
    {i in I: i <> j} y[j,i];
13: rini: sum{i in I: i <> 1} y[1,i] = 50;

```

4.12 Problema de asignación de enfermeros 2

La gerente del hospital *Ve Hacia la Luz*, la Dra. Cecilia Grierson, muy satisfecha con la asignación de enfermeros que usted hizo para los fines de semana (Problema 3.12), lo contrata nuevamente para realizar la asignación de enfermeros, pero para toda la semana. Debido a que tanto la demanda, como las normativas para cada día son las mismas, basta con realizar la programación de un solo día.

Los turnos disponibles y las demandas de enfermeros en cada turno se muestran en la Tabla 4.21. Por otra parte, la base de datos del hospital tiene 10 enfermeros disponibles para realizar contrataciones. Sus nombres y salarios esperados por hora se muestran en la Tabla 4.22.

Las condiciones de trabajo exigen solo dos restricciones: (1) se tiene un presupuesto máximo de 600 USD diarios para los honorarios de los enfermeros, y (2) en cada turno se pagará, a todos los enfermeros contratados, un salario con un valor máximo o igual al menor salario esperado entre los enfermeros contratados para dicho turno. Ejemplo: si en el turno 1 se contrata a Nestor (40 USD) y a Amaya (30 USD), a los dos se les pagará un salario como máximo de 30 USD (el menor de ambos).

Turno	Demanda estimada
1 $\rightarrow$ 4:00 - 8:00	2 enfermeros
2 $\rightarrow$ 8:00 - 12:00	3 enfermeros
3 $\rightarrow$ 12:00 - 16:00	2 enfermeros
4 $\rightarrow$ 16:00 - 20:00	4 enfermeros

Tabla 4.21: Turnos disponibles y demandas de enfermeros.

Nombre	Costo esperado por hora	Nombre	Salario esperado por hora
1 Nestor	40 USD	6 Raquel	44 USD
2 Josue	62 USD	7 Aurea	32 USD
3 Julen	60 USD	8 María	55 USD
4 Alfredo	42 USD	9 Amaya	30 USD
5 Martín	65 USD	10 Micaela	54 USD

Tabla 4.22: Enfermeros disponibles en la base de datos.

Debido a la restricción (2), las personas pueden recibir un salario menor del que esperan. Por tanto, se busca una asignación que minimice la diferencia entre el salario esperado y el salario realmente pagado entre todos los enfermeros. Formular un modelo de programación lineal que dé respuesta a esta problemática. Tener en cuenta que no conviene pagar 0 USD a los enfermeros, dado que la diferencia entre lo esperado y lo real sería muy alta.

4.12.1 Solución

Definición de conjuntos

- I : conjunto de enfermeros $\{1, \dots, 10\}$.
- J : turnos disponibles $\{1, \dots, 4\}$.

Definición de parámetros

- C_i : salario esperado del enfermero $i \in I$.
- DEM_j : demanda de enfermeros en el turno $j \in J$.
- M : parámetro constante muy grande en comparación con los demás.

Definición de variables de decisión

- $x_{ij} : \begin{cases} 1, & \text{si el enfermero } i \in I \text{ es asignado al turno } j \in J \\ 0, & \text{en otro caso} \end{cases}$
- P_j : salario real pagado a los enfermeros contratados en el turno $j \in J$.
- y_{ij}^+ y y_{ij}^- : desviación por encima y por debajo del salario esperado, con respecto al realmente pagado de cada enfermero $i \in I$ en el turno $j \in J$.

Píldora informativa 4.14: Variable no restringida en signo

En este problema se podrían plantear las desviaciones de forma muy similar a como se hizo en el Problema 4.2:

$$C_i - P_j = y_{ij}^+ - y_{ij}^-, \quad \forall i \in I, \forall j \in J$$

Así, las diferencias entre el salario esperado de cada enfermero i y el realmente pagado por turno j , se almacenarían en las variables y_{ij}^+ y y_{ij}^- . Sin embargo, esto incluiría a los enfermeros que no han sido contratados en un turno.

Para remediar este inconveniente, se podría usar la variable de asignación x_{ij} en el lado izquierdo de la anterior restricción, quedando de la forma: $x_{ij} \cdot (C_i - P_j)$. Pero, como P_j y x_{ij} son variables, el modelo resultante sería no lineal. Por esta razón, se crea una variable auxiliar que tomará el valor de la desviación correspondiente (positiva o negativa, por tanto **debe ser no restringida en signo**), solo cuando el enfermero i sea asignado al turno j ($x_{ij} = 1$).

- AUX_{ij} : variable de decisión auxiliar que almacena el valor (positivo o negativo) de la diferencia entre el salario esperado y el realmente pagado a cada enfermero $i \in I$ en el turno $j \in J$.

Modelo algebraico de programación lineal

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} (y_{ij}^+ + y_{ij}^-) \quad (4.81)$$

Sujeto a:

$$\sum_{i \in I} x_{ij} \geq DEM_j, \quad \forall j \in J \quad (4.82)$$

$$AUX_{ij} = y_{ij}^+ - y_{ij}^-, \quad \forall i \in I, \forall j \in J \quad (4.83)$$

$$AUX_{ij} \leq C_i - P_j + M \cdot (1 - x_{ij}), \quad \forall i \in I, \forall j \in J \quad (4.84)$$

$$AUX_{ij} \geq C_i - P_j - M \cdot (1 - x_{ij}), \quad \forall i \in I, \forall j \in J \quad (4.85)$$

$$P_j \leq C_i \cdot x_{ij} + M \cdot (1 - x_{ij}), \quad \forall i \in I, \forall j \in J \quad (4.86)$$

$$\sum_{j \in J} DEM_j \cdot P_j \leq 600 \quad (4.87)$$

$$x_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J \quad (4.88)$$

$$y_{ij}^+, y_{ij}^- \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (4.89)$$

$$P_j \in \mathbb{Z}^+ \cup \{0\}, \quad \forall j \in J \quad (4.90)$$

$$AUX_{ij} \in \mathbb{Z} \cup \{0\}, \quad \forall i \in I, \forall j \in J \quad (4.91)$$

La expresión (4.81) es la función objetivo, que busca minimizar las variables de desviación del salario esperado y el salario realmente pagado.

La expresión (4.82) garantiza contratar al menos el mínimo número de enfermeros requeridos en cada turno. La expresión (4.83) establece la relación entre la variable auxiliar de desviación AUX_{ij} y las variables de desviación y_{ij}^+ y y_{ij}^- . Las restricciones (4.84) y (4.85) permiten que la variable AUX_{ij} tome un valor de cero cuando el enfermero i no esté asignado al turno j , y tome el valor correspondiente a la diferencia cuando sí esté asignado (tener en cuenta que la variable AUX_{ij} es no restringida en signo). La restricción (4.86) establece que el pago real no puede superar el mínimo del salario esperado de los enfermeros asignados.

La restricción (4.87) garantiza no superar el presupuesto. Nótese la utilización del parámetro de demanda en lugar de la variable de asignación. Esto se debe a que el salario real pagado es una variable que, al multiplicarla por la variable de asignación, generaría un modelo cuadrático. Por otra parte, se sabe que el modelo solo asignará a los enfermeros con base en la demanda por turno, dado que es un modelo de minimización. Finalmente, las restricciones (4.88) a (4.91) exponen la naturaleza de las variables.

El Algoritmo 43 contiene el código del modelo algebraico en AMPL. La solución obtenida por CPLEX en AMPL alcanza un valor de 23 USD. La asignación y el salario por turno se muestran detalladamente en la Tabla 4.23.

Algoritmo 43: archivo AsignacionEnfer2.mod

```

1: set I := {1..10};
2: set J := {1..4};
3: param C{I};
4: param DEM{J};
5: param M;
6: var x{I,J} binary;
7: var P{J} >= 0;
8: var yma{I,J} >= 0;
9: var yme{I,J} >= 0;
10: var aux{I,J};
11: minimize z: sum{i in I, j in J} (yma[i,j] + yme[i,j]);
12: s.t.
13: Dem{j in J}: sum{i in I} x[i,j] >= DEM[j];
14: costos{i in I, j in J}: aux[i,j] = yma[i,j] - yme[i,j];
15: aux1{i in I, j in J}: aux[i,j] <= C[i] - P[j] + M*(1-x[i,j]);
16: aux2{i in I, j in J}: aux[i,j] >= C[i] - P[j] - M*(1-x[i,j]);
17: salarioTurno{i in I, j in J}: P[j] <= C[i]*x[i,j] + M*(1-x[i,j]);
18: CosT: sum{j in J} DEM[j]*P[j] <= 600;

```

	Turno 1, salario real $P_1 = 54$ USD			
Enfermero	8 (María)	10 (Micaela)		
Salario esperado	55 USD	54 USD		
Desviación	1	N/A		
	Turno 2, salario real $P_2 = 40$ USD			
Enfermero	1 (Nestor)	4 (Alfredo)	6 (Raquel)	
Salario esperado	40 USD	42 USD	44 USD	
Desviación	N/A	2	4	
	Turno 3, salario real $P_3 = 54$ USD			
Enfermero	8 (María)	10 (Micaela)		
Salario esperado	55 USD	54 USD		
Desviación	1	N/A		
	Turno 4, salario real $P_4 = 54$ USD			
Enfermero	2 (Josue)	3 (Julen)	8 (María)	10 (Micaela)
Salario esperado	62 USD	60 USD	55 USD	54 USD
Desviación	8	6	1	N/A

Tabla 4.23: Asignación óptima de enfermeros a los turnos y sus salarios reales.

4.13 Problema del aeropuerto

La compañía aérea *El Cielo que Ves cada Vez* (CVV) tiene graves problemas para programar las rutas de los vuelos en su aeropuerto desde Aglarond (origen) a Ithil (destino). Esto se debe a que, según la ruta elegida entre estas dos ciudades, la distancia que recorra la aeronave cambiará, y por tanto, los costos del viaje. A pesar de esto, el precio de venta del billete de vuelo no puede incrementarse, puesto que el origen y el destino son los mismos. El problema radica en que cada vez que se llega a una nueva ciudad, se tiene que aterrizar para que los pasajeros realicen un registro en la aduana y puedan continuar con el viaje.

A usted le comentan la situación actual: se tiene una flota de 4 aviones con una capacidad máxima de 15 personas cada uno, y se debe transportar a 30 personas desde un origen a un destino establecido, en el menor número de días posible. Para esta planeación se cuenta con un máximo de 6 días.

El gerente le proporciona un mapa que muestra todas las ciudades que se pueden visitar en las rutas de los vuelos. El nodo 1 representa el origen y el nodo 10 el destino (Figura 4.7).

Este mapa está compuesto por 10 ciudades, cada una de ellas está representada por un nodo. Los aviones partirán del nodo 1 y no pueden regresar a él. De igual forma, una vez se llegue al nodo 10 no se puede regresar a otro nodo. El resto de ellos pueden viajar a cualquiera de sus nodos vecinos. Considere que cada ruta de origen hasta el destino toma 1 día, sin importar la ruta. Todos los aviones usados regresan en la noche al nodo origen.

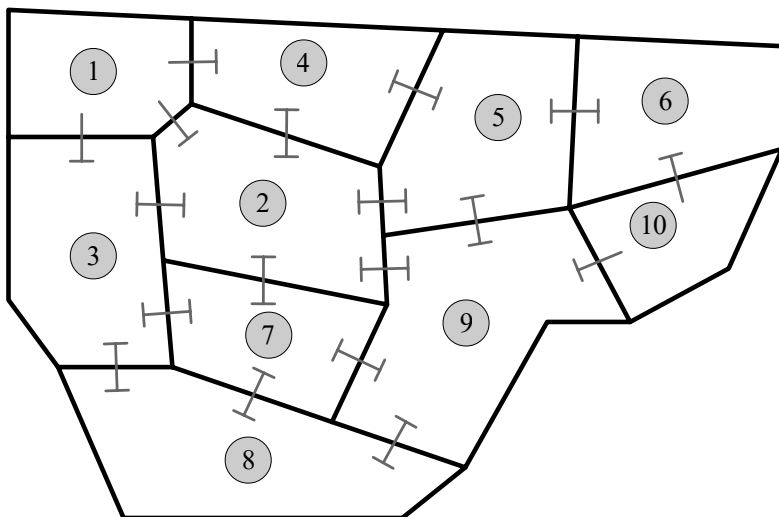


Figura 4.7: Mapa de las ciudades visitadas por la aerolínea.

Los vuelos en cualquier ruta se deben realizar de nodo a nodo, es decir, que todos los pasajeros deben visitar todas las ciudades de la ruta para registrarse en la aduana. Adicionalmente, cada nodo (excepto el origen y el destino) tiene un número máximo de personas por día que pueden recibir, para hacer trámites en las embajadas. Toda persona que llega a un nodo de paso debe salir de este el mismo día. Cuando se alcanza el límite diario, no se puede recibir a más pasajeros y se debe programar un nuevo vuelo al siguiente día o bien tomar una ruta diferente. El límite de personas que puede atender cada ciudad se muestra en la Tabla 4.24.

Nodo	Número máximo de personas permitidas por día
2	5
3	8
4	10
5	5
6	14
7	10
8	12
9	15

Tabla 4.24: Capacidad máxima en las aduanas de cada ciudad de paso.

El gerente ha estimado con precisión las distancias en kilómetros entre nodos. Estos datos se pueden apreciar en la Tabla 4.25. Los puntos en la tabla representan arcos que no son posibles en las rutas.

	1	2	3	4	5	6	7	8	9	10
1	.	4	2	5	.	.	.	.	.	.
2	2	.	3	6	7	.	10	.	10	.
3	6	3	.	.	.	.	6	9	.	.
4	5	7	.	.	10	.	.	.	.	.
5	.	8	.	6	.	9	.	.	17	.
6	.	.	.	.	3	.	.	.	.	4
7	.	3	7	.	.	.	.	10	4	.
8	.	.	15	.	.	.	7	.	13	.
9	.	8	.	.	4	.	10	13	.	8
10	.	.	.	.	.	3	.	.	9	.

Tabla 4.25: Distancia en kilómetros entre parejas de ciudades.

Los costos de la compañía aérea son generados por dos elementos: distancia y tiempo. Para poder compararlos, el gerente le comenta que tiene un factor de conversión de días a distancias: cada día utilizado es equivalente a 100 kilómetros. De esta forma, por ejemplo, si se programan los viajes para 3 días, a la distancia real recorrida, se le debe sumar $3 \cdot 100$ kilómetros.

Finalmente, el gerente le pide que encuentre una programación de vuelos que permita decidir qué avión usar, con qué cantidad de pasajeros viajar y en qué día. Además, debe establecer qué secuencia de nodos usar en el viaje (de cada avión), para transportar 30 pasajeros desde el nodo origen al nodo destino con la menor distancia posible (incluyendo los días transformados en kilómetros con el factor de conversión).

4.13.1 Solución

Definición de conjuntos

- M : conjunto de aviones $\{1, \dots, 4\}$.
- I : conjunto de nodos (ciudades) $\{1, \dots, 10\}$.
- A : conjunto de arcos factibles $\{(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (2, 5), (2, 7), (2, 9), (3, 2), (3, 7), (3, 8), (4, 2), (4, 5), (5, 2), (5, 4), (5, 6), (5, 9), (6, 5), (6, 10), (7, 2), (7, 3), (7, 8), (7, 9), (8, 3), (8, 7), (8, 9), (9, 2), (9, 5), (9, 7), (9, 10), (9, 8)\}$.
- ES : conjunto de escalas (los nodos excepto el origen y el destino) $I \setminus \{1, 10\}$.
- T : conjunto de días disponibles para hacer la programación de vuelos $\{1, \dots, DD\}$, donde DD es el número de días máximos disponibles.
- INI : conjunto de arcos factibles desde el origen $\{(1, 2), (1, 3), (1, 4)\}$.
- FIN : conjunto de arcos factibles que llegan al destino $\{(6, 10), (9, 10)\}$.

Definición de parámetros

- Dem : número de pasajeros a enviar (30).
- $MaxPer_i$: máxima cantidad de personas permitidas en el nodo $i \in ES$.
- $CapAv$: capacidad máxima de personas en los aviones (15).
- Dis_{ij} : distancia entre cada pareja de nodos $(i, j) \in A$.

Definición de variables de decisión

- $x_{mijt} : \begin{cases} 1, & \text{si el avión } m \in M \text{ viaja de } (i, j) \in A \text{ el día } t \in T \\ 0, & \text{en otro caso} \end{cases}$
- $y_{mijt} : \text{cantidad de personas que van en el avión } m \in M, \text{ a través de los nodos } (i, j) \in A \text{ en el día } t \in T.$
- $Days : \text{cantidad de días utilizados en la programación de rutas.}$

*Modelo algebraico de programación lineal***Píldora informativa 4.15: Complementar la relación entre variables continuas y binarias**

En otros problemas se ha visto cómo una variable continua y una variable binaria están relacionadas y se debe establecer una restricción que las represente. Sea y la variable continua de producción, x la variable binaria que represente la apertura de la planta y M un parámetro constante con un valor tendiente a infinito, la restricción quedaría de la siguiente forma:

$$y \leq x \cdot M$$

En este caso, si y tiene un valor positivo (si hay producción), x tiene que tomar el valor de 1 (hay apertura). Sin embargo, el caso contrario no está garantizado, es decir, que haya apertura $x = 1$, pero no haya producción $y = 0$, no viola la restricción anterior y puede ocasionar interpretaciones incorrectas. Esto se puede resolver principalmente de dos formas:

- Asignando un valor de penalidad a x , e incluirla en la función objetivo sumando, si se busca minimizar, o bien restando, si se busca maximizar.
- Creando otra restricción tipo mayor o igual:

$$y \geq x$$

$$\text{Minimizar } z = \sum_{m \in M} \sum_{(i,j) \in A} \sum_{t \in T} x_{mijt} \cdot Dis_i + 10 \cdot Days \quad (4.92)$$

Sujeto a:

$$Days \geq x_{mijt} \cdot t, \quad \forall m \in M, \forall (i, j) \in A, \forall t \in T \quad (4.93)$$

$$\sum_{(i,j) \in A: j=k} \sum_{m \in M} y_{mijt} \leq MaxPer_k, \quad \forall k \in ES, \forall t \in T \quad (4.94)$$

$$\sum_{m \in M} \sum_{(i,j) \in INI} \sum_{t \in T} y_{mijt} = Dem \quad (4.95)$$

$$\sum_{m \in M} \sum_{(i,j) \in FIN} \sum_{t \in T} y_{mijt} = Dem \quad (4.96)$$

$$\sum_{(i,j) \in A: j=k} y_{mijt} = \sum_{(j,i) \in A: j=k} y_{mjit}, \quad \forall k \in ES, \forall m \in M, \forall t \in T \quad (4.97)$$

$$y_{mijt} \leq x_{mijt} \cdot CapAv, \quad \forall m \in M, \forall (i, j) \in A, \forall t \in T \quad (4.98)$$

$$y_{mijt} \geq x_{mijt}, \quad \forall m \in M, \forall (i, j) \in A, \forall t \in T \quad (4.99)$$

$$\sum_{(i,j) \in A: i=k} x_{mijt} \leq 1, \quad \forall m \in M, \forall k \in I, \forall t \in T \quad (4.100)$$

$$x_{mijt} \in \{0, 1\}, \quad \forall m \in M, \forall (i, j) \in A, \forall t \in T \quad (4.101)$$

$$y_{mijt} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall m \in M, \forall (i, j) \in A, \forall t \in T \quad (4.102)$$

$$days \in \mathbb{Z}^+ \cup \{0\} \quad (4.103)$$

La expresión (4.92) es la función objetivo del problema, que busca minimizar las distancias y los días asociados a los viajes (transformados a kilómetros).

La restricción (4.93) permite calcular el número de días usados para los viajes. La restricción (4.94) asegura que la cantidad de personas que llegan a cada escala no sobrepase la capacidad máxima de personas permitidas. Por otro lado, las ecuaciones (4.95) y (4.96) garantizan que salgan 30 pasajeros del origen y lleguen 30 pasajeros al destino final, respectivamente. La restricción (4.97) establece el balance de pasajeros por avión en cada nodo, es decir, que la cantidad de personas que arriben a un nodo debe ser la misma que salen de él. Las restricciones (4.98) y (4.99) establecen la relación entre la variable asociada al uso del avión y la cantidad de pasajeros que viajan en él. La restricción (4.100) establece que cada avión solo puede salir una vez de cada nodo. Finalmente, las expresiones (4.101), (4.102) y (4.103) exponen la naturaleza de las variables.

El Algoritmo 44 contiene el código del modelo en AMPL. La Tabla 4.26 muestra las rutas óptimas realizadas por los aviones, que alcanza un valor

en la función objetivo de 312 kilómetros, usando solo 2 días para realizar todos los vuelos.

Avión núm.	Día 1	Pasajeros
	Ruta	
2	1-3-7-9-10	8
4	1-2-9-10	5
Avión núm.	Día 2	Pasajeros
	Ruta	
1	1-3-7-9-10	8
3	1-2-9-10	5
4	1-4-5-6-10	4

Tabla 4.26: Plan de vuelo de aviones para dos días.

Algoritmo 44: archivo AeropuertoRutas.mod

```

1: set M := {1..4};
2: set I := {1..10};
3: set ES := I diff {1,10};
4: param DD = 6;
5: set T := {1..DD};
6: set A := {(1,2), (1,3), (1,4), (2,3), (2,4), (2,5), (2,7), (2,9),
  (3,2), (3,7), (3,8), (4,2), (4,5), (5,2), (5,4), (5,6), (5,9),
  (6,5), (6,10), (7,2), (7,3), (7,8), (7,9), (8,3), (8,7), (8,9),
  (9,2), (9,5), (9,7), (9,10), (9,8)};
7: set INI := {(1,2), (1,3), (1,4)};
8: set FIN := {(9,10), (6,10)};
9: param Dem;
10: param MaxPer{ES};
11: param CapAv;
12: param Dis{I,I};
13: var x{M,(i,j) in A,T} >= 0 binary;
14: var y{M,(i,j) in A,T} >= 0 integer;
15: var Days >= 0;
16: minimize z: sum{m in M, (i,j) in A, t in T} x[m,i,j,t]*Dis[i,j] +
  100*Days;
17: s.t.
18: rdias{m in M, (i,j) in A, t in T}: Days >= x[m,i,j,t]*t;

```

```

19: rMaxPer{k in ES, t in T}: sum{(i,j) in A, m in M : j=k} y[m,i,j,t]
    <= MaxPer[k];
20: rIni: sum{m in M, (i,j) in INI, t in T} y[m,i,j,t] = Dem;
21: rFIN: sum{m in M, (i,j) in FIN, t in T} y[m,i,j,t] = Dem;
22: rBalanceNodos{k in ES, m in M, t in T}: sum{(i,j) in A: j=k}
    y[m,i,j,t] = sum{(j,i) in A: j=k} y[m,j,i,t];
23: rAviPeople1{m in M, (i,j) in A, t in T}: y[m,i,j,t] <= x[m,i,j,t]
    *CapAv;
24: rAviPeople2{m in M, (i,j) in A, t in T}: y[m,i,j,t] >= x[m,i,j,t];
25: avionesNodo{k in I, t in T, m in M}: sum{(i,j) in A : i=k}
    x[m,i,j,t] <= 1;

```

4.13.2 Variación propuesta para el estudiante

Si se considera que los pasajeros pueden quedarse en los destinos intermedios a la espera de otro avión en otro día, ¿cómo se afectaría el modelo actual?

4.14 Problema de las maderas

El aserradero *Los Maderos de San Juan* fabrica tres productos: puertas, ventanas y sillas. Para ello, compra madera a un proveedor local, quien vende tres tipos de madera: roble, pino y cedro. La reserva de madera del proveedor para cada tipo de madera es de 500 tablones.

Para la producción de los artículos no es relevante saber qué tipo de madera se usa, puesto que cualquier madera es apropiada para fabricar cualquiera de los productos. El proceso de transformación tiene una eficiencia del 90 %. Por ejemplo, si se tienen 10 tablones de materia prima, luego de pasar por procesos de corte, pulido y similares, se obtienen solo 9 tablones útiles para la fabricación de un artículo.

Los precios de cada tipo de madera son diferentes. Como es usual en el mercado, el proveedor cambia el precio de la materia prima (de todos los tipos de madera) según el número de unidades. La Tabla 4.27 resume el costo por tablones de cada tipo de madera.

$[USD/tablón]$	Roble	Pino	Cedro
≤ 100 tablones	6	7	3
> 100 tablones	5	4	5

Tabla 4.27: Precio de los tablones de madera en función a su cantidad.



Es usual que cuanto mayor sea la cantidad a comprar de un artículo, su precio disminuya. Sin embargo, pueden existir políticas que disuadan a los compradores a que adquieran un mayor número de artículos, como un aumento en el precio (en este problema ocurre con el cedro). Eso puede ocurrir cuando existe una escasez de dicho producto, o bien por políticas medioambientales.

Para producir una puerta se requieren 3 tablones de madera, mientras que una ventana requiere 4 tablones de madera y una silla requiere 1,5 tablones de madera. Las demandas anuales estimadas son 100, 120 y 50 unidades de puertas, ventanas y sillas, respectivamente.

El gerente lo contrata a usted para que modele este problema mediante programación lineal, cuyo objetivo sea minimizar los costos de producción cumpliendo la demanda estimada.

4.14.1 Solución

Definición de conjuntos

- I : tipos de madera $\{1, 2, 3\}$, para roble, pino y cedro, respectivamente.
- J : artículos a producir $\{1, 2, 3\}$, para puertas, ventanas y sillas, respectivamente.
- K : formas de comprar $\{1, 2\}$, para compras de 100, o menos tablones y para compras de más de 100 tablones, respectivamente.

Definición de parámetros

- Dem_j : demanda estimada del artículo $j \in J$.
- $Costos_{ki}$: costo de la madera tipo $i \in I$ comprada de la forma $k \in K$.

- $CuanMad_j$: cantidad de tablonos que se requiere para fabricar artículo $j \in J$.

Definición de variables de decisión

- x_{ijk} : cantidad de tablonos de madera tipo $i \in I$ para fabricar artículo $j \in J$, comprada de la forma $k \in K$.
- $y_{ik} : \begin{cases} 1, & \text{si la materia prima tipo } i \in I \text{ es comprada de la forma } k \in K \\ 0, & \text{en otro caso} \end{cases}$

Modelo algebraico de programación lineal

Píldora informativa 4.16: Modos de ejecución

En los problemas donde existan **diferentes modos de ejecutar una acción** (formas de comprar, formas de producir, alternativas a tomar, entre otros), **se pueden modelar usando variables binarias** que representen cada modo.

Como ejemplo, al igual que en este problema, asumamos que se busca comprar un producto, cuyo precio cambia en función de una cantidad C_1 y C_2 . Para esto se definen dos variables: y_i variable binaria que representa la forma de comprar $i \in \{1, 2, 3\}$, y x_i que representa las unidades a comprar de la forma $i \in \{1, 2, 3\}$. Las restricciones necesarias para garantizar la dinámica serían:

- Garantizar que si se compra de la forma 1, se debe comprar una cantidad de madera menor o igual a C_1 .

$$x_1 \leq y_1 \cdot C_1$$

- Garantizar que si se compra de la forma 2, se debe comprar una cantidad de madera mayor C_1 , pero menor que C_2 .

$$x_2 \geq y_2 \cdot (C_1 + 1)$$

$$x_2 \leq y_2 \cdot C_2$$

- Garantizar que si se compra de la forma 3, se debe comprar una cantidad de madera mayor a C_2 .

$$x_3 \geq y_3 \cdot (C_2 + 1)$$

- Garantizar que solo se pueda comprar de una sola forma.

$$y_1 + y_2 + y_3 \leq 1$$

$$\text{Minimizar } z = \sum_{i \in I} \sum_{j \in J} \sum_{k \in K} \text{Costos}_{ki} \cdot x_{ijk} \quad (4.104)$$

Sujeto a:

$$0,9 \cdot \left(\sum_{i \in I} \sum_{k \in K} x_{ijk} \right) / \text{CuanMad}_j \geq \text{Dem}_j, \quad \forall j \in J \quad (4.105)$$

$$\sum_{j \in J} x_{ij1} \leq 100 \cdot y_{i1}, \quad \forall i \in I \quad (4.106)$$

$$\sum_{j \in J} x_{ij2} \geq 101 \cdot y_{i2}, \quad \forall i \in I \quad (4.107)$$

$$\sum_{j \in J} x_{ij2} \leq 500 \cdot y_{i2}, \quad \forall i \in I \quad (4.108)$$

$$\sum_{k \in K} y_{ik} \leq 1, \quad \forall i \in I \quad (4.109)$$

$$y_{ik} \in \{0, 1\}, \quad \forall i \in I, \forall k \in K \quad (4.110)$$

$$x_{ijk} \in \mathbb{Z}^+ \cup \{0\}, \quad \forall i \in I, \forall j \in J, \forall k \in K \quad (4.111)$$

La expresión (4.104) es la función objetivo del problema, que busca minimizar el dinero usado para comprar tablones de madera.

La restricción (4.105) tiene doble función: la primera es comprar suficiente materia prima como para cumplir la demanda. La segunda es garantizar la eficiencia del 90 %. Las restricción (4.106) garantiza la dinámica de las compras menores o iguales a 100 tablones. Por su parte, las restricciones (4.107) y (4.108) garantizan que la segunda forma de comprar sea con al menos 101 tablones. La restricción (4.109) establece que solo se puede comprar de una forma (menor o igual a 100 o mayor a 100). Finalmente, las expresiones (4.110) y (4.111) exponen la naturaleza de las variables.

El Algoritmo 45 contiene el código del modelo en AMPL. La solución obtenida en AMPL usando CPLEX alcanza un z de 4060 USD. Para esto se deben comprar 352 tablones de roble de la forma 2 (> 100), 500 tablones de pino de la forma 2 (> 100) y 100 tablones de cedro de la forma 1 (≤ 100).

Algoritmo 45: archivo Maderas.mod

```

1: set I := {1..3};
2: set J := {1..3};
3: set K := {1..2};
4: param Dem{J};
5: param Costos{K,I};
6: param CuanMad{J};
7: var x{I,J,K} >= 0 integer;
8: var y{I,K} binary;
9: minimize z: sum{i in I, j in J, k in K} Costos[k,i]*x[i,j,k];
10: s.t.
11: dem{j in J}: 0.9*(sum{i in I, k in K} x[i,j,k]) / CuanMad[j]
    >= Dem[j];
12: pre1{i in I}: sum{j in J} x[i,j,1] <= 100*y[i,1];
13: pre2{i in I}: sum{j in J} x[i,j,2] >= 101*y[i,2];
14: pre3{i in I}: sum{j in J} x[i,j,2] <= 500*y[i,2];
15: soloUno{i in I}: sum{k in K} y[i,k] <= 1;

```

4.15 Problema del túnel y la mafia

Usted lidera un grupo de criminales de la mafia siciliana llamado *La Familia Corleone*. Sus informantes le comentan que la gran mayoría de sus integrantes han sido capturados y puestos en una penitenciaría de máxima seguridad, luego de fracasar en el robo a un banco. Adicionalmente, los informantes le facilitan un estudio sobre las diferentes alternativas para realizar el escape de la prisión.

La única opción viable es hacer un túnel en el patio de la prisión, que conecta con las tuberías de la instalación. El terreno ha sido clasificado en 20 bloques, debido a su composición y dureza. Además, se ha estimado el tiempo que le llevará a sus hombres el excavar cada bloque, como se puede apreciar en la Tabla 4.28. Un esquema del terreno donde debe ser cavado el túnel se aprecia en la Figura 4.8.

El Padrino, confiando en su formación como ingeniero y especialista en optimización, le encarga diseñar el plan de escape. Para ello, usted debe crear la ruta de extracción de rocas, de tal manera que empiece en el patio de la prisión (cautividad en el plano) y que finalice en las tuberías (libertad en el plano), en el menor tiempo posible y extrayendo la menor cantidad de piedras.

Como condiciones especiales, considere que no se puede retirar una roca sin antes retirar aquellas que están encima de ella (por ejemplo, no se puede retirar la roca 6 sin haber retirado antes las rocas 1 y 2. Igualmente, la roca 14 no puede ser retirada sin haber extraído la roca 11). Respecto a la roca 15, considere que se requieren extraer las rocas 14 y 16 antes de retirarla. Los bloques que no están enumerados no pueden ser extraídos, por ejemplo, para retirar la piedra 17 solo hay que extraer la piedra 14.

Bloque de piedra	Tiempo de extracción [h]	Bloque de piedra	Tiempo de extracción [h]
1	2	11	1
2	1	12	3
3	3	13	2
4	1	14	3
5	2	15	2
6	1	16	3
7	3	17	2
8	1	18	1
9	2	19	1
10	3	20	3

Tabla 4.28: Tiempo de extracción de las piedras (horas).

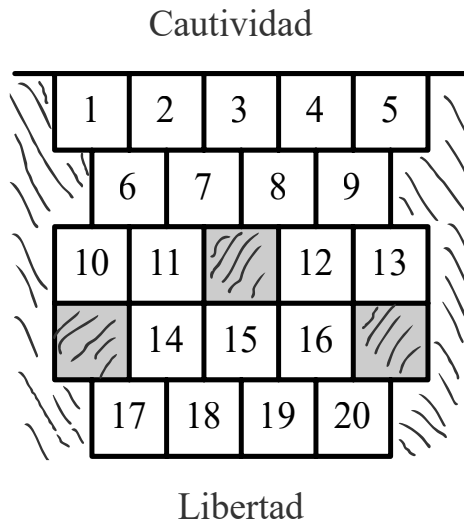


Figura 4.8: Esquema del terreno donde se planea hacer el túnel.

Formule el problema mediante un modelo de programación lineal que dé solución al escape de la prisión teniendo en cuenta las exigencias del Padrino.

4.15.1 Solución

Definición de conjuntos

- I : conjuntos de piedras $\{1, \dots, 20\}$.

Píldora informativa 4.17: Cotas no explícitas

En algunos problemas se requiere de cotas superiores (o inferiores), es decir, **valores máximos (o mínimos) de los que se está seguro que no se superarán**. En este problema, por ejemplo, es muy útil saber el número de horas máximo para hacer el escape, pues este valor se usará en la definición del número de variables de decisión. Una cota para este problema podría ser la suma del tiempo de extracción (D_i) de todas las piedras:

$$\text{Cota: } \sum_{i \in I} D_i = 40$$

Este valor representa el peor escenario del escape, por tanto, se tiene la seguridad de que la ruta óptima tendrá un valor menor o igual a 40 horas. Así, 40 es una cota superior al problema.

Por supuesto, se puede mejorar mucho el valor de la cota, por ejemplo, en lugar de usar el peor escenario, se puede hacer una ruta manualmente y tomar su valor de las horas de escape como la cota; con seguridad la ruta de escape óptima será menor o igual a dicho valor.

- T : conjunto de periodos (en horas) para realizar la extracción $\{1, \dots, T_{\max}\}$. Donde $T_{\max} = 40$ es una cota superior del tiempo (límite máximo).
- $SUCE$: conjunto de todas las relaciones de precedencias (i, j) entre las piedras. De modo que, para extraer la piedra i se debe extraer la piedra j : $\{(6, 1), (6, 2), (7, 2), (7, 3), (8, 3), (8, 4), (9, 4), (9, 5), (10, 6), (11, 6), (11, 7), (12, 8), (12, 9), (13, 9), (14, 11), (15, 14), (15, 16), (16, 12), (17, 14), (18, 14), (18, 15), (19, 15), (19, 16), (20, 16)\}$.
- FIN : conjunto de piedras conectadas directamente con la libertad $\{17, 18, 19, 20\}$.

Definición de parámetros

- D_i : tiempo (en horas) de extracción de la piedra $i \in I$.
- M : parámetro con un valor muy grande en comparación con los demás parámetros.

Definición de variables de decisión

- $x_{it} : \begin{cases} 1, & \text{si la piedra } i \in I \text{ se extrae en el periodo } t \in T \\ 0, & \text{en otro caso} \end{cases}$

Píldora informativa 4.18: Variables de decisión como contenedores

Como se explicará en la Píldora Informativa 4.19, la función objetivo de este problema no almacenará las horas requeridas para el escape, por tanto, sería útil guardar este valor para reportarlo en la solución. Para estos casos se usan unas **variables de decisión, cuyo único propósito en el modelo es servir como contenedores y almacenar información**. Es decir, no influyen en las decisiones tomadas en el modelo matemático.

- *horas* : número de horas usadas para hacer el escape.

Modelo algebraico de programación lineal

Píldora informativa 4.19: Funciones objetivo que no representan exactamente el objetivo

El propósito de una función objetivo (FO) es medir la calidad de una solución y con base en su valor, ayudar a un modelo matemático a hacer la búsqueda de la mejor solución que maximice o minimice su valor. Sin embargo, esto no obliga a que el valor numérico de la FO realmente represente el objetivo de un problema. Esto se puede apreciar principalmente en modelos con restricciones penalizadas en la FO o bien en FO con múltiples objetivos que se encuentran normalizados en una escala adimensional.

Por ejemplo, en este caso se busca minimizar el tiempo para escapar de la prisión. La ecuación que representa esto sería:

$$\text{Minimizar } z = \sum_{i \in FIN} \sum_{t \in T} x_{it} \cdot (t + D_i) \quad (FO\alpha)$$

Aunque correcta, esta FO solo minimizaría el tiempo en llegar a algún nodo perteneciente al conjunto *FIN*, más su tiempo de extracción. Sin embargo, nada impide extraer más piedras de las necesarias, siempre y cuando no afecten al tiempo final. Pero lo anterior no es lo que busca el problema; el objetivo es solo escaparse lo más rápido, no minimizar el número de rocas extraídas. Para resolver esta problemática se plantea la siguiente expresión:

$$\text{Minimizar } z = \sum_{i \in I} \sum_{t \in T} x_{it} \cdot (t + D_i) \quad (FO\beta)$$

En este caso, la nueva FO no representa el tiempo tomado para el escape, pero parece ser más adecuada. Sin embargo, aún tiene problemas, pues minimizar el tiempo de inicio de la última piedra más su duración ($FO\alpha$) no es igual que minimizar el inicio de cada piedra extraída más la duración correspondiente ($FO\beta$). Por tanto, es necesario unificar ambos enfoques (ecuación 4.113). La FO resultante tampoco representa el tiempo requerido para realizar el escape, pero es adecuada.

$$\text{Minimizar } z = \sum_{i \in I} \sum_{t \in T} x_{it} \cdot D_i + \text{horas} \quad (4.112)$$

Sujeto a:

$$\sum_{t \in T} x_{it} \leq 1, \quad \forall i \in I \quad (4.113)$$

$$\sum_{t \in T} \sum_{i \in FIN} x_{it} \geq 1 \quad (4.114)$$

$$\sum_{t \in T} x_{jt} \cdot (t + D_j) \leq M \cdot (1 - \sum_{t \in T} x_{it}) + \sum_{t \in T} x_{it} \cdot t, \quad \forall (i, j) \in SUCE \quad (4.115)$$

$$\sum_{t \in T} x_{jt} \geq \sum_{t \in T} x_{it}, \quad \forall (i, j) \in SUCE \quad (4.116)$$

$$\text{horas} = \sum_{i \in FIN} \sum_{t \in T} x_{it} \cdot (t + D_i) \quad (4.117)$$

$$x_{it} \in \{0, 1\}, \quad \forall i \in I, \forall t \in T \quad (4.118)$$

$$\text{horas} \in \mathbb{Z}^+ \cup \{0\} \quad (4.119)$$

La expresión (4.112) es la función objetivo del problema, está compuesta por dos componentes: el primero ($\sum_{i \in I} \sum_{t \in T} x_{it} \cdot D_i$) busca minimizar el número de piedras extraídas; el segundo (horas) busca minimizar el tiempo requerido en la extracción.

La restricción (4.113) asegura que una piedra solo puede ser extraída una única vez. Las ecuación (4.114) obliga al sistema a terminar el túnel en

alguna de las piedras 17, 18, 19 o 20. La restricción (4.115) establece que para cada pareja de piedras (i, j) que tengan una relación de precedencia, la piedra j sea extraída antes que la i . Para que esta restricción funcione es indispensable que las piedras relacionadas hayan sido extraídas; esto se expone en la restricción (4.116). La expresión (4.117) define la variable *horas* como el número de horas necesarias para realizar el escape. Finalmente, las restricciones (4.118) y (4.119) exponen la naturaleza de las variables.

El Algoritmo 46 contiene el código del modelo en AMPL. La solución obtenida en AMPL usando CPLEX logra el escape en 13 horas. La ruta de escape se muestra en la Figura 4.9.

Algoritmo 46: archivo Corleone.mod

```

1: set I := {1..20};
2: param Tmax = 40;
3: set T := {1..Tmax};
4: set FIN := {17, 18, 19, 20};
5: set SUCE := {(6, 1), (6, 2), (7, 2), (7, 3), (8, 3), (8, 4),
    (9, 4), (9, 5), (10, 6), (11, 6), (11, 7), (12, 8), (12, 9),
    (13, 9), (14, 11), (15, 14), (15, 16), (16, 12), (17, 14), (18, 14),
    (18, 15), (19, 15), (19, 16), (20, 16)};
6: param D{I};
7: param M = 100000;
8: var x{I,T} binary;
9: var horas >= 0;
10: minimize z : sum{i in I, t in T} x[i,t]*D[i] + horas;
11: s.t.
12: r1{i in I}: sum{t in T} x[i,t] <= 1;
13: r2: sum{t in T, i in FIN} x[i,t] >= 1;
14: r3{(i,j) in SUCE}: sum{t in T} x[j,t]*(t+D[j]) <= M*(1 - sum
    {t in T} x[i,t]) + sum{t in T} x[i,t]*t;
15: r4{(i,j) in SUCE}: sum{t in T} x[j,t] >= sum{t in T} x[i,t];
16: r5: horas = sum{i in FIN, t in T} x[i,t]*(t+D[i]);

```

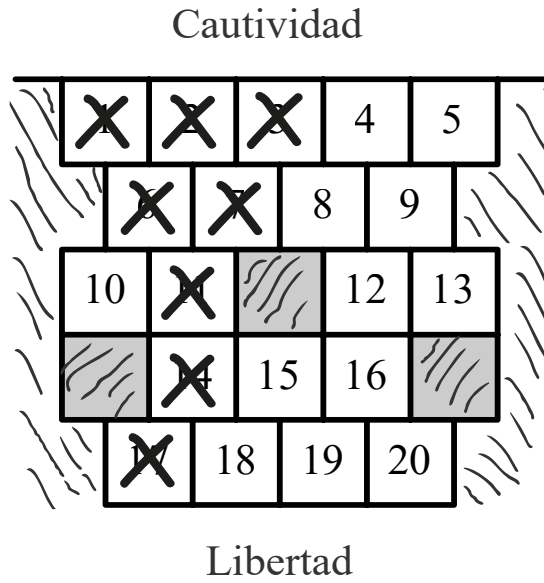


Figura 4.9: Esquema de la ruta de escape óptima.

Capítulo 5

Guía Rápida - Uso de AMPL

Se presenta una guía de consulta rápida e introductoria, con las principales funciones del lenguaje AMPL y su Entorno de Desarrollo Integrado (IDE, por su sigla en inglés). Para mayor detalle se remite al lector al manual oficial “The AMPL Book - AMPL”. [Online]. Disponible en: <https://ampl.com/resources/the-ampl-book/> [6].

5.1 Preguntas frecuentes

✓ ¿Qué es AMPL?

AMPL (*A Mathematical Programming Language*) es un lenguaje de programación matemática, especializado en el modelado de problemas de optimización lineales y no lineales. Dentro de sus principales características está la completa compatibilidad con la notación algebraica y su sintaxis natural para operaciones aritméticas (uso de sumatorias y demás operadores).

AMPL ofrece una versión ejecutable, tipo consola, y un IDE (*Integrated Development Environment*) en MacOS, Linux y Windows, para modelar y resolver los modelos de programación matemática.

✓ ¿Qué es un IDE?

Un IDE (*Integrated Development Environment*) es una aplicación que facilita herramientas para facilitar la programación al usuario. El IDE de AMPL proporciona un editor de texto, una consola y la conexión con los paquetes de *software* para optimización (motores de solución).

✓ ¿Cómo resuelve AMPL los problemas de programación matemática?

AMPL opera como un intérprete entre el modelo matemático (en un lenguaje compatible con AMPL) y los diferentes motores de solución disponibles en el mercado (*solvers*), como CPLEX o Gurobi. Un esquema del funcionamiento se muestra en la Figura 5.1.

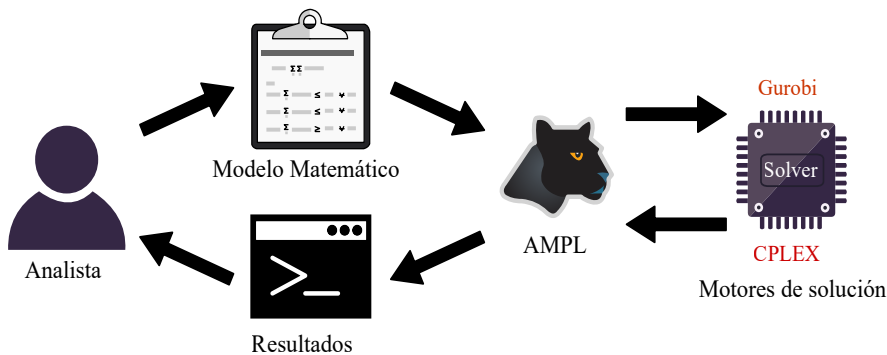


Figura 5.1: Dinámica de AMPL al resolver un problema.

✓ ¿Qué es NEOS Server?

NEOS (*Network-Enabled Optimization System*) Server es un servicio gratuito dedicado a la resolución de problemas de optimización numérica, incluyendo modelos de programación lineal. Usado tanto a nivel científico como industrial. El servidor está ubicado en la Universidad de Wisconsin en Madison. Neos Server incluye más de 60 *solvers*, incluidos CPLEX y Gurobi. El envío de trabajos al servidor es compatible con el lenguaje AMPL. Más información en “NEOS Server: State-of-the-Art Solvers for Numerical Optimization”. [Online]. Disponible en: <https://neos-server.org/neos/> [7]-[10].

✓ ¿Por qué AMPL?

La selección de AMPL en este libro responde a tres premisas:

1. Es uno de los lenguajes más cercanos a la escritura matemática formal, por tanto, pasar el modelo matemático al lenguaje AMPL es fluido e intuitivo.
2. Basándose en las estadísticas de uso en NEOS Server, AMPL es el lenguaje más usado en el mundo.
3. Se puede integrar fácilmente con Python, C++ o Java mediante sus APIs (*Application Programming Interface*).

✓ ¿AMPL es gratis? ¿Dónde lo puedo descargar?

AMPL tiene una versión de demostración con un límite de 500 variables y 500 restricciones para problemas lineales [11]. Para fines académicos, se puede solicitar una licencia gratuita. Para fines comerciales consultar directamente la página oficial [12].

✓ ¿Existe alguna alternativa completamente gratis sin restricciones?

Como AMPL es un lenguaje, existen varias alternativas open-source compatibles. Por ejemplo, el IDE GUSEK, junto con el *solver* de modelos lineales gratuito GLPK (GNU Linear Programming Kit) [13].

5.2 Instalación y ejecución

5.2.1 Instalación

AMPL es compatible con Windows, Linux y MacOS, en plataformas de 32 y 64 bits. Por lo general, una instalación limpia es suficiente para ejecutar correctamente AMPL. Para todas las plataformas el instalador se descarga en un archivo comprimido (con extensión `<.zip>` para Windows o `<.tgz>` para Linux y MacOS), que solo requiere ser descomprimido en la carpeta donde se desea tener la copia local de AMPL.

En el caso particular de MacOS, se requiere proporcionar los permisos al ejecutable para ser detectado como un desarrollador de confianza. Para mayor detalle de la instalación en cada plataforma consultar la web oficial [11].

5.2.2 Ejecución

Para su ejecución se debe buscar el archivo ejecutable con nombre `<amplide>` (en Windows se encuentra dentro de la carpeta con el mismo nombre); no confundir con el archivo con nombre similar `<ampl>`. Este último es solo la consola de AMPL, mientras que `<amplide>` es todo el entorno de desarrollo, que incluye la consola y el editor.

5.3 Interfaz de AMPL IDE

La interfaz de AMPL IDE está compuesta por tres pestañas (Figura 5.2):

1. **Directorio activo:** aquí se encontrarán los archivos generados según el uso de la aplicación. Se debe tener en cuenta que lo más recomendable es crear una carpeta en su computadora, en donde se puedan guardar únicamente los archivos de AMPL. Cuando se ejecute un modelo AMPL, buscará los archivos en esta carpeta.
2. **Consola:** definida como la interfaz de línea de comando, la consola se usa para establecer la comunicación entre AMPL y el usuario. Aquí usted podrá ejecutar su modelo y también ver la respuesta que se ha encontrado.

También, en caso de haber un error de escritura en el código (como el modelo o los datos), el programa indicará en qué línea se ha detectado dicho error y de qué tipo.

3. **Editor:** es donde se va a realizar la escritura del código de los archivos necesarios. Aquí se escribirán los modelos, datos y la rutina de ejecución.

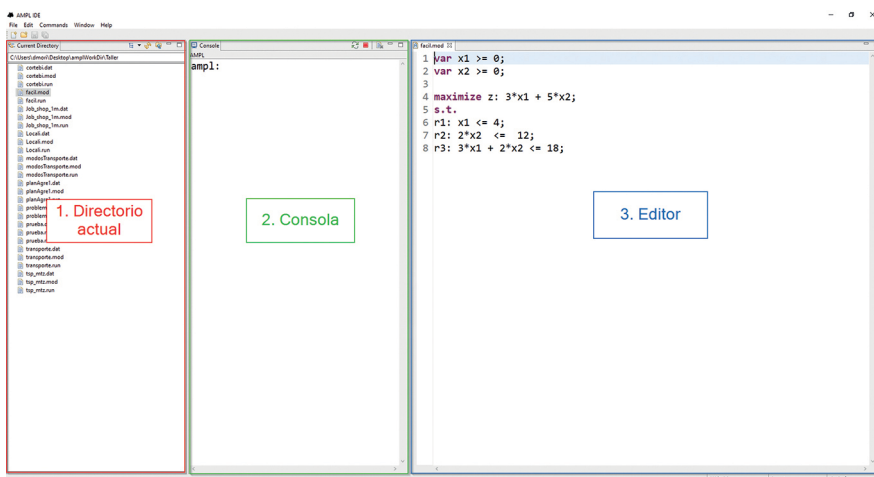


Figura 5.2: Partes de AMPL IDE.

5.4 Archivos necesarios

Para ejecutar un modelo en AMPL se deben crear de forma obligatoria dos archivos. El primero debe tener la extensión **<.mod>**. En este archivo es donde se realizará la escritura del modelo matemático del problema a resolver. El segundo debe tener terminación **<.run>**. Este es la rutina de ejecución, es la encargada de comunicarse con AMPL, tanto para cargar los modelos, configurar los *solvers* y hacer la impresión de los resultados.

Por otra parte, existe un último archivo que es opcional, con una extensión **<.dat>**. En él se consignan los parámetros que son necesarios para la ejecución del modelo. No es obligatorio la creación de este archivo como en los casos anteriores, pero es especialmente útil cuando se realizan formulaciones de forma algebraicas.

5.4.1 Creación de archivos

Los archivos **<.mod>**, **<.run>** y **<.dat>** se crean de la siguiente forma:

1. Primero se debe de ir a la esquina superior izquierda y dar clic en ícono de **<New File>** (Figura 5.3a), o bien ir al menú **<File>**, luego en el opción **<New>** y, finalmente, dar clic en **<File>** (Figura 5.3b).
2. Luego aparecerá la ventana de asistente para la creación de un archivo (**<New File Wizard>**), como se ilustra en la Figura 5.4. En esta ventana se encuentran tres partes:

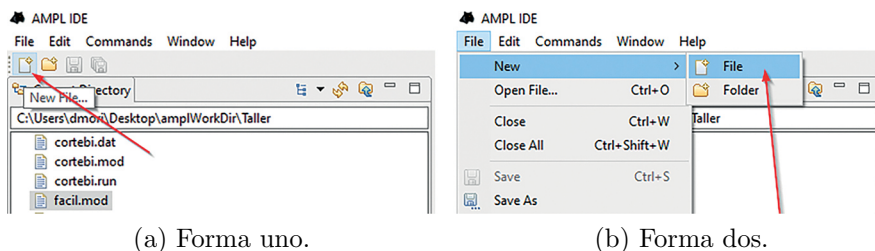


Figura 5.3: Formas de crear archivos en AMPL IDE.

- **<Directory>**: en esta caja de texto se muestra la ubicación en el disco duro donde se creará el archivo. Se debe verificar que se encuentre el nombre de la carpeta que usted ha creado para guardar los archivos de AMPL. De no ser así, puede ir a **<Browse>**, dar clic y buscar la carpeta.
- **<File Name>**: en esta caja de texto se muestra el nombre del archivo a crear. No incluir de forma explícita la extensión, pues esta se definirá en la siguiente parte. Igualmente, se recomienda no usar espacios ni caracteres de acento.
- **<File Extension>**: lista desplegable con las posibles extensiones del archivo a crear. Se recomienda tener solo un archivo de cada extensión para cada modelo.

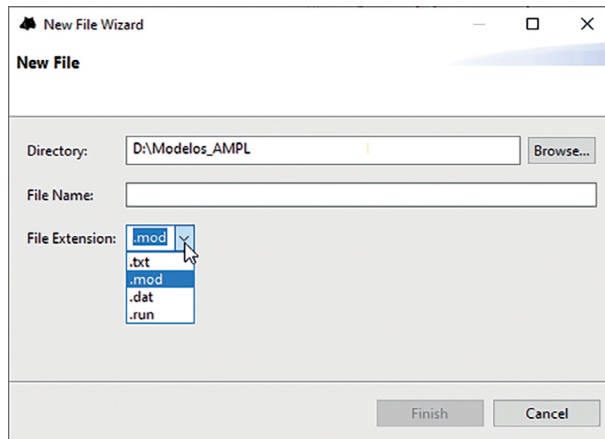


Figura 5.4: Asistente para la creación de archivos en AMPL IDE.

Para crear un archivo basta con diligenciar esta ventana (con un nombre y extensión válidos) y darle clic en el botón **<Finish>**.

Píldora informativa 5.1: Nombres de los archivos

Como se manejarán dos o tres archivos por problema, **se recomienda usar el mismo nombre** en cada uno. Así, se tendrían los archivos: **<ejemplo.mod>**, **<ejemplo.dat>** y **<ejemplo.run>**.

5.5 Código para el archivo `<.run>`

Este archivo es el encargado de establecer una comunicación entre el usuario y AMPL. Será a través de este que se darán las órdenes de ejecutar, resolver y mostrar la solución de los problemas. Además, en este archivo se pueden establecer las opciones del *solver*.

El código base es muy similar en cada problema. Generalmente solo varía el tipo de *solver* a utilizar y lo que se desea mostrar como resultados (y su formato). A continuación, se presenta una lista de los comandos¹³ básicos usados en el archivo `<.run>`.



Nota: AMPL utiliza el carácter de punto y coma (`<;>`) para indicar el final de una instrucción. Así, cada instrucción debe estar delimitada con este carácter.

5.5.1 Comandos básicos

`</> reset:` borra todos los elementos y configuraciones anteriores; indispensable para cada nueva ejecución.

Ejemplo: **`reset;`**

`</> model:` permite cargar el archivo donde está el modelo matemático.

Ejemplo: **`model nombre.mod;`**

`</> option solver:` permite seleccionar el *solver* a utilizar para resolver el problema, como CPLEX o Gurobi.

Ejemplo: **`option solver gurobi;`**

`</> solve:` es la instrucción que ejecuta el *solver* para resolver el problema. Si no se usa esta instrucción, los resultados de los modelos serán 0, pues no se ejecuta ningún *solver* y por tanto no se resuelve el problema.

¹³Son instrucciones u órdenes que el usuario proporciona mediante la consola.

Ejemplo: `solve;`

</> `display`: permite recuperar la solución proporcionada por el *solver*. Se usa para mostrar por pantalla el valor de las variables, parámetros, conjuntos, función objetivo y cualquier otro elemento del modelo.

Ejemplo: `display z, x, y;`

</> `_nvars` y `_ncons`: muestra el número de variables y restricciones, respectivamente.

Ejemplo:
`display _nvars;`
`display _ncons;`

Finalmente, en el Algoritmo 47 se muestra un ejemplo completo.

Algoritmo 47: archivo <ejemplo.run>

```
1: reset;  
2: model <archivo.mod>;  
3: data <archivo.dat>;  
4: option solver gurobi;           ► También se puede usar cplex  
5: <insertar aquí opciones del solver>  
6: solve;  
7: display <variables o parámetros>;
```

5.5.2 Comandos para la visualización de resultados

Además, en el archivo <.run> se pueden incluir algunos comandos para mejorar la visualización de los resultados. Estos comandos se pueden ubicar luego de la línea <`reset`>. Los más utilizados se resumen a continuación:

</> `omit_zero_cols` y `omit_zero_rows`: elimina aquellos valores que tienen un valor de cero al mostrar los resultados.

Ejemplo:

```
option omit_zero_cols 1;
option omit_zero_rows 1;
```

</> display__transpose: activa (usar un valor grande positivo) o desactiva (usar un valor grande negativo) la visualización de vectores y matrices de forma transpuesta.

Ejemplo: **option** display_transpose -20;

</> timelimit o **timelim:** establece un límite de tiempo (en segundos) en la ejecución del *solver*, de no poder resolver el problema en dicho límite, AMPL retornará la mejor solución encontrada.

Nota: los comandos **<logfreq>**¹⁴ y **<mipdisplay>**¹⁵ especifican el nivel de detalle en la impresión de resultados. Por otra parte, el comando **<outlev>** habilita la impresión de dichos resultados.

Ejemplo:

Usando Gurobi:

```
option solver gurobi;
option gurobi_options 'timelim=60 outlev=1 logfreq=2';
```

Usando CPLEX:

```
option solver cplex;
option cplex_options 'timelimit=5 outlev=1 mipdisplay=2';
```

</> display_precision: número de dígitos de precisión mostrados. La precisión máxima se consigue con el valor 0.

Ejemplo: **option** display_precision 6;

¹⁴Más detalle en “AMPL-Gurobi Parameter Reference”. [Online]. Disponible en: <https://www.gurobi.com/documentation/9.1/ampl-gurobi/parameters.html>

¹⁵Más detalle en “CPLEX Options for AMPL”. [Online]. Disponible en <https://ampl.com/products/solvers/solvers-we-sell/cplex/options/>

5.5.3 Comandos avanzados

Finalmente, a continuación, se muestran algunos comandos avanzados de visualización.

</> expand: muestra, por consola, de forma extendida, una expresión algebraica, en formato estándar, es decir, las variables al lado izquierdo de la igualdad (o desigualdad) y las constantes al lado derecho.

Ejemplo: `expand z;`

</> body: muestra el valor actual de una restricción, es decir, las sumas de las variables y sus parámetros (lado izquierdo de la desigualdad). Por actual, se entiende el valor de la restricción evaluado en la solución encontrada.

Ejemplo: `display r1.body;`

</> dual: muestra el *precio sombra* o *precio dual* (PD) de una restricción. Este valor se puede interpretar como el incremento (o decremento) de la función objetivo si se aumenta (o disminuye) en una unidad del recurso asociado a dicha restricción.

Ejemplo: `display r1.dual;`

</> rc: muestra el *costo reducido* (CR) de una variable. Este valor existe únicamente cuando una variable tiene el valor de cero en la solución óptima. El CR se puede interpretar como el costo a reducir (o utilidad a aumentar) para que el modelo incluya la variable en la solución óptima.

Ejemplo: `display x1.rc;`

</> up y down: muestra el intervalo donde es válida la interpretación del *costo reducido* (para las variables) o del *precio sombra* (para las restricciones).

Nota: se debe incluir la siguiente opción de sensibilidad para usar estos comandos:

```
option cplex_options 'sensitivity';
option presolve 0;
```

Ejemplo:

```
display x1.up;
display x1.down;
```

</> solve_result_num: muestra un número (llamado *status*) que representa el estado luego de la ejecución del *solver*. Este número informará si el resultado fue óptimo, o si solo se encontró un valor factible, se llegó a límite de tiempo, es un problema no acotado o infactible, entre otros.

La interpretación del *status* depende del *solver* a utilizar:

- Para CPLEX consultar “Netlib Repository at UTK and ORNL - CPLEX”. [Online]. Disponible en: <http://www.netlib.org/ampl/solvers/cplex/README.cplex>.
- Para Gurobi consultar “Netlib Repository at UTK and ORNL - Gurobi”. [Online]. Disponible en: <http://www.netlib.org/ampl/solvers/gurobi/README.gurobi>.

Ejemplo: **display solve_result_num;**

</> _solve_elapsed_time: muestra los segundos utilizados para resolver el problema.

Ejemplo: **display _solve_elapsed_time;**

</> relmipgap: muestra el *gap* obtenido luego de ejecutar el *solver*.

Píldora informativa 5.2: El *gap* de un modelo entero

El *gap* es la diferencia porcentual entre la mejor solución entera obtenida (*MS*) y la **mejor cota** encontrada hasta ese momento (*MC*). Esta última generalmente es el valor óptimo de la **relajación del modelo entero**. La mejor cota es un valor límite para un modelo, del que se tiene la seguridad que no se puede superar.

La expresión para calcular el *gap* es:

$$GAP = \frac{MS - MC}{MC} \cdot 100 \quad (\text{para minimización})$$

$$GAP = \frac{MC - MS}{MC} \cdot 100 \quad (\text{para maximización})$$

Cuanto más cercano sea el valor del *gap* a cero, mejor es la solución encontrada. Si el *gap* es cero, se tiene la seguridad que se ha alcanzado el óptimo.

Píldora informativa 5.3: Relajación de un modelo de programación lineal

La relajación de un modelo de programación lineal es el modelo obtenido al eliminar las restricciones de integralidad de cada variable. Por ejemplo, si se tienen variables binarias de la forma:

$$x_{ij} \in \{0, 1\}, \quad \forall i \in I, \forall j \in J,$$

esta restricción se reemplazaría por:

$$0 \leq x_{ij} \leq 1, \quad \forall i \in I, \forall j \in J.$$

Para obtener el *gap* se debe incluir un comando especial en las opciones del *solver*:

CPLEX: `option cplex_options 'return_mipgap=1';`
Gurobi: `option gurobi_options 'bestbound=1 return_mipgap=3';`

Para consultar el valor del *gap* se realiza igual en ambos *solvers*. Por ejemplo si *z* el nombre de la función objetivo de un problema, la forma de consultar este valor sería:

Ejemplo: `display z.relmpgap;`

5.6 Código para el archivo <.mod>

Este archivo debe contener todo lo referente al modelo matemático y se puede escribir de forma algebraica o extendida.



Caracteres básicos

- **Llaves** $\{ \cdot \cdot \cdot \}$: se usan para hacer definiciones dentro del modelo. Por ejemplo, definir un conjunto, una variable (vector o matriz), una sumatoria, entre otros. Las condiciones especiales de una sumatoria, por ejemplo, también estarían dentro de unas llaves.
- **Corchetes** $[\cdot \cdot \cdot]$: se usan para acceder a los valores de las variables o parámetros mediante subíndices en la función objetivo o en las restricciones.

5.6.1 Definición de conjuntos

La declaración de conjuntos en el archivo <.mod> tiene la siguiente forma:

```
set <nombre>:= {<elementos>:<condiciones>;}
```

Ejemplo 1:

set I := {1..10};	← elementos de 1 a 10.
set SUC := {(1.2), (2.3), (0,8)};	← los elementos son las parejas.
set P := {'tv', 'radio', 'micro'};	← elementos no numéricos.
set YEAR := {10..100 by 5};	← elementos: 10, 15, ..., 95 y 100.



Se puede usar los dos puntos $<:>$ al final de una expresión para indicar condiciones (expresiones lógicas) de los conjuntos.

Ejemplo 2:

```
set S := {1..3};
```

```
set A := {i in S, j in S: i <> j};
```

El conjunto A representaría todas las parejas (i, j) con valores de 1 a 3 con $i \neq j$: $(1, 2)$, $(1, 3)$, $(2, 1)$, $(2, 3)$, $(3, 1)$ y $(3, 2)$.

Ejemplo 3:

```
set T := {1..2};
```

```
set S{T};
```

Define un vector de conjuntos S . De forma que $S[1]$ es un conjunto y $S[2]$ sería otro conjunto. Tanto $S[1]$ y $S[2]$ se deben definir en el archivo **<.dat>**. Para mayor información consultar la Sección [5.7.1](#).

AMPL incluye las operaciones básicas de conjuntos:

</> diff: operador de diferencia entre conjuntos. Equivalente al símbolo $(\setminus)$ en notación matemática. Se usa para sustraer elementos entre conjuntos.

Ejemplo:

```
set A := {1,2,3,4,5}; set B := {3,4};
```

```
set C := A diff B;
```

En este caso $C = \{1, 2, 5\}$.

</> union: operador de unión entre conjuntos. Equivalente al símbolo $(\cup)$ en notación matemática. Se usa para adicionar elementos entre conjuntos.

Ejemplo:

```
set A := {1,2,3}; set B := {4,5};
```

```
set C := A union B;
```

En este caso $C = \{1, 2, 3, 4, 5\}$.

</> within: operador de subconjunto. Equivalente al símbolo $(\subseteq)$ en notación matemática.

Ejemplo:**set** A := {1..10};**B within** A;Todos los elementos de B deben estar en el conjunto A .

Cuando se define un conjunto como el subconjunto de otro, en este ejemplo, $B \subseteq A$, B puede estar definido en el archivo <.dat>. Para mayor información consultar la Sección 5.7.1.

</> **inter:** operador de intersección entre conjuntos. Equivalente al símbolo ($\cap$) en notación matemática. Se usa para establecer elementos comunes entre conjuntos.

Ejemplo:**set** A := {1,2,3,4,5}; **set** B := {4,5,6,7,8};**set** C := A **inter** B;En este caso $C = \{4, 5\}$.

5.6.2 Definición de parámetros

La declaración de parámetros en el archivo <.mod> tiene la siguiente forma:

```
param <nombre>{<conjuntos>:<condiciones>;
```

Se pueden declarar parámetros definidos o indefinidos. Los primeros están numéricamente escritos en el archivo <.mod>; los segundos están numéricamente escritos en el archivo <.dat>.



Un parámetro puede estar indexado en un conjunto. Así se transforma en un vector (una dimensión), una matriz (dos dimensiones) o un contenedor (tres o más dimensiones). Para esto se acompaña al nombre del parámetro con los conjuntos a indexar entre llaves ($\{\}$).

Ejemplo 1:

param B;	← parámetro indefinido.
param B = 100;	← parámetro definido.
param B{I};	← vector de parámetros indefinidos.
param B{I,J};	← matriz de parámetros indefinidos.
param B{I,J,K};	← contenedor de parámetros indefinidos.
param B{I,J,K} default 0;	← igual al anterior pero lleno de ceros.



Al igual que en los conjuntos, se pueden usar los dos puntos `<:>` al final de una expresión para indicar condiciones (expresiones lógicas) de los conjuntos indexados. Por ejemplo, establecer que un subíndice debe ser diferente a otro.

Ejemplo 2:

```
set S := {1..3};
param B{i in S, j in S: i>j};
```

La matriz B está definida para las parejas (i, j) con $i > j$: $B = \{(2, 1), (3, 1), (3, 2)\}$.



A veces es útil crear un parámetro del mismo tamaño que un conjunto (como un vector o una matriz). Para estos casos puede usar el módulo de un conjunto. En AMPL el comando correspondiente es **<card>**. También se puede consultar el valor del módulo con ayuda del comando **<display>**.

Ejemplo 3:

```
set S := {'fresa', 'naranja', 'mango', 'piña'};
param B{card(S)};
```

El contenedor indefinido B se define con una capacidad de 4 elementos.

5.6.3 Definición de variables

La declaración de variables en el archivo `<.mod>` tiene la siguiente forma:

```
var <nombre>{<conjuntos>:<condiciones>} <dominio>;
```

En <dominio> se debe establecer la naturaleza de la(s) variable(s) definidas. Por ejemplo, mayores a cero, binarias, sin restricción de signo, enteras, entre otras.

Ejemplo 1:

```
set S := {1..3};
var x{S} >= 0;
var x{S} >= 0 integer;
var x{S,S} binary;
var x{S} <= 0;
```



Al igual que en los conjuntos y parámetros, se pueden usar los dos puntos <:> al final de una expresión para indicar condiciones (expresiones lógicas) de los conjuntos indexados.

Ejemplo 2:

```
set S := {1..3};
var x{i in S, j in S: i = j} binary;
```

Variables binarias definidas: x_{11} , x_{22} y x_{33} .

5.6.4 Definición de expresiones lineales

Las expresiones lineales están compuestas por parejas de dos elementos: un parámetro (constante) y una variable de decisión, ambos multiplicados entre sí. Estas parejas se suman para conformar una expresión completa.

Expresiones extendidas. Ejemplo 1:

$$3 \cdot x_1 + 4 \cdot x_2 + 10 \cdot x_3;$$

$$\leftarrow 3 \cdot x_1 + 4 \cdot x_2 + 10 \cdot x_3$$


Para definir una sumatoria (Σ), en AMPL se puede usar el comando <sum>. Para sumatorias dobles, triples o de orden superior, basta con separar los límites de la sumatoria mediante comas <,>, pero se deben establecer los subíndices y los conjuntos a los que pertenecen.

Expresiones algebraicas. Ejemplo 2:

$$\begin{aligned}
 \text{sum}\{i \text{ in } I\} c[i]*x[i]; & \leftarrow \sum_{i \in I} c_i \cdot x_i \\
 \text{sum}\{i \text{ in } I, j \text{ in } J\} c[i,j]*x[i,j]; & \leftarrow \sum_{i \in I} \sum_{j \in J} c_{ij} \cdot x_{ij} \\
 \text{sum}\{i \text{ in } I, j \text{ in } J, k \text{ in } K\} c[i,j,k]*x[i,j,k]; & \leftarrow \sum_{i \in I} \sum_{j \in J} \sum_{k \in K} c_{ijk} \cdot x_{ijk}
 \end{aligned}$$



Al igual que en los conjuntos y parámetros, se pueden usar los dos puntos <:> al final de una expresión para indicar condiciones (expresiones lógicas) de los conjuntos indexados.

Expresiones algebraicas. Ejemplo 3:

$$\text{sum}\{i \text{ in } I, k \text{ in } K: i>2, k<>i\} c[i,k]*x[i,k]; \leftarrow \sum_{i \in I, i>2} \sum_{k \in K, k \neq i} c_{ijk} \cdot x_{ijk}$$

5.6.5 Definición de la función objetivo

La declaración de función objetivo en el archivo <.mod>. Se acostumbra a usar como <nombre> la letra z, pero se puede usar cualquier otra e incluso se puede usar una cadena de caracteres. Esta declaración tiene la siguiente forma:

<maximize o minimize> <nombre>: <expresiones>;

Luego de la función objetivo se debe usar un comando para indicar que, seguida a esta función, se escribirán las restricciones. En los modelos matemáticos se acostumbra escribir la frase sujeto a. En AMPL también se usa esta frase, pero escrita en inglés.

subject to o subj to o s.t.

5.6.6 Definición de las restricciones

La declaración de las restricciones en el archivo <.mod> tiene la siguiente forma:

<nombre> <para todo>: <expresiones> <símbolo> <expresiones>;

Cada restricción se debe definir usando un nombre de referencia, con el cual podemos consultar sus propiedades con el comando <display>. Este nombre se puede reconocer fácilmente, pues se encuentra al principio de una línea y seguido de dos puntos <:>.

Ejemplo 1:

r1: $\text{sum}\{i \text{ in } I\} x[i]*c[i] \leq 100$



Para usar el símbolo matemático “para todo” ($\forall$), se deben poner los conjuntos referentes a él justo antes de los dos puntos <:>, luego del nombre.

Ejemplo 2:

r1{i in I}: $\text{sum}\{j \text{ in } J\} x[i,j] \leq 1$; $\leftarrow r1: \sum_{j \in J} x_{ij} \leq 1, \forall i \in I$

r1{i in I, t in T}: $y[i,t] \leq 1$; $\leftarrow r1: y_{it} \leq 1, \forall i \in I, \forall t \in T$

5.7 Código para el archivo <.dat>

Este archivo puede contener todo lo referente a los valores constantes del modelo como los parámetros o las definiciones de los conjuntos. Se trata de un archivo **opcional**, pero altamente útil para manejar modelos cuando se usa notación algebraica.

5.7.1 Declarar conjuntos

Se puede declarar un conjunto en el archivo `<.dat>`, en los siguientes casos:

- Si se ha definido en el archivo `<.mod>` un conjunto, sin especificar sus valores:

```
set K;
```

En este caso el código correspondiente en el archivo `<.dat>` podría ser:

Ejemplo 1:

```
set K := 1 2 3 4 5 6;  
set K := 15 20 25 30;  
set K := 'A' 'B' 'C';
```

- Si se ha definido en el archivo `<.mod>` un conjunto dependiente de otros conjuntos, ambos sin especificar sus valores:

```
set T := {1..2};  
set S{T};  
set L{T};
```

En este caso el código correspondiente en el archivo `<.dat>` podría ser:

Ejemplo 2:

```
set S[1] := 1 2 3;  
set S[2] := 4 5 6;  
set L[1] := 1 2 3;  
set L[2] := 4 5 3;
```

5.7.2 Declarar parámetros

Para declarar un parámetro en el archivo `<.dat>` se deben tener en cuenta las dimensiones de este: puntual (un solo valor), unidimensional (un vector), bidimensional (una matriz) y tridimensional u orden superior (un contenedor).

</> Puntual (un solo valor): si se ha definido en el archivo <.mod> un parámetro de la siguiente forma:

```
param Dem;
```

El código para definir el parámetro en el archivo <.dat> podría ser:

Ejemplo 1:

```
param dem = 8;
```

</> Unidimensional (un vector): si se ha definido en el archivo <.mod> un parámetro de la siguiente forma:

```
set T:= {1..4};  
param Dem{T};
```

El código para definir el parámetro en el archivo <.dat> podría ser:

Ejemplo 1:

```
param dem :=  
  1    10  
  2    15  
  3    35  
  4     9  
;
```

</> Bidimensional (una matriz): si se ha definido en el archivo <.mod> un parámetro de la siguiente forma:

```
set T:= {1..4};  
param Dem{T,T};
```

El código para definir el parámetro en el archivo <.dat> podría ser:

Ejemplo 2:

```
param dem :
      1      2      3      4 :=
1    19    31    41    44
2    41    18    36    17
3    22    42    13    37
4    22    37    18    15
;
```

</> Tridimensional o orden superior (un contenedor): si se ha definido en el archivo `<.mod>` un parámetro de la siguiente forma:

```
set T:= {1..3};
param Dem{T,T,T};
```

El código para definir el parámetro en el archivo `<.dat>` podría ser:

Ejemplo 3:

```
param dem :=
[*,* ,1]:
      1      2      3 :=
1    11    32    28
2    23    95    48
3    50    63    95
[*,* ,2]:
      1      2      3 :=
1     5    32    28
2    50    96     8
3    41    14     5
[*,* ,3]:
      1      2      3 :=
1    95    58     4
2    23    75     9
3    41    40     5
;
```

El uso de `<*>`, tal como la expresión `[*, *, 1]` en el ejemplo pasado, tiene como objeto indicarle a AMPL qué subíndices estarán en la tabla de datos (que se muestra por consola) asociada a `[*, *, 1]` y cuáles subíndices estarán fijos. En dicho ejemplo, el primer `<*>` implica que las filas de los datos se representarán por el primer subíndice, el segundo `<*>` implica que las

columnas representan el segundo subíndice, y el 1 implica que el tercer subíndice permanecerá constante. Esta interpretación se puede expandir a $[*, *, 2]$ y $[*, *, 3]$.



El orden de los $\langle * \rangle$ en el archivo se puede cambiar a preferencia del usuario, recordando que el primer $\langle * \rangle$ (en cualquier posición) será representado por las filas y el segundo $\langle * \rangle$ (en cualquier posición) será representado por las columnas de la tabla de datos asociada.

5.7.3 Declarar contenedores llenos por defecto

Si en el archivo $\langle .mod \rangle$ se ha definido un contenedor lleno por defecto (con un valor determinado), se pueden declarar de nuevo valores específicos en el archivo $\langle .dat \rangle$. Por ejemplo, supongamos que se tiene el siguiente parámetro (lleno de cero en todos sus elementos):

set S:= {1..3}; **param** B{I, I, I, I} **default** 0;

En el archivo $\langle .dat \rangle$ se podrían definir algunos valores, con se ilustra a continuación:

Ejemplo 7:

param B :=

1 1 1 2 5 $\leftarrow$ equivale a $B_{1,1,1,2} = 5$

2 3 2 1 6 $\leftarrow$ equivale a $B_{2,3,2,1} = 6$

;

5.8 ¿Cómo ejecutar el modelo en AMPL?

Para ejecutar un modelo se requiere cumplir las siguientes condiciones:

- (Obligatorio) tener mínimo dos archivos: el $\langle .mod \rangle$ (asociado el modelo matemático) y el $\langle .run \rangle$ (asociado a los comandos para AMPL).

- (Opcional) tener el archivo `<.dat>` (asociado a los parámetros del modelo matemático).
- Los archivos deben estar en el **directorio activo** (véase en la sección 5.3).
- Asegurarse que los archivos se encuentren **guardados**, es decir, que los archivos no tengan modificaciones no guardadas.

Una vez cumplidas las condiciones, se puede ejecutar el modelo mediante dos vías:

1. Usando el **directorio activo**: basta con dar clic derecho sobre el archivo `<.run>` y luego clic en la opción `<Sent To AMPL>` (Figura 5.5), o bien usar la combinación de teclas `Ctrl` + `R`.

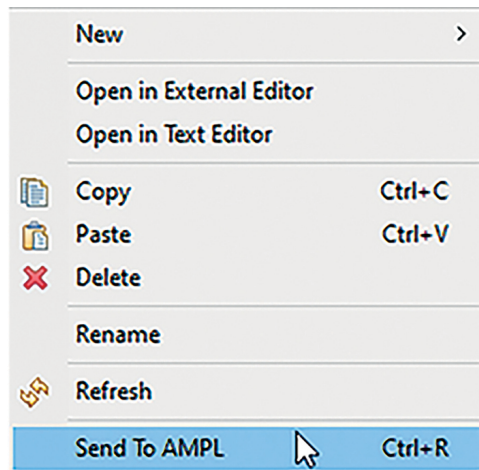


Figura 5.5: Ejecutar un modelo en AMPL usando el directorio activo.

2. Usando la **consola** (sección 5.3): usando el comando `include` y presionando la tecla enter (`↵`):

```
include nombre.run
```

5.9 Un ejemplo completo

A continuación, se tomará el problema 2.7 en su notación algebraica, para ilustrar un ejemplo completo de los archivos necesarios para ejecutar AMPL y resolver dicho problema. Para este ejemplo se usará el *solver* CPLEX, pero se podría usar cualquier otro solver disponible como Gurobi. En el Algoritmo 48 se encuentra el archivo `<.run>`, en el Algoritmo 49 el archivo `<.mod>`, y en el Algoritmo 50 el archivo `<.dat>`.

Algoritmo 48: ProblemaDeTransporte.run

```
1: reset;
2: model ProblemaTransporte.mod;
3: data ProblemaTransporte.dat;
4: option solver cplex;
5: solve;
6: display X,Z;
```

Algoritmo 49: ProblemaDeTransporte.mod

```
1: set I := {1..3};
2: set J := {1..6};
3: param Cap{I};
4: param Dem{J};
5: param C{I,J};
6: var X{I,J} >=0 integer;
7: minimize Z: sum{i in I, j in J} X[i,j]*C[i,j];
8: s.t.
9: r1{j in J}: sum{i in I} X[i,j] >= Dem[j];
10: r2{i in I}: sum{j in J} X[i,j] <= Cap[i];
```

Algoritmo 50: ProblemaDeTransporte.dat

```
1: param C :
2:      1      2      3      4      5      6 :=
3:      1      14      24      21      20      21.5      19
4:      2      17      30      24      28      25      18
5:      3      24      15      28      20      18.5      19.5
6: ;
7: param Cap:=
8:      1      60
9:      2      60
10:     3      25
```

```
11: ;  
12: param Dem:=  
13:    1    28  
14:    2    25  
15:    3    24  
16:    4    17  
17:    5    21  
18:    6    30  
19: ;
```



Los espacios entre los datos con tabulador () son estéticos, por tanto, podrían ser reemplazados por espacios simples, e incluso se pueden poner todos los datos en una sola línea. Sin embargo, el uso de los espacios  facilita la visualización de la información para el usuario.

Bibliografía

- [1] H. Prado, *Ejercicios de Investigación Operativa [Material de clase]*, Pontificia Universidad Javeriana-Cali, Colombia (vid. [pág. 15](#)).
- [2] F. Hillier, *Introduction to Operations Research*, 10th. McGraw-Hill Education, 2014, isbn: 9781259162985 (vid. [pág. 17](#)).
- [3] C. Vidal-Holguín, *Introducción a la Modelación Matemática y Optimización*. Universidad del Valle, Colombia, 2011 (vid. págs. [34](#), [37](#), [79](#), [82](#), [101](#), [104](#), [138](#), [140](#), [146](#), [149](#), [151](#), [158](#), [161](#)).
- [4] J. Escobar-Velasquez, *Ejercicios de Investigación Operativa [Material de clase]*, Pontificia Universidad Javeriana-Cali, Colombia (vid. págs. [69](#), [71](#)).
- [5] F. Chediak-Pinzón, *Investigación de operaciones, Volumen I*. Universidad de Ibagué, 2012, isbn: 9587540786 (vid. págs. [72](#), [74](#), [83](#), [86-89](#), [91](#)).
- [6] R. Fourer, D. M. Gay y B. W. Kernighan, *AMPL: A Modeling Language for Mathematical Programming*. Cengage Learning; 2nd edition, 2002, isbn: 0534388094 (vid. [pág. 211](#)).
- [7] Gropp y J. J. Moré, «Optimization Environments and the NEOS Server,» *Approximation Theory and Optimization*, M. D. Buhman y A. Iserles, eds., Cambridge University Press, 1997, págs. 167-182 (vid. [pág. 212](#)).
- [8] Czyzyk, M. P. Mesnier y J. J. Moré, «The NEOS Server,» *IEEE Journal on Computational Science and Engineering*, vol. 5, n.o 3, págs. 68 -75, 1998 (vid. [pág. 212](#)).
- [9] E. D. Dolan, «The NEOS Server 4.0 Administrative Guide», Mathematics y Computer Science Division, Argonne National Laboratory, Technical Memorandum ANL/MCS-TM-250, 2001 (vid. [pág. 212](#)).

- [10] NEOS Server. «NEOS Server: State-of-the-Art Solvers for Numerical Optimization.» (2021), dirección: <https://neos-server.org/neos/>. Accedido:05-01-2021(vid. pág. 212).
- [11] AMPL Optimization Inc. «Download a Free Demo - AMPL.» (2021), dirección: <https://ampl.com/try-ampl/download-a-free-demo/>. Accedido:05-01-2021(vid. págs. 212, 213).
- [12] AMPL Optimization Inc. «Buy AMPL Products - AMPL.» (2021), dirección: <https://ampl.com/try-ampl/buy-ampl/>. Accedido:05-01-2021 (vid. pág. 213).
- [13] L. Bettoni. «GUSEK (GLPK Under Scite Extended Kit).» (2018), dirección: <http://gusek.sourceforge.net/gusek.html/>. Accedido: 05-01-2021 (vid. pág. 213).

APRENDE MODELANDO: Programación Lineal Entera-Mixta Guía de aprendizaje desde el problema a la solución AMPL

Mientras que la mayoría de los libros relativos a los problemas de optimización se dedican principalmente al diseño de algoritmos heurísticos y exactos para la solución efectiva de los problemas considerados, el objetivo de este libro es facilitar la difícil tarea de la adecuada modelización de los problemas a abordar.

La tarea de enseñar a construir un modelo matemático correcto es muy difícil, mucho más que la de enseñar teoremas y cómo demostrarlos. La mejor técnica didáctica es guiar gradualmente a los alumnos, por un camino que les permita comprender las similitudes y las diferencias entre los componentes de los distintos modelos que se han presentado. Y esta es, efectivamente, la técnica que se sigue en este excelente libro.

Paolo Toth
Profesor emérito Università di Bologna



Pontificia Universidad
JAVERIANA
Cali



**Universidad
Andrés Bello®**